

CS 185/285: Deep Reinforcement Learning

Comprehensive Study Notes

UC Berkeley, Spring 2026

Instructor: Sergey Levine

Synthesized from: 22 lecture transcripts, 22 lecture slide decks,
8 discussion sections, 7 course staff note documents, and 20 quizzes (68 questions).

Generated April 14, 2026

Contents

1	Introduction & Foundations	19
1.1	What is Reinforcement Learning?	19
1.2	Intellectual Roots of RL	19
1.3	Sequential Decision Making and Non-i.i.d. Data	20
1.4	What Problems Can Be Framed as RL?	20
1.5	Standard Assumptions in RL	20
1.6	Overview of RL Algorithm Types	21
1.7	Check Your Understanding – Chapter 1	22
2	Imitation Learning & Behavioral Cloning	23
2.1	From Supervised Learning to Behavioral Cloning	23
2.1.1	Notation Change	23
2.1.2	Behavioral Cloning Algorithm	23
2.1.3	Action Representations	24
2.2	The Distributional Shift Problem	24
2.2.1	Why Behavioral Cloning Can Fail	24
2.2.2	The Tightrope Walker Example (Informal Bound)	25
2.2.3	Full Distributional Shift Derivation	25
2.3	DAgger: Dataset Aggregation	28
2.4	Why Imitation Learning Fails: Modeling Challenges	29
2.4.1	Non-Markovian Experts	29
2.4.2	Multimodal Distributions	30
2.4.3	Solutions to Multimodality	30
2.5	Additional Practical Techniques	31
2.5.1	Action Chunking	31
2.5.2	Data Collection Strategies	32

2.5.3	Pre-training vs. Post-training	32
2.6	Check Your Understanding – Chapter 2	32
3	MDP & POMDP Foundations	34
3.1	Markov Chains	34
3.1.1	Definition	34
3.1.2	The Transition Operator as a Matrix	34
3.1.3	Stationary Distribution	35
3.2	Markov Decision Processes (MDPs)	35
3.2.1	Definition	35
3.2.2	How Policy + Dynamics = Markov Chain over (s, a) Pairs	36
3.2.3	Trajectory Distribution	37
3.2.4	State-Action Marginals	37
3.3	Partially Observed MDPs (POMDPs)	37
3.3.1	Definition	37
3.3.2	Key Differences: POMDP vs. MDP	38
3.3.3	The Fully Observed Special Case	38
3.4	Finite vs. Infinite Horizon	39
3.4.1	Finite Horizon	39
3.4.2	Infinite Horizon	39
3.5	The RL Objective	40
3.5.1	Value Functions and Q Functions	40
3.5.2	Using Q Functions to Improve Policies	41
3.6	Worked Example: Markov Chain Computation	41
3.7	Check Your Understanding – Chapter 3	41
4	Policy Gradients	43
4.1	The RL Objective as Expectation over Trajectories	43
4.1.1	Why Not Just Use Autodiff?	44
4.2	REINFORCE Derivation – Full Step-by-Step with Log-Derivative Trick	44
4.2.1	Step 1: Write the objective as an integral	44
4.2.2	Step 2: Differentiate under the integral sign	44
4.2.3	Step 3: Apply the log-derivative trick	45
4.2.4	Step 4: Substitute back	45

4.2.5	Step 5: Simplify $\nabla_{\theta} \log p_{\theta}(\tau)$ – why dynamics terms drop out	45
4.2.6	Step 6: Final policy gradient formula	46
4.2.7	Step 7: Sample-based approximation	46
4.3	The REINFORCE Algorithm	46
4.3.1	Example: Gaussian Policies	47
4.3.2	Partial Observability	47
4.4	Variance Reduction	47
4.4.1	The Problem: High Variance	47
4.4.2	Baselines – Prove Unbiasedness	47
4.4.3	Reward-to-Go (Causality Trick)	49
4.4.4	Discounting	50
4.5	All Valid Policy Gradient Estimator Variants	50
4.6	Practical Implementation	51
4.6.1	Pseudo-Loss for Automatic Differentiation	51
4.6.2	Practical Tips	51
4.7	Check Your Understanding	52
5	Actor-Critic Methods	53
5.1	Motivation: From Monte Carlo Returns to Learned Value Functions	53
5.2	Value Functions: $V(s)$, $Q(s,a)$, $A(s,a)$ – Definitions and Relationships	54
5.2.1	State-Action Value Function (Q-function)	54
5.2.2	State Value Function (V-function)	54
5.2.3	Advantage Function	54
5.2.4	Key Relationships	54
5.2.5	ASCII Backup Diagram	55
5.3	Discount Factors	56
5.3.1	Why Discount?	56
5.3.2	Equivalent Interpretation as Modified Transition Probabilities (Death State)	56
5.3.3	Consequences of Discounting	57
5.4	Policy Evaluation and Value Function Fitting	57
5.4.1	Monte Carlo Policy Evaluation	57
5.4.2	Bootstrapped (TD) Policy Evaluation	58
5.5	The Actor-Critic Algorithm	58
5.5.1	Bias-Variance Tradeoff	58

5.5.2	Batch Actor-Critic Algorithm	59
5.5.3	Online Actor-Critic Algorithm (e.g., A3C)	59
5.6	Which Advantage Estimators Are Unbiased vs. Biased?	59
5.7	On-Policy vs. Off-Policy Actor-Critic	60
5.7.1	On-Policy Actor-Critic	60
5.7.2	Off-Policy Actor-Critic	60
5.7.3	Solution: Estimate Q Instead of V	61
5.7.4	Off-Policy Actor-Critic Algorithm	61
5.7.5	Reparameterization Trick (for continuous actions)	61
5.8	Generalized Advantage Estimation (GAE)	62
5.8.1	N-Step Returns: Controlling the Bias-Variance Tradeoff	62
5.8.2	GAE: Weighted Combination of All N-Step Returns	62
5.8.3	Efficient Computation via TD Errors	62
5.8.4	Policy Gradient with GAE	63
5.9	Summary: The Spectrum of Estimators	63
5.10	Check Your Understanding	64
6	Value-Based Methods	66
6.1	Motivation: Actor-Critic Without the Actor	66
6.2	Recovering the Optimal Policy from Value Functions	66
6.2.1	How to recover the optimal policy	67
6.3	Dynamic Programming: Policy Iteration and Value Iteration	67
6.3.1	Policy Iteration	67
6.3.2	Value Iteration	68
6.3.3	Convergence of Tabular Value Iteration	69
6.4	Fitted Value Iteration – and Why It Does Not Work Well	70
6.4.1	Two critical problems	70
6.5	Q-Learning and Fitted Q-Iteration	70
6.5.1	The Key Idea: Learn Q Instead of V	70
6.5.2	Why Q-Functions Enable Off-Policy Learning	71
6.5.3	Fitted Q-Iteration Algorithm	71
6.5.4	What Is Fitted Q-Iteration Optimizing?	71
6.6	Online Q-Learning	72
6.7	Exploration: Why Greedy Fails	72

6.7.1	Epsilon-Greedy Exploration	72
6.7.2	Boltzmann (Softmax) Exploration	72
6.8	Deep Q-Networks (DQN)	73
6.8.1	Experience Replay	73
6.8.2	Target Networks	73
6.8.3	The DQN Algorithm	74
6.8.4	The General View: Three Processes	74
6.8.5	Value Propagation and N-Step Returns	75
6.9	Double Q-Learning	75
6.9.1	The Overestimation Problem	75
6.9.2	Double Q-Learning: The Fix	76
6.9.3	Double DQN in Practice	76
6.9.4	Clipped Double Q-Learning	77
6.10	Continuous Actions with Q-Learning	77
6.10.1	Option 1: Optimization-Based	77
6.10.2	Option 2: DDPG / Actor Network	77
6.11	Convergence: NONE of These Converge with Neural Nets	78
6.11.1	Tabular Value Iteration Converges	78
6.11.2	Fitted Value Iteration Does NOT Converge	78
6.11.3	The Definitive Statement	79
6.12	Soft Actor-Critic (SAC)	79
6.12.1	Maximum Entropy Objective	80
6.12.2	SAC Components	80
6.12.3	SAC Losses Summary	80
6.13	Practical Tips for Deep Q-Learning	81
6.14	Summary Table	82
6.15	Check Your Understanding	82
7	Advanced Policy Gradients	84
7.1	Why REINFORCE Is Impractical	84
7.1.1	What We Would Like	85
7.2	Importance Sampling for Policy Gradients	85
7.2.1	Importance Sampling Review	85
7.2.2	Applying IS to the RL Objective	86

7.2.3	Trajectory Probability Ratio – Cancellation of Dynamics	86
7.2.4	Deriving the Off-Policy Policy Gradient	86
7.2.5	Expanding the Off-Policy Gradient ($\theta \neq \theta'$)	87
7.2.6	First-Order Approximation	87
7.3	Clipping Importance Weights	88
7.3.1	The Variance Problem	88
7.3.2	The Surrogate Objective	88
7.3.3	Clipped Weights	88
7.3.4	Asymmetric Clipping (PPO Detail)	89
7.4	Off-Policy Policy Gradient and the Surrogate Objective	90
7.4.1	Policy Gradient as Policy Iteration	90
7.4.2	The Policy Improvement Identity	90
7.4.3	Converting to an Off-Policy Objective	91
7.5	Trust Regions and Policy Constraints	91
7.5.1	Bounding the State Distribution Mismatch	91
7.5.2	Bounding the Objective Value	92
7.5.3	The Constrained Optimization Problem	93
7.5.4	From Total Variation to KL Divergence	93
7.5.5	KL Divergence: Forward vs. Reverse	94
7.6	Natural Policy Gradient	94
7.6.1	Motivation	94
7.6.2	KL Divergence as a Quadratic Form (Fisher Information Matrix)	95
7.6.3	The Natural Gradient Update	95
7.6.4	The Fisher Information Matrix	96
7.7	PPO (Proximal Policy Optimization)	96
7.7.1	PPO-Clip (Clipped Surrogate Objective)	96
7.7.2	PPO-KL (KL Penalty Variant)	97
7.8	TRPO (Trust Region Policy Optimization)	98
7.8.1	The TRPO Objective	98
7.8.2	TRPO Algorithm	98
7.8.3	TRPO vs. PPO	99
7.9	Summary: The Landscape of Advanced Policy Gradient Methods	99
7.10	Check Your Understanding	100

8	Variational Inference	102
8.1	Latent Variable Models	102
8.1.1	What Are They?	102
8.1.2	Concrete Example: Mixture of Gaussians	102
8.1.3	Continuous Latent Variables	103
8.1.4	Latent Variable Models in RL	103
8.2	The Intractability Problem	103
8.2.1	Why Can't We Just Maximize $\log p(x)$?	103
8.2.2	The Expected Log-Likelihood Alternative	104
8.3	The Evidence Lower Bound (ELBO)	104
8.3.1	Derivation 1: From Jensen's Inequality	104
8.3.2	Derivation 2: Via KL Divergence	105
8.3.3	When Is the ELBO Tight?	106
8.3.4	Why Maximize ELBO w.r.t. BOTH p and q ?	107
8.3.5	ELBO Decomposition Diagram	107
8.4	Entropy and KL Divergence Review	107
8.4.1	Entropy	107
8.4.2	KL Divergence	108
8.5	Variational Distributions $q(z)$	108
8.5.1	Basic Variational Inference Algorithm	108
8.5.2	The Problem: Parameter Explosion	108
8.6	Amortized Variational Inference	109
8.6.1	The Key Idea	109
8.6.2	The Amortized ELBO	109
8.6.3	Computing the Gradient w.r.t. ϕ	109
8.7	The Reparameterization Trick	110
8.7.1	What It Is	110
8.7.2	Why It's Needed	110
8.7.3	Comparison: Policy Gradient vs. Reparameterization Trick	110
8.8	Variational Autoencoders (VAEs)	111
8.8.1	Architecture	111
8.8.2	Training Objective	111
8.8.3	Why Sampling Works at Test Time	112
8.8.4	Concrete Walkthrough Example	112

8.8.5	Conditional VAEs	112
8.9	Connection to Diffusion Models	113
8.10	KL Divergence Deep Dive	114
8.10.1	Forward KL: $D_{\text{KL}}(p q)$ – Mode-Covering	114
8.10.2	Reverse KL: $D_{\text{KL}}(q p)$ – Mode-Seeking	114
8.10.3	The Asymmetry	114
8.10.4	When to Use Which	114
8.11	Check Your Understanding	115
9	Control as Inference	116
9.1	Motivation: Modeling Approximately Optimal Behavior	116
9.1.1	The Problem with Perfect Rationality	116
9.1.2	Why Model Suboptimality?	116
9.2	The Graphical Model for Control as Inference	117
9.2.1	The MDP with Optimality Variables	117
9.2.2	Optimality Variables $\mathcal{O}_t$	117
9.2.3	Trajectory Distribution Under Optimality	118
9.3	Running Inference in the Graphical Model	118
9.3.1	Backward Messages	118
9.3.2	Interpreting as Value Functions	119
9.3.3	The Soft Max	120
9.3.4	The Optimistic Backup Problem	120
9.4	Computing the Policy	121
9.4.1	Temperature Parameter	121
9.4.2	The Action Prior	122
9.5	Fixing the Optimistic Backup with Variational Inference	122
9.5.1	The Problem Revisited	122
9.5.2	The Variational Fix	123
9.6	Maximum Entropy RL	123
9.6.1	The Objective	123
9.6.2	Soft Value Functions	124
9.6.3	The Soft Bellman Equation	124
9.6.4	Full Derivation: Control-as-Inference to Max-Ent RL	124
9.6.5	Why Entropy Regularization Helps	125

9.6.6	Connection to SAC (Soft Actor-Critic)	125
9.7	Forward Messages and State Distributions	126
9.7.1	Forward Messages	126
9.7.2	State Distribution Along Optimal Trajectories	126
9.8	Inverse RL Through Control as Inference	126
9.9	Worked Examples	127
9.9.1	Worked Example 1: Grid World with Deterministic Dynamics	127
9.9.2	Worked Example 2: Stochastic Dynamics – The Optimism Problem	127
9.10	Summary: The Full Picture	128
9.11	Check Your Understanding	128
10	Inverse Reinforcement Learning & Reward Learning	130
10.1	Motivation: Why Learn Rewards?	130
10.2	Maximum Entropy IRL (MaxEnt IRL)	131
10.2.1	Setting Up the Problem	131
10.2.2	Maximum Likelihood Objective	131
10.2.3	Computing the Gradient	132
10.2.4	Estimating the Gradient in Practice	132
10.2.5	Importance Sampling for Efficient Updates	133
10.2.6	Algorithm 10.1: MaxEnt IRL (Practical Version)	133
10.3	Connection to GANs and GAIL	134
10.4	Learning Rewards from Human Preferences	135
10.4.1	The Bradley-Terry Model	135
10.4.2	Training the Reward Model	136
10.5	Check Your Understanding	136
11	Reinforcement Learning for Large Language Models	138
11.1	Background: The Anatomy of LLM Training	138
11.1.1	Two Paradigms for RL with LLMs	138
11.2	LLMs as MDPs	139
11.2.1	The One-Step (Contextual Bandit) Formulation	139
11.2.2	The Multi-Step (Token-Level) Formulation	139
11.2.3	Why LLMs Are Technically POMDPs	140
11.3	Policy Gradient for LLMs	140

11.3.1	The REINFORCE Estimator	140
11.3.2	Importance-Sampled Policy Gradient / PPO	141
11.3.3	Algorithm 11.1: PPO for LLMs	141
11.4	Baselines for LLMs	141
11.4.1	Standard Approach: GAE with a Value Function	141
11.4.2	GRPO: Baselines Without Value Functions	142
11.4.3	Algorithm 11.2: GRPO	142
11.5	Reference Model Regularization (KL Penalty)	143
11.5.1	The Problem: Reward Hacking	143
11.5.2	The Solution: KL Divergence Penalty	143
11.6	The Full RLHF Pipeline	144
11.6.1	Stage 1: Supervised Fine-Tuning (SFT)	144
11.6.2	Stage 2: Reward Model Training	144
11.6.3	Stage 3: RL Optimization	144
11.6.4	Algorithm 11.3: Full RLHF Pipeline	145
11.7	RL from Verifiers (Bonus)	145
11.8	Check Your Understanding	146
12	Model-Based Reinforcement Learning	147
12.1	Motivation: Why Learn a Model?	147
12.2	The Basic Model-Based RL Loop	148
12.2.1	Step 1: Collect data	148
12.2.2	Step 2: Learn the model	148
12.2.3	Step 3: Recover a policy	149
12.2.4	Step 4: Iterate	149
12.3	Distributional Shift in Model-Based RL	149
12.3.1	The Problem	149
12.3.2	The Fix: Iterate (Step 4)	149
12.4	Uncertainty Estimation	150
12.4.1	Aleatoric vs. Epistemic Uncertainty	150
12.4.2	Bayesian Neural Networks	151
12.4.3	Bootstrap Ensembles	151
12.4.4	Why Uncertainty-Aware Models Prevent Policy Exploitation	151
12.5	Planning with Learned Models	152

12.5.1	Open-Loop vs. Closed-Loop Planning	152
12.5.2	Random Shooting	153
12.5.3	Cross-Entropy Method (CEM)	153
12.5.4	Planning with Ensembles	154
12.5.5	Why Backpropagating Through Dynamics is Ill-Conditioned	154
12.6	Dyna-Style Methods: Short Model Rollouts	155
12.6.1	The Curse of Long Rollouts	155
12.6.2	The Dyna Framework	155
12.6.3	Parallel Process View	156
12.7	Latent State Space Models / Sequence VAEs	156
12.7.1	Motivation	156
12.7.2	The Variational Framework	157
12.7.3	The ELBO for Sequence Models	157
12.7.4	Deterministic Encoder (Simplified Version)	158
12.7.5	Using Latent Models for RL	158
12.8	Check Your Understanding	158
13	Offline Reinforcement Learning	160
13.1	What is Offline RL?	160
13.1.1	Why Study Offline RL?	160
13.1.2	How is Offline RL Different from Imitation Learning?	160
13.2	Why Standard Q-Learning Fails Offline: OOD Action Overestimation	161
13.2.1	The Problem	161
13.2.2	Empirical Evidence	162
13.2.3	This Affects All RL Methods	162
13.2.4	Why Q-Function Methods are Preferred for Offline RL	162
13.3	Approach 1: Explicit Policy Constraint Methods	163
13.3.1	The Constrained Actor Objective	163
13.3.2	Forward KL vs. Reverse KL	163
13.3.3	Reverse KL is Convenient: Add $\log \pi_\beta$ to Reward	164
13.3.4	AC+BC (Actor-Critic + Behavioral Cloning)	164
13.3.5	Advantage-Weighted Regression (AWR)	165
13.4	Approach 2: Implicit Policy Constraints — Implicit Q-Learning (IQL)	166
13.4.1	Core Idea: In-Sample Bellman Backup	166

13.4.2	Expectile Regression	167
13.4.3	Why This Isn't Over-Optimistic	167
13.4.4	The Complete IQL Algorithm	167
13.5	Approach 3: Conservative Q-Learning (CQL)	168
13.5.1	Core Idea: Push Down OOD Action Values	168
13.5.2	CQL Objective (Version 1 – Naive)	168
13.5.3	CQL Objective (Version 2 – Improved)	168
13.5.4	Practical Implementation	169
13.5.5	Flow Matching + Q Ranking (“Flow Q-Learning”)	169
13.6	Model-Based Offline RL	170
13.6.1	Why It's Challenging	170
13.6.2	Solutions	171
13.7	Summary and Comparison of Offline RL Methods	171
13.8	Connection to Previous Chapters	172
13.9	Check Your Understanding	172
14	Exploration	174
14.1	The Exploration Problem	174
14.1.1	Why Exploration Matters	174
14.1.2	Exploration vs. Exploitation	174
14.1.3	The Spectrum of Tractability	175
14.2	Multi-Armed Bandits	175
14.2.1	Setup	175
14.2.2	Bandits as POMDPs	176
14.2.3	Regret	176
14.3	Upper Confidence Bound (UCB) – Optimism in the Face of Uncertainty	177
14.3.1	The UCB Algorithm	177
14.3.2	The Classic UCB1 Formula	177
14.4	From Bandits to Deep RL: Count-Based Exploration	178
14.4.1	Exploration Bonuses in MDPs	178
14.4.2	Common Bonus Functions	178
14.5	Why Count-Based Exploration Fails in Deep RL	178
14.5.1	The Problem: $N(s) = 1$ Almost Always	178
14.5.2	The Insight: Similarity Matters	179

14.6	Pseudo-Counts and Density Models	179
14.6.1	The Idea	179
14.6.2	From Densities to Pseudo-Counts	179
14.6.3	Solving for the Pseudo-Count	179
14.6.4	The Pseudo-Count Algorithm (Bellemare et al., 2016)	180
14.6.5	What Kind of Model?	180
14.7	Prediction Error as Exploration Bonus	180
14.7.1	The Key Insight	180
14.7.2	General Framework	180
14.7.3	Dynamics Prediction Error	180
14.7.4	Random Network Distillation (RND)	181
14.8	Thompson Sampling	181
14.8.1	The Idea	181
14.8.2	Properties	182
14.9	Information Gain and Bayesian Exploration	182
14.9.1	Information Gain	182
14.9.2	Bayesian Exploration in MDPs	182
14.9.3	Connection to Other Methods	182
14.10	Check Your Understanding	183
15	RL Theory	185
15.1	What is the Point of RL Theory?	185
15.1.1	Types of Theoretical Questions	185
15.1.2	The Nature of Theoretical Results	185
15.1.3	Important Notational Distinction	186
15.1.4	Assumptions in RL Theory	186
15.1.5	What We Learn From Theory	186
15.2	Useful Identities and Notation	186
15.2.1	Vector Notation for Value Functions	186
15.2.2	Bellman Equation in Matrix Form	186
15.2.3	State-Action Transition Matrix	187
15.2.4	Closed-Form Policy Evaluation	187
15.2.5	Size of Q-values	187
15.3	Value Iteration Convergence Proof – Contraction Mapping	188

15.3.1	The Bellman Optimality Operator	188
15.3.2	Key Inequality	188
15.3.3	Proof: The Bellman Operator is a γ -Contraction	188
15.3.4	Proof: Value Iteration Converges to V^*	189
15.4	Sample Complexity Without Exploration	190
15.4.1	The Setup	190
15.4.2	Concentration Inequalities	190
15.4.3	The Simulation Lemma	191
15.4.4	Bounding the Evaluation Operator	192
15.5	The Main Error Bound	192
15.5.1	Putting It All Together	192
15.5.2	Interpretation of the Error Bound	193
15.6	Implications for Q^* and the Optimal Policy	194
15.6.1	From Q^π to Q^*	194
15.6.2	From Q^* to the Policy Performance	194
15.7	Approximation Error vs. Sampling Error in Fitted Q-Iteration	194
15.7.1	The Setting	194
15.7.2	Two Sources of Error	195
15.7.3	Analyzing Sampling Error	195
15.7.4	Analyzing Approximation Error	195
15.7.5	Putting Sampling and Approximation Error Together	196
15.8	Check Your Understanding	197
16	Algorithm Summary & Comparison	198
16.1	Imitation Learning Algorithms	198
16.1.1	Behavioral Cloning (BC)	198
16.1.2	DAGger (Dataset Aggregation)	199
16.2	Policy Gradient Algorithms	199
16.2.1	REINFORCE	199
16.2.2	REINFORCE with Baseline	199
16.2.3	Vanilla Actor-Critic	200
16.2.4	A2C / A3C (Advantage Actor-Critic / Asynchronous)	200
16.2.5	GAE (Generalized Advantage Estimation)	201
16.2.6	TRPO (Trust Region Policy Optimization)	201

16.2.7	Natural Policy Gradient	202
16.2.8	PPO (Proximal Policy Optimization)	202
16.3	Value-Based Algorithms	203
16.3.1	Q-Learning (Tabular)	203
16.3.2	DQN (Deep Q-Network)	203
16.3.3	Double DQN	203
16.3.4	Value Iteration	204
16.3.5	Policy Iteration	204
16.3.6	Fitted Value Iteration (Fitted VI)	205
16.3.7	Fitted Q-Iteration (FQI)	205
16.4	Continuous Action Algorithms	206
16.4.1	DDPG (Deep Deterministic Policy Gradient)	206
16.4.2	TD3 (Twin Delayed DDPG)	206
16.4.3	SAC (Soft Actor-Critic)	207
16.5	Model-Based RL Algorithms	207
16.5.1	Dyna	207
16.5.2	CEM / Random Shooting (Model Predictive Control)	208
16.5.3	MBPO (Model-Based Policy Optimization)	208
16.6	Inverse RL and Reward Learning	209
16.6.1	MaxEnt IRL (Maximum Entropy Inverse RL)	209
16.7	Offline RL Algorithms	209
16.7.1	AWR (Advantage-Weighted Regression)	209
16.7.2	CQL (Conservative Q-Learning)	210
16.7.3	IQL (Implicit Q-Learning)	210
16.7.4	Flow Matching + Q Ranking (“Flow Q-Learning”)	211
16.8	LLM/RLHF-Related Algorithms	212
16.8.1	GRPO (Group Relative Policy Optimization)	212
16.9	Exploration Algorithms	212
16.9.1	RND (Random Network Distillation)	212
16.10	Master Comparison Table	213
16.11	Decision Flowchart: Which Algorithm Should I Use?	215
16.12	Key Themes Across Algorithms	215
16.12.1	The Bias-Variance Tradeoff (Chapters 5-6)	215
16.12.2	On-Policy vs. Off-Policy (Chapters 5-8)	215

16.12.3	The Deadly Triad (Chapter 7)	215
16.12.4	Exploration vs. Exploitation (Chapter 14)	216
16.12.5	The $1/(1 - \gamma)$ Factor (Chapter 15)	216
17	Appendix: Key Identities and Formulas	217
17.1	A.1 The Log-Derivative Trick (REINFORCE Trick)	217
17.2	A.2 Score Function Has Zero Expectation	217
17.3	A.3 Baselines Don't Introduce Bias	218
17.4	A.4 Policy Gradient Theorem	218
17.5	A.5 Importance Sampling	219
17.6	A.6 KL Divergence	219
17.7	A.7 Total Variation Divergence	219
17.8	A.8 Jensen's Inequality	220
17.9	A.9 Evidence Lower Bound (ELBO)	220
17.10	A.10 Bellman Equations	220
17.11	A.11 Bellman Operator Contraction	221
17.12	A.12 Entropy	221
17.13	A.13 Soft Bellman Equations (Maximum Entropy RL)	222
17.14	A.14 Offline RL: KL Constraint as Reward Bonus	222
17.15	A.15 Simulation Lemma (Model Error $\rightarrow$ Policy Regret)	222
17.16	A.16 Distributional Shift Bound (Behavioral Cloning)	223
17.17	A.17 Value Function Notation Guide	223
17.17.1	The Core Definitions	223
17.17.2	Optimal Value Functions	223
17.17.3	Learned / Approximate Value Functions	224
17.17.4	Policy-Specific Q-Functions	224
17.17.5	Soft / Entropy-Regularized Value Functions	225
17.17.6	Summary: How They All Relate	225
18	Appendix B: Practice Midterm Solutions and Additional Problems	226
18.1	B.1 Answer Key (Practice Midterm)	226
18.2	B.2 Detailed Solutions	227
18.2.1	Problem 1: Distribution Shift for Success/Failure Tasks	227
18.2.2	Problem 2: Open Loop Control	228

18.2.3	Problem 3: Entropy-Regularized Policy Gradient	228
18.2.4	Problem 4: Learning in Simulation	229
18.2.5	Problem 5: Computing the Policy Gradient	229
18.2.6	Problem 6: Imitation Learning Analysis	230
18.2.7	Problem 7: IQL Variants	230
18.2.8	Problem 8: LLM MDPs	230
18.2.9	Problem 9: Multi-Step Dynamics Models	231
18.2.10	Problem 10: Managing Distribution Shift in Offline RL	231
18.3	B.3 Additional Practice Problems	232
18.3.1	Problem A1: Baselines in Policy Gradient	232
18.3.2	Problem A2: Bellman Operator Contraction	233
18.3.3	Problem A3: Variational Inference	233
18.3.4	Problem A4: Off-Policy Actor-Critic	234
18.3.5	Problem A5: Soft Actor-Critic	234
18.3.6	Problem A6: Importance Sampling in PPO	235
18.3.7	Problem A7: Distributional Shift in Model-Based RL	235
18.3.8	Problem A8: Inverse Reinforcement Learning	236
18.3.9	Problem A9: GRPO (Group Relative Policy Optimization)	237
18.3.10	Problem A10: Control as Inference	237
18.3.11	Problem A11: Exploration	238
18.3.12	Problem A12: Advantage Estimators	238
18.3.13	Problem A13: Offline RL OOD Problem	239
18.3.14	Problem A14: Natural Policy Gradient	239
18.3.15	Problem A15: RLHF Reward Hacking	240

Chapter 1

Introduction & Foundations

1.1 What is Reinforcement Learning?

Reinforcement learning (RL) is a paradigm for learning from **experience** rather than from **explicit supervision**. Unlike supervised learning, where a model learns to map inputs to correct outputs provided by a labeler, RL learns from a **reward signal** that indicates how good or bad an outcome was – without being told the correct action.

Motivating example. Consider a robotic arm that must pick up objects from a bin. In supervised learning, you would need a dataset of images labeled with the correct XYZ coordinates to grasp each object – but obtaining such labels is extremely difficult (humans do not know how to optimally control a robot arm). Instead, you can let the robots **practice**: try many grasps, observe which ones succeed and which fail, and use that success/failure signal to improve.

This is a fundamentally different learning problem:

Aspect	Supervised Learning	Reinforcement Learning
Data	(input, correct output) pairs	(state, action, reward) tuples
Signal	Correct labels	Reward / cost signal
Goal	Reproduce what is in the data	Discover behavior that maximizes reward
Data dependency	Samples are i.i.d.	Samples are non-i.i.d. – current actions affect future states

Professor's Note

“Generative AI is impressive because it reproduces some aspect of human behavior. Reinforcement learning is interesting because it leads to emergent behaviors that are different from what people would have done.” – AlphaGo discovered Go strategies that humans hadn’t thought of in thousands of years.

1.2 Intellectual Roots of RL

RL has two intellectual ancestors:

1. **Psychology** – B.F. Skinner’s study of how animals learn from reward and punishment. This is a model of learning from successes and failures, not from explicit instruction.
2. **Computer science / optimization** – Early work on evolutionary optimization (e.g., Karl Sims’ virtual creatures) showed that optimization could produce emergent behaviors. These methods were computationally expensive, but combining them with ideas from RL theory produced much more efficient algorithms.

Modern RL combines the algorithmic foundations from psychology-inspired RL (e.g., Q-learning) with optimization techniques to achieve emergent behavior on complex, long-horizon problems.

1.3 Sequential Decision Making and Non-i.i.d. Data

The defining feature that separates RL from supervised learning is **sequential decision making**:

- In supervised learning, each data point is drawn independently from a fixed distribution (i.i.d. assumption).
- In RL, each action affects the next state, which affects the next observation, which affects the next action. Data is **not i.i.d.**

This creates a fundamentally different challenge: small mistakes can **compound** over time. A tiny error at time step 1 shifts the state distribution at time step 2, causing a slightly larger error, which shifts the distribution further, and so on. We formalize this in Chapter 2.

1.4 What Problems Can Be Framed as RL?

RL is extremely general. Virtually any sequential decision-making problem can be framed as an RL problem, including:

- **Inventory management** – deciding how much stock to order over time
- **Robotics** – controlling a robot arm, legged locomotion, drones
- **Traffic control** – managing traffic signals to optimize flow
- **Game playing** – Go, chess, Atari, poker
- **Language model alignment** – RLHF for fine-tuning LLMs

Common Misconception

“Some of these problems (e.g., inventory management) cannot be framed as RL.” This is incorrect. **All** of the above can be framed as RL problems. The RL framework is general enough to encompass any problem where an agent takes actions in an environment and receives feedback.

1.5 Standard Assumptions in RL

The standard RL assumption is:

You are given a **reward signal** (a scalar function of state and/or action). You must discover the optimal behavior through interaction with the environment.

What you are **not** given: - Demonstrations of correct behavior (that would be imitation learning) - The optimal action at each state (that would be supervised learning) - A simulator of the environment (model-based RL can learn one, but it is not assumed)

Common Misconception

Students sometimes believe that RL assumes access to demonstrations, optimal actions, or a simulator. The standard assumption is **only** that a reward signal is provided.

1.6 Overview of RL Algorithm Types

Every RL algorithm has three components:

(See Figure 1.1)

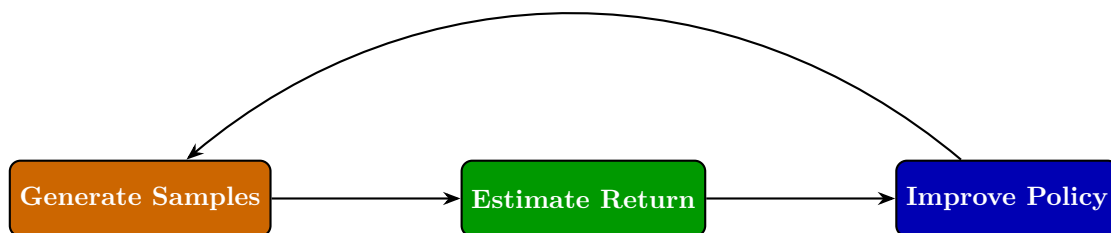


Figure 1.1: The RL Loop — generate samples, estimate return, improve policy.

Algorithm types (see Chapter 3 for details on value functions and Q functions):

1. **Policy gradient methods** – directly compute gradient of RL objective w.r.t. policy parameters
2. **Value-based methods** – estimate Q-function, derive policy from it (e.g., argmax)
3. **Actor-critic methods** – combine policy gradient with a learned value function
4. **Model-based methods** – learn a model of the environment, use it for planning or policy improvement

Sample efficiency ordering (least to most efficient):

(See Figure 1.2)

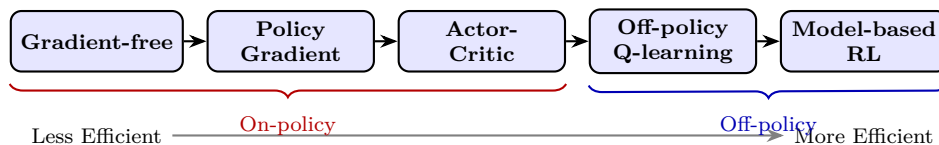


Figure 1.2: Sample Efficiency Spectrum.

Common Misconception

Students sometimes think more sample-efficient algorithms are always better. In practice, less efficient algorithms (e.g., policy gradients) can be **computationally** cheaper if you have access to a fast simulator. More efficient algorithms (e.g., model-based RL) trade compute for sample efficiency.

1.7 Check Your Understanding – Chapter 1

1. **What makes RL data fundamentally different from supervised learning data?** RL data is non-i.i.d. – the current action affects the next state, so samples are temporally correlated. The task is not to match labels but to maximize cumulative reward.
 2. **Can inventory management, robot control, traffic optimization, and Go all be framed as RL problems?** Yes – all of them involve sequential decision making with a reward signal.
 3. **What is the standard assumption in RL?** That a reward signal is given. Not demonstrations, not optimal actions, not a simulator.
-

Chapter 2

Imitation Learning & Behavioral Cloning

2.1 From Supervised Learning to Behavioral Cloning

2.1.1 Notation Change

We start with standard supervised learning and gradually transform it into a decision-making problem:

Supervised Learning	RL / Imitation Learning
Input X	Observation o_t
Output Y	Action a_t
Model $p_\theta(Y X)$	Policy $\pi_\theta(a_t o_t)$

The policy π_θ is a conditional distribution over actions given observations (or states), parameterized by neural network weights θ .

2.1.2 Behavioral Cloning Algorithm

Input: Demonstration trajectories $\{(o_1^i, a_1^i, o_2^i, a_2^i, \dots)\}_{i=1}^N$ collected by an expert.

Objective:

$$\theta^* = \arg \max_{\theta} \sum_{i=1}^N \sum_{t=1}^H \log \pi_\theta(a_t^i | o_t^i) \quad (1)$$

This is exactly **maximum likelihood estimation** applied to action prediction. It is identical to supervised learning with cross-entropy loss (discrete actions) or Gaussian log-likelihood (continuous actions).

Algorithm: > **Algorithm: Behavioral Cloning** > > — > > 1. Collect demonstrations $\mathcal{D} = \{\tau_1, \tau_2, \dots, \tau_N\}$ from expert > 2. Train policy π_θ by maximizing log-likelihood of actions in $\mathcal{D}$

(equivalent to standard supervised learning on (observation, action) pairs) > 3. Deploy π_θ at test time

2.1.3 Action Representations

For **discrete actions** (e.g., Atari keys, LLM tokens): use softmax output

$$\pi_\theta(a_t = k \mid o_t) = \frac{\exp(\text{logit}_k)}{\sum_j \exp(\text{logit}_j)} \quad (2)$$

For **continuous actions** (e.g., robot joint torques): use multivariate Gaussian

$$\pi_\theta(a_t \mid o_t) = \mathcal{N}(a_t; \mu_\theta(o_t), \Sigma_\theta(o_t)) \quad (3)$$

where the network outputs the mean μ and (often diagonal) covariance Σ . With identity covariance, this reduces to MSE regression.

2.2 The Distributional Shift Problem

2.2.1 Why Behavioral Cloning Can Fail

In standard supervised learning, train and test data come from the same distribution (i.i.d. assumption). In behavioral cloning, this assumption is violated:

- We **train** on states from $p_{\text{data}}(s_t)$ – the distribution visited by the expert
- We **test** on states from $p_{\pi_\theta}(s_t)$ – the distribution visited by our learned policy

Because our policy makes small mistakes, it drifts into states the expert never visited. In these unfamiliar states, the policy makes larger mistakes, drifting further, compounding errors.

(See Figure 2.1)

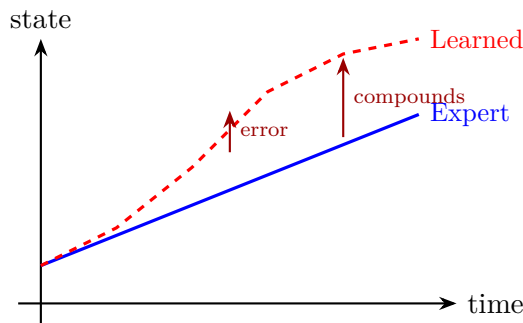


Figure 2.1: Distributional shift — errors compound as the learned policy drifts from expert trajectories.

Intuition

Think of a tightrope walker. On the rope (training distribution), mistakes are small. One step off the rope, you are in territory never seen during training, and every subsequent action is essentially random.

2.2.2 The Tightrope Walker Example (Informal Bound)

Consider a grid world where the agent must walk along a single row of H cells. At each step, with probability ϵ it makes a mistake and falls off the rope. Once off the rope, it makes a mistake at every subsequent step.

- At time step 1: expected mistakes = $\epsilon \cdot H$ (fall off, then H mistakes)
- At time step 2 (given survived step 1): expected mistakes = $\epsilon \cdot (H - 1)$
- ...

Total expected mistakes:

$$\epsilon H + (1 - \epsilon)\epsilon(H - 1) + \dots \approx \epsilon \cdot \frac{H(H + 1)}{2} = O(\epsilon H^2) \quad (4)$$

2.2.3 Full Distributional Shift Derivation

We now prove the $O(\epsilon H^2)$ bound rigorously.

Setup: - Expert policy $\pi^*(s_t)$ is deterministic - Fully observed case: $o_t = s_t$ - Cost function:

$$c(s_t, a_t) = \begin{cases} 0 & \text{if } a_t = \pi^*(s_t) \\ 1 & \text{otherwise} \end{cases} \quad (5)$$

- Expected cost at state s_t equals the probability of making a mistake:

$$\mathbb{E}_{a_t \sim \pi_\theta} [c(s_t, a_t)] = \pi_\theta(a_t \neq \pi^*(s_t) \mid s_t) \quad (6)$$

Assumption: Under the training distribution, the expected probability of mistake is at most ϵ :

$$\mathbb{E}_{s_t \sim p_{\text{train}}} [\pi_\theta(a_t \neq \pi^*(s_t) \mid s_t)] \leq \epsilon \quad (7)$$

Goal: Bound the total expected number of mistakes over a trajectory of length H :

$$J_{\text{cost}} = \sum_{t=1}^H \mathbb{E}_{a_t \sim \pi_\theta, s_t \sim p_{\pi_\theta}} [c(s_t, a_t)] \quad (8)$$

Note: states are sampled from p_{π_θ} (the policy's own distribution), NOT from training data.

Step 1: State Distribution Decomposition

We decompose $p_{\pi_\theta}(s_t)$ into two cases: - Case A: No mistakes made in first t steps $\rightarrow$ still on p_{train} - Case B: At least one mistake $\rightarrow$ on some unknown distribution p_{mistake}

$$p_{\pi_\theta}(s_t) = (1 - \epsilon)^t \cdot p_{\text{train}}(s_t) + (1 - (1 - \epsilon)^t) \cdot p_{\text{mistake}}(s_t) \quad (9)$$

where: - $(1 - \epsilon)^t$ = probability of zero mistakes over t steps - $p_{\text{mistake}}(s_t)$ = unknown distribution after a mistake (could be anything)

Step 2: Bounding Total Variation Divergence

Total Variation (TV) divergence measures how different two distributions are:

$$D_{\text{TV}}(p, q) = \frac{1}{2} \sum_x |p(x) - q(x)| \quad (\text{bounded by } 1) \quad (10)$$

Compute $D_{\text{TV}}(p_{\text{train}}, p_{\pi_\theta})$:

$$\begin{aligned} D_{\text{TV}}(p_{\text{train}}, p_{\pi_\theta}) &= \frac{1}{2} \sum_{s_t} |p_{\text{train}}(s_t) - p_{\pi_\theta}(s_t)| \\ &\quad // \text{ Substitute Eq. (9):} \\ &= \frac{1}{2} \sum_{s_t} |p_{\text{train}}(s_t) - (1 - \epsilon)^t p_{\text{train}}(s_t) - (1 - (1 - \epsilon)^t) p_{\text{mistake}}(s_t)| \\ &\quad // \text{ Factor out from the } p_{\text{train}} \text{ terms:} \\ &\quad // \quad p_{\text{train}} - (1 - \epsilon)^t p_{\text{train}} = (1 - (1 - \epsilon)^t) p_{\text{train}} \\ &= \frac{1}{2} \sum_{s_t} |(1 - (1 - \epsilon)^t) p_{\text{train}}(s_t) - (1 - (1 - \epsilon)^t) p_{\text{mistake}}(s_t)| \quad (11) \\ &\quad // \text{ Factor out the constant:} \\ &= (1 - (1 - \epsilon)^t) \cdot \frac{1}{2} \sum_{s_t} |p_{\text{train}}(s_t) - p_{\text{mistake}}(s_t)| \\ &\quad // \text{ The remaining sum is } D_{\text{TV}}(p_{\text{train}}, p_{\text{mistake}}) \leq 1: \\ &\leq (1 - (1 - \epsilon)^t) \cdot 1 \\ &= 1 - (1 - \epsilon)^t \end{aligned}$$

Bernoulli's inequality: For $\epsilon \in [0, 1]$:

$$\begin{aligned} (1 - \epsilon)^t &\geq 1 - \epsilon t \\ \Rightarrow 1 - (1 - \epsilon)^t &\leq \epsilon t \end{aligned} \quad (12)$$

(This follows from $(1 + x)^n \geq 1 + nx$ for $x \geq -1$.)

Therefore:

$$D_{\text{TV}}(p_{\text{train}}, p_{\pi_\theta}) \leq \epsilon t \quad (13)$$

Intuition

The distributions diverge **linearly** with time. At each time step, there is an ϵ probability of making a mistake that pushes you off the training distribution, so after t steps, the total divergence is at most ϵt .

Step 3: Bounding the Total Cost

Write out the expected cost at time t :

$$\mathbb{E}_{s_t \sim p_{\pi_\theta}} [\mathbb{E}_{a_t} [c(s_t, a_t)]] = \sum_{s_t} p_{\pi_\theta}(s_t) \cdot \mathbb{E}_{a_t} [c(s_t, a_t)] \quad (14)$$

Add and subtract $p_{\text{train}}(s_t)$:

$$\begin{aligned} &= \sum_{s_t} (p_{\text{train}}(s_t) + p_{\pi_\theta}(s_t) - p_{\text{train}}(s_t)) \cdot \mathbb{E}_{a_t} [c(s_t, a_t)] \\ &= \underbrace{\sum_{s_t} p_{\text{train}}(s_t) \cdot \mathbb{E}_{a_t} [c(s_t, a_t)]}_{\text{Term A}} + \underbrace{\sum_{s_t} (p_{\pi_\theta}(s_t) - p_{\text{train}}(s_t)) \cdot \mathbb{E}_{a_t} [c(s_t, a_t)]}_{\text{Term B}} \end{aligned} \quad (15)$$

Step 4: Bounding Each Term

Term A is the expected cost under the training distribution:

$$\text{Term A} = \mathbb{E}_{s_t \sim p_{\text{train}}} [\pi_\theta(a_t \neq \pi^*(s_t) \mid s_t)] \leq \epsilon \quad (16)$$

by our assumption (Eq. 7).

Term B is the extra cost due to distributional shift:

$$\begin{aligned} \text{Term B} &= \sum_{s_t} (p_{\pi_\theta}(s_t) - p_{\text{train}}(s_t)) \cdot \mathbb{E}_{a_t} [c(s_t, a_t)] \\ &\quad // \text{ Take absolute value (can only increase):} \\ &\leq \sum_{s_t} |p_{\pi_\theta}(s_t) - p_{\text{train}}(s_t)| \cdot \mathbb{E}_{a_t} [c(s_t, a_t)] \\ &\quad // \text{ Cost is bounded by 1:} \\ &\leq \sum_{s_t} |p_{\pi_\theta}(s_t) - p_{\text{train}}(s_t)| \cdot 1 \\ &= 2 \cdot D_{\text{TV}}(p_{\pi_\theta}, p_{\text{train}}) \\ &\leq 2\epsilon t \end{aligned} \quad (17)$$

Step 5: Final Result

Combining Terms A and B and summing over all time steps:

$$\begin{aligned}
 \sum_{t=1}^H \mathbb{E}[c(s_t, a_t)] &\leq \sum_{t=1}^H (\epsilon + 2\epsilon t) \\
 &= \epsilon H + 2\epsilon \sum_{t=1}^H t \\
 &= \epsilon H + 2\epsilon \cdot \frac{H(H+1)}{2} \\
 &= \epsilon H + \epsilon H(H+1) \\
 &= \epsilon H(1 + H + 1) \\
 &= O(\epsilon H^2)
 \end{aligned} \tag{18}$$

Intuition

The total expected number of mistakes has two components: - ϵH – mistakes you would make even without distributional shift (same as supervised learning) - ϵH^2 – additional mistakes from distributional shift (the compounding error term)

The quadratic term dominates. Compare with i.i.d. supervised learning where total error is $O(\epsilon H)$ – linear in H . The quadratic scaling is the **price of non-i.i.d. data**.

Common Misconception: “ $O(\epsilon H^2)$ is an exact prediction of how many mistakes you will make.” No – it is an **upper bound** on the worst case. Real-world problems are often much better because: (a) neural networks generalize beyond the training distribution, (b) many systems allow recovery from mistakes (unlike the tightrope), and (c) careful data collection can mitigate the problem.

2.3 DAgger: Dataset Aggregation

DAgger (Ross et al., 2011) provably addresses distributional shift by making p_{data} match p_{π_θ} .

Algorithm: DAgger (Dataset Aggregation)

1. Collect initial demonstrations $\mathcal{D}_0$ from expert
2. Train initial policy π_θ on $\mathcal{D}_0$
3. For iteration $n = 1, 2, \dots$:
 - (a) Run π_θ in the environment, collect observations $\{o_t\}$
 - (b) Query expert for optimal actions: $a_t^* = \pi^*(o_t)$ for each o_t
 - (c) Aggregate: $\mathcal{D}_n = \mathcal{D}_{n-1} \cup \{(o_t, a_t^*)\}$
 - (d) Retrain π_θ on $\mathcal{D}_n$

4. Return π_θ

Why it works: As more data is collected from p_{π_θ} and added to the dataset, eventually most of D consists of on-policy data. The distributional shift between p_{data} and p_{π_θ} vanishes asymptotically, and the error bound improves to $O(\epsilon H)$ – linear, not quadratic.

Challenges with DAgger: - Requires an expert who can label **arbitrary** observations (not just ones from their own trajectories) - For many tasks (e.g., labeling robot joint torques from arbitrary states), humans cannot provide good labels - Non-Markovian human labelers struggle when shown isolated frames without context

Professor’s Note

“The challenge with DAgger is really the step of asking the human to label the actions. It’s very problem dependent – for chess, a good player can look at any board and pick the right move. But for a humanoid robot doing a backflip, you’d have a really hard time getting a human demonstrator to figure out the right joint torques.”

2.4 Why Imitation Learning Fails: Modeling Challenges

Even with enough data and no distributional shift, behavioral cloning can fail due to two fundamental modeling problems.

2.4.1 Non-Markovian Experts

A Markovian policy $\pi(a_t|o_t)$ depends only on the current observation. But human experts are **non-Markovian** – they use memory of past observations.

Why humans are non-Markovian: - **Bounded rationality** – humans cannot recompute optimal actions from scratch at each time step; they maintain internal state - **Reaction to past events** – if a car cuts you off, you may swerve even after the danger has passed

Solutions: History-dependent policies

Use sequence models (transformers) to process observation histories:

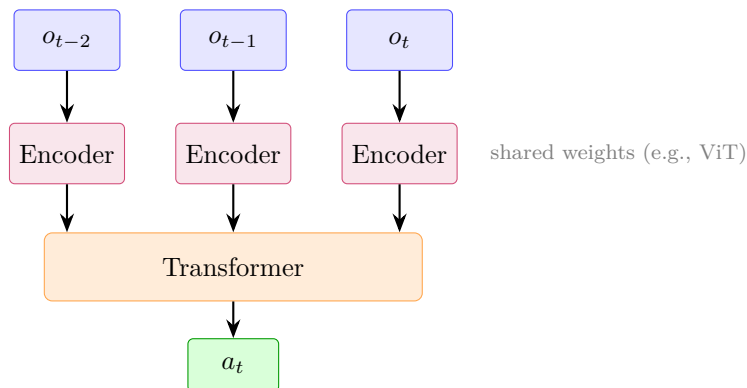


Figure 2.2: History-dependent policy via transformer: observations o_{t-2}, o_{t-1}, o_t are encoded (shared weights) and passed through a transformer to produce action a_t .

Caution: History can **worsen** distributional shift because: - More inputs = more ways for inputs to be out-of-distribution - Spurious correlations with past actions (causal confusion)

Professor's Note

Professor's Note (Causal Confusion): Imagine a car with a brake light on the dashboard. In training data, the light is on 49/50 times when you brake. The policy learns to brake when it sees the light – a **spurious correlation**. The light was *caused* by braking, not the other way around. With history, these spurious cues are even more prevalent.

2.4.2 Multimodal Distributions

If the expert sometimes goes left around a tree and sometimes goes right, a unimodal Gaussian policy will **average** the two modes and drive straight into the tree.

(See Figure 2.3)

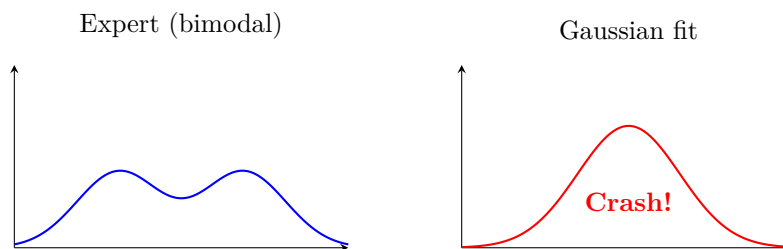


Figure 2.3: Multimodal expert distribution leads to averaging behavior when fit with a unimodal Gaussian.

This is only a problem for **continuous actions** with unimodal distributions. Discrete actions with softmax naturally handle multimodality (50% left, 50% right, 0% straight).

2.4.3 Solutions to Multimodality

Solution 1: Autoregressive Discretization

Discretize each action dimension sequentially using the chain rule of probability:

$$p(a_0, a_1, a_2 \mid s_t) = p(a_0 \mid s_t) \cdot p(a_1 \mid s_t, a_0) \cdot p(a_2 \mid s_t, a_0, a_1) \quad (19)$$

Implementation: Use a transformer where each time step outputs a softmax over bins for one dimension, conditioned on **samples** from previous dimensions.

(See Figure 2.4)

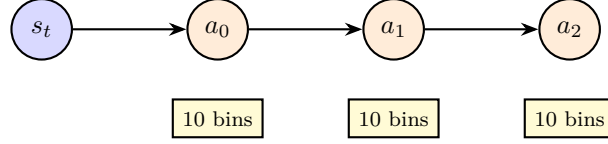


Figure 2.4: Autoregressive discretization — each action dimension is predicted sequentially with 10 bins.

- Complexity: $O(d \cdot K)$ instead of $O(K^d)$ for d dimensions with K bins each
- Captures correlations between dimensions via autoregressive conditioning
- **Critical:** you must pass the **sample** (not the probabilities) from earlier dimensions

Solution 2: Flow Matching

Flow matching (a modern variant of diffusion models) learns a **vector field** that transforms a simple base distribution (e.g., Gaussian noise) into the complex action distribution.

Core idea: Instead of changing the output to have multiple modes, add a **random input** (noise) that determines which mode to select.

(See Figure 2.5)

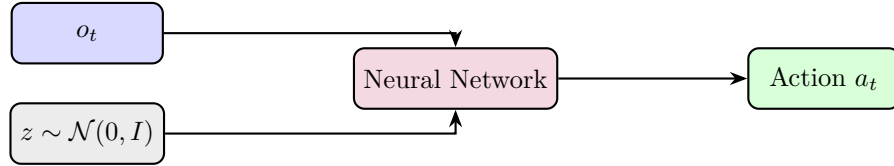


Figure 2.5: Flow Matching — observation and noise are combined by a neural network to produce an action.

Training procedure: > **Algorithm: Flow Matching Training** > > — > > For each mini-batch element: > 1. Sample (o_t, a_t) from dataset > 2. Sample noise $z \sim \mathcal{N}(0, I)$ > 3. Sample flow time $\tau \sim \text{Uniform}(0, 1)$ > 4. Compute intermediate point: $x_\tau = (1 - \tau)z + \tau a_t$ > 5. Target velocity: $v_{\text{target}} = a_t - z$ > 6. Loss: $\|v_\theta(o_t, x_\tau, \tau) - v_{\text{target}}\|^2 \dots (20)$

At inference: sample z , then integrate the learned vector field from $\tau = 0$ to $\tau = 1$ (typically ~ 10 steps for actions, ~ 100 for images).

Solution 3: Mixture Density Networks

Output parameters for a mixture of Gaussians: weights, means, and covariances for K mixture components. Less commonly used in modern practice.

2.5 Additional Practical Techniques

2.5.1 Action Chunking

Instead of predicting one action at a time, predict a **sequence** of K future actions:

$$o_t \longrightarrow [a_t, a_{t+1}, \dots, a_{t+K-1}]$$

Then execute some subset (e.g., 10 of 50). This helps because: - Effectively reduces the horizon (fewer decision points) - Provides stronger training signal per prediction - State-of-the-art for robotic manipulation (e.g., Diffusion Policy)

2.5.2 Data Collection Strategies

- **Include imperfect demonstrations** – showing mistakes and recoveries helps the policy learn corrective behaviors
- **Data augmentation** – e.g., NVIDIA’s three-camera trick (left/center/right cameras with adjusted action labels)
- **Goal-conditioned behavioral cloning** – condition policy on the goal state; relabel diverse trajectories with their actual endpoints

2.5.3 Pre-training vs. Post-training

This mirrors the recipe used for large language models:

Phase	Data	Quality	Purpose
Pre-training	Massive, heterogeneous	Low-to-medium	Learn diverse representations; see many situations (including failures)
Post-training (fine-tuning)	Small, curated	High	Learn correct behavior; narrow to desired task

Common Misconception

“Pre-training uses high-quality data and post-training uses low-quality data.” It is the **opposite**. Pre-training uses broad, lower-quality data to build general knowledge. Post-training uses narrow, high-quality data to refine behavior.

Example: For a robotic foundation model (e.g., Pi Zero), pre-training uses ~10,000 hours of diverse robot data, and post-training uses carefully selected demonstrations of specific tasks (folding laundry, assembling boxes).

2.6 Check Your Understanding – Chapter 2

1. **What is distributional shift in behavioral cloning?** The policy is trained on the expert’s state distribution p_{data} , but at test time it visits its own distribution p_{π_θ} . Because the policy makes small mistakes, it drifts into states not seen during training, causing larger mistakes. The error bound is $O(\epsilon H^2)$.
2. **Is $O(\epsilon H^2)$ an exact error or an upper bound?** It is an **upper bound** on the worst-case total number of mistakes. Actual performance is often much better.

3. **When does imitation learning fail even with perfect data coverage?** When (a) the expert is non-Markovian but the policy is Markovian (cannot capture temporal dependencies), or (b) the expert’s behavior is multimodal but the policy uses a unimodal distribution (averages modes).
 4. **Which methods effectively handle multimodal continuous action distributions?** Autoregressive discretization, flow matching (diffusion), and mixture density networks. A simple Gaussian does NOT handle multimodality.
 5. **How does DAgger fix distributional shift?** By iteratively collecting data from the learned policy’s distribution and relabeling it with expert actions, so the training distribution converges to the policy’s distribution.
-

Chapter 3

MDP & POMDP Foundations

3.1 Markov Chains

3.1.1 Definition

A **Markov chain** is the simplest probabilistic model of a dynamical system. It consists of:

- **S** – a set of states (discrete or continuous)
- **T** – a transition operator: $p(s_{t+1}|s_t)$

(See Figure 3.1)

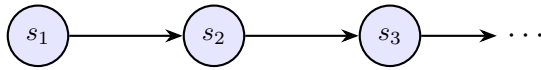


Figure 3.1: Markov Chain — a sequence of states where each state depends only on the previous one.

The **Markov property** states that the future is conditionally independent of the past given the present:

$$p(s_{t+1}|s_t, s_{t-1}, \dots, s_1) = p(s_{t+1}|s_t) \quad (21)$$

Intuition

If you know the present state completely, knowing the past gives you no additional information about the future.

3.1.2 The Transition Operator as a Matrix

For **discrete states** with $|S| = n$, the transition probabilities can be written as an $n \times n$ matrix:

$$T_{ij} = p(s_{t+1} = i \mid s_t = j) \quad (22)$$

Define a **state distribution vector** μ_t of dimension n , where entry i is $p(s_t = i)$. Then:

$$\mu_{t+1} = T \cdot \mu_t \quad (23)$$

This is a **matrix-vector multiply** – we can run the dynamical system forward using linear algebra.

For k steps:

$$\mu_{t+k} = T^k \cdot \mu_t \quad (24)$$

Intuition

Exponentiating the transition matrix propagates the state distribution forward by k steps. This is why we write things with linear algebra – it gives us concise formulas for analyzing multi-step behavior.

3.1.3 Stationary Distribution

A **stationary distribution** $\bar{\mu}$ satisfies:

$$\bar{\mu} = T \cdot \bar{\mu} \quad (25)$$

Rearranging:

$$(T - I) \cdot \bar{\mu} = 0 \quad (26)$$

This means $\bar{\mu}$ is an **eigenvector of T with eigenvalue 1**.

Existence conditions: If the Markov chain is: - **Ergodic** – any state can be reached from any other state - **Aperiodic** – does not cycle with a fixed period

then there exists a **unique** stationary distribution, and the chain converges to it regardless of the initial state.

Intuition

“If I go for a random walk around the UC Berkeley campus, and I’ve had a few too many drinks, after a few hours I won’t remember where I started.” – Prof. Levine. Eventually, you forget your starting point and enter the stationary distribution.

3.2 Markov Decision Processes (MDPs)

3.2.1 Definition

An MDP adds **actions** and **rewards** to a Markov chain. It is defined by a tuple (S, A, T, r) :

- S – state space

- $\mathbf{A}$ – action space
- $\mathbf{T}$ – transition operator: $p(s_{t+1}|s_t, a_t)$ – now conditioned on actions
- $\mathbf{r}$ – reward function: $r(s_t, a_t) \rightarrow \mathbb{R}$

(See Figure 3.2)

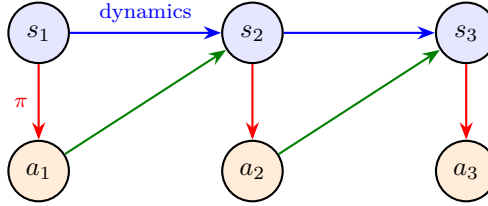


Figure 3.2: MDP as a Bayes Net — states, actions, and transition dynamics.

The reward function tells us which state-action pairs we **prefer**: - Driving happily on the road: $r = 0$ - Arriving at destination: $r = +1$ - Car accident: $r = -1,000,000$

Professor's Note

“The reward function doesn’t tell you what you *should do*. It tells you which *outcomes* you prefer. Figuring out the actions that lead to good outcomes – that’s the RL problem.”

3.2.2 How Policy + Dynamics = Markov Chain over (s, a) Pairs

Given a fixed policy $\pi(a|s)$, an MDP reduces to a Markov chain over **state-action pairs** (s, a) .

Define the augmented state as the tuple (s_t, a_t) . The transition probability for this Markov chain is:

$$p(s_{t+1}, a_{t+1} \mid s_t, a_t) = p(s_{t+1} \mid s_t, a_t) \cdot \pi(a_{t+1} \mid s_{t+1}) \quad (27)$$

This is key because: - The first factor is the **environment dynamics** (physics) - The second factor is the **policy** (our choice) - Together, they define a complete Markov chain with no free parameters

Transition operator as a matrix: If we have n states and m actions, the augmented state vector μ has $n \cdot m$ entries:

$$\mu = [p(s_1, a_1), p(s_1, a_2), \dots, p(s_1, a_m), p(s_2, a_1), \dots]^\top \quad (28)$$

The transition matrix T_θ has dimension $(nm) \times (nm)$, with entries:

$$T_{\theta,ij} = p(s' = s_i, a' = a_i \mid s = s_j, a = a_j) = p(s_i \mid s_j, a_j) \cdot \pi_\theta(a_i \mid s_i) \quad (29)$$

And just as before:

$$\mu_{t+1} = T_\theta \cdot \mu_t \quad (30)$$

Intuition

Once you fix the policy, the MDP becomes “just” a Markov chain, and all the Markov chain machinery (matrix multiplication, stationary distributions, eigenvectors) applies. This is the foundation for analyzing RL objectives.

3.2.3 Trajectory Distribution

For a fixed policy π_θ , the probability of a trajectory $\tau = (s_1, a_1, s_2, a_2, \dots, s_H, a_H)$ is:

$$p_\theta(\tau) = p(s_1) \prod_{t=1}^H [\pi_\theta(a_t | s_t) \cdot p(s_{t+1} | s_t, a_t)] \quad (31)$$

This follows directly from the chain rule of probability applied to the MDP Bayes net.

3.2.4 State-Action Marginals

The **state-action marginal** at time t is the probability of being in a particular (s_t, a_t) :

$$p_\theta(s_t, a_t) = \sum_{s_1, a_1, \dots, s_{t-1}, a_{t-1}} p_\theta(\tau) \quad (32)$$

This is ugly to write but **easy to sample**: just run the policy in the environment and record (s_t, a_t) .

Professor’s Note

“We work with probabilities when we want to do theory, and we work with samples when we want to actually code things up. The probabilities are never stored in the computer.”

3.3 Partially Observed MDPs (POMDPs)**3.3.1 Definition**

A POMDP extends the MDP with an **observation model**. The agent does not directly observe states – it observes noisy, partial observations.

A POMDP is defined by (S, A, O, T, E, r) :

- **O** – observation space
- **E** – emission/observation probabilities: $p(o_t | s_t)$
- Everything else as in the MDP

(See Figure 3.3)

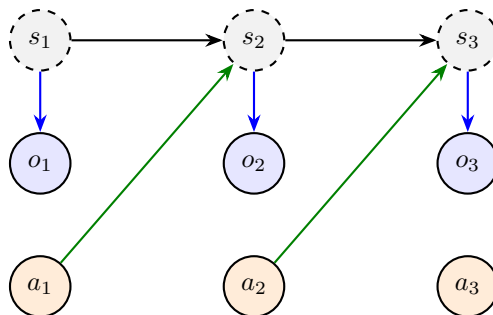


Figure 3.3: POMDP — hidden states (dashed), observations, and actions.

The policy now conditions on observations: $\pi_\theta(a_t|o_t)$.

3.3.2 Key Differences: POMDP vs. MDP

Common Misconception

“Observations satisfy the Markov property just like states.” **No.** States satisfy the Markov property by definition. Observations do **not** in general – the future observation is **not** conditionally independent of past observations given the current observation. This is because observations are a noisy, partial view of the underlying state.

Important consequences:

1. **Stochastic optimal policies may be necessary.** In a fully observed MDP, there always exists an optimal deterministic policy. In a POMDP, the optimal policy may need to be **stochastic** because the agent is uncertain about the true state.
2. **Information-gathering actions become valuable.** In a POMDP, it may be worth taking an action that yields no immediate reward but helps reduce uncertainty about the state (e.g., looking around before crossing a street).
3. **History may be needed.** Because observations are not Markovian, the optimal policy may need to depend on a **history** of observations, not just the current one. Maintaining a **belief state** (probability distribution over possible states) can serve as a sufficient statistic.

3.3.3 The Fully Observed Special Case

When $o_t = s_t$ (observation equals state), the POMDP reduces to an MDP. The Bayes net simplifies:

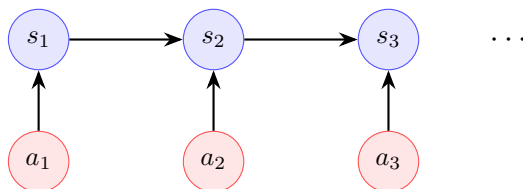


Figure 3.4: Fully observed MDP: states connected sequentially, actions influence transitions.

Many RL algorithms assume or require the fully observed case. When the professor writes $\pi(a|s)$ instead of $\pi(a|o)$, it implies full observability.

Common Misconception

“The transition probabilities in a POMDP depend on observations.” No – transitions depend only on states and actions: $p(s_{t+1}|s_t, a_t)$. Observations are a **consequence** of states, not a cause of transitions.

3.4 Finite vs. Infinite Horizon

3.4.1 Finite Horizon

For a trajectory of fixed length H , the RL objective is:

$$\begin{aligned} J(\theta) &= \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t=1}^H r(s_t, a_t) \right] \\ &= \sum_{t=1}^H \mathbb{E}_{(s_t, a_t) \sim p_\theta(s_t, a_t)} [r(s_t, a_t)] \end{aligned} \tag{33}$$

(by linearity of expectation)

Using the Markov chain representation:

$$\mathbb{E}[r_t] = \mu_t^\top r = (T_\theta^{t-1} \mu_1)^\top r \tag{34}$$

where r is the vector of rewards for each (s, a) pair.

3.4.2 Infinite Horizon

When $H \rightarrow \infty$, the sum of rewards diverges unless we normalize. We use the **average reward**:

$$J(\theta) = \lim_{H \rightarrow \infty} \frac{1}{H} \sum_{t=1}^H \mathbb{E}[r(s_t, a_t)] \tag{35}$$

Under ergodicity and aperiodicity conditions, the Markov chain converges to a unique **stationary distribution** $\bar{\mu}$. Almost all terms in the average come from this distribution, so:

$$J(\theta) = \mathbb{E}_{(s,a) \sim \bar{\mu}} [r(s, a)] = \bar{\mu}^\top r \tag{36}$$

To find $\bar{\mu}$, solve:

$$T_\theta \cdot \bar{\mu} = \bar{\mu} \quad \Rightarrow \quad \bar{\mu} \text{ is eigenvector of } T_\theta \text{ with eigenvalue 1} \tag{37}$$

Professor's Note

“We usually don’t solve for the stationary distribution directly in our algorithms. The place where it helps is to characterize solutions theoretically.”

3.5 The RL Objective

The RL objective is to find the policy parameters θ that maximize expected cumulative reward:

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^H r(s_t, a_t) \right] \quad (38)$$

3.5.1 Value Functions and Q Functions

These provide convenient shorthand for the “reward-to-go” from a given state or state-action pair.

Q function (state-action value function) for policy π :

$$Q^{\pi}(s_t, a_t) = \mathbb{E} \left[\sum_{t'=t}^H r(s_{t'}, a_{t'}) \mid s_t, a_t, \pi \right] \quad (39)$$

“Expected total reward if you start in state s_t , take action a_t , and follow π thereafter.”

Value function (state value function) for policy π :

$$V^{\pi}(s_t) = \mathbb{E}_{a_t \sim \pi} [Q^{\pi}(s_t, a_t)] = \sum_a \pi(a|s_t) \cdot Q^{\pi}(s_t, a) \quad (40)$$

“Expected total reward if you start in state s_t and follow π .”

Key relationships:

$$V^{\pi}(s_t) = \mathbb{E}_{a_t \sim \pi} [Q^{\pi}(s_t, a_t)] \quad (41)$$

$$J(\theta) = \mathbb{E}_{s_1 \sim p(s_1)} [V^{\pi}(s_1)] \quad (42)$$

Common Misconception

“All policies share the same V and Q functions.” No – **different policies have different V and Q functions**. The value function V^{π} and Q function Q^{π} are specific to the policy π . Changing the policy changes the expected future rewards and hence changes V and Q.

3.5.2 Using Q Functions to Improve Policies

If you know Q^π , you can construct a better policy π' by:

1. **Argmax method:** $\pi'(s) = \arg \max_a Q^\pi(s, a)$ – always choose the best action
2. **Policy gradient:** increase probability of actions where $Q^\pi(s, a) > V^\pi(s)$ – make above-average actions more likely

If $Q^\pi(s, a) > V^\pi(s)$, then action a is BETTER than average $\implies$ Increase $\pi(a|s)$ to improve the policy

3.6 Worked Example: Markov Chain Computation

Problem: Consider a Markov chain with 3 states {A, B, C} and transition matrix:

$$T = \begin{pmatrix} 0.5 & 0.3 & 0.2 \\ 0.1 & 0.6 & 0.3 \\ 0.4 & 0.1 & 0.5 \end{pmatrix} \quad \text{where } T_{ij} = p(s_{t+1} = i \mid s_t = j)$$

(Rows = next state, columns = current state)

Starting from $\mu_1 = [1, 0, 0]^\top$ (start in state A), find μ_2 .

Solution:

$$\mu_2 = T \cdot \mu_1 = \begin{bmatrix} 0.5 \cdot 1 + 0.3 \cdot 0 + 0.2 \cdot 0 \\ 0.1 \cdot 1 + 0.6 \cdot 0 + 0.3 \cdot 0 \\ 0.4 \cdot 1 + 0.1 \cdot 0 + 0.5 \cdot 0 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 0.1 \\ 0.4 \end{bmatrix}$$

After one step: 50% chance in A, 10% in B, 40% in C.

To find the **stationary distribution**, solve $(T - I) \cdot \bar{\mu} = 0$ with $\|\bar{\mu}\|_1 = 1$.

3.7 Check Your Understanding – Chapter 3

1. **Do MDP states obey the Markov property? Do POMDP states? Do POMDP transitions depend on observations?**
 - MDP states: Yes (by definition).
 - POMDP states: Yes (the underlying states still satisfy Markov property).
 - POMDP transitions: No, they depend only on states and actions, not observations.
2. **In the infinite horizon case, how do we express the RL objective?** Using the stationary distribution of the Markov chain formed by combining the transition operator with the policy. The average reward equals $\bar{\mu}^\top r$, where $\bar{\mu}$ is the eigenvector of T_θ with eigenvalue 1.
3. **Do different policies share the same V and Q functions?** No. Different policies induce different state-action distributions and hence have different expected future rewards, leading to different V and Q functions.

4. **What is the sample efficiency ordering of RL algorithms?** Model-based > Off-policy actor-critic / Q-learning > On-policy actor-critic > Policy gradient > Gradient-free. More efficient algorithms generally trade off computational cost for fewer required environment samples.

Cross-references: The distributional shift analysis in Section 2.2 uses Markov chain concepts from Section 3.1 (state distribution propagation). The trajectory distribution in Section 3.2 is the formal object underlying the behavioral cloning objective in Section 2.1. The pre-training/post-training paradigm in Section 2.5 can be viewed through the lens of the RL objective in Section 3.5 – pre-training optimizes a broad objective, while post-training (including RLHF) optimizes for the true reward.

Chapter 4

Policy Gradients

4.1 The RL Objective as Expectation over Trajectories

Recall from Chapter 3 that the goal of reinforcement learning is to find a parameterized policy π_θ that maximizes the expected total reward:

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t) \right] \quad (4.1)$$

We define the **RL objective** (also called the **expected return**) as:

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t) \right] \quad (4.2)$$

where the **trajectory distribution** $p_\theta(\tau)$ factorizes by the chain rule of probability and the Markov property:

$$p_\theta(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_H, \mathbf{a}_H) = p(\mathbf{s}_1) \prod_{t=1}^H \pi_\theta(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t) \quad (4.3)$$

We can **evaluate** the objective by sampling N trajectories from the policy and averaging:

$$J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_t r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) \quad (4.4)$$

To **improve** the policy, we want to compute $\nabla_\theta J(\theta)$ and perform gradient ascent:

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta) \quad (4.5)$$

Professor’s Note

“Policy gradient methods have a very simple modeling step. They basically just compute the reward of each generated trajectory. So the modeling step is just a summation sign.” The three parts of any RL algorithm – (1) generate samples (orange), (2) fit a model/estimate the return (green), (3) improve the policy (blue) – are all present here, with step (2) being trivially a sum.

4.1.1 Why Not Just Use Autodiff?

A natural first idea: code up Eq. (4.4) in PyTorch and call `.backward()`. This **does not work**. The reason is that generating trajectories requires passing actions through a simulator/environment, which is a black box to PyTorch. As far as the autodiff engine is concerned, the transition $\mathbf{s}_{t+1} \sim p(\cdot | \mathbf{s}_t, \mathbf{a}_t)$ is non-differentiable. PyTorch would report a gradient of zero with respect to θ because the computation graph is broken at every environment step.

Professor’s Note

“If you are extremely clever, you can actually implement a simulator that is itself differentiable. People have actually done this. There’s all sorts of other problems that will happen if you do that... you will often have numerically very ill-behaved gradients.”

4.2 REINFORCE Derivation – Full Step-by-Step with Log-Derivative Trick

We need a way to compute $\nabla_{\theta} J(\theta)$ that does not require differentiating through the environment. The key mathematical tool is the **log-derivative trick** (also called the **REINFORCE trick** or **likelihood ratio method**).

4.2.1 Step 1: Write the objective as an integral

For notational convenience, let $r(\tau) = \sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t)$ denote the total reward of trajectory τ .

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)}[r(\tau)] = \int p_{\theta}(\tau) r(\tau) d\tau \quad (4.6)$$

4.2.2 Step 2: Differentiate under the integral sign

Since differentiation is a linear operator, we can push the gradient inside:

$$\nabla_{\theta} J(\theta) = \int \nabla_{\theta} p_{\theta}(\tau) r(\tau) d\tau \quad (4.7)$$

4.2.3 Step 3: Apply the log-derivative trick

Here is the **key identity**. For any probability distribution $p_\theta(\tau) > 0$:

$$p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) = p_\theta(\tau) \cdot \frac{\nabla_\theta p_\theta(\tau)}{p_\theta(\tau)} = \nabla_\theta p_\theta(\tau) \quad (4.8)$$

This follows directly from the chain rule for logarithms: $\frac{d}{dx} \log f(x) = \frac{f'(x)}{f(x)}$.

4.2.4 Step 4: Substitute back

Replacing $\nabla_\theta p_\theta(\tau)$ with $p_\theta(\tau) \nabla_\theta \log p_\theta(\tau)$ in Eq. (4.7):

$$\nabla_\theta J(\theta) = \int p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) r(\tau) d\tau = \mathbb{E}_{\tau \sim p_\theta(\tau)} [\nabla_\theta \log p_\theta(\tau) r(\tau)] \quad (4.9)$$

Intuition

We have converted the gradient of an expectation (hard, because the distribution depends on θ) into an expectation of a gradient (easy, because we can estimate expectations by sampling). This is the **policy gradient theorem**, and it is arguably the single most important formula in reinforcement learning.

Professor's Note: “If there’s nothing else you remember, just remember this slide. And that’s like a hint. That’s code for it’ll be on the exam. This is probably the single most important formula in reinforcement learning.”

4.2.5 Step 5: Simplify $\nabla_\theta \log p_\theta(\tau)$ – why dynamics terms drop out

Now we need to evaluate $\nabla_\theta \log p_\theta(\tau)$. Start by taking the log of the trajectory distribution (Eq. 4.3):

$$\log p_\theta(\tau) = \log p(\mathbf{s}_1) + \sum_{t=1}^H [\log \pi_\theta(\mathbf{a}_t | \mathbf{s}_t) + \log p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)] \quad (4.10)$$

Now differentiate with respect to θ :

$$\nabla_\theta \log p_\theta(\tau) = \underbrace{\nabla_\theta \log p(\mathbf{s}_1)}_{=0} + \sum_{t=1}^H \left[\nabla_\theta \log \pi_\theta(\mathbf{a}_t | \mathbf{s}_t) + \underbrace{\nabla_\theta \log p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)}_{=0} \right] \quad (4.11)$$

Why do the terms drop out? - $\nabla_\theta \log p(\mathbf{s}_1) = 0$ because the initial state distribution does not depend on θ . - $\nabla_\theta \log p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t) = 0$ because the environment dynamics do not depend on θ .

Therefore:

$$\nabla_{\theta} \log p_{\theta}(\tau) = \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) \quad (4.12)$$

Common Misconception

Common Misconception (Quiz 5, Q1): “The identity $\nabla_{\theta} \log p_{\theta}(\tau) = \sum_t \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$ is incorrect.” This statement is **FALSE** – the identity **IS** correct. The dynamics and initial state terms vanish under differentiation w.r.t. θ .

4.2.6 Step 6: Final policy gradient formula

Substituting Eq. (4.12) into Eq. (4.9):

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\left(\sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) \right) \left(\sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t) \right) \right] \quad (4.13)$$

4.2.7 Step 7: Sample-based approximation

Approximate the expectation by sampling N trajectories $\{\tau^{(i)}\}$ from π_{θ} :

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \right) \left(\sum_{t=1}^H r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) \right) \quad (4.14)$$

Common Misconception

Common Misconception (Quiz 5, Q2): To implement REINFORCE, you need access to $\nabla_{\theta} \log p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$, the gradient of the dynamics model. **FALSE.** You only need $\nabla_{\theta} \log \pi_{\theta}(\mathbf{a} | \mathbf{s})$. The dynamics terms drop out of the gradient entirely.

Intuition (Comparison to Maximum Likelihood): The policy gradient looks like **reward-weighted maximum likelihood**. Maximum likelihood would be $\nabla_{\theta} J_{ML}(\theta) \approx \frac{1}{N} \sum_i \nabla_{\theta} \log \pi_{\theta}(\tau^{(i)})$, which makes all sampled trajectories more likely. The policy gradient multiplies by $r(\tau)$, making **good trajectories more likely** and **bad trajectories less likely**. It formalizes “trial and error.”

4.3 The REINFORCE Algorithm

Algorithm: REINFORCE

Repeat:

1. Sample $\{\tau^{(i)}\}$ from $\pi_{\theta}(a_t | s_t)$ (run policy)
2. Compute gradient: $\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_i (\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)) (\sum_t r(s_t, a_t))$
3. Update: $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

4.3.1 Example: Gaussian Policies

For continuous action spaces, a common choice is $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t) = \mathcal{N}(f_{\text{neural net}}(\mathbf{s}_t); \Sigma)$.

Then:

$$\log \pi_\theta(\mathbf{a}_t|\mathbf{s}_t) = -\frac{1}{2}\|f(\mathbf{s}_t) - \mathbf{a}_t\|_\Sigma^2 + \text{const} \quad (4.15)$$

$$\nabla_\theta \log \pi_\theta(\mathbf{a}_t|\mathbf{s}_t) = -\frac{1}{2}\Sigma^{-1}(f(\mathbf{s}_t) - \mathbf{a}_t)\frac{df}{d\theta} \quad (4.16)$$

4.3.2 Partial Observability

An important property: the Markov assumption is **never used** in the REINFORCE derivation. The policy can condition on observations $\mathbf{o}_t$ instead of states $\mathbf{s}_t$, and the algorithm works identically for POMDPs:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^H \nabla_\theta \log \pi_\theta(\mathbf{a}_t^{(i)}|\mathbf{o}_t^{(i)}) \right) \left(\sum_{t=1}^H r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) \right) \quad (4.17)$$

4.4 Variance Reduction

4.4.1 The Problem: High Variance

The basic REINFORCE estimator suffers from **high variance**. Consider a chess example where $R = +1$ for winning, -1 for losing. Problems include:

- **Lucky/unlucky starts:** Positive multipliers on trajectories with lucky initial states (even if the agent played poorly), negative multipliers on trajectories with unlucky starts (even if the agent played well).
- **Credit assignment failure:** Good moves get negative multipliers when a mistake was made later; bad moves get positive multipliers when the opponent blundered later.

These issues “average out” with enough samples, but you may need a **very large** number of samples. This is the core challenge of policy gradient methods.

4.4.2 Baselines – Prove Unbiasedness

Idea: Subtract a constant baseline b from the reward:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_\theta \log p_\theta(\tau^{(i)}) [r(\tau^{(i)}) - b] \quad (4.18)$$

A common choice is $b = \frac{1}{N} \sum_{i=1}^N r(\tau^{(i)})$ (the average reward).

But are we allowed to do this? We must prove that subtracting b does not introduce bias.

Proof of unbiasedness (constant baseline):

We need to show $\mathbb{E}[\nabla_\theta \log p_\theta(\tau) \cdot b] = 0$:

$$\mathbb{E}[\nabla_\theta \log p_\theta(\tau) \cdot b] = \int p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) \cdot b d\tau \quad (4.19)$$

$$= \int \nabla_\theta p_\theta(\tau) \cdot b d\tau \quad (\text{by the log-derivative trick, Eq. 4.8}) \quad (4.20)$$

$$= b \nabla_\theta \int p_\theta(\tau) d\tau \quad (\text{pull constant } b \text{ and } \nabla_\theta \text{ out}) \quad (4.21)$$

$$= b \nabla_\theta 1 \quad (4.22)$$

$$= 0 \quad (4.23)$$

The key step is that $\int p_\theta(\tau) d\tau = 1$ for any θ (it is a probability distribution), so its gradient is zero.

Intuition

Subtracting a baseline shifts the rewards so that roughly half are positive and half are negative. This means some trajectories get their probability increased and others decreased, rather than having all trajectories pushed in the same direction with different magnitudes. The expected gradient is unchanged, but the variance decreases dramatically.

Professor's Note: "Average reward is not the best baseline, but it's pretty good!"

State-dependent baseline: We can go further and use a baseline $b(s_t)$ that depends on the state (but NOT on the action):

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_{t=0}^{H-1} \nabla_\theta \log \pi_\theta(a_t | s_t) \left(\sum_{t'=t}^{H-1} r(s_{t'}, a_{t'}) - b(s_t) \right) \right] \quad (4.24)$$

Proof of unbiasedness (state-dependent baseline):

For any fixed t , we need to show:

$$\mathbb{E}_{\tau \sim p_\theta(\tau)} [\nabla_\theta \log \pi_\theta(a_t | s_t) \cdot b(s_t)] = 0 \quad (4.25)$$

Expand the expectation by splitting over states and actions:

$$= \mathbb{E}_{s_t \sim p_\theta(s_t)} \left[\mathbb{E}_{a_t \sim \pi_\theta(a_t | s_t)} [\nabla_\theta \log \pi_\theta(a_t | s_t) \cdot b(s_t)] \right] \quad (4.26)$$

$$= \mathbb{E}_{s_t \sim p_\theta(s_t)} \left[b(s_t) \cdot \mathbb{E}_{a_t \sim \pi_\theta(a_t | s_t)} [\nabla_\theta \log \pi_\theta(a_t | s_t)] \right] \quad (4.27)$$

Now, the inner expectation:

$$\mathbb{E}_{a_t \sim \pi_\theta(a_t|s_t)}[\nabla_\theta \log \pi_\theta(a_t|s_t)] = \int \pi_\theta(a_t|s_t) \nabla_\theta \log \pi_\theta(a_t|s_t) da_t \quad (4.28)$$

$$= \int \nabla_\theta \pi_\theta(a_t|s_t) da_t \quad (\text{log-derivative trick again}) \quad (4.29)$$

$$= \nabla_\theta \int \pi_\theta(a_t|s_t) da_t = \nabla_\theta 1 = 0 \quad (4.30)$$

Therefore Eq. (4.27) = 0. The key insight: $b(s_t)$ can be pulled out of the expectation over a_t because it does not depend on a_t , and $\mathbb{E}_a[\nabla_\theta \log \pi_\theta(a|s)] = 0$ always.

Common Misconception

Common Misconception (Quiz 5, Q3): “Adding a baseline changes the bias of the policy gradient estimator.” **FALSE.** Both the baseline and non-baseline estimators have the **same bias (zero bias)**. The baseline only reduces variance while preserving unbiasedness.

4.4.3 Reward-to-Go (Causality Trick)

Key insight: The action at time t' cannot affect the reward at time $t < t'$. Therefore, we can replace the total trajectory reward with only the **future rewards from time t onward**:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_\theta \log \pi_\theta(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \left(\sum_{t'=t}^H r(\mathbf{s}_{t'}^{(i)}, \mathbf{a}_{t'}^{(i)}) - b_t \right) \quad (4.31)$$

The quantity $\hat{Q}_t^{(i)} = \sum_{t'=t}^H r(\mathbf{s}_{t'}^{(i)}, \mathbf{a}_{t'}^{(i)})$ is called the **reward-to-go**.

Why is this valid? Formally, for $t' < t$:

$$\mathbb{E}_{\tau \sim p_\theta(\tau)}[\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot r(s_{t'}, a_{t'})] = 0 \quad (4.32)$$

Proof sketch (from discussion Section 3): Define sub-trajectories $\tau_{0,t}$ and $\tau_{t'+1,t}$. By the tower property of conditional expectations, we can split the expectation and show that the inner expectation over a_t conditioned on the sub-trajectory up to t' yields $\nabla_\theta \int \pi_\theta(a_t|s_t) da_t = \nabla_\theta 1 = 0$, since $r(s_{t'}, a_{t'})$ does not depend on a_t when $t' < t$.

Intuition

It makes no sense to credit or blame the action at step 15 for a reward received at step 3. The reward-to-go removes this spurious correlation, reducing variance without introducing bias.

4.4.4 Discounting

Another variance reduction technique is to introduce a **discount factor** $\gamma \in [0, 1)$. This modifies the objective to:

$$J^\gamma(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_{t=1}^H \gamma^t r(\mathbf{s}_t, \mathbf{a}_t) \right] \quad (4.33)$$

The discounted reward-to-go policy gradient becomes:

$$\nabla_\theta J^\gamma(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_{t=0}^{H-1} \nabla_\theta \log \pi_\theta(a_t | s_t) \left(\sum_{t'=t}^{H-1} \gamma^{t'-t} r(s_{t'}, a_{t'}) \right) \right] \quad (4.34)$$

Note: In practice, we apply discounting to the reward-to-go using $\gamma^{t'-t}$ (not $\gamma^{t'}$) so that we only discount future rewards relative to the current time step, rather than downweighting the gradient at later states (see Section 5.3 for why this matters). The full interpretation of γ as modified transition probabilities is discussed in Section 5.3.

4.5 All Valid Policy Gradient Estimator Variants

Common Misconception

Common Misconception (Quiz 5, Q4): “Only some of the following four estimators are valid unbiased policy gradient estimators.” **FALSE.** All four of the following are **valid, unbiased** policy gradient estimators:

Variant 1 – Full trajectory reward:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_\theta \log p_\theta(\tau^{(i)}) r(\tau^{(i)}) \quad (4.35)$$

Variant 2 – With baseline:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_\theta \log p_\theta(\tau^{(i)}) [r(\tau^{(i)}) - b] \quad (4.36)$$

Variant 3 – Expanded sum form:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^H \nabla_\theta \log \pi_\theta(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \right) \left(\sum_{t=1}^H r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) \right) \quad (4.37)$$

Variant 4 – Reward-to-go:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \left(\sum_{t'=t}^H r(\mathbf{s}_{t'}^{(i)}, \mathbf{a}_{t'}^{(i)}) \right) \quad (4.38)$$

All four have the same expected value (the true policy gradient). They differ only in **variance**: Variant 4 has the lowest variance, Variant 1 has the highest.

4.6 Practical Implementation

4.6.1 Pseudo-Loss for Automatic Differentiation

Computing $\nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$ explicitly is inefficient. Instead, define a **pseudo-loss** whose gradient equals the policy gradient:

$$\tilde{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \hat{Q}_t^{(i)} \quad (4.39)$$

Then $\nabla_{\theta} \tilde{J}(\theta)$ is exactly the policy gradient estimator. This is just **reward-weighted maximum likelihood**: a weighted cross-entropy loss (for discrete actions) or weighted squared error (for Gaussian actions).

```
# Policy gradient implementation (discrete actions):

# Given:
#   actions   - (N*T) x Da tensor of actions
#   states    - (N*T) x Ds tensor of states
#   q_values  - (N*T) x 1 tensor of estimated state-action values
#               (reward-to-go, with baseline subtracted)

# Build the graph:
logits = policy.predictions(states)
negative_likelihoods = tf.nn.softmax_cross_entropy_with_logits(
    labels=actions, logits=logits)
weighted_negative_likelihoods = tf.multiply(
    negative_likelihoods, q_values)
loss = tf.reduce_mean(weighted_negative_likelihoods)
gradients = loss.gradients(loss, variables)
```

4.6.2 Practical Tips

- **High variance**: Policy gradient is NOT the same as supervised learning. Gradients will be very noisy.

- **Large batches:** Use much larger batch sizes than you would in supervised learning.
 - **Learning rates:** Tweaking learning rates is very hard. Adaptive step size rules like Adam can be OK. Policy gradient-specific methods (e.g., natural gradient, TRPO, PPO) are better.
 - **On-policy:** REINFORCE is inherently **on-policy** – after each gradient update, you must discard old samples and collect new ones from the updated policy, because the gradient estimator requires samples from $p_\theta(\tau)$. This makes REINFORCE highly **sample-inefficient**.
-

4.7 Check Your Understanding

1. **Starting from $J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)}[r(\tau)]$, derive the policy gradient theorem. At which step does the log-derivative trick appear, and why is it needed?** *Answer: Write J as an integral, differentiate under the integral to get $\int \nabla_\theta p_\theta(\tau) r(\tau) d\tau$, then apply the log-derivative trick $\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau)$ to convert it back into an expectation $\mathbb{E}[\nabla_\theta \log p_\theta(\tau) r(\tau)]$. The trick is needed because it converts the gradient of an expectation (hard, since the distribution depends on θ) into an expectation of a gradient (easy, estimable by sampling).*
 2. **Explain why $\nabla_\theta \log p_\theta(\tau)$ simplifies to $\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)$. What property of the environment dynamics makes this possible?** *Answer: Taking the log of $p_\theta(\tau)$ decomposes it into $\log p(s_1) + \sum_t [\log \pi_\theta(a_t|s_t) + \log p(s_{t+1}|s_t, a_t)]$. When differentiating w.r.t. θ , the initial state and transition dynamics terms vanish because they do not depend on θ . Only the policy terms survive.*
 3. **Prove that subtracting a state-dependent baseline $b(s_t)$ from the reward-to-go preserves unbiasedness of the policy gradient. Why must b not depend on a_t ?** *Answer: $\mathbb{E}_{a_t}[\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot b(s_t)] = b(s_t) \cdot \mathbb{E}_{a_t}[\nabla_\theta \log \pi_\theta(a_t|s_t)]$. The inner expectation equals $\int \nabla_\theta \pi_\theta(a_t|s_t) da_t = \nabla_\theta 1 = 0$. We can only pull $b(s_t)$ out of the expectation over a_t if it does not depend on a_t ; otherwise it would interact with the gradient and not vanish.*
 4. **Consider two estimators: one using the full trajectory reward $r(\tau)$ and one using the reward-to-go $\hat{Q}_t$. Both are unbiased. Which has lower variance, and why?** *Answer: The reward-to-go estimator has lower variance because it removes the spurious correlation between the action at time t and rewards received at earlier times $t' < t$. Past rewards are independent of the current action and contribute only noise, so removing them reduces variance without affecting the expected value.*
 5. **Why is REINFORCE on-policy? What happens if you use trajectories collected from an old policy π_{old} to estimate $\nabla_\theta J(\theta)$?** *Answer: The policy gradient estimator requires samples from $p_\theta(\tau)$ – the current policy’s trajectory distribution. Using trajectories from an old policy π_{old} means the samples come from the wrong distribution, making $\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot r(\tau)$ a biased estimate of the true gradient. After each parameter update, old samples must be discarded and new ones collected.*
-
-

Chapter 5

Actor-Critic Methods

5.1 Motivation: From Monte Carlo Returns to Learned Value Functions

In REINFORCE, we use the **single-sample Monte Carlo reward-to-go** $\hat{Q}_t^{(i)} = \sum_{t'=t}^H r(\mathbf{s}_{t'}, \mathbf{a}_{t'}^{(i)})$ as the multiplier in the policy gradient. This is a **random variable** – it has an expected value and variance.

Professor's Note

“If you could somehow teleport back to that state and complete that trajectory multiple times, you will get multiple different answers. So the fact that $\hat{Q}$ is this random variable induces variance.”

The **ideal** quantity to use would be the **true expected reward-to-go**:

$$Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^H \mathbb{E}_{\pi_\theta} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t, \mathbf{a}_t] \quad (5.1)$$

If we could plug this into the policy gradient, we would have zero variance from the reward estimation (the only remaining variance would be from sampling states and actions). The Monte Carlo estimate $\hat{Q}_t^{(i)}$ is an unbiased but high-variance estimator of Q^π .

The core idea of **actor-critic** methods: replace the noisy Monte Carlo estimate with a **learned approximation** of the value function, trading a small amount of bias for a large reduction in variance.

5.2 Value Functions: $V(s)$, $Q(s,a)$, $A(s,a)$ – Definitions and Relationships

5.2.1 State-Action Value Function (Q-function)

$$Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^H \mathbb{E}_{\pi_\theta} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t, \mathbf{a}_t] \quad (5.2)$$

“The total expected reward from taking action $\mathbf{a}_t$ in state $\mathbf{s}_t$ and then following policy π thereafter.”

5.2.2 State Value Function (V-function)

$$V^\pi(\mathbf{s}_t) = \mathbb{E}_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} [Q^\pi(\mathbf{s}_t, \mathbf{a}_t)] \quad (5.3)$$

“The total expected reward from state $\mathbf{s}_t$ when following policy π . It is the average Q-value over actions.”

Quiz 6, Q2: A valid example of a value function is the “probability of winning given a board state.” This is exactly $V^\pi(s)$ when the reward is +1 for winning and 0 otherwise, as used in TD-Gammon and AlphaGo.

5.2.3 Advantage Function

$$A^\pi(\mathbf{s}_t, \mathbf{a}_t) = Q^\pi(\mathbf{s}_t, \mathbf{a}_t) - V^\pi(\mathbf{s}_t) \quad (5.4)$$

“How much **better** action $\mathbf{a}_t$ is compared to the **average** action from state $\mathbf{s}_t$.”

Intuition

The advantage is positive for actions that are better than average and negative for actions that are worse than average. When used as the multiplier in the policy gradient, it increases the probability of above-average actions and decreases the probability of below-average actions. This is exactly the right behavior for credit assignment.

Professor’s Note: “V is the average Q, so Q minus the average Q represents how much better than average the action AT is. We call it the advantage because it is representing how advantageous the action is.”

5.2.4 Key Relationships

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(a|s)} [Q^\pi(s, a)] \quad (5.5)$$

$$\mathbb{E}_{a \sim \pi(a|s)} [A^\pi(s, a)] = 0 \quad (\text{the average advantage is always zero}) \quad (5.6)$$

Recursive (Bellman) relationship for Q:

$$Q^\pi(s_t, \mathbf{a}_t) = r(s_t, \mathbf{a}_t) + \mathbb{E}_{s_{t+1} \sim p(s_{t+1}|s_t, \mathbf{a}_t)} [V^\pi(s_{t+1})] \quad (5.7)$$

Single-sample approximation of the advantage:

$$A^\pi(s_t, \mathbf{a}_t) \approx r(s_t, \mathbf{a}_t) + V^\pi(s_{t+1}) - V^\pi(s_t) \quad (5.8)$$

This replaces the expected value over s_{t+1} with the single observed next state, introducing a small amount of variance (but only over one step, not the entire trajectory).

Intuition

Eq. (5.8) says the advantage is “how did my value function change over this transition?” It decomposes into: reward I got NOW + how good my NEW state is - how good my OLD state was.

5.2.5 ASCII Backup Diagram

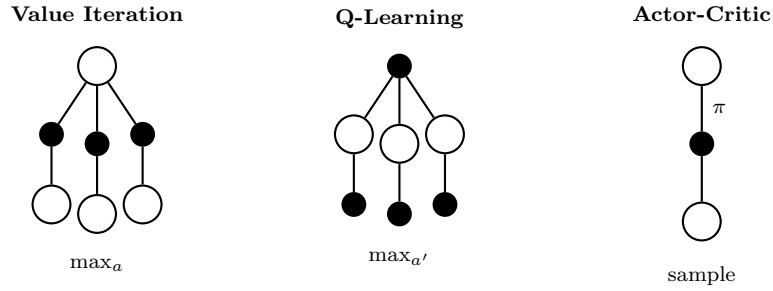


Figure 5.1: Backup diagrams: Value Iteration (max over actions), Q-Learning (max over next actions), Actor-Critic (sample).

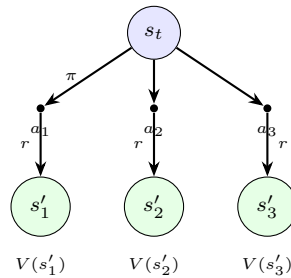


Figure 5.2: Actor-critic backup: sample actions from $\pi(a|s_t)$, observe rewards, bootstrap with $V(s')$.

The key relationships:

$$Q(s_t, a_t) = r(s_t, a_t) + \mathbb{E}[V(s_{t+1})]$$

$$V(s_t) = \mathbb{E}_a[Q(s_t, a)] \quad A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$$

Backup diagram showing the relationship between value functions: from state s_t , the agent can take different actions a_1, a_2, a_3 , each yielding an immediate reward $r(s_t, a_t)$ and transitioning to a next

state s' . The Q -function combines the immediate reward with the expected future value $V(s')$, the V -function averages Q over actions, and the advantage A measures how much better an action is compared to that average.

5.3 Discount Factors

5.3.1 Why Discount?

When using bootstrapped value estimation (Section 5.4), the time horizon H does not appear explicitly. For infinite-horizon or very long-horizon tasks, the value function $V^\pi(s)$ can grow without bound. The discount factor $\gamma \in [0, 1)$ resolves this:

$$y_t^{(i)} = r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \gamma \hat{V}_\phi^\pi(\mathbf{s}_{t+1}^{(i)}) \quad (5.9)$$

5.3.2 Equivalent Interpretation as Modified Transition Probabilities (Death State)

IMPORTANT (Quiz 6, Q3): The discount factor γ has an equivalent interpretation as modified transition probabilities.

Full derivation: Define a **modified MDP** $\tilde{\mathcal{M}}$ that is identical to the original MDP but with modified transition dynamics:

$$\tilde{p}(\mathbf{s}'|\mathbf{s}, \mathbf{a}) = \gamma \cdot p(\mathbf{s}'|\mathbf{s}, \mathbf{a}) \quad (5.10)$$

$$\tilde{p}(\text{death}|\mathbf{s}, \mathbf{a}) = 1 - \gamma \quad (5.11)$$

The “death state” is an **absorbing state** with zero reward: once you enter it, you stay there forever and receive reward 0.

Now consider the undiscounted expected return in $\tilde{\mathcal{M}}$:

$$\tilde{J}(\theta) = \mathbb{E}_{\tau \sim \tilde{p}_\theta(\tau)} \left[\sum_{t=1}^{\infty} r(\mathbf{s}_t, \mathbf{a}_t) \right] \quad (5.12)$$

At each step, there is a probability γ of continuing and $(1 - \gamma)$ of “dying.” The probability of surviving for exactly t steps is γ^t . Therefore, the expected return from step t is discounted by γ^t :

$$\tilde{J}(\theta) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(\mathbf{s}_t, \mathbf{a}_t) \right] = J^\gamma(\theta) \quad (5.13)$$

This is exactly the **discounted return** in the original MDP.

Intuition

Discounting is NOT just an arbitrary mathematical trick. It is equivalent to solving an undiscounted problem in a world where you might “die” at each step with probability $(1 - \gamma)$. Since the probability of being alive decreases exponentially with time, you naturally prefer rewards sooner rather than later.

5.3.3 Consequences of Discounting

Quiz 6, Q3 (all three are true):

1. γ can be interpreted as modified transition probabilities (the death-state construction above).
2. **Discounting changes the optimal policy.** Because we are solving a different objective $J^\gamma(\theta) \neq J(\theta)$, the optimal policy may differ. An agent may take a suboptimal-in-the-long-run action if it yields higher immediate reward.
3. **Discounting makes the agent prefer sooner rewards.** By the death-state interpretation, the agent “fears mortality” – it prefers to collect rewards before the episode might end.

Professor’s Note

“Fear of one’s own mortality is the key to making infinite horizon work. Very philosophical.” And: “If I asked all of you if you want \$10 now versus \$10 100 years from now, chances are most of you prefer to get it now.”

5.4 Policy Evaluation and Value Function Fitting**5.4.1 Monte Carlo Policy Evaluation**

Use sampled trajectories to estimate V^π :

$$V^\pi(\mathbf{s}_t) \approx \sum_{t'=t}^H r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \quad (5.14)$$

To fit a neural network $\hat{V}_\phi^\pi(\mathbf{s})$ with parameters ϕ , create training data:

$$\text{Training set: } \left\{ \left(\mathbf{s}_t^{(i)}, y_t^{(i)} = \sum_{t'=t}^H r(\mathbf{s}_{t'}^{(i)}, \mathbf{a}_{t'}^{(i)}) \right) \right\} \quad (5.15)$$

Minimize the supervised regression loss:

$$\mathcal{L}(\phi) = \frac{1}{2} \sum_{i=1}^N \sum_{t=1}^H \left\| \hat{V}_\phi^\pi(\mathbf{s}_t^{(i)}) - y_t^{(i)} \right\|^2 \quad (5.16)$$

Intuition

Even though the labels $y_t^{(i)}$ are noisy single-sample estimates, the neural network **generalizes** across similar states, effectively averaging together the noisy estimates for nearby states. This gives a better estimate than any single trajectory.

5.4.2 Bootstrapped (TD) Policy Evaluation

Instead of using the full Monte Carlo return, use the **bootstrapped target**:

$$y_t^{(i)} = r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \gamma \hat{V}_\phi^\pi(\mathbf{s}_{t+1}^{(i)}) \quad (5.17)$$

This uses the previously fitted value function to estimate the future return. It is “circular” (learning V from V), but under appropriate conditions converges to the true value function. This is called **temporal difference (TD) learning**.

The supervised regression loss with bootstrapped targets:

$$\mathcal{L}(\phi) = \frac{1}{2} \sum_{i=1}^N \sum_{t=1}^H \left\| \hat{V}_\phi^\pi(\mathbf{s}_t^{(i)}) - \left[r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \gamma \hat{V}_\phi^\pi(\mathbf{s}_{t+1}^{(i)}) \right]_\times \right\|^2 \quad (5.18)$$

where $[\cdot]_\times$ denotes the **stop-gradient operator** (treat the target as a constant when computing gradients w.r.t. ϕ).

5.5 The Actor-Critic Algorithm

5.5.1 Bias-Variance Tradeoff

The actor-critic introduces a fundamental tradeoff:

Estimator	Bias	Variance
MC return $\sum_{t'=t}^H r(s_{t'}, a_{t'}) - b_t$	Unbiased	High (single-sample)
TD estimate $r(s_t, a_t) + \gamma \hat{V}_\phi^\pi(s_{t+1}) - \hat{V}_\phi^\pi(s_t)$	Biased (if $\hat{V} \neq V$)	Lower (uses learned function)
MC return with $\hat{V}$ baseline: $\sum_{t'=t}^H \gamma^{t'-t} r(s_{t'}, a_{t'}) - \hat{V}_\phi^\pi(s_t)$	Unbiased	Medium (MC return + good baseline)

Quiz 6, Q1: Actor-critic is preferred over standard policy gradient because it has **lower variance**.

5.5.2 Batch Actor-Critic Algorithm

Algorithm: Batch Actor-Critic

Repeat:

1. Sample $\{s_i, a_i, s'_i\}$ from $\pi_\theta(a|s)$ (run policy)
2. Evaluate $y_i = r(s_i, a_i) + \gamma V_\phi(s'_i)$
3. Refit $V_\phi(s)$ to targets $\{y_i\}$ (minimize $\mathcal{L}(\phi) = \sum \|V_\phi(s_i) - y_i\|^2$)
4. Evaluate $\hat{A}(s_i, a_i) = r(s_i, a_i) + \gamma V_\phi(s'_i) - V_\phi(s_i)$
5. $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(a_i|s_i) \cdot \hat{A}(s_i, a_i)$
6. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

Two neural networks:

$s \longrightarrow [\text{Neural Net } \theta] \longrightarrow \pi_\theta(a|s)$ (Actor)

$s \longrightarrow [\text{Neural Net } \phi] \longrightarrow \hat{V}(s)$ (Critic)

5.5.3 Online Actor-Critic Algorithm (e.g., A3C)

Algorithm: Online Actor-Critic (e.g., A3C)

Repeat:

1. Get (s_i, a_i, s'_i) by taking one step with $a \sim \pi_\theta(a|s)$
2. Evaluate $y_i = r(s_i, a_i) + \gamma V_\phi(s'_i)$
3. Update ϕ using $\nabla_\phi \mathcal{L}_i(\phi)$ where $\mathcal{L}_i(\phi) = \|V_\phi(s_i) - y_i\|^2$
4. Evaluate $\hat{A}(s_i, a_i) = r(s_i, a_i) + \gamma V_\phi(s'_i) - V_\phi(s_i)$
5. $\nabla_\theta J(\theta) \approx \nabla_\theta \log \pi_\theta(a_i|s_i) \cdot \hat{A}(s_i, a_i)$
6. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

Note: Getting this to work in practice typically requires multiple parallel workers (e.g., A3C uses asynchronous workers).

5.6 Which Advantage Estimators Are Unbiased vs. Biased?

Quiz 6, Q5 and Discussion Section 3 (Equations 46-49): Consider these four variants of the actor-critic policy gradient estimator, differing only in where the “hats” (learned approximations) appear:

$$(1) \nabla_{\theta} J(\theta) \approx \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \left(r(s_t, a_t) + \gamma V^{\pi}(s_{t+1}) - V_{\phi}^{\pi}(s_t) \right) \right]$$

– Uses true $V^{\pi}(s_{t+1})$ but learned $\hat{V}_{\phi}^{\pi}(s_t)$. **UNBIASED**, because the baseline $\hat{V}_{\phi}^{\pi}(s_t)$ is a state-dependent function subtracted from the reward, and any state-dependent baseline preserves unbiasedness (Section 4.4.2).

$$(2) \nabla_{\theta} J(\theta) \approx \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \left(r(s_t, a_t) + \gamma V^{\pi}(s_{t+1}) - \hat{V}_{\phi}^{\pi}(s_t) \right) \right]$$

– Same as (1) with different notation. **UNBIASED** for the same reason.

$$(3) \nabla_{\theta} J(\theta) \approx \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \left(r(s_t, a_t) + \gamma \hat{V}_{\phi}^{\pi}(s_{t+1}) - V^{\pi}(s_t) \right) \right]$$

– Uses learned $\hat{V}_{\phi}^{\pi}(s_{t+1})$ inside the Q-function estimate. **BIASED**, because $\hat{V}_{\phi}^{\pi}(s_{t+1}) \neq V^{\pi}(s_{t+1})$ in general, and this error is NOT in the baseline position.

$$(4) \nabla_{\theta} J(\theta) \approx \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \left(r(s_t, a_t) + \gamma \hat{V}_{\phi}^{\pi}(s_{t+1}) - \hat{V}_{\phi}^{\pi}(s_t) \right) \right]$$

– Uses learned functions for BOTH terms. **BIASED** (because the $\hat{V}_{\phi}(s_{t+1})$ term introduces bias).

Summary of unbiased advantage estimators (Quiz 6, Q5): - MC return minus constant baseline: **Unbiased** - MC return minus learned $\hat{V}_{\phi}^{\pi}(s_t)$: **Unbiased** (state-dependent baseline is always unbiased) - Single-step TD: $r + \gamma \hat{V}_{\phi}(s') - \hat{V}_{\phi}(s)$: **Biased** (unless $\hat{V}_{\phi} = V^{\pi}$ exactly) - $Q - V$ with learned functions: **Biased**

Key principle: Learned functions can be subtracted as baselines (unbiased), but learned functions used to **replace the reward-to-go itself** introduce bias.

5.7 On-Policy vs. Off-Policy Actor-Critic

5.7.1 On-Policy Actor-Critic

The batch and online algorithms above are **on-policy**: they require samples from the current policy π_{θ} . After updating θ , old samples must be discarded.

5.7.2 Off-Policy Actor-Critic

Motivation: Reuse old data via a **replay buffer** to improve sample efficiency.

Problem with naive replay: If we store transitions (s_i, a_i, s'_i) in a buffer and reuse them, two things break:

1. **The value target is wrong:** We want $V^{\pi}(s_i) = \mathbb{E}_{a \sim \pi_{\theta}(a|s_i)}[r(s_i, a) + \gamma V^{\pi}(s')]$, but the stored action a_i came from an **old policy** $\bar{\pi}$, not π_{θ} . So $y_i = r(s_i, a_i) + \gamma \hat{V}_{\phi}^{\pi}(s'_i)$ estimates V under the wrong action distribution.
2. **The policy gradient uses the wrong actions:** $\nabla_{\theta} \log \pi_{\theta}(a_i | s_i)$ uses the buffered action a_i , which is not from π_{θ} .

Quiz 6, Q4: Off-policy actor-critic requires: (1) a **Q-function** (not V-function) and (2) sampling actions from the **latest policy** π_{θ} .

5.7.3 Solution: Estimate Q Instead of V

Why Q fixes the value target: The Q-function is defined for a specific state-action pair. The Bellman equation for Q is:

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim p(s'|s, a)} \left[\mathbb{E}_{a' \sim \pi_\theta(a'|s')} [Q^\pi(s', a')] \right] \quad (5.19)$$

The key insight: the inner expectation over a' must be under the **current** policy π_θ , but the outer expectation over s' does not depend on which policy generated the data (the transition dynamics are the same regardless of which policy chose a).

So we can use buffered transitions (s_i, a_i, s'_i) to compute:

$$y_i = r(s_i, a_i) + \gamma \mathbb{E}_{a' \sim \pi_\theta(a'|s'_i)} [\hat{Q}_\phi^\pi(s'_i, a')] \quad (5.20)$$

We sample $a'_i \sim \pi_\theta(a'|s'_i)$ from the **latest policy** (not the buffer) to estimate this expectation.

5.7.4 Off-Policy Actor-Critic Algorithm

Algorithm: Off-Policy Actor-Critic with Replay Buffer

Repeat:

1. Get (s_i, a_i, s'_i) by taking one step with $a \sim \pi_\theta(a|s)$, store in replay buffer $\mathcal{R}$
2. Sample batch $\{(s_i, a_i, s'_i)\}$ from $\mathcal{R}$
3. Evaluate $y_i = r(s_i, a_i) + \gamma \mathbb{E}_{a' \sim \pi_\theta(a'|s'_i)} [Q_\phi(s'_i, a')]$ (sample $a'_i \sim \pi_\theta$ to estimate the expectation)
4. Update ϕ using $\nabla_\phi \sum \|Q_\phi(s_i, a_i) - y_i\|^2$
5. Sample $a_i^\pi \sim \pi_\theta(a|s_i)$ [from LATEST policy!] $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(a_i^\pi | s_i) \cdot Q_\phi(s_i, a_i^\pi)$
6. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

Important details: - Step 3 uses $a' \sim \pi_\theta$ (latest policy) to compute Q-targets, NOT the buffered actions. - Step 5 samples **fresh actions** from π_θ for the policy gradient, NOT from the buffer. - The states s_i come from the buffer (older policy), which means we are optimizing the policy on a broader distribution of states. This is accepted as a practical approximation.

5.7.5 Reparameterization Trick (for continuous actions)

For Gaussian policies $\pi_\theta(a|s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta(s))$, we can use the **reparameterization trick**:

$$a = \mu_\theta(s) + \sigma_\theta(s)\epsilon, \quad \epsilon \sim \mathcal{N}(0, 1) \quad (5.21)$$

Then:

$$\nabla_\theta \mathbb{E}_{a \sim \pi_\theta(a|s)} [Q(s, a)] \approx \nabla_\theta Q(s, \mu_\theta(s) + \sigma_\theta(s)\epsilon) \quad (5.22)$$

This is a **pathwise gradient** that can be computed directly through PyTorch autodiff, without needing the log-derivative trick. This approach is used in algorithms like **Soft Actor-Critic (SAC)**.

5.8 Generalized Advantage Estimation (GAE)

5.8.1 N-Step Returns: Controlling the Bias-Variance Tradeoff

The two extremes of advantage estimation:

1-step (critic) estimate – low variance, high bias:

$$\hat{A}_C^\pi(s_t, a_t) = r(s_t, a_t) + \gamma \hat{V}_\phi^\pi(s_{t+1}) - \hat{V}_\phi^\pi(s_t) \quad (5.23)$$

Monte Carlo estimate – no bias, high variance:

$$\hat{A}_{MC}^\pi(s_t, a_t) = \sum_{t'=t}^{\infty} \gamma^{t'-t} r(s_{t'}, a_{t'}) - \hat{V}_\phi^\pi(s_t) \quad (5.24)$$

N-step return – interpolates between the two:

$$\hat{A}_n^\pi(s_t, a_t) = \sum_{t'=t}^{t+n-1} \gamma^{t'-t} r(s_{t'}, a_{t'}) + \gamma^n \hat{V}_\phi^\pi(s_{t+n}) - \hat{V}_\phi^\pi(s_t) \quad (5.25)$$

When $n = 1$, this is the 1-step bootstrapped estimate. When $n = H - t$, this is the Monte Carlo estimate (assuming $\hat{V}_\phi^\pi(s_H) = 0$). Choosing $n > 1$ often works better than either extreme.

5.8.2 GAE: Weighted Combination of All N-Step Returns

Rather than choosing a single n , GAE takes an **exponentially weighted average** over all n :

$$\hat{A}_{GAE}^\pi(s_t, a_t) = \sum_{n=1}^{\infty} w_n \hat{A}_n^\pi(s_t, a_t), \quad w_n \propto \lambda^{n-1} \quad (5.26)$$

where $\lambda \in [0, 1]$ controls the bias-variance tradeoff: - $\lambda = 0$: reduces to 1-step TD (low variance, high bias) - $\lambda = 1$: reduces to Monte Carlo (no bias, high variance)

5.8.3 Efficient Computation via TD Errors

Define the **TD error** (or **TD residual**):

$$\delta_{t'} = r(s_{t'}, a_{t'}) + \gamma \hat{V}_\phi^\pi(s_{t'+1}) - \hat{V}_\phi^\pi(s_{t'}) \quad (5.27)$$

Then GAE has a remarkably simple closed form:

$$\hat{A}_{GAE}^{\pi}(s_t, a_t) = \sum_{t'=t}^{\infty} (\gamma\lambda)^{t'-t} \delta_{t'} \quad (5.28)$$

This can be computed efficiently via a single backward pass over the trajectory using the recursion:

$$\hat{A}_{GAE}^{\pi}(s_t, a_t) = \delta_t + \gamma\lambda \hat{A}_{GAE}^{\pi}(s_{t+1}, a_{t+1}) \quad (5.29)$$

Intuition

The product $\gamma\lambda$ acts similarly to the discount factor. While γ controls how much we care about future rewards, λ controls how much we trust our value function versus actual observed rewards. With $\lambda < 1$, we “cut off” the trajectory early and substitute the learned value function, reducing variance at the cost of bias.

5.8.4 Policy Gradient with GAE

Algorithm: Policy Gradient with GAE

Repeat:

1. Sample $\{\tau^{(i)}\}$ from π_{θ} (run the policy)
2. Evaluate $y_t = r(s_t, a_t) + \gamma V_{\phi}(s_{t+1})$
3. Fit $V_{\phi}(s)$ to targets $\{y_t\}$
4. Evaluate GAE advantages: $\delta_t = r(s_t, a_t) + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$ $\hat{A}_{GAE}(s_t, a_t) = \sum_{t'=t}^{\infty} (\gamma\lambda)^{t'-t} \delta_{t'}$
5. Normalize advantages: $\mu = \text{mean}(\hat{A})$, $\sigma = \text{std}(\hat{A})$ $\bar{A} = (\hat{A} - \mu)/\sigma$
6. $\nabla_{\theta} J(\theta) \approx \sum_i \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot \bar{A}(s_t, a_t)$
7. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

Note: This is the first part of implementing PPO. The other part is a slightly different gradient estimator (clipped surrogate objective).

Professor’s Note

GAE is more popular than n-step returns in policy gradient methods because λ is typically easier to tune than n and is more robust across tasks. In off-policy RL algorithms, n-step returns are more popular because off-policy bias can be better controlled with n-step returns.

5.9 Summary: The Spectrum of Estimators

Method	Advantage Estimator	Bias	Variance	Sample Efficiency
REINFORCE	$\sum_{t'=t}^H r_{t'}$	None	Very high	Low (on-policy)
REINFORCE	$\sum_{t'=t}^H r_{t'} - b$	None	High	Low (on-policy)
+ baseline				
REINFORCE	$\sum_{t'=t}^H \gamma^{t'-t} r_{t'} - \hat{V}_\phi(s_t)$	None	Medium	Low (on-policy)
+ V				
baseline				
Actor-Critic (1-step TD)	$r_t + \gamma \hat{V}_\phi(s_{t+1}) - \hat{V}_\phi(s_t)$	Yes	Low	Low (on-policy)
GAE	$\sum_{t'=t} (\gamma \lambda)^{t'-t} \delta_{t'}$	Yes (tunable)	Tunable	Low (on-policy)
Off-policy AC	$\hat{Q}_\phi(s, a)$ with replay	Yes	Low	Higher (off-policy)

5.10 Check Your Understanding

- 1. Explain the difference between $V^\pi(s)$, $Q^\pi(s, a)$, and $A^\pi(s, a)$. Why is the advantage function particularly useful for policy gradients?** *Answer: $V^\pi(s)$ is the expected total reward from state s following policy π . $Q^\pi(s, a)$ is the expected total reward from taking action a in state s then following π . $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ measures how much better action a is compared to the average. The advantage is useful because it directly tells the policy gradient to increase the probability of above-average actions and decrease below-average ones, providing effective credit assignment.*
- 2. In the actor-critic algorithm, what are the two neural networks, and what does each one output? Why do we use separate parameters θ and ϕ ?** *Answer: The actor network (parameters θ) outputs the policy $\pi_\theta(a|s)$, and the critic network (parameters ϕ) outputs the value function $\hat{V}_\phi^\pi(s)$. Separate parameters are used because they have different objectives: the actor maximizes expected return via policy gradients, while the critic minimizes a regression loss to fit value targets. Coupling them would create conflicting gradient signals.*
- 3. Derive the equivalent interpretation of the discount factor γ as modified transition probabilities with a “death state.” Why does this construction make the agent prefer sooner rewards?** *Answer: Define a modified MDP where at each step, with probability γ the agent transitions normally and with probability $(1 - \gamma)$ it enters an absorbing “death state” with zero reward. The undiscounted return in this modified MDP equals the discounted return $\sum_t \gamma^t r_t$ in the original MDP, because the probability of surviving t steps is γ^t . The agent prefers sooner rewards because the probability of being “alive” to collect them decreases exponentially with time.*
- 4. Why does off-policy actor-critic require a Q-function rather than a V-function? What goes wrong if you use $\hat{V}$ with a replay buffer?** *Answer: The V-function target $y_i = r(s_i, a_i) + \gamma \hat{V}(s'_i)$ depends on the action a_i that was taken, but V^π should reflect the expected value under the current policy’s action distribution. With a replay buffer, a_i came from an old policy, so the target estimates V under the wrong action distribution. The Q-function avoids*

this because its Bellman target uses $a' \sim \pi_\theta$ (sampled fresh from the current policy) rather than relying on the buffered action.

5. **In GAE, what happens when $\lambda = 0$ and when $\lambda = 1$? How does the parameter λ control the bias-variance tradeoff, and how does it relate to the discount factor γ ?**

Answer: When $\lambda = 0$, GAE reduces to the 1-step TD advantage estimate (low variance, high bias). When $\lambda = 1$, it reduces to the Monte Carlo advantage estimate (no bias, high variance). The parameter λ controls how much we trust the learned value function vs. actual observed rewards: lower λ substitutes the critic earlier (less variance, more bias). The effective discount in the GAE sum is $\gamma\lambda$, where γ controls how much we care about future rewards and λ controls how far we look ahead before bootstrapping.

Chapter 6

Value-Based Methods

CS 185/285 – Deep Reinforcement Learning, Decision Making, and Control
Based on Lectures 7–8 (Sergey Levine, Spring 2026)

6.1 Motivation: Actor-Critic Without the Actor

In actor-critic, we maintain two models: a **critic** (value function) and an **actor** (policy). Value-based methods ask: *can we drop the actor entirely?*

If the action space is discrete and small (e.g., 4 Atari buttons), we can enumerate all actions, evaluate Q for each, and simply pick the best one:

$$\pi(a|s) = \begin{cases} 1 & \text{if } a = \arg \max_{a'} Q(s, a') \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

This **implicitly** defines the policy through the Q-function – no separate policy network needed.

Intuition

Instead of learning *what to do* (actor) and *how good it is* (critic) separately, we only learn how good each action is and always pick the best one. The policy is just the argmax of Q.

6.2 Recovering the Optimal Policy from Value Functions

We have three closely related quantities:

State value function:

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, \pi \right] \quad (2)$$

Action-value (Q) function:

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim p(\cdot|s, a)} [V^\pi(s')] \quad (3)$$

Advantage function:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s) \quad (4)$$

6.2.1 How to recover the optimal policy

Given the **optimal** versions of these functions (denoted with a star superscript):

Expression	Recovers optimal policy?	Why / Why not
$\arg \max_a Q^*(s, a)$	Yes	Directly selects the best action at each state
$\arg \max_a A^*(s, a)$	Yes	$A^* = Q^* - V^*$; since V^* does not depend on a , the argmax over a is the same as for Q^*
$\arg \max_s V^*(s)$	No	This maximizes over <i>states</i> , not actions; it tells you the best state to be in, not what action to take
$\arg \min_a Q^*(s, a)$	No	This selects the <i>worst</i> action

[QUIZ – Lecture 7, Q1] You can recover the optimal policy from $\arg \max_a Q^*$ or $\arg \max_a A^*$, but NOT from $\arg \max_s V^*$ and NOT from $\arg \min$.

Common Misconception

“I can recover the policy from V^* alone.” You cannot – $V^*(s)$ tells you the value of each state, but to pick an action you need to know which action leads to the highest-value next state. That requires knowing the transition dynamics $p(s'|s, a)$, which we generally do not have in model-free RL.

6.3 Dynamic Programming: Policy Iteration and Value Iteration

These are **tabular** methods that assume: - Discrete, small state and action spaces - Known transition dynamics $p(s'|s, a)$

6.3.1 Policy Iteration

High-level idea: Alternate between evaluating a policy and improving it.

Algorithm: Policy Iteration (Tabular)

Initialize: policy π arbitrarily

Repeat until convergence: 1. **Policy Evaluation:** Compute $V^\pi(s)$ for all s - Repeat until convergence: - For all s : $V^\pi(s) \leftarrow r(s, \pi(s)) + \gamma \sum_{s'} p(s'|s, \pi(s)) \cdot V^\pi(s')$ 2. **Policy Improvement:** Set new policy - For all s : $\pi(s) \leftarrow \arg \max_a [r(s, a) + \gamma \sum_{s'} p(s'|s, a) \cdot V^\pi(s')]$

Backup diagram (Policy Evaluation):

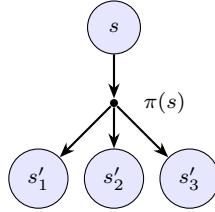


Figure 6.1: Policy evaluation backup: action from $\pi(s)$, then branch over next states weighted by $p(s'|s, a)$.

Professor's Note

“For a fixed policy, evaluating V^π is actually a system of linear equations – you could solve it in closed form. But if you want the *optimal* policy, the max introduces nonlinearity, so you need iteration.”

6.3.2 Value Iteration

Key insight: We can skip the explicit policy improvement step by folding the max directly into the value update.

Derivation (step by step):

Start from policy iteration. The improved policy is:

$$\pi'(s) = \arg \max_a Q^\pi(s, a) \quad (5)$$

The value of this improved policy satisfies:

$$V^{\pi'}(s) = \max_a Q^\pi(s, a) \quad (6)$$

Substituting the definition of Q :

$$V^{\pi'}(s) = \max_a \left[r(s, a) + \gamma \mathbb{E}_{s' \sim p(\cdot|s, a)} [V^\pi(s')] \right] \quad (7)$$

This suggests a single-step update that combines evaluation and improvement:

$$V(s) \leftarrow \max_a \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V(s') \right] \quad (8)$$

Or equivalently using Q-values:

$$Q(s, a) \leftarrow r(s, a) + \gamma \mathbb{E}_{s'}[V(s')] \quad (9)$$

$$V(s) \leftarrow \max_a Q(s, a) \quad (10)$$

Algorithm: Value Iteration (Tabular)

Initialize: $V(s) = 0$ for all s

Repeat until convergence: 1. For all s : $V(s) \leftarrow \max_a [r(s, a) + \gamma \sum_{s'} p(s'|s, a) \cdot V(s')]$

Extract policy: $\pi(s) = \arg \max_a [r(s, a) + \gamma \sum_{s'} p(s'|s, a) \cdot V(s')]$

Backup diagram (Value Iteration):

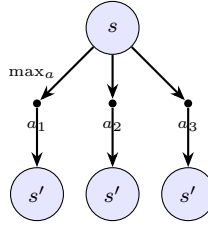


Figure 6.2: Value iteration backup: try all actions, take the max.

6.3.3 Convergence of Tabular Value Iteration

Define the **Bellman backup operator** $\mathcal{B}$:

$$(\mathcal{B}V)(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V(s') \right] \quad (11)$$

Theorem (Contraction): $\mathcal{B}$ is a **contraction** in the ℓ_∞ norm:

$$\|\mathcal{B}V - \mathcal{B}\bar{V}\|_\infty \leq \gamma \|V - \bar{V}\|_\infty \quad (12)$$

Since $\gamma < 1$, repeated application of $\mathcal{B}$ brings any V closer to the unique fixed point V^* :

$$\|\mathcal{B}V - V^*\|_\infty \leq \gamma \|V - V^*\|_\infty \quad (13)$$

Result: Tabular value iteration converges to V^* (and thus to π^*).

6.4 Fitted Value Iteration – and Why It Does Not Work Well

When state spaces are large (e.g., images), we cannot store $V(s)$ in a table. Instead, we use a neural network $V_\theta(s)$.

Fitted Value Iteration: > **Algorithm: Fitted Value Iteration** > > — > > **Repeat:**>

1. For each sample s_i , set target: $y_i \leftarrow \max_{a_i} [r(s_i, a_i) + \gamma \mathbb{E}[V_\theta(s'_i)]]$ >
2. Update θ : $\theta \leftarrow \arg \min_\theta \frac{1}{2} \sum_i \|V_\theta(s_i) - y_i\|^2$

6.4.1 Two critical problems

Problem 1 – Requires transition dynamics: Computing $\max_a$ in step 1 requires evaluating *every possible action* and knowing what next state each would produce. This requires knowing $p(s'|s, a)$, which we typically do not have.

Problem 2 – Cannot do off-policy learning: Even if we approximate the max using samples, fitted V-iteration needs to evaluate outcomes of *different actions from the same state*. With only sampled transitions (s, a, s') , we only know the outcome of the action that was actually taken.

[QUIZ – Lecture 7, Q2] Fitted Q-iteration can use off-policy data. Fitted V-iteration, plain value iteration, and policy iteration with V all cannot (they either require the dynamics model or on-policy data).

Common Misconception

“Fitted value iteration is just value iteration with a neural net, so it should work fine.” It has fundamental problems: (1) it needs a dynamics model for the max, and (2) even the regression step introduces errors that can compound (see Section 6.10 on convergence).

6.5 Q-Learning and Fitted Q-Iteration

6.5.1 The Key Idea: Learn Q Instead of V

If we learn $Q(s, a)$ instead of $V(s)$, we can replace the problematic expected value with a max:

$$V(s) = \max_{a'} Q(s, a') \tag{14}$$

This eliminates the need to know transition dynamics! The target becomes:

$$y_i = r(s_i, a_i) + \gamma \max_{a'_i} Q_\theta(s'_i, a'_i) \tag{15}$$

Everything in this expression – r, s' – comes from a single observed transition (s_i, a_i, s'_i) . We do not need to simulate alternative actions.

6.5.2 Why Q-Functions Enable Off-Policy Learning

Critical insight: Given s and a , the transition to s' is governed by $p(s'|s, a)$, which is independent of the policy π . The target value $r(s, a) + \gamma \max_{a'} Q(s', a')$ only depends on (s, a, s') , not on which policy generated the data.

The $\max_{a'} Q_\theta(s', a')$ implicitly evaluates the *greedy* (improved) policy:

$$\pi'(a|s) = \begin{cases} 1 & \text{if } a = \arg \max_{a'} Q(s, a') \\ 0 & \text{otherwise} \end{cases} \quad (16)$$

So even if the data was collected by an old, different policy, the Q-learning update bootstraps using the best action at s' , which is the action the *new* policy would take.

Intuition

In contrast, if we used $V(s)$ and wanted to evaluate the target $r(s, \pi(s)) + \gamma V(s')$, we would need to know $\pi(s)$ to determine which action to take, and we would need the data to have been collected under that specific policy. Q-learning sidesteps this entirely because the max operator replaces the policy.

6.5.3 Fitted Q-Iteration Algorithm

Algorithm: Fitted Q-Iteration (Batch / Offline)

1. Collect dataset $\{(s_i, a_i, s'_i)\}$ using some policy (any policy)

Repeat K times: 2. For each (s_i, a_i, s'_i) : $y_i \leftarrow r(s_i, a_i) + \gamma \max_{a'} Q_\theta(s'_i, a')$ 3. $\theta \leftarrow \arg \min_\theta \frac{1}{2} \sum_i \|Q_\theta(s_i, a_i) - y_i\|^2$ (run S gradient steps)

Parameters: dataset size N , number of outer iterations K , gradient steps S .

Properties: - (+) Works with off-policy samples - (+) Only one network, no high-variance policy gradient - (-) No convergence guarantees with nonlinear function approximation

6.5.4 What Is Fitted Q-Iteration Optimizing?

The Bellman error is:

$$\mathcal{E} = \frac{1}{2} \mathbb{E}_{(s,a) \sim \beta} \left[\left(Q_\theta(s, a) - \left[r(s, a) + \gamma \max_{a'} Q_\theta(s', a') \right] \right)^2 \right] \quad (17)$$

If $\mathcal{E} = 0$, then:

$$Q_\theta(s, a) = r(s, a) + \gamma \max_{a'} Q_\theta(s', a') \quad (18)$$

This is the **Bellman optimality equation**. A Q-function satisfying this corresponds to the **optimal** policy π^* – independent of what policy collected the data.

6.6 Online Q-Learning

Online Q-learning interleaves data collection and learning:

Algorithm: Online Q-Learning

Repeat:

1. Take action a_i (from some exploration policy), observe (s_i, a_i, s'_i)
 2. Compute target: $y_i = r(s_i, a_i) + \gamma \max_{a'} Q_\theta(s'_i, a'_i)$
 3. Update: $\theta \leftarrow \theta - \alpha \cdot \frac{\partial Q_\theta}{\partial \theta}(s_i, a_i) \cdot (Q_\theta(s_i, a_i) - y_i)$
-

6.7 Exploration: Why Greedy Fails

If we always act greedily ($a = \arg \max_a Q(s, a)$), we may never discover high-reward states.

[QUIZ – Lecture 7, Q3] A purely greedy policy fails because Q-learning can fail to find high-reward states that require exploration to discover. If the agent never visits rewarding regions, Q-values there stay at their initial (low) values.

6.7.1 Epsilon-Greedy Exploration

$$\pi(a|s) = \begin{cases} 1 - \epsilon & \text{if } a = \arg \max_{a'} Q_\theta(s, a') \\ \epsilon / (|\mathcal{A}| - 1) & \text{otherwise} \end{cases} \quad (19)$$

With probability $1 - \epsilon$, take the best action. With probability ϵ , take a random action.

Common schedule: Start with $\epsilon = 1.0$ (fully random) and decay to $\epsilon = 0.01$ over the first N environment steps.

6.7.2 Boltzmann (Softmax) Exploration

$$\pi(a|s) \propto \exp(Q_\theta(s, a)) \quad (20)$$

Actions with higher Q-values are exponentially more likely. This is smoother than epsilon-greedy – it assigns higher probability to actions that Q thinks are better, rather than treating all non-greedy actions equally.

Professor's Note

“Exploration is a really big topic and we’ll discuss it in detail in a later lecture. For now, just know that greedy is bad and epsilon-greedy is a simple fix.”

6.8 Deep Q-Networks (DQN)

If you implement basic Q-learning with a neural network, **it almost certainly will not work**. DQN (Mnih et al., 2013/2015) introduced key tricks to stabilize deep Q-learning.

6.8.1 Experience Replay

Problem: Training on sequential transitions $(s_t, s_{t+1}, s_{t+2}, \dots)$ produces highly correlated gradients. SGD assumes i.i.d. samples – correlated samples destabilize learning.

Solution: Store transitions in a **replay buffer** $\mathcal{D}$ (ring buffer, e.g., capacity 1 million) and sample random mini-batches.

Benefits: 1. **Breaks correlations** between consecutive samples 2. **Reuses data** efficiently (each transition used multiple times) 3. **Reduces non-stationarity** of the training data distribution

6.8.2 Target Networks

Problem: The target $y = r + \gamma \max_{a'} Q_{\theta}(s', a')$ keeps changing as we update θ . Q-learning is **not gradient descent** on any well-defined objective – the targets depend on the parameters being optimized. This creates a moving target problem.

Professor’s Note

“Even though Q-learning looks like it’s taking steps on a gradient with something that resembles mini-batch SGD, Q-learning is actually not gradient descent in the conventional sense. The targets change every single gradient step – and when you actually implement this, don’t forget to put a stop gradient to make sure gradients don’t flow through the target value.”

Solution: Use a separate **target network** $Q_{\bar{\theta}}$ with frozen parameters. Compute targets using $Q_{\bar{\theta}}$, update only Q_{θ} , and periodically sync:

[QUIZ – Lecture 8, Q1] The target network is used specifically when computing $y = r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a')$. The target network parameters $\bar{\theta}$ are frozen during training and only updated periodically.

Two update strategies:

Method	Update Rule	Typical Value
Hard update	$\bar{\theta} \leftarrow \theta$ every N steps	$N = 10,000$
Soft update (Polyak)	$\bar{\theta} \leftarrow \tau \bar{\theta} + (1 - \tau)\theta$ each step	$\tau = 0.999$

Intuition

Intuition (Hard update): For 10,000 steps, you have a perfectly stable regression target. Then you swap in the new network. This is like running 10,000 iterations of a well-defined regression problem, then switching to a new regression problem.

Intuition (Polyak averaging): Instead of the discrete “flip” every 10,000 steps (which creates uneven lag), smoothly blend the current network into the target. With $\tau = 0.999$, after about $1/(1 - \tau) = 1000$ steps, the old target network is mostly replaced.

6.8.3 The DQN Algorithm

Algorithm: DQN (Deep Q-Network)

Initialize: replay buffer $\mathcal{D}$, Q-network Q_θ , target network $Q_{\bar{\theta}} \leftarrow Q_\theta$

Repeat:

1. Select action a using ϵ -greedy w.r.t. Q_θ
2. Execute a , observe r, s' ; store (s, a, s', r) in $\mathcal{D}$
3. Sample mini-batch $\{(s_i, a_i, s'_i, r_i)\}$ from $\mathcal{D}$ uniformly
4. Compute target: $y_i = r_i + \gamma(1 - d_i) \max_{a'} Q_{\bar{\theta}}(s'_i, a')$ where $d_i = 1$ if s'_i is terminal, 0 otherwise
5. Update θ by minimizing: $L = \frac{1}{B} \sum_i (Q_\theta(s_i, a_i) - y_i)^2$ (gradient does NOT flow through y_i - use stop gradient!)
6. Update target network: **Hard:** $\bar{\theta} \leftarrow \theta$ every N steps **OR Soft:** $\bar{\theta} \leftarrow \tau \bar{\theta} + (1 - \tau)\theta$

Common Misconception

“The target network is just a regularization trick.” It is more fundamental than that – without it, Q-learning is not even performing gradient descent on a well-defined objective. The targets y_i depend on θ , so the “loss” is not a fixed function of θ .

6.8.4 The General View: Three Processes

DQN can be viewed as three concurrent processes running at different rates:

Process 1: Data Collection — collect (s, a, s') , add to replay buffer

Process 2: Target Update — periodically copy $\phi \rightarrow \phi'$

Process 3: Q-Regression — sample batch, minimize Bellman error

Figure 6.3: DQN: Three concurrent processes running at different rates.

Process 1 – Data Collection: - Collect (s, a, s') transitions, add to replay buffer - Remove oldest transitions (ring buffer)

Process 2 – Target Update: - Periodically set $\bar{\theta} \leftarrow \theta$ (slow)

Process 3 – Q-Function Regression: - Sample batch from buffer - Compute targets using $Q_{\bar{\theta}}$ - Take gradient step on Q_θ

Algorithm	Process 1	Process 2	Process 3
Online Q-learning	Same speed as 3	Same speed as 3 (buffer size = 1)	Same speed as 1
DQN ($N = 1, K = 1$)	Same speed as 3	Slow (every 10,000 steps)	Same speed as 1
Fitted Q-iteration	Outermost loop	Middle loop	Innermost loop

The **update-to-data (UTD) ratio** K controls how many gradient steps per data point collected:
- Higher UTD = faster learning (but more compute) - Too high UTD = **overfitting** (overoptimistic Q-values due to the max) - Practical range: UTD of 1 to 10 is safe; past 10, test carefully

6.8.5 Value Propagation and N-Step Returns

With target networks, values only propagate one step per target network update. In a grid world with reward only at the end, if the horizon is H steps, you may need H target network updates for the value to reach the start state.

N-step return target speeds up propagation:

$$y_t^{(i)} = \sum_{t'=t}^{t+n} \gamma^{t'-t} r(s_{t'}^{(i)}, a_{t'}^{(i)}) + \gamma^n \max_{a'} Q_{\bar{\theta}}(s_{t+n}^{(i)}, a') \quad (21)$$

Why good: Values propagate n steps at a time instead of 1.

Why bad: The intermediate rewards $r(s_{t'}, a_{t'})$ for $t' > t$ depend on the behavior policy. If that policy is old/different, you get a **downward bias** (underestimation). As $n \rightarrow \infty$, the max disappears and you are just estimating the value of the data-collection policy, not improving. Practical values: $n \approx 5$.

6.9 Double Q-Learning

6.9.1 The Overestimation Problem

Problem: The max operator in the target systematically overestimates Q-values.

Mathematical reason: For any noisy random variables X_1, X_2 :

$$\mathbb{E}[\max(X_1, X_2)] \geq \max(\mathbb{E}[X_1], \mathbb{E}[X_2]) \quad (22)$$

Since $Q_{\bar{\theta}}(s', a')$ is an imperfect approximation (noisy), $\max_{a'} Q_{\bar{\theta}}(s', a')$ systematically overestimates the true maximum value. This is because the max picks out the action with the largest positive error.

[QUIZ – Lecture 8, Q2] Double Q-learning specifically reduces **overestimation bias** that comes from the max operator.

Root cause: In standard Q-learning:

$$\max_{a'} Q_{\bar{\theta}}(s', a') = Q_{\bar{\theta}}(s', \arg \max_{a'} Q_{\bar{\theta}}(s', a')) \quad (23)$$

The **same** network is used to both *select* the action (argmax) and *evaluate* its value. If the network has positive error at some action, that action gets selected AND evaluated at its inflated value.

6.9.2 Double Q-Learning: The Fix

Idea: Decorrelate the action selection from value evaluation by using two different networks.

Original Double Q-learning (Hasselt, 2010): Train two Q-networks Q_{θ_A} and Q_{θ_B} in parallel:

$$Q_{\theta_A}(s, a) \leftarrow r + \gamma Q_{\theta_B}(s', \arg \max_{a'} Q_{\theta_A}(s', a')) \quad (24)$$

$$Q_{\theta_B}(s, a) \leftarrow r + \gamma Q_{\theta_A}(s', \arg \max_{a'} Q_{\theta_B}(s', a')) \quad (25)$$

Each network selects its best action, but the *other* network evaluates that action's value.

6.9.3 Double DQN in Practice

Practical shortcut: We already have two networks – Q_{θ} (current) and $Q_{\bar{\theta}}$ (target). Just use them!

Standard Q-learning target:

$$y = r + \gamma Q_{\bar{\theta}}(s', \arg \max_{a'} Q_{\bar{\theta}}(s', a')) \quad (26)$$

Double Q-learning target:

$$y = r + \gamma Q_{\bar{\theta}}\left(s', \arg \max_{a'} Q_{\theta}(s', a')\right) \quad (27)$$

The only change: use the **current** network Q_{θ} (not target) to select the action, but still use the **target** network $Q_{\bar{\theta}}$ to evaluate.

Professor's Note

“This is so easy to implement that some people do it by accident – literally the only difference is removing the bar from θ in the argmax. Sometimes students have a bug in their HW3 code and instead of regular Q-learning they implement double Q-learning.”

Algorithm: Double DQN

Same as DQN, except step 4 becomes: 4. $a^* = \arg \max_{a'} Q_{\theta}(s'_i, a')$ (current network selects action)
 $y_i = r_i + \gamma(1 - d_i) \cdot Q_{\bar{\theta}}(s'_i, a^*)$ (target network evaluates)

6.9.4 Clipped Double Q-Learning

For even more aggressive overestimation reduction (common in actor-critic, e.g., TD3, SAC):

$$y = r + \gamma \mathbb{E}_{a' \sim \pi_\theta(\cdot|s')} \left[\min_{j \in \{1,2\}} Q_{\bar{\theta}_j}(s', a') \right] \quad (28)$$

Use an ensemble of 2 target Q-networks and take the **minimum**. This can underestimate, but in practice works well for actor-critic.

6.10 Continuous Actions with Q-Learning

In continuous action spaces, we cannot enumerate actions to compute $\max_a Q(s, a)$.

[**QUIZ – Lecture 8, Q3**] Two approaches for continuous actions: (1) optimize over actions (e.g., sampling-based), or (2) use a separate actor network. NOT looking up actions from the replay buffer, NOT using GAE.

6.10.1 Option 1: Optimization-Based

Sample candidate actions and pick the best:

1. **Simple random sampling:** Generate N actions uniformly, evaluate $Q(s, a_i)$ for each, take the max. Works for low-dimensional actions (1-2D).
2. **Cross-Entropy Method (CEM):** Iteratively refine a sampling distribution. Generate samples, keep the best, update the distribution. Works up to ~40 dimensions.
3. **CMA-ES:** Similar iterative stochastic optimization.

These are **wide but short** optimizers – many parallel samples, few iterations. They parallelize well on GPU.

6.10.2 Option 2: DDPG / Actor Network

Train a **deterministic** actor $\mu_\theta(s)$ to approximate the argmax:

$$\mu_\theta(s) \approx \arg \max_a Q_\phi(s, a) \quad (29)$$

DDPG (Deep Deterministic Policy Gradient):

The actor is trained by maximizing Q:

$$\max_{\theta} \mathbb{E}_{s \sim \mathcal{D}} [Q_\phi(s, \mu_\theta(s))] \quad (30)$$

Gradient via chain rule:

$$\frac{\partial Q_\phi(s, \mu_\theta(s))}{\partial \theta} = \frac{\partial \mu_\theta(s)}{\partial \theta} \cdot \frac{\partial Q_\phi(s, a)}{\partial a} \Big|_{a=\mu_\theta(s)} \quad (31)$$

The gradient flows through Q into the actor – no REINFORCE needed.

Algorithm: DDPG (Deep Deterministic Policy Gradient)

Repeat:

1. Take action a_i (with exploration noise), observe (s_i, a_i, s'_i) ; store in replay buffer $\mathcal{D}$
2. Sample mini-batch from $\mathcal{D}$
3. Compute target: $y_i = r_i + \gamma \cdot Q_{\bar{\phi}}(s'_i, \mu_{\bar{\theta}}(s'_i))$
4. Update critic ϕ : minimize $\frac{1}{B} \sum_i (Q_\phi(s_i, a_i) - y_i)^2$
5. Update actor θ : maximize $\frac{1}{B} \sum_i Q_\phi(s_i, \mu_\theta(s_i))$ (gradient does NOT flow through Q to update ϕ here)
6. Update target networks: $\bar{\phi} \leftarrow \tau \bar{\phi} + (1 - \tau)\phi$, $\bar{\theta} \leftarrow \tau \bar{\theta} + (1 - \tau)\theta$

Intuition

DDPG is simultaneously Q-learning (for the critic) and actor-critic (for the actor). One way to view actor-critic: it is Q-learning with an approximate maximizer. Another view: it is policy gradient with a Q-function providing advantages. They converge to the same thing.

6.11 Convergence: NONE of These Converge with Neural Nets

6.11.1 Tabular Value Iteration Converges

The Bellman operator $\mathcal{B}$ is a contraction in $\|\cdot\|_\infty$:

$$\|\mathcal{B}V - \mathcal{B}\bar{V}\|_\infty \leq \gamma \|V - \bar{V}\|_\infty \quad (32)$$

Repeated application: $V \rightarrow \mathcal{B}V \rightarrow \mathcal{B}^2V \rightarrow \dots \rightarrow V^*$.

6.11.2 Fitted Value Iteration Does NOT Converge

With function approximation, the regression step is a **projection** Π onto the set of representable functions Ω :

$$V \leftarrow \Pi \mathcal{B}V \quad (33)$$

- $\mathcal{B}$ is a contraction in $\|\cdot\|_\infty$

- Π is a contraction in $\|\cdot\|_2$ (L2 projection)
- But the composition $\Pi\mathcal{B}$ is **not a contraction in any norm**

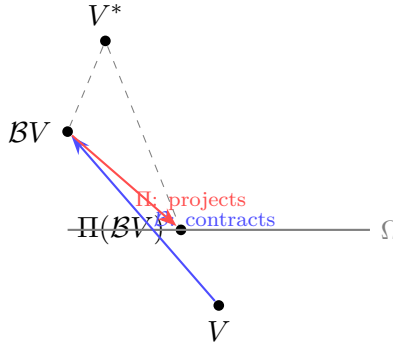


Figure 6.4: Why fitted VI may not converge: $\mathcal{B}$ contracts toward V^* in $\|\cdot\|_\infty$, but Π projects onto Ω in $\|\cdot\|_2$, potentially increasing distance to V^* .

The backup $\mathcal{B}$ brings V closer to V^* in ℓ_∞ , but the projection Π back onto Ω can push it further away in ℓ_∞ . The algorithm can diverge or enter a cycle around V^* .

6.11.3 The Definitive Statement

[QUIZ – Lecture 8, Q4] NONE of the following are guaranteed to converge when using neural network function approximation: - Fitted value iteration - Fitted Q-iteration - Actor-critic with bootstrap targets

All of these involve composing a Bellman backup (contraction in ℓ_∞) with a regression/projection step (contraction in ℓ_2), which is not a contraction in any norm.

Professor's Note

“Most guarantees are lost when we leave the tabular case. This is kind of a problem – we know gradient descent converges but we don’t necessarily know this thing converges. In practice, all the tricks we discussed (target networks, experience replay, double Q-learning) help stabilize things enough that they work, but there are no formal guarantees.”

6.12 Soft Actor-Critic (SAC)

SAC addresses the continuous action problem with a principled framework based on maximum entropy RL.

6.12.1 Maximum Entropy Objective

Standard RL: $\max_{\pi} \sum_t \mathbb{E}[r(s_t, a_t)]$

Maximum entropy RL adds an **entropy bonus**:

$$J(\pi) = \sum_t \mathbb{E}[r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t)))] \quad (34)$$

where $\mathcal{H}(\pi(\cdot|s)) = -\mathbb{E}_{a \sim \pi(\cdot|s)}[\log \pi(a|s)]$ is the conditional entropy.

Why entropy regularization? - Encourages exploration: prevents premature collapse to a deterministic policy - **Improves robustness:** captures multiple good solutions rather than committing to one - **Improves convergence** in practice

The **temperature** α controls the tradeoff between reward maximization and entropy: - Large α : prioritize exploration (high entropy) - Small α : prioritize exploitation (reward)

6.12.2 SAC Components

1. Stochastic Policy with Reparameterization:

SAC uses a Gaussian policy with a tanh squashing function:

$$a = \tanh(\mu_{\phi}(s) + \sigma_{\phi}(s) \odot \epsilon), \quad \epsilon \sim \mathcal{N}(0, I) \quad (35)$$

The tanh bounds actions to $[-1, 1]$. The reparameterization trick allows backpropagation through sampling.

2. Clipped Double-Q (from TD3):

SAC uses two Q-networks and takes the minimum to reduce overestimation:

$$y = r + \gamma \left(\min_{j=1,2} Q_{\bar{\theta}_j}(s', a') - \alpha \log \pi_{\phi}(a'|s') \right), \quad a' \sim \pi_{\phi}(\cdot|s') \quad (36)$$

3. Automatic Temperature Tuning:

Instead of hand-tuning α , SAC learns it by minimizing:

$$\mathcal{L}_{\alpha} = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_{\phi}(\cdot|s)} [-\alpha (\log \pi_{\phi}(a|s) + \mathcal{H}_{\text{tgt}})] \quad (37)$$

where $\mathcal{H}_{\text{tgt}}$ is a target entropy, typically $-\dim(\mathcal{A})$.

Implementation tip: Optimize $\log \alpha$ instead of α directly to ensure $\alpha > 0$.

6.12.3 SAC Losses Summary

Critic loss (for each Q_{θ_i} , $i \in \{1, 2\}$):

$$\mathcal{L}_Q(\theta_i) = \mathbb{E}_{(s,a,s') \sim \mathcal{D}} [(Q_{\theta_i}(s, a) - y)^2] \quad (38)$$

Actor loss:

$$\mathcal{L}_\pi(\phi) = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_\phi(\cdot|s)} \left[\alpha \log \pi_\phi(a|s) - \min_{i=1,2} Q_{\theta_i}(s, a) \right] \quad (39)$$

Temperature loss:

$$\mathcal{L}_\alpha = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_\phi(\cdot|s)} [-\alpha (\log \pi_\phi(a|s) + \mathcal{H}_{\text{tgt}})] \quad (40)$$

Algorithm: Soft Actor-Critic (SAC)

Initialize: $Q_{\theta_1}, Q_{\theta_2}, \pi_\phi, \alpha$; target networks $\bar{\theta}_1, \bar{\theta}_2$; replay buffer $\mathcal{D}$

Repeat:

1. Sample $a \sim \pi_\phi(\cdot|s)$, execute, observe (s, a, r, s') , store in $\mathcal{D}$
2. Sample mini-batch from $\mathcal{D}$
3. Compute target: sample $a' \sim \pi_\phi(\cdot|s')$, then $y = r + \gamma(1-d) \left(\min_{j=1,2} Q_{\bar{\theta}_j}(s', a') - \alpha \log \pi_\phi(a'|s') \right)$
4. Update both critics: $\theta_i \leftarrow \theta_i - \text{lr} \cdot \nabla \mathcal{L}_Q(\theta_i)$ for $i = 1, 2$
5. Update actor: $\phi \leftarrow \phi - \text{lr} \cdot \nabla \mathcal{L}_\pi(\phi)$
6. Update temperature: $\alpha \leftarrow \alpha - \text{lr} \cdot \nabla \mathcal{L}_\alpha$
7. Update target networks (Polyak): $\bar{\theta}_j \leftarrow \tau \bar{\theta}_j + (1 - \tau) \theta_j$ for $j = 1, 2$

Intuition

SAC = DDPG + stochastic policy + entropy bonus + clipped double-Q + automatic temperature. It is currently one of the most popular off-policy continuous-action RL algorithms.

6.13 Practical Tips for Deep Q-Learning

From lecture and course staff notes:

1. **Test on easy tasks first** (e.g., CartPole, Pong) before moving to harder ones
2. **Large replay buffers** (1 million transitions) stabilize training by decorrelating gradient updates
3. **Expect S-shaped learning curves** – early plateau while exploring, then rapid improvement
4. **Schedule exploration:** ϵ starts high (~ 1.0) and decays to low (~ 0.01)
5. **Use Huber loss** instead of MSE to handle large Bellman errors (clip gradients)
6. **Double Q-learning** is free and always helps – no reason not to use it
7. **Handle terminal states:** $y = r + \gamma(1-d) \max_{a'} Q_{\bar{\theta}}(s', a')$ where $d = 1$ if terminal
8. **Run multiple random seeds** – Q-learning has high variance from exploration and initialization
9. **Use Adam** optimizer (or schedule learning rate)
10. **N-step returns** help propagation speed but introduce bias – use $n \approx 3-5$

6.14 Summary Table

Method	On/Off-Policy	Discrete/Continuous	Convergence (tabular)	Convergence (neural nets)
Policy Iteration	On-policy (needs dynamics)	Either	Yes	N/A (needs full MDP)
Value Iteration	N/A (needs dynamics)	Either	Yes	No
Fitted V Iteration	Needs dynamics	Either	–	No
Fitted Q Iteration	Off-policy	Discrete	Yes (tabular)	No
DQN	Off-policy	Discrete	–	No (but works with tricks)
Double DQN	Off-policy	Discrete	–	No (but more stable)
DDPG	Off-policy	Continuous	–	No
SAC	Off-policy	Continuous	–	No

6.15 Check Your Understanding

Q1. Why can fitted Q-iteration use off-policy data, but fitted value iteration cannot?

Answer: In fitted Q-iteration, the target $y_i = r(s_i, a_i) + \gamma \max_{a'} Q_\theta(s'_i, a')$ only requires a transition (s, a, s') . Given s and a , the transition to s' is independent of the policy – it is determined by the dynamics $p(s'|s, a)$. The max over a' at s' implicitly evaluates the greedy (optimal) policy, not the data-collection policy. In fitted V-iteration, computing the target requires $\max_a [r(s, a) + \gamma E[V(s')]]$, which means evaluating what would happen for every action from the same state – this requires knowing the transition model.

Q2. In the DQN algorithm, where exactly is the target network used, and why is a stop-gradient necessary?

Answer: The target network $Q_{\bar{\theta}}$ is used to compute the target value: $y = r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a')$. A stop-gradient is necessary because without it, the gradient from the loss $(Q_\theta(s, a) - y)^2$ would flow through y back into Q_θ (since $Q_{\bar{\theta}}$ and Q_θ share the same architecture). But the target should be treated as a constant for the purpose of the gradient – we only want to update $Q_\theta(s, a)$ to move toward the target, not also change

the target. Q-learning is not gradient descent on any fixed objective; it is an iterative process where targets are periodically refreshed.

Q3. Explain why $\mathbb{E}[\max(X_1, X_2)] \geq \max(\mathbb{E}[X_1], \mathbb{E}[X_2])$ leads to overestimation in Q-learning, and how double Q-learning addresses it.

Answer: In standard Q-learning, $\max_{a'} Q(s', a') = Q(s', \arg \max_{a'} Q(s', a'))$. The same noisy function is used to both select the action (argmax) and evaluate its value. If the Q-function has positive error at some action, that action is preferentially selected AND evaluated at its inflated value, systematically overestimating the true max. Double Q-learning decorrelates selection and evaluation: use the current network Q_θ to select the best action, but evaluate that action using the target network $Q_{\bar{\theta}}$. Since the two networks have different errors, the positive error from one is unlikely to be present in the other, reducing the bias.

Q4. Why does fitted value iteration with neural networks fail to converge, even though tabular value iteration converges?

Answer: Tabular value iteration converges because the Bellman operator $\mathcal{B}$ is a contraction in $\|\cdot\|_\infty$ – each application brings the value function closer to V^* . With function approximation, the regression step is a projection Π onto the set of representable functions, which is a contraction in $\|\cdot\|_2$. The composition $\Pi\mathcal{B}$ of an ℓ_∞ -contraction followed by an ℓ_2 -contraction is NOT a contraction in any norm. The backup can bring V closer to V^* in ℓ_∞ , but the projection back onto the function class can push it further away. The algorithm can diverge or oscillate.

Q5. In SAC, what role does the entropy bonus play, and how does automatic temperature tuning work?

Answer: The entropy bonus $\alpha \mathcal{H}(\pi(\cdot|s))$ encourages the policy to remain stochastic, which promotes exploration and prevents premature convergence to a suboptimal deterministic policy. The temperature α controls the exploration-exploitation tradeoff. Automatic tuning learns α by minimizing $\mathcal{L}_\alpha = \mathbb{E}[-\alpha(\log \pi(a|s) + \mathcal{H}_{\text{tgt}})]$. This adjusts α so that the average log-probability of actions under the policy matches the target entropy $\mathcal{H}_{\text{tgt}} = -\dim(\mathcal{A})$. If the policy is too deterministic (low entropy), α increases to add more entropy bonus; if too random, α decreases.

Chapter 7

Advanced Policy Gradients

Lectures covered: Lecture 9 (Off-Policy Policy Gradient), Lecture 10 (Advanced Policy Gradient) **Key algorithms:** PPO (Proximal Policy Optimization), TRPO (Trust Region Policy Optimization), Natural Policy Gradient **Prerequisites:** Policy gradients (Ch. 4), Actor-Critic / GAE (Ch. 5), Value-based methods (Ch. 6)

7.1 Why REINFORCE Is Impractical

Recall the on-policy batch actor-critic algorithm (policy gradient with GAE):

Algorithm: Policy Gradient with GAE

1. Sample $\{\tau^{(i)}\}$ from π_θ (run the policy)
2. Evaluate $y_t^{(i)} = r(s_t^{(i)}, a_t^{(i)}) + V_\phi^\pi(s_{t+1}^{(i)})$
3. Fit $V_\phi^\pi(s)$ to targets $\{y_t^{(i)}\}$
4. Evaluate $\hat{A}_{\text{GAE}}^\pi(s_t^{(i)}, a_t^{(i)}) = \sum_{t'=t}^\infty (\gamma\lambda)^{t'-t} \delta_{t'}^{(i)}$
5. $\nabla_\theta J(\theta) \approx \sum_i \sum_t \nabla_\theta \log \pi_\theta(a_t^{(i)} | s_t^{(i)}) \cdot \hat{A}^\pi(s_t^{(i)}, a_t^{(i)})$
6. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

The fundamental problem: Each iteration takes exactly **one** gradient step, yet each iteration requires generating entirely new samples (steps 1-4). With neural network policies, we typically need hundreds of thousands or millions of gradient steps. Generating fresh samples for every single gradient step is extraordinarily wasteful.

Additionally, the high variance of policy gradient estimators means we usually need: - **Bigger batches** than in standard supervised learning - **Smaller learning rates** (which means more gradient steps needed)

This quickly becomes intractable, even in settings where samples are cheap (e.g., simulation, LLM training, game environments).

Intuition

Imagine training a neural network classifier with standard SGD, but you are forced to collect a brand new dataset before every single gradient step. That is essentially what vanilla REINFORCE does – it throws away all data after one gradient update.

[QUIZ] Q1 (Lecture 9): REINFORCE is impractical because it requires generating new samples for every gradient step. The samples from the previous iteration cannot be reused because the expectation in the policy gradient is over trajectories from the *current* policy π_θ , and after a gradient step θ has changed.

7.1.1 What We Would Like

We want an algorithm that: 1. Generates samples (expensive steps 1-4) **once** 2. Takes **K** gradient steps on the policy using those same samples (cheap steps 5-6) 3. Only then goes back to generate fresh samples

The naive approach of simply iterating steps 5-6 multiple times is **incorrect**. After the first gradient update, θ has changed, so the samples no longer come from the current policy distribution. The gradient estimate becomes biased – it is no longer a valid estimate of the expected gradient under the new policy.

Professor's Note

“If you push [the policy] away from multiple gradient steps, it can actually go very, very far away from all of your samples. It can go off to the stratosphere. It'll assign extremely low probability to that bad sample, also extremely low probability to everything else. But since it's not getting any more samples in the location where it's currently located, your policy can get arbitrarily bad.”

Solution: Use **importance sampling** to correct for the distribution mismatch.

7.2 Importance Sampling for Policy Gradients

7.2.1 Importance Sampling Review

Given an expectation under distribution $p(x)$ that we want to estimate using samples from a different distribution $q(x)$:

$$\mathbb{E}_{x \sim p(x)}[f(x)] = \int p(x) f(x) dx \quad (7.1)$$

$$= \int \frac{q(x)}{q(x)} p(x) f(x) dx \quad (7.2)$$

$$= \int q(x) \frac{p(x)}{q(x)} f(x) dx \quad (7.3)$$

$$= \mathbb{E}_{x \sim q(x)} \left[\frac{p(x)}{q(x)} f(x) \right] \quad (7.4)$$

The ratio $p(x)/q(x)$ is called the **importance weight**. It corrects for the fact that samples came from the “wrong” distribution.

7.2.2 Applying IS to the RL Objective

Our RL objective is:

$$\theta^* = \arg \max_{\theta} J(\theta), \quad J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [r(\tau)] \quad (7.5)$$

If we have samples from some other distribution $\bar{p}(\tau)$ (e.g., a previous policy), we can write:

$$J(\theta) = \mathbb{E}_{\tau \sim \bar{p}(\tau)} \left[\frac{p_{\theta}(\tau)}{\bar{p}(\tau)} r(\tau) \right] \quad (7.6)$$

7.2.3 Trajectory Probability Ratio – Cancellation of Dynamics

The trajectory probability factorizes as:

$$p_{\theta}(\tau) = p(\mathbf{s}_1) \prod_{t=1}^H \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t) \quad (7.7)$$

When we form the ratio $p_{\theta}(\tau)/\bar{p}(\tau)$, the initial state probabilities $p(\mathbf{s}_1)$ and transition dynamics $p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$ **cancel out** because they do not depend on the policy:

$$\frac{p_{\theta}(\tau)}{\bar{p}(\tau)} = \frac{\prod_{t=1}^H \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)}{\prod_{t=1}^H \bar{\pi}(\mathbf{a}_t | \mathbf{s}_t)} \quad (7.8)$$

This is crucial – we do not need to know the environment dynamics to compute importance weights.

7.2.4 Deriving the Off-Policy Policy Gradient

Now suppose we want to evaluate the objective for *new* parameters θ' using samples collected under θ :

$$J(\theta') = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\frac{p_{\theta'}(\tau)}{p_{\theta}(\tau)} r(\tau) \right] \quad (7.9)$$

Taking the gradient with respect to θ' :

$$\nabla_{\theta'} J(\theta') = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\frac{\nabla_{\theta'} p_{\theta'}(\tau)}{p_{\theta}(\tau)} r(\tau) \right] \quad (7.10)$$

Using the log-derivative trick ($p_{\theta}(\tau) \cdot \nabla \log p_{\theta}(\tau) = \nabla p_{\theta}(\tau)$):

$$= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\frac{p_{\theta'}(\tau)}{p_{\theta}(\tau)} \nabla_{\theta'} \log p_{\theta'}(\tau) r(\tau) \right] \quad (7.11)$$

Intuition

A convenient identity used here: $p_{\theta}(\tau) \cdot \nabla_{\theta} \log p_{\theta}(\tau) = \nabla_{\theta} p_{\theta}(\tau)$. This is the same log-derivative trick from the vanilla policy gradient derivation.

When $\theta' = \theta$ (evaluate locally), the importance weights become 1 and we recover the standard policy gradient:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \log p_{\theta}(\tau) r(\tau)] \quad (7.12)$$

7.2.5 Expanding the Off-Policy Gradient (theta != theta')

Substituting the trajectory ratio (Eq. 7.8) into Eq. 7.11:

$$\nabla_{\theta'} J(\theta') = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\left(\prod_{t=1}^H \frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)} \right) \left(\sum_{t=1}^H \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t) \right) \left(\sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t) \right) \right] \quad (7.13)$$

Applying causality (future actions don't affect current weights, and past rewards don't matter):

$$= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^H \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t) \left(\prod_{t'=1}^t \frac{\pi_{\theta'}(\mathbf{a}_{t'} | \mathbf{s}_{t'})}{\pi_{\theta}(\mathbf{a}_{t'} | \mathbf{s}_{t'})} \right) \left(\sum_{t'=t}^H r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \right) \right] \quad (7.14)$$

Problem: The product of importance weights is **exponential in T**. Even with moderate horizons, this product can become astronomically large or vanishingly small, leading to extreme variance.

7.2.6 First-Order Approximation

To make this practical, we make two simplifications:

Simplification 1: Ignore the importance weights on the reward-to-go (the future IS weights). This corresponds to using the advantage of π_{θ} instead of $\pi_{\theta'}$.

Simplification 2: Ignore the state marginal ratio ($\pi_{\theta'}(s_t)/\pi_{\theta}(s_t)$), keeping only the action ratio.

This gives us the approximate off-policy policy gradient:

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \frac{\pi_{\theta'}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})}{\pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})} \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \hat{A}_t^{(i)} \quad (7.15)$$

where $(s_t^{(i)}, a_t^{(i)}) \sim \pi_{\theta}(s_t, a_t)$ are samples from the *old* policy.

[QUIZ] Q2 (Lecture 9): The intractable term in the importance-sampled policy gradient is the **state marginal ratio** $p_{\theta'}(s_t)/p_{\theta}(s_t)$. Unlike the action probability ratio $\pi_{\theta'}(a|s)/\pi_{\theta}(a|s)$, the gradient of log pi, or the advantages, the state marginal ratio requires knowing the transition dynamics of the MDP and is generally impossible to compute.

Common Misconception

Students sometimes think the transition dynamics ratio $p(s'|s,a)$ is the intractable term. But the dynamics cancel in the trajectory ratio (Eq. 7.8). The intractable term is the *state marginal* $p_{\theta}(s_t)$ – the marginal probability of visiting state s_t at time t under policy π_{θ} – which depends on the entire history of the policy’s interactions with the environment.

7.3 Clipping Importance Weights

7.3.1 The Variance Problem

The multi-step off-policy algorithm uses importance weights $\pi_{\theta'}(a|s)/\pi_{\theta}(a|s)$. As θ' drifts from θ over multiple gradient steps, these weights can become extreme: - Very large weights ($\gg 1$): θ' assigns much higher probability than θ to some actions - Very small weights ($\ll 1$): θ' assigns much lower probability

Formally, the **variance of the importance-sampled estimator becomes too high** when the weights are far from 1.0. The estimator is technically unbiased, but in practice it becomes useless because a few samples with extreme weights dominate the entire estimate.

7.3.2 The Surrogate Objective

The importance-sampled objective can be written as:

$$J(\theta') = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\underbrace{\frac{p_{\theta'}(\tau)}{p_{\theta}(\tau)}}_{w(\tau)} r(\tau) \right] \quad (7.16)$$

This is called the **surrogate objective** because we are estimating $J(\theta')$ using samples from θ .

7.3.3 Clipped Weights

To control variance and prevent the policy from changing too much, we **clip** the importance weights:

$$w_c(\tau) = \max \left\{ 1 - \epsilon, \min \left\{ 1 + \epsilon, \frac{p_{\theta'}(\tau)}{p_{\theta}(\tau)} \right\} \right\} \quad (7.17)$$

where ϵ is a small constant (commonly 0.1 or 0.2).

This puts a **hard limit** on how much probabilities can increase or decrease: - If the ratio exceeds $1 + \epsilon$, it is clamped to $1 + \epsilon$ - If the ratio falls below $1 - \epsilon$, it is clamped to $1 - \epsilon$

[QUIZ] Q3 (Lecture 9): Clipping importance weights helps with TWO things: (1) it reduces the **high variance** that comes from extreme importance weights, and (2) it **prevents the policy from changing too much** in a single update, which is important because our approximation (ignoring the state marginal ratio) is only valid when the policies are close.

7.3.4 Asymmetric Clipping (PPO Detail)

There is a subtle but important detail. Simply using clipped weights $w_c(\tau) \cdot r(\tau)$ has a problem: when $r(\tau) < 0$ (negative advantage), the clipped objective provides **no penalty for moving away from the data**. The policy could drift arbitrarily far in the direction of decreasing probability of bad trajectories without any cost.

The fix is to use the **minimum** of the clipped and unclipped objectives:

$$J(\theta') = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\min\{w(\tau)r(\tau), w_c(\tau)r(\tau)\}] \quad (7.18)$$

This ensures: - When $A > 0$ (good actions): the objective is clipped above at $(1 + \epsilon) * A$, so there is no incentive to increase the ratio beyond $1 + \epsilon$ - When $A < 0$ (bad actions): the objective is clipped below at $(1 - \epsilon) * A$, so there is no incentive to decrease the ratio below $1 - \epsilon$

(See Figure 7.1 for visualization of asymmetric clipping behavior.)

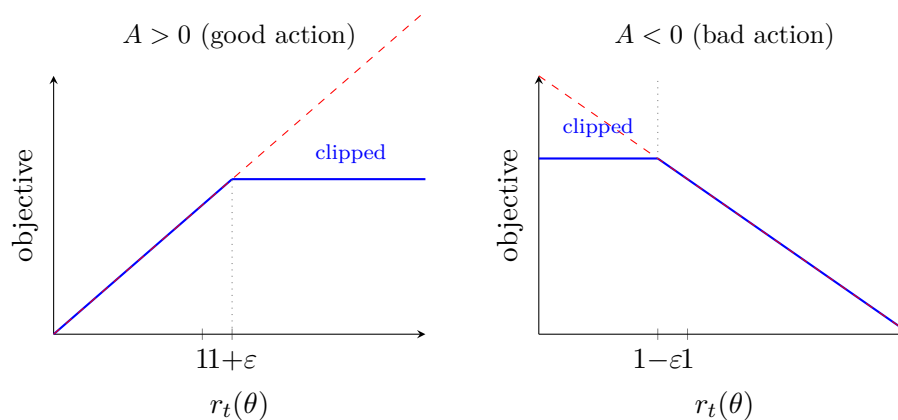


Figure 7.1: PPO clipping. Left: positive advantage clips at $1 + \epsilon$. Right: negative advantage clips at $1 - \epsilon$.

7.4 Off-Policy Policy Gradient and the Surrogate Objective

7.4.1 Policy Gradient as Policy Iteration

A key insight from Lecture 10 is that policy gradient can be viewed as a form of **policy iteration**:

Classic policy iteration: 1. Evaluate $A^\pi(s, a)$ for the current policy π 2. Set $\pi \leftarrow \pi'$ where π' improves on the advantages of π

Policy gradient: 1. Estimate $\hat{A}^\pi(s_t, a_t)$ for the current policy π 2. Use $\hat{A}^\pi$ to get an *improved* policy π' via gradient ascent

The crucial observation: in policy iteration, we evaluate the advantage of the **old** policy and use it to find a **better** new policy. The new policy is judged by how well it does under the advantage of the old policy, not its own advantage.

7.4.2 The Policy Improvement Identity

Claim: The difference in performance between the new and old policy equals the expected advantage of the old policy under the new policy's trajectory distribution:

$$J(\theta') - J(\theta) = \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_t \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \quad (7.19)$$

Proof (sketch):

Start with $J(\theta') - J(\theta)$. Express $J(\theta)$ as the expected value function at the initial state:

$$J(\theta') - J(\theta) = J(\theta') - \mathbb{E}_{\mathbf{s}_0 \sim p(\mathbf{s}_0)} [V^{\pi_\theta}(\mathbf{s}_0)] \quad (7.20)$$

Since the initial state distribution is independent of the policy, we can take the expectation under $p_{\theta'}(\tau)$:

$$= J(\theta') - \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} [V^{\pi_\theta}(\mathbf{s}_0)] \quad (7.21)$$

Now apply the telescoping trick. Write $V^{\pi_\theta}(\mathbf{s}_0)$ as the difference of two sums:

$$= J(\theta') + \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (\gamma V^{\pi_\theta}(\mathbf{s}_{t+1}) - V^{\pi_\theta}(\mathbf{s}_t)) \right] \quad (7.22)$$

Combine with $J(\theta') = \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} [\sum_t \gamma^t r(s_t, a_t)]$:

$$= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (r(\mathbf{s}_t, \mathbf{a}_t) + \gamma V^{\pi_\theta}(\mathbf{s}_{t+1}) - V^{\pi_\theta}(\mathbf{s}_t)) \right] \quad (7.23)$$

$$= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \quad \blacksquare \quad (7.24)$$

Intuition

This result says: “the improvement of the new policy over the old one is exactly equal to how much better the new policy’s actions are, on average, according to the old policy’s advantage function.” This justifies using the old policy’s advantage estimates (which we can compute) rather than the new policy’s advantage estimates (which we cannot).

[QUIZ] Q1 (Lecture 10): The surrogate objective (using $p_\theta(s_t)$ instead of $p_{\theta'}(s_t)$ in the expectation) is valid because $p_\theta(s_t)$ is approximately equal to $p_{\theta'}(s_t)$ when the two policies are close. The exact objective requires expectations under $p_{\theta'}$, but when the policies don’t differ much, their state visitation distributions are similar.

7.4.3 Converting to an Off-Policy Objective

From Eq. 7.24, we can decompose the expectation over trajectories into expectations over states and actions:

$$J(\theta') - J(\theta) = \sum_t \mathbb{E}_{\mathbf{s}_t \sim p_{\theta'}(\mathbf{s}_t)} \left[\mathbb{E}_{\mathbf{a}_t \sim \pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)} \left[\gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right] \quad (7.25)$$

Apply importance sampling on the action distribution to switch from $\pi_{\theta'}$ to π_θ :

$$= \sum_t \mathbb{E}_{\mathbf{s}_t \sim p_{\theta'}(\mathbf{s}_t)} \left[\mathbb{E}_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right] \quad (7.26)$$

Note: this is exact – we used importance sampling only on the action, which involves a **single** probability ratio (not a product over time), so the variance is controlled.

But we still have the state distribution under $p_{\theta'}(s_t)$, which we cannot sample from. This motivates the need for the surrogate objective.

7.5 Trust Regions and Policy Constraints

7.5.1 Bounding the State Distribution Mismatch

We want to show that replacing $p_{\theta'}(s_t)$ with $p_\theta(s_t)$ introduces only bounded error when the policies are close. The key result:

Claim: $p_\theta(s_t)$ is close to $p_{\theta'}(s_t)$ when π_θ is close to $\pi_{\theta'}$.

Proof (deterministic case for intuition):

Assume π_θ is deterministic. Define “close” as: $\pi_{\theta'}$ takes a different action from π_θ with probability at most ϵ .

Then the state distribution under θ' can be decomposed:

$$p_{\theta'}(\mathbf{s}_t) = (1 - \epsilon)^t p_\theta(\mathbf{s}_t) + (1 - (1 - \epsilon)^t) p_{\text{mistake}}(\mathbf{s}_t) \quad (7.27)$$

where: - $(1 - \epsilon)^t$ is the probability of making no mistakes in t steps - $p_mistake(s_t)$ is some unknown distribution (where we end up if we deviate)

Taking the total variation divergence:

$$\frac{1}{2} \sum_{\mathbf{s}_t} |p_{\theta'}(\mathbf{s}_t) - p_{\theta}(\mathbf{s}_t)| = (1 - (1 - \epsilon)^t) \frac{1}{2} \sum_{\mathbf{s}_t} |p_{mistake}(\mathbf{s}_t) - p_{\theta}(\mathbf{s}_t)| \leq 1 - (1 - \epsilon)^t \quad (7.28)$$

Using the identity $(1 - \epsilon)^t \geq 1 - \epsilon t$ for ϵ in $[0, 1]$:

$$\frac{1}{2} \sum_{\mathbf{s}_t} |p_{\theta'}(\mathbf{s}_t) - p_{\theta}(\mathbf{s}_t)| \leq \epsilon t \quad (7.29)$$

Professor's Note

“It’s not a great bound, but it is a bound!” The bound grows linearly with the horizon t , which is pessimistic but finite. In practice, small policy changes typically lead to much smaller state distribution shifts than this worst-case bound suggests.

General case (stochastic policies): Define “close” via total variation divergence:

$$\pi_{\theta'} \text{ is close to } \pi_{\theta} \text{ if } D_{TV}(\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t), \pi_{\theta}(\mathbf{a}_t|\mathbf{s}_t)) = \frac{1}{2} \sum_{\mathbf{a}_t} |\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t) - \pi_{\theta}(\mathbf{a}_t|\mathbf{s}_t)| \leq \epsilon \quad \forall \mathbf{s}_t \quad (7.30)$$

Using a coupling lemma (if $D_{TV}(p_X, p_Y) = \epsilon$, there exists a joint $p(x, y)$ such that $p(x) = p_X(x)$, $p(y) = p_Y(y)$, and $p(x = y) = 1 - \epsilon$), the same bound holds:

$$\frac{1}{2} \sum_{\mathbf{s}_t} |p_{\theta'}(\mathbf{s}_t) - p_{\theta}(\mathbf{s}_t)| \leq \epsilon t \quad (7.31)$$

7.5.2 Bounding the Objective Value

Using the state distribution bound, we can relate expectations under $p_{\theta'}$ to expectations under p_{θ} :

$$\mathbb{E}_{p_{\theta'}(\mathbf{s}_t)}[f(\mathbf{s}_t)] \geq \mathbb{E}_{p_{\theta}(\mathbf{s}_t)}[f(\mathbf{s}_t)] - 2\epsilon t \max_{\mathbf{s}_t} |f(\mathbf{s}_t)| \quad (7.32)$$

Applying this to our objective (Eq. 7.26):

$$J(\theta') - J(\theta) \geq \sum_t \mathbb{E}_{\mathbf{s}_t \sim p_{\theta}(\mathbf{s}_t)} \left[\mathbb{E}_{\mathbf{a}_t \sim \pi_{\theta}(\mathbf{a}_t|\mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)}{\pi_{\theta}(\mathbf{a}_t|\mathbf{s}_t)} A^{\pi_{\theta}}(\mathbf{s}_t, \mathbf{a}_t) \right] \right] - \sum_t 2\epsilon t C \quad (7.33)$$

where $C = O(H * r_max)$ or $O(r_max / (1 - \gamma))$ in the discounted case.

Key insight: Maximizing the first term on the RHS (which only requires samples from π_{θ}) maximizes a **lower bound** on the thing we actually care about ($J(\theta') - J(\theta)$), provided ϵ is small enough.

7.5.3 The Constrained Optimization Problem

[QUIZ] Q2 (Lecture 10): A valid constraint on the policy update is the total variation divergence: $|\pi_{\theta'} - \pi_{\theta}| \leq \epsilon$. This is the natural constraint that falls out of the bound derivation.

This gives us the constrained optimization:

$$\theta' \leftarrow \arg \max_{\theta'} \sum_t \mathbb{E}_{\mathbf{s}_t \sim p_{\theta}(\mathbf{s}_t)} \left[\mathbb{E}_{\mathbf{a}_t \sim \pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)}{\pi_{\theta}(\mathbf{a}_t|\mathbf{s}_t)} A^{\pi_{\theta}}(\mathbf{s}_t, \mathbf{a}_t) \right] \right] \quad (7.34)$$

$$\text{subject to } D_{\text{TV}}(\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t), \pi_{\theta}(\mathbf{a}_t|\mathbf{s}_t)) \leq \epsilon \quad (7.35)$$

For small enough ϵ , this is **guaranteed** to improve $J(\theta') - J(\theta)$.

7.5.4 From Total Variation to KL Divergence

Total variation divergence is inconvenient to optimize with neural networks because it involves absolute differences of probabilities rather than log-probabilities. We prefer KL divergence because:

1. It can be expressed in terms of log-probabilities: $D_{\text{KL}}(p_1||p_2) = \mathbb{E}_{x \sim p_1} [\log p_1(x) - \log p_2(x)]$
2. It is differentiable as long as $\text{supp}(p) \subseteq \text{supp}(q)$
3. It is tractable to compute for common distributions (Gaussians, categoricals)

The relationship between TV and KL divergence is given by **Pinsker's inequality**:

$$D_{\text{TV}}(p, q) \leq \sqrt{\frac{1}{2} D_{\text{KL}}(p||q)} \quad (7.36)$$

Therefore, constraining KL divergence implies a constraint on TV divergence:

$$D_{\text{KL}}(\pi_{\theta}(\mathbf{a}_t|\mathbf{s}_t)||\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)) \leq \epsilon \implies D_{\text{TV}} \leq \sqrt{\epsilon/2} \quad (7.37)$$

This gives us the more practical **KL-constrained** objective:

$$\theta' \leftarrow \arg \max_{\theta'} \sum_t \mathbb{E}_{\mathbf{s}_t \sim p_{\theta}(\mathbf{s}_t)} \left[\mathbb{E}_{\mathbf{a}_t \sim \pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)}{\pi_{\theta}(\mathbf{a}_t|\mathbf{s}_t)} A^{\pi_{\theta}}(\mathbf{s}_t, \mathbf{a}_t) \right] \right] \quad (7.38)$$

$$\text{subject to } D_{\text{KL}}(\pi_{\theta}(\mathbf{a}_t|\mathbf{s}_t)||\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)) \leq \epsilon \quad (7.39)$$

[QUIZ] Q3 (Lecture 10): KL divergence in PPO serves two purposes: it (1) **approximately enforces** the total variation constraint (via Pinsker's inequality) that guarantees policy improvement, and (2) **prevents the policy from moving too far** from the data-collecting policy, keeping the surrogate objective a valid approximation.

7.5.5 KL Divergence: Forward vs. Reverse

KL divergence is **asymmetric**: $D_{\text{KL}}(p||q) \neq D_{\text{KL}}(q||p)$ in general.

Forward KL: $D_{\text{KL}}(p || q) = \mathbb{E}_{\{x \sim p\}}[\log p(x) - \log q(x)]$ - Also called the “inclusive” or “mean-seeking” KL - Penalizes q for placing low probability where p has mass - Because $\log q$ goes to $-\infty$ when q assigns low probability to places where p does, the forward KL encourages q to *cover* all of p ’s mass - Result: **mode-covering** behavior – q will be broad to cover all modes of p

Reverse KL: $D_{\text{KL}}(q || p) = \mathbb{E}_{\{x \sim q\}}[\log q(x) - \log p(x)]$ - Also called the “exclusive” or “mode-seeking” KL - Penalizes q for placing mass where p has low probability - Because $\log p$ goes to $-\infty$ where p is small, q is encouraged not to place probability mass where p says it’s unlikely - Result: **mode-seeking** behavior – q will concentrate on a single mode of p

(See Figure 7.2 for visualization of forward vs. reverse KL.)

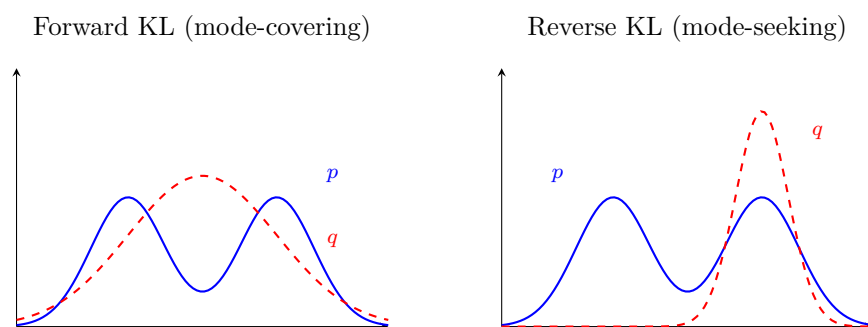


Figure 7.2: Forward KL produces mode-covering q ; Reverse KL produces mode-seeking q .

In the context of policy optimization: - $D_{\text{KL}}(\pi_{\theta}||\pi_{\theta'})$ is used in the TRPO/PPO constraint. The expectation is over the old policy π_{θ} (which we have samples from), making it easy to estimate. - Minimizing $D_{\text{KL}}(\pi_{\theta}||\pi_{\theta'})$ w.r.t. θ' simply means **maximizing the log-likelihood** of samples from π_{θ} under $\pi_{\theta'}$ – this is exactly supervised learning!

Common Misconception

Students often confuse which direction of KL is used. In the trust region constraint, we use $D_{\text{KL}}(\pi_{\theta}||\pi_{\theta'})$ where the expectation is under π_{θ} (the old policy we have samples from). The first argument is the distribution we sample from. This makes estimation straightforward.

7.6 Natural Policy Gradient

7.6.1 Motivation

The standard policy gradient $\theta' = \theta + \alpha \cdot \nabla_{\theta} J(\theta)$ uses the Euclidean geometry of the parameter space. But a small change in parameters could cause a large change in the policy distribution (or vice versa), depending on the parameterization.

The **natural policy gradient** asks: what is the steepest ascent direction in the space of *distributions* rather than *parameters*? This is achieved by approximately enforcing a KL divergence constraint.

7.6.2 KL Divergence as a Quadratic Form (Fisher Information Matrix)

For two distributions that are close in parameter space (θ and $\theta + \delta$), the KL divergence can be approximated via a 2nd-order Taylor expansion:

$$D_{\text{KL}}(p_\theta \| p_{\theta+\delta}) \approx \frac{1}{2} \delta^T F(\theta) \delta \quad (7.40)$$

where $F(\theta)$ is the **Fisher Information Matrix**:

$$F(\theta) = \mathbb{E}_{x \sim p_\theta} \left[\nabla_\theta \log p_\theta(x) \nabla_\theta \log p_\theta(x)^T \right] \quad (7.41)$$

Derivation sketch (from the discussion section, Eqs. 2-20):

Start with $D_{\text{KL}}(p_\theta \| p_{\theta+\delta}) = \mathbb{E}_{x \sim p_\theta} [\log p_\theta(x) - \log p_{\theta+\delta}(x)]$.

Taylor expand $\log p_{\theta+\delta}(x)$ around θ :

$$\log p_{\theta+\delta}(x) \approx \log p_\theta(x) + \delta^T \nabla_\theta \log p_\theta(x) + \frac{1}{2} \delta^T \nabla_\theta^2 \log p_\theta(x) \delta \quad (7.42)$$

Substituting:

$$D_{\text{KL}} \approx \mathbb{E}_{x \sim p_\theta} \left[-\delta^T \nabla_\theta \log p_\theta(x) - \frac{1}{2} \delta^T \nabla_\theta^2 \log p_\theta(x) \delta \right] \quad (7.43)$$

The first-order term vanishes because $\mathbb{E}_{x \sim p_\theta} [\nabla_\theta \log p_\theta(x)] = \nabla_\theta \int p_\theta(x) dx = \nabla_\theta(1) = 0$.

The second-order term can be rewritten using the identity:

$$-\mathbb{E}_{x \sim p_\theta} \left[\nabla_\theta^2 \log p_\theta(x) \right] = \mathbb{E}_{x \sim p_\theta} \left[\nabla_\theta \log p_\theta(x) \nabla_\theta \log p_\theta(x)^T \right] = F(\theta) \quad (7.44)$$

Therefore:

$$D_{\text{KL}}(p_\theta \| p_{\theta+\delta}) \approx \frac{1}{2} \delta^T F(\theta) \delta \quad (7.45)$$

7.6.3 The Natural Gradient Update

The natural policy gradient solves:

$$\max_{\delta} \delta^T \nabla_\theta J(\theta) \quad \text{subject to} \quad \frac{1}{2} \delta^T F(\theta) \delta \leq \epsilon \quad (7.46)$$

Using Lagrange multipliers, the solution is:

$$\delta^* \propto F(\theta)^{-1} \nabla_\theta J(\theta) \quad (7.47)$$

So the natural gradient update is:

$$\theta \leftarrow \theta + \alpha F(\theta)^{-1} \nabla_\theta J(\theta) \quad (7.48)$$

[QUIZ] Q4 (Lecture 10): The natural gradient **approximately enforces a KL constraint** on the policy update. By pre-multiplying the gradient by the inverse Fisher information matrix, it rescales the gradient so that a fixed step size corresponds to a fixed amount of change in the *distribution* (as measured by KL divergence) rather than in the *parameters*.

Intuition

Think of walking on a curved surface. The standard gradient tells you the direction of steepest ascent in the *coordinate system*, but the natural gradient tells you the direction of steepest ascent on the *surface itself*. The Fisher information matrix encodes the local curvature of the distribution manifold.

7.6.4 The Fisher Information Matrix

Properties of $F(\theta)$: - It is always **positive semi-definite** (it is an outer product expectation) - It measures the **sensitivity** of the distribution to parameter changes - It serves as a Riemannian metric on the space of distributions - In practice, it is often approximated (e.g., via conjugate gradient methods in TRPO)

For policy optimization specifically:

$$F(\theta) = \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \pi_\theta} \left[\nabla_\theta \log \pi_\theta(\mathbf{a}_t | \mathbf{s}_t) \nabla_\theta \log \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)^T \right] \quad (7.49)$$

7.7 PPO (Proximal Policy Optimization)

PPO is the most widely used policy gradient algorithm in practice (used for training LLMs, robotics, games). There are two main variants.

7.7.1 PPO-Clip (Clipped Surrogate Objective)

The clipped surrogate objective operates at the per-timestep level:

$$\mathcal{L}_{\text{CLIP}}(\theta') = \sum_{i=1}^N \sum_{t=1}^H \min \left\{ \frac{\pi_{\theta'}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})}{\pi_\theta(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})} \hat{A}_t^{(i)}, \text{clip} \left(\frac{\pi_{\theta'}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})}{\pi_\theta(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t^{(i)} \right\} \quad (7.50)$$

where ϵ is typically 0.1 or 0.2.

Algorithm: PPO-Clip

Input: Initial policy parameters θ , clipping parameter ϵ , number of inner iterations K

Repeat:

1. Sample trajectories $\{\tau^{(i)}\}$ from π_θ (run the policy)
2. Evaluate $y_t^{(i)} = r(s_t^{(i)}, a_t^{(i)}) + V_\phi^\pi(s_{t+1}^{(i)})$
3. Fit $V_\phi^\pi(s)$ to targets $\{y_t^{(i)}\}$ [can also use GAE targets]
4. Evaluate $\hat{A}_{\text{GAE}}^\pi(s_t^{(i)}, a_t^{(i)}) = \sum_{t'=t}^\infty (\gamma\lambda)^{t'-t} \delta_{t'}^{(i)}$
5. $\theta' \leftarrow \theta$
6. **For** $k = 1$ to K : [multiple gradient steps!] - Compute $\mathcal{L}_{\text{CLIP}}(\theta')$ using Eq. 7.50 -
 $\theta' \leftarrow \theta' + \alpha \nabla_{\theta'} \mathcal{L}_{\text{CLIP}}(\theta')$
7. $\theta \leftarrow \theta'$

Until convergence

Practical details: - Use GAE for computing value function targets $y_t^{(i)}$ as well (not just advantages)
 - Add an **entropy bonus** $\sum_i \sum_t \mathcal{H}(\pi_{\theta'}(\cdot|s_t^{(i)}))$ to the objective to encourage exploration - K is typically 50-100 gradient steps per batch of samples

7.7.2 PPO-KL (KL Penalty Variant)

Instead of clipping, use a KL divergence penalty with an adaptive coefficient:

$$\mathcal{L}(\theta', \beta) = \sum_t \mathbb{E}_{\mathbf{s}_t \sim p_\theta(\mathbf{s}_t)} \left[\mathbb{E}_{\mathbf{a}_t \sim \pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)} A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right] - \beta (D_{\text{KL}}(\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t) \parallel \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)) - \epsilon) \quad (7.51)$$

The coefficient β is updated via dual gradient descent:

Algorithm: PPO-KL (Penalty Variant)

Input: Initial policy parameters θ , KL target ϵ , initial penalty coefficient β , number of inner iterations K

Repeat:

1. Sample trajectories $\{\tau^{(i)}\}$ from π_θ 2–4. Compute value targets and GAE advantages (same as PPO-Clip)
5. $\theta' \leftarrow \theta$
6. **For** $k = 1$ to K : - Maximize $\mathcal{L}(\theta', \beta)$ w.r.t. θ' (gradient step on surrogate + KL penalty) - $\beta \leftarrow \beta + \alpha_\beta (D_{\text{KL}}(\pi_{\theta'} \parallel \pi_\theta) - \epsilon)$ - If constraint violated too much: β increases - If constraint satisfied easily: β decreases
7. $\theta \leftarrow \theta'$

Until convergence

Professor’s Note

“The algorithm from Wednesday [PPO-Clip] is an algorithm that you can and should use if you want a good policy gradient method. And it’s not necessarily the case that the stuff we’ll cover today [PPO-KL, TRPO] is a better method. It’s just a better way of understanding these methods and what it is that they do and why they are, in fact, pretty reasonable things to use.”

7.8 TRPO (Trust Region Policy Optimization)

TRPO solves the KL-constrained optimization problem more directly than PPO, using second-order methods.

7.8.1 The TRPO Objective

$$\theta' \leftarrow \arg \max_{\theta'} \mathbb{E}_{\mathbf{s}_t, \mathbf{a}_t \sim \pi_{\theta}} \left[\frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)} A^{\pi_{\theta}}(\mathbf{s}_t, \mathbf{a}_t) \right] \quad (7.52)$$

$$\text{subject to } \mathbb{E}_{\mathbf{s}_t \sim \pi_{\theta}} [D_{\text{KL}}(\pi_{\theta}(\cdot | \mathbf{s}_t) \| \pi_{\theta'}(\cdot | \mathbf{s}_t))] \leq \epsilon \quad (7.53)$$

7.8.2 TRPO Algorithm

TRPO uses the following approach: 1. Linearize the objective (first-order Taylor approximation) 2. Approximate the constraint as a quadratic (second-order Taylor, giving the Fisher information matrix) 3. Solve the resulting constrained quadratic program analytically

The linearized objective + quadratic constraint:

$$\max_{\delta} g^T \delta \quad \text{subject to} \quad \frac{1}{2} \delta^T F \delta \leq \epsilon \quad (7.54)$$

where $g = \nabla_{\theta} L(\theta)$ is the policy gradient and F is the Fisher information matrix.

The analytic solution is:

$$\delta^* = \sqrt{\frac{2\epsilon}{g^T F^{-1} g}} F^{-1} g \quad (7.55)$$

Algorithm: TRPO

Input: Initial policy parameters θ , KL constraint ϵ

Repeat:

1. Sample trajectories $\{\tau^{(i)}\}$ from π_θ 2–4. Compute value targets and GAE advantages
5. Compute policy gradient $g = \nabla_\theta \mathcal{L}(\theta)$
6. Compute Fisher-vector products Fv using conjugate gradient (avoids explicitly forming the full Fisher matrix)
7. Compute step direction: $\delta = F^{-1}g$ (via conjugate gradient)
8. Compute step size: $\alpha = \sqrt{2\epsilon/(g^T \delta)}$
9. $\theta \leftarrow \theta + \alpha \delta$
10. (Optional) Line search with backtracking to ensure KL constraint is satisfied and objective improves

Until convergence

7.8.3 TRPO vs. PPO

Property	TRPO	PPO-Clip	PPO-KL
Constraint enforcement	Hard (via conjugate gradient + line search)	Soft (via clipping)	Soft (via adaptive penalty)
Computational cost per step	High (conjugate gradient, line search)	Low (standard SGD)	Low (standard SGD)
Theoretical guarantees	Monotonic improvement (approx.)	Heuristic	Dual gradient descent
Implementation complexity	High	Low	Medium
Practical performance	Good	Often better	Comparable

Common Misconception

Students sometimes think TRPO is strictly better than PPO because it has stronger theoretical guarantees. In practice, PPO-Clip often performs as well or better because: (1) the theoretical bounds are conservative anyway, (2) PPO-Clip is much simpler to implement and tune, and (3) PPO-Clip naturally handles the constraint through the clipping mechanism without requiring second-order optimization.

7.9 Summary: The Landscape of Advanced Policy Gradient Methods

All of these methods share the same core idea: optimize a surrogate objective that uses old samples, but **constrain how far the new policy can deviate** from the old one.

(See Figure 7.3 for the relationship flowchart of advanced policy gradient methods.)

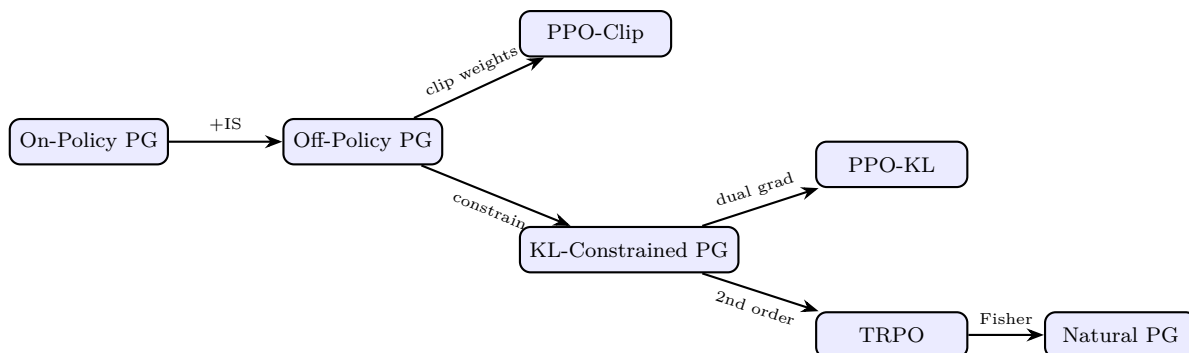


Figure 7.3: Landscape of advanced policy gradient methods.

7.10 Check Your Understanding

Q1. In the importance-sampled off-policy policy gradient, we decompose the trajectory importance weight into a product of per-timestep action ratios. Why do the initial state probabilities and transition dynamics cancel? What would happen if the two policies used different environment dynamics?

Answer: The trajectory probability factorizes as $p_\theta(\tau) = p(s_1) \prod_t \pi_\theta(a_t|s_t) p(s_{t+1}|s_t, a_t)$. In the ratio $p_\theta(\tau)/\bar{p}(\tau)$, the initial state distribution $p(s_1)$ and transition dynamics $p(s_{t+1}|s_t, a_t)$ appear identically in numerator and denominator because they are properties of the environment, not the policy. They cancel, leaving only the product of action probability ratios. If the two policies used different environment dynamics (e.g., different simulators), the dynamics terms would NOT cancel, and we would need to know both sets of dynamics to compute the importance weights – making the approach intractable.

Q2. Consider the following claim: “PPO-Clip is theoretically justified because clipping the importance weights is equivalent to enforcing a KL divergence constraint.” Is this claim accurate? Explain the precise relationship between clipping and trust region constraints.

Answer: The claim is not quite accurate. Clipping is NOT equivalent to a KL divergence constraint; it is a heuristic approximation. The theoretical justification comes from the trust region bound: if $D_{TV}(\pi_\theta, \pi_{\theta'}) \leq \epsilon$, then the surrogate objective is a lower bound on true improvement. PPO-Clip enforces this indirectly by capping the importance ratio to $[1 - \epsilon, 1 + \epsilon]$, which limits how much any single action probability can change. This roughly constrains the policy change but does not exactly enforce a KL or TV constraint. TRPO enforces the KL constraint directly via second-order optimization. PPO-Clip works well in practice despite weaker theoretical guarantees.

Q3. Suppose you are optimizing the surrogate objective and the importance weight $\pi_{\theta'}(a|s)/\pi_{\theta}(a|s)$ for a particular (s, a) pair is 50. The advantage $\hat{A}$ for this pair is positive. What does this tell you about the policies, and why is this problematic? How does PPO-Clip handle this situation?

Answer: A ratio of 50 means $\pi_{\theta'}$ assigns 50x more probability to action a in state s than π_{θ} did. The new policy has drifted far from the old one. This is problematic because: (1) the surrogate objective approximation (which ignores the state marginal ratio) is only valid when the policies are close, so with a ratio of 50 the approximation is unreliable; (2) the variance of the importance-sampled estimate becomes extremely high, as this single sample dominates the entire gradient. PPO-Clip would clamp this ratio to $1 + \epsilon$ (e.g., 1.2), so the effective weight is 1.2 instead of 50. Since $\hat{A} > 0$, the clipped objective equals $(1 + \epsilon)\hat{A}$, removing any incentive to increase the ratio further.

Q4. The Fisher information matrix $F(\theta)$ appears in both the natural policy gradient and the KL divergence approximation. Derive the relationship: show that $D_{\text{KL}}(p_{\theta}||p_{\theta+\delta})$ is approximately $(1/2)\delta^T F(\theta)\delta$ for small δ . What assumption is critical for this approximation to hold?

Answer: Start with $D_{\text{KL}}(p_{\theta}||p_{\theta+\delta}) = \mathbb{E}_{x \sim p_{\theta}}[\log p_{\theta}(x) - \log p_{\theta+\delta}(x)]$. Taylor-expand $\log p_{\theta+\delta}(x)$ around θ : $\log p_{\theta+\delta}(x) \approx \log p_{\theta}(x) + \delta^T \nabla_{\theta} \log p_{\theta}(x) + \frac{1}{2} \delta^T \nabla_{\theta}^2 \log p_{\theta}(x) \delta$. Substituting, the zeroth-order terms cancel. The first-order term vanishes because $\mathbb{E}_{x \sim p_{\theta}}[\nabla_{\theta} \log p_{\theta}(x)] = 0$ (score function identity). The remaining second-order term gives $D_{\text{KL}} \approx -\frac{1}{2} \delta^T \mathbb{E}[\nabla_{\theta}^2 \log p_{\theta}(x)] \delta = \frac{1}{2} \delta^T F(\theta) \delta$, using the identity $-\mathbb{E}[\nabla^2 \log p] = \mathbb{E}[\nabla \log p \cdot (\nabla \log p)^T] = F$. The critical assumption is that δ is small enough for the second-order Taylor expansion to be accurate.

Q5. In the policy improvement identity $J(\theta') - J(\theta) = \mathbb{E}_{\tau \sim p_{\theta'}}[\sum_t \gamma^t A^{\pi_{\theta}}(s_t, a_t)]$, the advantages are computed under π_{θ} (the OLD policy) but the expectation is under $p_{\theta'}$ (the NEW policy's trajectory distribution). Why is it correct to use the old policy's advantages here? Connect this to the concept of policy iteration.

Answer: This is an exact identity, not an approximation. The advantage $A^{\pi_{\theta}}(s, a) = Q^{\pi_{\theta}}(s, a) - V^{\pi_{\theta}}(s)$ measures how much better action a is compared to the average action under the old policy π_{θ} . The identity says: the improvement of $\pi_{\theta'}$ over π_{θ} equals the expected advantage of $\pi_{\theta'}$'s actions as judged by π_{θ} 's value function. This directly parallels policy iteration: in policy iteration, we evaluate the current policy (compute $A^{\pi_{\theta}}$) and then find a new policy that improves on it ($\pi_{\theta'}$ that makes $A^{\pi_{\theta}}$ positive in expectation). We do NOT need the new policy's own advantage because the identity tells us the old policy's advantages are sufficient to measure improvement.

Chapter 8

Variational Inference

Lecture source: Lecture 11 (Feb 25) and Lecture 12 (Feb 27), Discussion Section 6, Staff Notes on Variational Inference **Key idea:** Variational inference is an optimization-based framework for training models with latent variables – any ML problem where not all the variables you care about are given in your data.

8.1 Latent Variable Models

8.1.1 What Are They?

A **latent variable model** is a probabilistic model that includes unobserved (hidden) variables in addition to the observed data. These hidden variables are called **latent variables**.

Simple probabilistic models have only observed variables: - **Density estimation:** Model $p(x)$ directly (e.g., fit a single Gaussian to 2D data points) - **Conditional distributions:** Model $p(y|x)$ (e.g., linear regression, or a policy $\pi(a|s)$)

Latent variable models introduce an unobserved variable z :

$$p(x) = \sum_z p(x|z) p(z) \quad [\text{discrete } z] \quad (8.1)$$

$$p(x) = \int p(x|z) p(z) dz \quad [\text{continuous } z] \quad (8.2)$$

The key property: even though $p(x|z)$ and $p(z)$ are individually simple distributions, their composition $p(x)$ can be arbitrarily complex.

8.1.2 Concrete Example: Mixture of Gaussians

Suppose you have 2D data that forms three distinct clusters. A single Gaussian would be a poor fit. Instead, use a **mixture of Gaussians**:

$$p(x) = \sum_{z=1}^3 p(x|z) p(z) \quad (8.3)$$

Here: - z is a **categorical** latent variable with 3 values (one per cluster) - $p(z)$ gives the mixing weights (how likely each cluster is) - $p(x|z)$ is a Gaussian for each cluster (with its own mean and covariance) - z is NOT in the training data – we only observe x values, not which cluster they belong to

Intuition

Think of z as a “label” telling you which cluster generated each data point. The trouble is that in your training set, data points are NOT labeled with z – they only contain x . So z is latent (hidden), and you have to figure it out yourself.

8.1.3 Continuous Latent Variables

For more complex distributions (e.g., natural images), we use continuous latent variables:

$$p(x) = \int p(x|z) p(z) dz \quad (8.4)$$

where: - $p(z)$ is an “easy” distribution (e.g., a standard Gaussian $\mathcal{N}(0, I)$) - $p(x|z)$ is a conditional Gaussian whose mean and variance are given by a neural network: $p(x|z) = \mathcal{N}(\mu_{nn}(z), \sigma_{nn}(z))$ - The neural network acts as a deterministic, nonlinear transformation of z

Intuition

Intuition (Professor’s Note): Think of a computer program that takes a random seed. The random seed is simple, the computer program is deterministic, but feeding a random seed into a deterministic computer program can represent basically any probability distribution. Here, $p(z)$ is the random seed, $p(x|z)$ is the computer program.

8.1.4 Latent Variable Models in RL

Two important applications: 1. **Multimodal policies:** $p(a|s) = \int p(a|s, z) p(z) dz$ – the latent z captures intent (e.g., go left or right around a tree). This is what you did with flow matching in HW1. 2. **Model-based RL with POMDPs:** The latent state s is unobserved; you only see observations o . The latent space has temporal structure with dynamics $p(z_{t+1}|z_t, a_t)$.

8.2 The Intractability Problem

8.2.1 Why Can’t We Just Maximize $\log p(x)$?

Given data $D = \{x_1, x_2, \dots, x_N\}$, the standard approach is maximum likelihood estimation:

$$\theta \leftarrow \arg \max_{\theta} \frac{1}{N} \sum_i \log p_{\theta}(x_i) \quad (8.5)$$

But for a latent variable model, $p_{\theta}(x_i) = \int p_{\theta}(x_i|z) p(z) dz$. Substituting:

$$\theta \leftarrow \arg \max_{\theta} \frac{1}{N} \sum_i \log \left(\int p_{\theta}(x_i|z) p(z) dz \right) \quad (8.6)$$

This is **completely intractable** because: 1. The integral over z may be over a high-dimensional space (possibly thousands of dimensions) 2. The log is OUTSIDE the integral, so we cannot use linearity of expectation 3. We cannot estimate the integral with samples naively (sampling inside a log gives a biased estimator)

Common Misconception

“Can’t we just sample z values and average?” No! The log of an average is NOT the average of logs. Because log is concave, $\log \mathbb{E}[f(z)]$ is not equal to $\mathbb{E}[\log f(z)]$. Sampling inside the log gives a biased estimate of $\log p(x)$.

8.2.2 The Expected Log-Likelihood Alternative

Instead of integrating over all z , we can “guess” which z goes with each x_i :

$$\theta \leftarrow \arg \max_{\theta} \frac{1}{N} \sum_i \mathbb{E}_{z \sim p(z|x_i)} [\log p_{\theta}(x_i, z)] \quad (8.7)$$

The intuition: if someone told you which z generated each x_i , everything would be easy. Since you don’t know, use the distribution $p(z|x_i)$ to guess.

But: How do we calculate $p(z|x_i)$? This is the posterior distribution, and computing it requires solving an inference problem – which is exactly the intractable computation we’re trying to avoid. This is called **probabilistic inference**, and it is the fundamental problem that variational inference solves.

8.3 The Evidence Lower Bound (ELBO)

8.3.1 Derivation 1: From Jensen’s Inequality

Since we can’t compute $p(z|x_i)$ exactly, we’ll approximate it with a simpler distribution $q_i(z)$. This is “incorrect but very convenient.”

QUIZ ANSWER (Lecture 11, Q1): q_i must be a distribution over $\mathbf{z}$ (the latent variable). NOT over $\mathbf{x}$, NOT over $(\mathbf{x}, \mathbf{z})$, NOT over $\mathbf{x}|z$, NOT over $z|\mathbf{x}$. It is an approximate posterior over z for each data point x_i .

Step-by-step derivation:

$$\log p(x_i) = \log \int p(x_i|z) p(z) dz \quad (8.8)$$

Multiply and divide by $q_i(z)$ inside the integral (equivalent to multiplying by 1):

$$= \log \int p(x_i|z) p(z) \frac{q_i(z)}{q_i(z)} dz \quad (8.9)$$

Rewrite as an expectation under $q_i(z)$:

$$= \log \mathbb{E}_{z \sim q_i(z)} \left[\frac{p(x_i|z) p(z)}{q_i(z)} \right] \quad (8.10)$$

Apply **Jensen's inequality** (for concave functions like $\log$: $\log \mathbb{E}[Y] \geq \mathbb{E}[\log Y]$):

$$\geq \mathbb{E}_{z \sim q_i(z)} \left[\log \left(\frac{p(x_i|z) p(z)}{q_i(z)} \right) \right] \quad (8.11)$$

Expand using log rules:

$$= \mathbb{E}_{z \sim q_i(z)} [\log p(x_i|z) + \log p(z)] - \mathbb{E}_{z \sim q_i(z)} [\log q_i(z)] \quad (8.12)$$

Recognize the second term as entropy:

$$= \mathbb{E}_{z \sim q_i(z)} [\log p(x_i|z) + \log p(z)] + \mathcal{H}(q_i) \quad (8.13)$$

This is the **Evidence Lower Bound (ELBO)**, denoted $\mathcal{L}_i(p, q_i)$:

$$\mathcal{L}_i(p, q_i) = \mathbb{E}_{z \sim q_i(z)} [\log p(x_i|z) + \log p(z)] + \mathcal{H}(q_i) \quad (8.14)$$

$$\log p(x_i) \geq \mathcal{L}_i(p, q_i)$$

Jensen's Inequality Reminder: For any convex function ϕ : $\phi(\mathbb{E}[X]) \leq \mathbb{E}[\phi(X)]$.
Since $-\log$ is convex (equivalently, $\log$ is concave): $\log \mathbb{E}[Y] \geq \mathbb{E}[\log Y]$.

8.3.2 Derivation 2: Via KL Divergence

We can also derive the ELBO by asking: “What makes a good $q_i(z)$?” It should approximate the true posterior $p(z|x_i)$. We measure this with KL divergence:

$$D_{\text{KL}}(q_i(z) \| p(z|x_i)) = \mathbb{E}_{z \sim q_i(z)} \left[\log \frac{q_i(z)}{p(z|x_i)} \right] \quad (8.15)$$

Rewrite $p(z|x_i) = p(x_i, z)/p(x_i)$ using Bayes' rule:

$$= \mathbb{E}_{z \sim q_i(z)} \left[\log \left(\frac{q_i(z) p(x_i)}{p(x_i, z)} \right) \right] \quad (8.16)$$

Expand:

$$= -\mathbb{E}_{z \sim q_i(z)} [\log p(x_i|z) + \log p(z)] + \mathbb{E}_{z \sim q_i(z)} [\log q_i(z)] + \mathbb{E}_{z \sim q_i(z)} [\log p(x_i)] \quad (8.17)$$

Since $\log p(x_i)$ doesn't depend on z :

$$= -\mathbb{E}_{z \sim q_i(z)} [\log p(x_i|z) + \log p(z)] - \mathcal{H}(q_i) + \log p(x_i) \quad (8.18)$$

Recognize the ELBO $\mathcal{L}_i(p, q_i)$:

$$= -\mathcal{L}_i(p, q_i) + \log p(x_i) \quad (8.19)$$

Rearranging:

$$\log p(x_i) = D_{\text{KL}}(q_i(z) \| p(z|x_i)) + \mathcal{L}_i(p, q_i) \quad (8.20)$$

Since KL divergence is always ≥ 0 :

$$\log p(x_i) \geq \mathcal{L}_i(p, q_i) \quad (8.21)$$

This confirms $\mathcal{L}_i$ is a lower bound on the log-evidence.

8.3.3 When Is the ELBO Tight?

QUIZ ANSWER (Lecture 11, Q2): The ELBO is tight (equality holds) when $q_i(z) = p(z|x_i)$, the **true posterior**.

Why: From Eq. (8.20), the gap between $\log p(x_i)$ and the ELBO is exactly $D_{\text{KL}}(q_i(z) \| p(z|x_i))$. KL divergence equals zero if and only if the two distributions are identical. Therefore:

$$q_i(z) = p(z|x_i) \implies D_{\text{KL}} = 0 \implies \log p(x_i) = \mathcal{L}_i(p, q_i) \quad (8.22)$$

Intuition

The ELBO is a “pessimistic” estimate of the true log-likelihood. The better q_i approximates the true posterior, the tighter the bound, and the closer we are to actually optimizing the true objective.

8.3.4 Why Maximize ELBO w.r.t. BOTH p and q ?

QUIZ ANSWER (Lecture 11, Q3): We maximize the ELBO with respect to BOTH the model parameters (p , parameterized by θ) AND the variational distribution (q_i).

From Eq. (8.20): $\log p(x_i) = D_{\text{KL}}(q_i \| p(z|x_i)) + \mathcal{L}_i(p, q_i)$

Maximizing $\mathcal{L}_i$ w.r.t. q_i : - Since $\log p(x_i)$ is independent of q_i (it's a fixed quantity for fixed θ), maximizing $\mathcal{L}_i$ w.r.t. q_i is equivalent to minimizing $D_{\text{KL}}(q_i \| p(z|x_i))$. - This makes q_i a better approximation of the true posterior. - This tightens the bound.

Maximizing $\mathcal{L}_i$ w.r.t. θ (the model parameters): - This pushes up the lower bound, which in turn pushes up $\log p(x_i)$. - The gradient w.r.t. θ only hits the $\log p_\theta(x_i|z)$ term.

Instead of the intractable $\max_\theta \frac{1}{N} \sum_i \log p_\theta(x_i)$, we solve:

$$\theta \leftarrow \arg \max_\theta \frac{1}{N} \sum_i \mathcal{L}_i(p, q_i) \quad (8.23)$$

8.3.5 ELBO Decomposition Diagram

(See Figure 8.2)

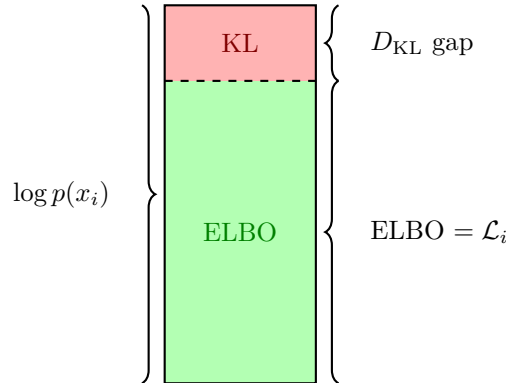


Figure 8.1: ELBO decomposition — $\log p(x_i) = \text{ELBO} + D_{\text{KL}}$.

$$\begin{aligned} \mathcal{L}_i &= \mathbb{E}_{z \sim q_i} [\log p(x_i|z) + \log p(z)] + \mathcal{H}(q_i) \\ &= \mathbb{E}_{z \sim q_i} [\log p(x_i|z)] - D_{\text{KL}}(q_i(z) \| p(z)) \end{aligned}$$

8.4 Entropy and KL Divergence Review

8.4.1 Entropy

$$\mathcal{H}(p) = -\mathbb{E}_{x \sim p(x)} [\log p(x)] = -\int p(x) \log p(x) dx \quad (8.24)$$

- **Intuition 1:** How “random” is the random variable? High entropy = spread out; low entropy = peaked.
- **Intuition 2:** How large is the log probability in expectation under itself?

8.4.2 KL Divergence

$$\begin{aligned}
 D_{\text{KL}}(q\|p) &= \mathbb{E}_{x \sim q(x)} \left[\log \frac{q(x)}{p(x)} \right] \\
 &= \mathbb{E}_{x \sim q(x)} [\log q(x)] - \mathbb{E}_{x \sim q(x)} [\log p(x)] \\
 &= -\mathbb{E}_{x \sim q(x)} [\log p(x)] - \mathcal{H}(q)
 \end{aligned} \tag{8.25}$$

- **Intuition 1:** How “different” are two distributions?
- **Intuition 2:** How small is the expected log probability of p under q , minus entropy?
- **Key property:** $D_{\text{KL}}(q\|p) \geq 0$, with equality iff $q = p$

Why subtract entropy? Consider q_i trying to cover a bimodal $p(z|x_i)$. Maximizing $\mathbb{E}_{z \sim q} [\log p(x_i|z) + \log p(z)]$ alone would collapse q onto the single best z . Adding entropy $\mathcal{H}(q_i)$ encourages q to be as wide as possible while still putting mass where $p(x_i, z)$ is large.

8.5 Variational Distributions $q(z)$

8.5.1 Basic Variational Inference Algorithm

For each data point x_i , maintain a separate $q_i(z) = \mathcal{N}(\mu_i, \sigma_i)$:

Algorithm: Basic Variational Inference

Initialize $\theta, \{\mu_i, \sigma_i\}$ for all i

For each x_i (or mini-batch): 1. Sample $z \sim q_i(z)$ 2. Compute $\nabla_{\theta} \mathcal{L} \approx \nabla_{\theta} \log p_{\theta}(x_i|z)$ 3. $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} \mathcal{L}$ 4. Compute $\nabla_{\mu_i} \mathcal{L}, \nabla_{\sigma_i} \mathcal{L}$ 5. $\mu_i \leftarrow \mu_i + \alpha \cdot \nabla_{\mu_i} \mathcal{L}$ 6. $\sigma_i \leftarrow \sigma_i + \alpha \cdot \nabla_{\sigma_i} \mathcal{L}$

8.5.2 The Problem: Parameter Explosion

Total number of parameters: $|\theta| + (|\mu_i| + |\sigma_i|) \times N$

If $N = 1,000,000$ data points, we have a million separate (μ, σ) pairs. This is: - Not incorrect (any q gives a valid lower bound) - But impractical for modern ML workflows expecting a fixed parameter count

Professor's Note

“Could we just share one μ and σ for all data points? Yes, it's a valid lower bound. Is it a good one? Probably not, because unless all your data points have the same posterior, you won't accurately approximate any of them. You'll probably get q equal to the prior, which averages everything out.”

8.6 Amortized Variational Inference

8.6.1 The Key Idea

Instead of separate parameters per data point, use a **neural network** that maps each x_i to its posterior parameters:

$$q_i(z) = q_\phi(z|x_i) = \mathcal{N}(\mu_\phi(x_i), \sigma_\phi(x_i)) \quad (8.26)$$

Now: - One network with parameters ϕ serves ALL data points - The number of parameters is $|\theta| + |\phi|$, independent of dataset size - Different x_i still get different posteriors because the network outputs different μ and σ for different inputs

Why “amortized”? The cost of learning the posterior is spread (amortized) over all data points working together to train one shared network, rather than each data point doing the work independently.

8.6.2 The Amortized ELBO

$$\mathcal{L}_i = \mathbb{E}_{z \sim q_\phi(z|x_i)} [\log p_\theta(x_i|z) + \log p(z)] + \mathcal{H}(q_\phi(z|x_i)) \quad (8.27)$$

Equivalently (grouping the $\log p(z)$ and entropy terms):

$$\mathcal{L}_i = \mathbb{E}_{z \sim q_\phi(z|x_i)} [\log p_\theta(x_i|z)] - D_{\text{KL}}(q_\phi(z|x_i) \| p(z)) \quad (8.28)$$

8.6.3 Computing the Gradient w.r.t. ϕ

The gradient w.r.t. ϕ is challenging because ϕ appears in two places: 1. In the entropy term $\mathcal{H}(q_\phi)$ – this is easy (look up formula for Gaussian entropy) 2. In the expectation $\mathbb{E}_{z \sim q_\phi}[\dots]$ – ϕ controls the distribution we sample from

For the expectation term, define $J(\phi) = \mathbb{E}_{z \sim q_\phi(z|x_i)} [r(x_i, z)]$ where $r(x_i, z) = \log p_\theta(x_i|z) + \log p(z)$.

Option 1: Policy gradient (valid but high variance):

$$\nabla_\phi J(\phi) \approx \frac{1}{M} \sum_j \nabla_\phi \log q_\phi(z_j|x_i) \cdot r(x_i, z_j) \quad (8.29)$$

- Works for both discrete and continuous z

- High variance, requires many samples and small learning rates

Option 2: Reparameterization trick (preferred for continuous z):

See Section 8.7 below.

8.7 The Reparameterization Trick

8.7.1 What It Is

For a Gaussian $q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi(x))$, any sample z can be written as:

$$z = \mu_\phi(x) + \epsilon \cdot \sigma_\phi(x), \quad \epsilon \sim \mathcal{N}(0, I) \quad (8.30)$$

This means we can rewrite the expectation:

$$\begin{aligned} J(\phi) &= \mathbb{E}_{z \sim q_\phi(z|x_i)}[r(x_i, z)] \\ &= \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)}[r(x_i, \mu_\phi(x_i) + \epsilon \cdot \sigma_\phi(x_i))] \end{aligned} \quad (8.31)$$

8.7.2 Why It's Needed

The key insight: ϵ **does not depend on** ϕ . So we can move the gradient inside the expectation:

$$\nabla_\phi J(\phi) \approx \frac{1}{M} \sum_j \nabla_\phi r(x_i, \mu_\phi(x_i) + \epsilon_j \cdot \sigma_\phi(x_i)) \quad (8.32)$$

This is just standard backpropagation! In PyTorch, you implement the computation $z = \mu + \epsilon \cdot \sigma$, pass z through the rest of the computation, and call `.backward()`.

Why this is better than policy gradient: The reparameterization trick produces a low-variance gradient estimator because it uses the analytical gradient of r rather than the noisy REINFORCE estimator. A single sample ($M = 1$) often works well.

Professor's Note

“In practice, you use one ϵ per data point. You could use more, but it's better to just get more x 's from your mini-batch instead – same computational cost, better coverage of the data distribution.”

8.7.3 Comparison: Policy Gradient vs. Reparameterization Trick

Property	Policy Gradient	Reparameterization Trick
Gradient formula	$(1/M) \sum \nabla_{\phi} \log q_{\phi}(z_j x_i) \cdot r(x_i, z_j)$	$(1/M) \sum \nabla_{\phi} r(x_i, \mu_{\phi}(x_i) + \epsilon_j \cdot \sigma_{\phi}(x_i))$
Variance	High	Low
Latent variable types	Continuous AND discrete	Continuous only
Implementation	Need $\log q_{\phi}$	Just backprop through computation graph

Rule of thumb: Use reparameterization trick for continuous latent variables. Use policy gradient only if you have discrete latent variables.

8.8 Variational Autoencoders (VAEs)

8.8.1 Architecture

A VAE consists of two neural networks:

(See Figure 8.1)

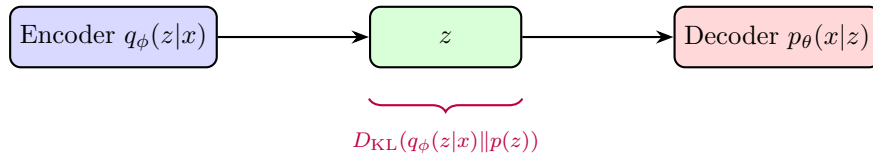


Figure 8.2: VAE Architecture — encoder, latent space, decoder with KL regularization.

- **Encoder** $q_{\phi}(z|x) = \mathcal{N}(\mu_{\phi}(x), \sigma_{\phi}(x))$: maps data x to latent code z
- **Decoder** $p_{\theta}(x|z) = \mathcal{N}(\mu_{\theta}(z), \sigma_{\theta}(z))$: maps latent code z back to data
- **Prior** $p(z) = \mathcal{N}(0, I)$: standard Gaussian (fixed, not learned)

8.8.2 Training Objective

Using the reparameterized ELBO (Eq. 8.28):

$$\max_{\theta, \phi} \frac{1}{N} \sum_i [\log p_{\theta}(x_i | \mu_{\phi}(x_i) + \epsilon \cdot \sigma_{\phi}(x_i)) - D_{KL}(q_{\phi}(z|x_i)||p(z))] \quad (8.33)$$

where $\epsilon \sim \mathcal{N}(0, I)$

Term 1 (Reconstruction): $\mathbb{E}[\log p_{\theta}(x_i|z)]$ – how well can the decoder reconstruct x from z ?

Term 2 (Regularization): $-D_{KL}(q_{\phi}(z|x_i)||p(z))$ – keep the posterior close to the prior. For two Gaussians, the KL divergence has a closed-form formula.

Intuition

Intuition for the KL term: “All else being equal, I prefer my posterior to look like my prior. But if $q(z|x) = p(z)$, then z carries no information about x , and reconstruction will fail. So the game is: find a posterior that contains as little information as possible while still allowing accurate reconstruction.” – Prof. Levine

8.8.3 Why Sampling Works at Test Time

At test time, we generate new data by: 1. Sample $z \sim p(z) = \mathcal{N}(0, I)$ 2. Sample $x \sim p_\theta(x|z)$

Paradox: During training, the decoder only ever sees z ’s from the encoder (from q_ϕ). At test time, we feed it z ’s from the prior $p(z)$. Why does this work?

Answer: The KL divergence term $D_{\text{KL}}(q_\phi(z|x_i)||p(z))$ forces the encoder to produce z ’s that look like they came from the prior. Each data point’s posterior $q_\phi(z|x_i)$ is a little Gaussian ball in z -space, and the KL term encourages these balls to collectively cover the prior $\mathcal{N}(0, I)$. So after training, any z sampled from $\mathcal{N}(0, I)$ will be near some data point’s encoding.

8.8.4 Concrete Walkthrough Example

Setting: Training a VAE on face images (e.g., CelebA dataset).

1. **Data:** x_i is a face image (say, 64x64 pixels = 4096 dimensions)
2. **Latent space:** z is a 64-dimensional vector
3. **Encoder:** ConvNet that takes a 64x64 image and outputs $\mu_\phi(x) \in \mathbb{R}^{64}$ and $\sigma_\phi(x) \in \mathbb{R}^{64}$
4. **Decoder:** Deconvolutional network that takes $z \in \mathbb{R}^{64}$ and outputs $\mu_\theta(z)$ (a 64x64 image)

One training step: - Take mini-batch of 256 face images - For each x_i : - Pass through encoder: get $\mu_\phi(x_i), \sigma_\phi(x_i)$ - Sample $\epsilon \sim \mathcal{N}(0, I)$ in $\mathbb{R}^{64}$ - Compute $z = \mu_\phi(x_i) + \epsilon \cdot \sigma_\phi(x_i)$ - Pass z through decoder: get reconstructed image $\mu_\theta(z)$ - Compute reconstruction loss: $\|x_i - \mu_\theta(z)\|^2$ (this is $\log p_\theta$ up to constants for Gaussian decoder) - Compute KL loss: $D_{\text{KL}}(\mathcal{N}(\mu_\phi, \sigma_\phi)||\mathcal{N}(0, I))$ using closed-form formula - Total loss = reconstruction loss + KL loss - Backpropagate through entire computation graph (gradients flow through $z = \mu + \epsilon \cdot \sigma$ because ϵ is treated as a constant) - Update θ and ϕ with gradient descent

8.8.5 Conditional VAEs

For modeling $p(y|x)$ (e.g., a policy $\pi(a|s)$):

$$\begin{aligned} \mathcal{L}_i = \mathbb{E}_{z \sim q_\phi(z|x_i, y_i)} [\log p_\theta(y_i|x_i, z) + \log p(z|x_i)] \\ + \mathcal{H}(q_\phi(z|x_i, y_i)) \end{aligned} \quad (8.34)$$

Architecture changes: - Encoder takes BOTH x and y as input: $q_\phi(z|x, y)$ - Decoder takes BOTH x and z as input: $p_\theta(y|x, z)$ - Prior can optionally depend on x : $p(z|x)$ or just $p(z)$

At test time: Sample $z \sim p(z)$ (or $p(z|x)$), then decode $y \sim p_\theta(y|x, z)$. The encoder is NOT needed at test time.

Application: Multimodal imitation learning – x = state/image, y = action, z captures which “mode” of behavior (e.g., go left vs. right around obstacle).

8.9 Connection to Diffusion Models

QUIZ ANSWER (Lecture 12, Q2): Diffusion models share with VAEs: they both have **latent variables** and optimize a **variational lower bound**. They do NOT share the property of backpropagating encoder $\rightarrow$ decoder.

Key insight: Diffusion can be thought of as a **hierarchical VAE** – a VAE with many layers of latent variables.

(See Figure 8.4)

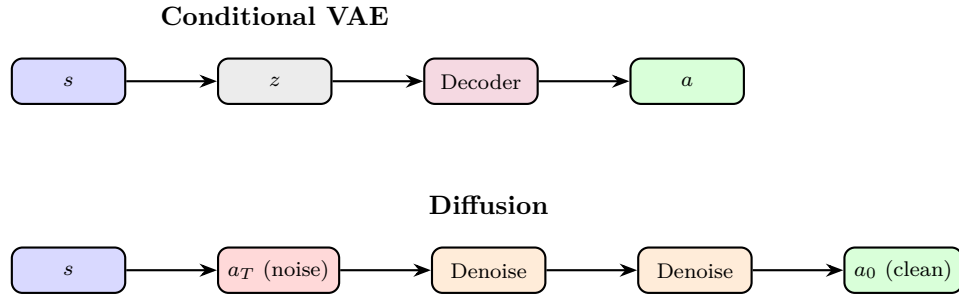


Figure 8.3: CVAE (single-shot latent) vs Diffusion (iterative denoising).

For a conditional VAE:

$$\log p(a|s) \geq \mathbb{E}_{z \sim q(z|s)} [\log p(a|s, z)] - D_{\text{KL}}(q(z|s) \| p(z))$$

For diffusion:

$$\log p(a_0|s) \geq \mathbb{E}_{a_{dt:1} \sim q(a_{dt:1}|s)} [\log p(a_0|s, a_{dt:1})] - D_{\text{KL}}(q(a_{dt:1}|a_0) \| p(a_{dt:1}))$$

The latent variables in diffusion are all the intermediate noisy versions of the action. The “encoder” (forward process) adds noise; the “decoder” (reverse process) removes noise. Both frameworks optimize a lower bound on the data likelihood.

8.10 KL Divergence Deep Dive

8.10.1 Forward KL: $D_{\text{KL}}(p\|q)$ – Mode-Covering

$$D_{\text{KL}}(p\|q) = \mathbb{E}_{x \sim p(x)} \left[\log \frac{p(x)}{q(x)} \right] \quad (8.35)$$

The expectation is under $\mathbf{p}$ (the true distribution). This means: - Wherever $p(x) > 0$, we need $q(x) > 0$ (otherwise $\log p/q \rightarrow \infty$) - q must “cover” all modes of p - Result: q tends to be **wider** than p , spreading mass to cover everything

(See Forward KL pgfplots figure above)

8.10.2 Reverse KL: $D_{\text{KL}}(q\|p)$ – Mode-Seeking

$$D_{\text{KL}}(q\|p) = \mathbb{E}_{x \sim q(x)} \left[\log \frac{q(x)}{p(x)} \right] \quad (8.36)$$

The expectation is under $\mathbf{q}$ (the approximate distribution). This means: - Wherever $q(x) > 0$, we need $p(x) > 0$ (otherwise the KL blows up) - If $p(x) = 0$, then $q(x)$ MUST be 0 - q can safely “ignore” modes of p – it just picks one - Result: q tends to “seek” a single mode of p

(See Reverse KL pgfplots figure above)

8.10.3 The Asymmetry

KL divergence is NOT symmetric: $D_{\text{KL}}(p\|q) \neq D_{\text{KL}}(q\|p)$ in general.

When does this matter in variational inference? - The ELBO derivation uses **reverse KL**: $D_{\text{KL}}(q\|p(z|x))$ - This means variational inference is inherently **mode-seeking** - If the true posterior $p(z|x)$ is multimodal, a Gaussian q will typically lock onto one mode

Professor’s Note

Professor’s Note (on the KL in VAEs): “This would be considered reverse KL because the parameters ϕ are in the first argument. So it’s mode seeking. But it kind of doesn’t matter because $p(z)$ is a unimodal Gaussian.”

8.10.4 When to Use Which

Situation	Use Forward KL	Use Reverse KL
Need to cover all modes	Yes	No
Can tolerate mass between modes	Yes	No
Simple q approximating complex p	Risky (may spread too thin)	Safer (picks best mode)
Variational inference	Not standard	Standard (ELBO uses this)

Situation	Use Forward KL	Use Reverse KL
Offline RL (policy constraint)	Mode-covering: safe but conservative	Mode-seeking: can exploit but risky

8.11 Check Your Understanding

1. **Why is $\log \int p(x|z)p(z)dz$ intractable, even though we can sample from $p(z)$?** *The log is outside the integral, preventing us from using linearity of expectation. Sampling z and averaging $\log p(x|z)p(z)$ gives $\mathbb{E}[\log f(z)]$, not $\log \mathbb{E}[f(z)]$.*
2. **If $q_i(z)$ is set to $p(z)$ (the prior), is the ELBO still a valid lower bound? Is it a tight bound?** *Yes, it's still a valid bound (any q gives a valid bound). No, it's typically not tight because $p(z) \neq p(z|x_i)$ in general.*
3. **In a VAE, why can't we just use a deterministic autoencoder (without the noise ϵ and KL term)?** *Without stochasticity and the KL regularizer, the latent space has no structure – nearby z values don't correspond to similar x values, so sampling from $p(z)$ at test time would produce garbage.*
4. **Why is the reparameterization trick necessary for training VAEs, and why can't we just differentiate $\mathbb{E}_{z \sim q}[f(z)]$ directly?** *Because the expectation is taken over a distribution that depends on the parameters ϕ . We can't differentiate through the sampling operation directly. The reparameterization trick moves the randomness to a fixed distribution $\epsilon \sim \mathcal{N}(0, I)$ so that ϕ only appears in deterministic operations.*
5. **A classmate claims that increasing the weight on the KL term in the VAE objective will always improve generation quality. Is this correct?** *No. Increasing the KL weight makes q closer to the prior (better sampling quality in theory), but it also reduces the information in z , harming reconstruction. With too much KL weight, the model may “ignore” z entirely, producing blurry/average outputs. This is known as posterior collapse.*

Chapter 9

Control as Inference

Lecture source: Lecture 12 (Feb 27), Discussion Section 6, Staff Notes on Control as Inference **Key idea:** We can frame the RL problem as probabilistic inference in a graphical model with “optimality variables.” This gives rise to maximum entropy RL and algorithms like Soft Actor-Critic (SAC).

9.1 Motivation: Modeling Approximately Optimal Behavior

9.1.1 The Problem with Perfect Rationality

Standard RL assumes a perfectly rational agent that maximizes expected reward. But real agents (humans, animals) are only **approximately rational**:

- A monkey trained to move a dot to a target takes a roughly straight path, but with random deviations
- You walk to class reliably, but take slightly different paths each day
- Good behavior is most likely, but suboptimal behavior has non-zero probability

Professor’s Note

“Some mistakes matter a lot more than others, and the monkey knows that it can be a little lazy and approximate as long as it gets to the target. So mostly the monkey is rational, but it’s only approximately rational.”

9.1.2 Why Model Suboptimality?

Three reasons to build a probabilistic model of approximately optimal behavior:

1. **Inverse RL:** If you model behavior as deterministic optimization and observe slightly suboptimal actions, you’ll infer an absurdly complex reward function. A probabilistic model lets you “disregard noise” and infer a simple reward.

2. **Control via inference:** Inference algorithms from probabilistic graphical models can solve control problems, giving us new algorithmic tools.
3. **Stochastic policies:** Explains why stochastic behavior can be beneficial (exploration, robustness).

9.2 The Graphical Model for Control as Inference

9.2.1 The MDP with Optimality Variables

We augment the standard MDP graphical model with binary **optimality variables** $\mathcal{O}_t$ at each time step:

(See Figure 9.1)

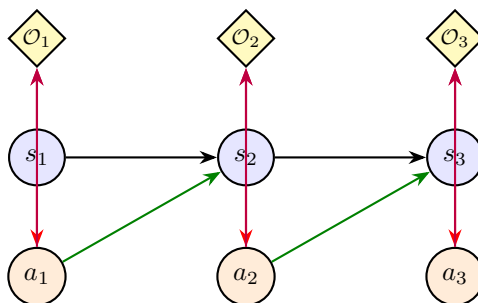


Figure 9.1: Control as Inference — MDP with optimality variables $\mathcal{O}_t$ (diamonds).

The key components: - **Dynamics:** $p(s_{t+1}|s_t, a_t)$ – same as standard MDP - **Action prior:** $p(a_t|s_t)$ – how likely actions are without any intent (typically uniform) - **Optimality variables:** $\mathcal{O}_t$ is a binary random variable

9.2.2 Optimality Variables $\mathcal{O}_t$

QUIZ ANSWER (Lecture 12, Q1): $p(\mathcal{O}_t|s_t, a_t)$ is a distribution over a **BINARY** variable related to **instantaneous** reward.

The optimality variable is defined as:

$$p(\mathcal{O}_t = 1 \mid s_t, a_t) = \exp(r(s_t, a_t)) \quad (9.1)$$

Assumption: $r(s_t, a_t) \leq 0$ so that $0 < \exp(r) \leq 1$

Why this choice? - We need rewards to be non-positive so $\exp(r)$ is between 0 and 1 (valid probability) - This is without loss of generality: adding a constant to all rewards doesn't change the optimal policy - The exponential form leads to elegant algebra

Interpretation: - $\mathcal{O}_t = 1$ (“good monkey”): the agent is being optimal at time step t - $\mathcal{O}_t = 0$ (“bad monkey”): the agent is acting randomly - Higher reward states/actions have higher probability of “being optimal”

9.2.3 Trajectory Distribution Under Optimality

What is $p(\tau|\mathcal{O}_{1:T} = 1)$? That is, what is the probability of a trajectory given the agent is optimal at ALL time steps?

By conditional probability:

$$p(\tau|\mathcal{O}_{1:T}) = \frac{p(\tau, \mathcal{O}_{1:T})}{p(\mathcal{O}_{1:T})} \quad (9.2)$$

$$p(\tau, \mathcal{O}_{1:T}) \propto p(\tau) \cdot \prod_t \exp(r(s_t, a_t)) = p(\tau) \cdot \exp\left(\sum_t r(s_t, a_t)\right) \quad (9.3)$$

Intuition

The probability of a trajectory given optimality is: - Proportional to $p(\tau)$ (physical possibility) times $\exp(\text{total reward})$ - The highest-reward trajectory is the most probable - As reward decreases, probability drops exponentially - This explains the monkey: indirect paths have slightly less reward, so they’re slightly less likely. Failing to reach the target has much less reward, so it’s very unlikely.

9.3 Running Inference in the Graphical Model

9.3.1 Backward Messages

Define the **backward message** $\beta_t(s_t, a_t)$ as:

$$\beta_t(s_t, a_t) = p(\mathcal{O}_{t:T}|s_t, a_t) \quad (9.4)$$

This is the probability of being optimal from time t to the end, given the current state and action.

Base case (at the last time step T):

$$\beta_T(s_T, a_T) = p(\mathcal{O}_T|s_T, a_T) = \exp(r(s_T, a_T)) \quad (9.5)$$

Recursive derivation:

Step 1: Introduce s_{t+1} by marginalization:

$$\beta_t(s_t, a_t) = \int p(\mathcal{O}_{t:T}, s_{t+1}|s_t, a_t) ds_{t+1} \quad (9.6)$$

Step 2: Factor using the Markov property (s_{t+1} d-separates future from past):

$$= \int p(\mathcal{O}_{t+1:T}|s_{t+1}) \cdot p(s_{t+1}|s_t, a_t) \cdot p(\mathcal{O}_t|s_t, a_t) ds_{t+1} \quad (9.7)$$

Step 3: The first term $p(\mathcal{O}_{t+1:T}|s_{t+1})$ requires marginalizing over a_{t+1} :

$$p(\mathcal{O}_{t+1:T}|s_{t+1}) = \int \beta_{t+1}(s_{t+1}, a_{t+1}) \cdot p(a_{t+1}|s_{t+1}) da_{t+1} \quad (9.8)$$

Step 4: Under a **uniform action prior** $p(a|s) = \text{const}$, define:

$$\beta_t(s_t) = \mathbb{E}_{a_t \sim p(a|s_t)}[\beta_t(s_t, a_t)] \quad (9.9)$$

The full backward recursion becomes:

$$\beta_t(s_t, a_t) = p(\mathcal{O}_t|s_t, a_t) \cdot \mathbb{E}_{s_{t+1} \sim p(s'|s_t, a_t)}[\beta_{t+1}(s_{t+1})] \quad (9.10)$$

$$\beta_t(s_t) = \mathbb{E}_{a_t \sim p(a|s_t)}[\beta_t(s_t, a_t)] \quad (9.11)$$

9.3.2 Interpreting as Value Functions

Define V and Q in terms of log backward messages:

$$V_t(s_t) = \log \beta_t(s_t) \quad (9.12)$$

$$Q_t(s_t, a_t) = \log \beta_t(s_t, a_t) \quad (9.13)$$

Then the recursion becomes:

$$V_t(s_t) = \log \int \exp(Q_t(s_t, a_t)) da_t \quad (9.14)$$

(this is a "soft max" – NOT the PyTorch softmax)

$$Q_t(s_t, a_t) = r(s_t, a_t) + \log \mathbb{E}_{s' \sim p(s'|s_t, a_t)}[\exp(V_{t+1}(s'))] \quad (9.15)$$

Comparison with standard value iteration:

Standard Value Iteration	Control-as-Inference
$V(s) = \max_a Q(s, a)$	$V(s) = \log \int \exp(Q(s, a)) da$ [soft max]
$Q(s, a) = r(s, a) + \mathbb{E}[V(s')]$	$Q(s, a) = r(s, a) + \log \mathbb{E}[\exp(V(s'))]$ [optimistic]

QUIZ ANSWER (Lecture 12, Q3, partial): Standard inference in control-as-inference gives a policy proportional to $\exp(Q)$, with Q -values that account for future entropy, AND it is erroneously optimistic due to the $\log \mathbb{E}[\exp(Q)]$ backup.

9.3.3 The Soft Max

The term $\log \int \exp(Q(s, a)) da$ is called a **soft max** (two words, not the PyTorch softmax function).

Why “soft max”? The exponential amplifies large values. For very large Q values, the log-integral-exp approaches the true max:

$$\lim_{\alpha \rightarrow 0} \alpha \cdot \log \int \exp(Q(s, a)/\alpha) da = \max_a Q(s, a)$$

For finite values, it’s a “softened” version of the max that gives some weight to all actions.

9.3.4 The Optimistic Backup Problem

In Eq. (9.15), the term $\log \mathbb{E}[\exp(V(s'))]$ is a **soft max over transition probabilities**. This is problematic because:

$$\log \mathbb{E}[\exp(V(s'))] \geq \mathbb{E}[V(s')] \tag{9.16}$$

(By Jensen’s inequality, since log-exp is convex)

This means the agent is being **erroneously optimistic** about stochastic outcomes! It acts as if it can “choose” the best next state from the stochastic transitions.

Professor’s Note

“The monkey is being optimistic over stochastic outcomes. It’s an optimistic monkey. And that maybe is not so rational, because irrational optimism can get you into trouble.”

Special case – deterministic dynamics: If $p(s'|s, a)$ assigns probability 1 to a single s' , then:

$$\log \mathbb{E}[\exp(V(s'))] = \log \exp(V(s')) = V(s') = \mathbb{E}[V(s')]$$

The optimistic backup disappears! In the deterministic case, control-as-inference gives us standard value iteration with soft max replacing hard max.

9.4 Computing the Policy

Using backward messages, we can compute the policy $\pi(a_t|s_t) = p(a_t|s_t, \mathcal{O}_{1:T})$:

Derivation:

Step 1: Since s_t d-separates past from future, past $\mathcal{O}$'s don't matter:

$$\pi(a_t|s_t) = p(a_t|s_t, \mathcal{O}_{t:T}) \quad (9.17)$$

Step 2: Apply conditional probability and Bayes' rule:

$$= \frac{p(a_t, s_t|\mathcal{O}_{t:T})}{p(s_t|\mathcal{O}_{t:T})} \quad (9.18)$$

Step 3: Apply Bayes' rule to numerator and denominator separately, cancel $p(\mathcal{O}_{t:T})$:

$$= \frac{p(\mathcal{O}_{t:T}|s_t, a_t) \cdot p(a_t, s_t)}{p(\mathcal{O}_{t:T}|s_t) \cdot p(s_t)} \quad (9.19)$$

Step 4: Recognize backward messages and action prior:

$$= \frac{\beta_t(s_t, a_t)}{\beta_t(s_t)} \cdot p(a_t|s_t) \quad (9.20)$$

Step 5: With uniform action prior, $p(a_t|s_t)$ is constant:

$$\begin{aligned} \pi(a_t|s_t) &= \frac{\beta_t(s_t, a_t)}{\beta_t(s_t)} \\ &= \frac{\exp(Q_t(s_t, a_t))}{\exp(V_t(s_t))} \\ &= \exp(Q_t(s_t, a_t) - V_t(s_t)) \\ &= \exp(A_t(s_t, a_t)) \end{aligned} \quad (9.21)$$

Intuition

The policy is proportional to the exponential of the advantage! Actions with high advantage are exponentially more likely. Actions with low advantage are exponentially less likely. This is exactly the Boltzmann exploration policy that we've seen before.

9.4.1 Temperature Parameter

We can introduce a temperature α to control the “sharpness” of the policy:

$$\pi(a_t|s_t) = \exp\left(\frac{1}{\alpha} A_t(s_t, a_t)\right) \quad (9.22)$$

- $\alpha \rightarrow 0$: Policy approaches the greedy (deterministic) optimal policy

- **Large α :** Policy is more random/exploratory
- This is equivalent to scaling the rewards by $1/\alpha$

9.4.2 The Action Prior

Common Misconception

“The action prior $p(a|s)$ must be uniform for control-as-inference to work.” Actually, a non-uniform action prior is equivalent to adding $\log p(a|s)$ to the reward function. So without loss of generality, we can always fold the action prior into the reward.

If the action prior is not uniform:

$$V_t(s_t) = \log \int \exp(Q_t(s_t, a_t) + \log p(a_t|s_t)) da_t \quad (9.23)$$

Define $\tilde{Q}_t(s_t, a_t) = Q_t(s_t, a_t) + \log p(a_t|s_t)$, and this becomes:

$$V_t(s_t) = \log \int \exp(\tilde{Q}_t(s_t, a_t)) da_t \quad (9.24)$$

which has the same form as before with a modified reward $\tilde{r}(s, a) = r(s, a) + \log p(a|s)$.

Intuition

If the monkey is naturally more likely to take small actions (Gaussian prior centered at zero), that’s the same as saying small actions have higher reward. So you can always absorb the action prior into the reward function.

9.5 Fixing the Optimistic Backup with Variational Inference

9.5.1 The Problem Revisited

The Q -value backup from naive inference has:

$$Q_t(s_t, a_t) = r(s_t, a_t) + \log \mathbb{E}_{s'}[\exp(V_{t+1}(s'))] \quad (9.25)$$

The $\log \mathbb{E}[\exp(V)]$ term is a **soft max over states** – the agent acts as if it can choose the best outcome from stochastic transitions. This is unreasonably optimistic.

What we WANT is:

$$Q_t(s_t, a_t) = r(s_t, a_t) + \mathbb{E}_{s'}[V_{t+1}(s')] \quad (9.26)$$

QUIZ ANSWER (Lecture 12, Q4): Replacing $\log \mathbb{E}[\exp(Q)]$ with $\mathbb{E}[Q]$ corresponds to using variational inference with a particular choice of variational distribution form.

9.5.2 The Variational Fix

The idea: instead of exact inference in the graphical model (which gives the optimistic backup), use **variational inference** with a structured variational distribution.

Specifically, choose the variational distribution to factorize as:

$$q(s_{1:T}, a_{1:T}) = \prod_t q(a_t|s_t) p(s_{t+1}|s_t, a_t) \quad (9.27)$$

This constrains the variational distribution to use the **true dynamics** for state transitions, but allows the action distribution $q(a_t|s_t)$ to be optimized.

When we optimize the ELBO with this structured q , the $\log \mathbb{E}[\exp(V)]$ term gets replaced by $\mathbb{E}[V]$, giving us the **soft Bellman backup**:

$$Q_t(s_t, a_t) = r(s_t, a_t) + \mathbb{E}_{s'}[V_{t+1}(s')] \quad (9.28)$$

$$\begin{aligned} V_t(s_t) &= \log \int \exp(Q_t(s_t, a_t)) da_t \\ &= \mathbb{E}_{a \sim \pi}[Q(s, a)] + \mathcal{H}(\pi(\cdot|s)) \end{aligned} \quad (9.29)$$

Intuition

By constraining the variational distribution to use the real dynamics, we prevent the agent from “cheating” by imagining favorable transitions. The resulting backup is honest about transition stochasticity while still being soft over actions.

9.6 Maximum Entropy RL

9.6.1 The Objective

Maximum entropy RL augments the standard RL objective with an entropy bonus:

$$\begin{aligned} J(\pi) &= \sum_t \mathbb{E}_{(s_t, a_t) \sim \pi} [r(s_t, a_t) + \alpha \cdot \mathcal{H}(\pi(\cdot|s_t))] \\ &= \sum_t \mathbb{E} [r(s_t, a_t) - \alpha \cdot \log \pi(a_t|s_t)] \end{aligned} \quad (9.30)$$

This says: maximize reward while also keeping the policy as random as possible (high entropy).

9.6.2 Soft Value Functions

The soft value functions satisfy:

$$\begin{aligned} V^{\text{soft}}(s) &= \mathbb{E}_{a \sim \pi}[Q^{\text{soft}}(s, a) - \alpha \log \pi(a|s)] \\ &= \mathbb{E}_{a \sim \pi}[Q^{\text{soft}}(s, a)] + \alpha \cdot \mathcal{H}(\pi(\cdot|s)) \end{aligned} \quad (9.31)$$

$$Q^{\text{soft}}(s, a) = r(s, a) + \gamma \mathbb{E}_{s'}[V^{\text{soft}}(s')] \quad (9.32)$$

$$V^{\text{soft}}(s) = \alpha \log \int \exp(Q^{\text{soft}}(s, a)/\alpha) da = \text{soft max}_a Q^{\text{soft}}(s, a) \quad (9.33)$$

9.6.3 The Soft Bellman Equation

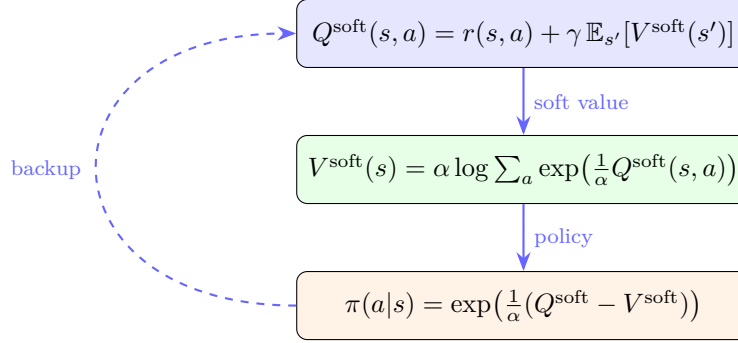


Figure 9.2: Soft Bellman equations: Q backs up with soft V, V uses log-sum-exp, policy is $\propto \exp(A/\alpha)$.

Combining:

$$Q^{\text{soft}}(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim p(s'|s, a)} \left[\alpha \log \int \exp(Q^{\text{soft}}(s', a')/\alpha) da' \right] \quad (9.34)$$

9.6.4 Full Derivation: Control-as-Inference to Max-Ent RL

Starting from control-as-inference with the variational fix:

Step 1: The variational lower bound on $\log p(\mathcal{O}_{1:T})$ with structured q gives:

$$\begin{aligned} \mathcal{L} &= \sum_t \mathbb{E}_{(s_t, a_t) \sim q} [r(s_t, a_t) - \log q(a_t|s_t)] \\ &= \sum_t \mathbb{E}_{(s_t, a_t) \sim q} [r(s_t, a_t)] + \sum_t \mathcal{H}(q(\cdot|s_t)) \end{aligned} \quad (9.35)$$

Step 2: Maximizing this ELBO over $q(a_t|s_t)$ is exactly max-entropy RL:

$$\max_q \sum_t \mathbb{E}[r(s_t, a_t) + \mathcal{H}(q(\cdot|s_t))] \quad (9.36)$$

Step 3: The optimal policy from this variational problem is:

$$\begin{aligned} \pi^*(a|s) &= \exp\left(\frac{1}{\alpha}(Q^{\text{soft}}(s, a) - V^{\text{soft}}(s))\right) \\ &= \exp\left(\frac{1}{\alpha}A^{\text{soft}}(s, a)\right) \end{aligned} \quad (9.37)$$

This closes the loop: control-as-inference with variational inference gives exactly maximum entropy RL.

9.6.5 Why Entropy Regularization Helps

1. **Exploration:** High-entropy policies explore more broadly, discovering new rewarding states
2. **Robustness:** Stochastic policies are more robust to perturbations – if the environment changes slightly, a diverse policy is more likely to still have good actions in its repertoire
3. **Multimodal policies:** Instead of committing to one solution, the agent maintains multiple strategies. If one fails, it can switch to another.
4. **Prevents premature convergence:** The entropy term prevents the policy from collapsing onto a single action too early in training

Professor's Note

“The monkey adds randomness in places where it doesn’t matter. So if you’re walking to class, you’re not going to go to a random classroom, but you might have some randomness in which path you take. As long as you end up in the right spot, you’re pretty happy.”

9.6.6 Connection to SAC (Soft Actor-Critic)

SAC is a practical algorithm that implements maximum entropy RL:

SAC Components:

- **Soft Q-function:** $Q_\theta(s, a)$ trained with soft Bellman backup
- **Policy:** $\pi_\phi(a|s)$ trained to maximize $\mathbb{E}[Q(s, a)] + \alpha \cdot \mathcal{H}(\pi)$
- **Temperature:** α can be fixed or automatically tuned
- Uses two Q-networks (clipped double-Q) for stability

The key insight from control-as-inference is that SAC’s entropy bonus is not just a heuristic – it arises naturally from treating control as probabilistic inference. The soft Bellman backup and the entropy-augmented policy are the direct consequences of running variational inference in the control-as-inference graphical model.

9.7 Forward Messages and State Distributions

9.7.1 Forward Messages

The forward message $\alpha_t(s_t)$ represents the probability of reaching state s_t given that the agent has been optimal at all past time steps:

$$\alpha_t(s_t) = p(s_t | \mathcal{O}_{1:t-1}) \quad (9.38)$$

Base case: $\alpha_1(s_1) = p(s_1)$

Recursive computation:

$$\alpha_t(s_t) = \int \frac{p(s_t | s_{t-1}, a_{t-1}) \cdot \exp(r(s_{t-1}, a_{t-1})) \cdot \alpha_{t-1}(s_{t-1})}{p(\mathcal{O}_{t-1} | \mathcal{O}_{1:t-2})} ds_{t-1} da_{t-1} \quad (9.39)$$

9.7.2 State Distribution Along Optimal Trajectories

The probability of being in state s_t given optimality at ALL time steps is:

$$p(s_t | \mathcal{O}_{1:T}) \propto \beta_t(s_t) \cdot \alpha_t(s_t) \quad (9.40)$$

This is the product of backward and forward messages, analogous to the forward-backward algorithm in HMMs.

9.8 Inverse RL Through Control as Inference

The control-as-inference framework provides an elegant perspective on inverse RL:

Given observed trajectories τ from an approximately optimal agent, we want to infer the reward function $r(s, a)$.

Under the model:

$$p(\tau | \mathcal{O}_{1:T}) \propto p(\tau) \cdot \exp\left(\sum_t r(s_t, a_t)\right)$$

This is a likelihood model for trajectories parameterized by the reward function r . We can: 1. Use maximum likelihood to find r that makes observed trajectories most probable 2. The probabilistic model naturally handles suboptimal demonstrations (they're less likely but non-zero probability) 3. This avoids the problem of deterministic IRL where any deviation from optimality requires a complex reward explanation

9.9 Worked Examples

9.9.1 Worked Example 1: Grid World with Deterministic Dynamics

Setup: 3-state grid: $[A] - [B] - [C]$, with goal at C. - $r(A, \text{right}) = -1$, $r(B, \text{right}) = -1$, $r(B, \text{left}) = -1$, $r(C, \text{stay}) = 0$ - All rewards negative (as required) - Deterministic dynamics

Step 1: Backward messages (from $T=2$)

At $T=2$, $\beta_2(s, a) = \exp(r(s, a))$: - $\beta_2(C, \text{stay}) = \exp(0) = 1$ - $\beta_2(B, \text{right}) = \exp(-1) = 0.37$ - $\beta_2(A, \text{right}) = \exp(-1) = 0.37$

At $T=1$, with deterministic dynamics: - $Q_1(A, \text{right}) = r(A, \text{right}) + V_2(B) = -1 + \log(\exp(Q_2(B, \text{right})) + \exp(Q_2(B, \text{left}))) = -1 + \log(\exp(-1) + \exp(-1)) = -1 + \log(0.74) = -1 + (-0.30) = -1.30$ - $V_1(A) = \log(\exp(Q_1(A, \text{right}))) = Q_1(A, \text{right}) = -1.30$

Step 2: Policy computation

$\pi(a|s) = \exp(Q(s, a) - V(s)) = \exp(A(s, a))$

At state B at time 2: - $\pi(\text{right}|B) = \exp(Q(B, \text{right}) - V(B)) = \exp(-1 - \log(\exp(-1) + \exp(-1))) = \exp(-1 - (-0.30)) = \exp(-0.70) = 0.50$ - $\pi(\text{left}|B) = \text{same by symmetry} = 0.50$

Both actions equally likely because both have the same Q -value (same reward). The agent is indifferent, which matches the soft optimal behavior.

9.9.2 Worked Example 2: Stochastic Dynamics – The Optimism Problem

Setup: State A, action “go right,” two possible outcomes: - $p(s' = \text{GOOD}|A, \text{right}) = 0.5$, $V(\text{GOOD}) = 10$ - $p(s' = \text{BAD}|A, \text{right}) = 0.5$, $V(\text{BAD}) = -10$

Naive inference (optimistic):

$$\begin{aligned} Q(A, \text{right}) &= r(A, \text{right}) + \log \mathbb{E}[\exp(V(s'))] \\ &= r + \log(0.5 \cdot \exp(10) + 0.5 \cdot \exp(-10)) \\ &= r + \log(0.5 \cdot 22026 + 0.5 \cdot 0.000045) \\ &= r + \log(11013) = r + 9.31 \end{aligned}$$

With variational fix (honest):

$$\begin{aligned} Q(A, \text{right}) &= r(A, \text{right}) + \mathbb{E}[V(s')] \\ &= r + 0.5 \cdot 10 + 0.5 \cdot (-10) \\ &= r + 0 \end{aligned}$$

The naive version assigns Q approximately $r + 9.31$, acting as if the agent will always get the GOOD outcome. The variational version correctly averages: $Q = r + 0$. The difference is dramatic when V values have high variance.

9.10 Summary: The Full Picture

(See Figure 9.2)

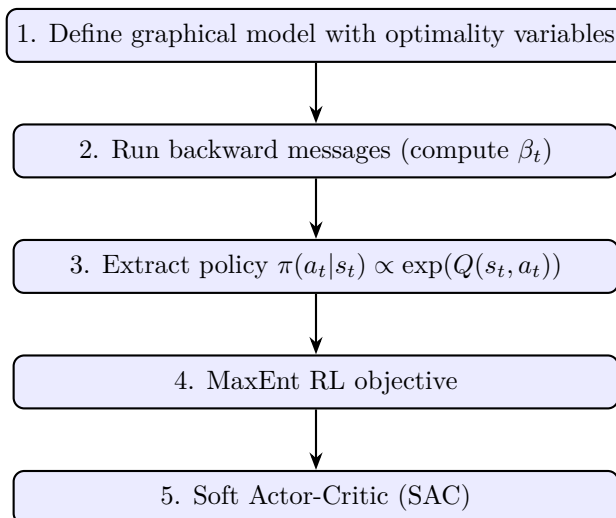


Figure 9.3: Control as Inference pipeline — from graphical model to SAC.

9.11 Check Your Understanding

1. **Why must we assume $r(s, a) \leq 0$ in the control-as-inference framework? Is this restrictive?** Because $p(\mathcal{O}_t|s_t, a_t) = \exp(r(s, a))$ must be a valid probability (between 0 and 1). This is NOT restrictive because adding any constant to all rewards doesn't change the optimal policy, so we can always shift rewards to be non-positive.
2. **In the deterministic dynamics case, what is the only difference between standard value iteration and the backup derived from control-as-inference?** The hard max over actions is replaced by a soft max ($\log \int \exp$). The transition backup $\mathbb{E}[V(s')]$ is identical because with deterministic dynamics, $\log \mathbb{E}[\exp(V)] = \log \exp(V(s')) = V(s') = \mathbb{E}[V(s')]$.
3. **A classmate says “the optimistic backup in control-as-inference makes the agent explore more, so it’s actually a feature, not a bug.” Evaluate this claim.** This is incorrect. The optimistic backup makes the agent overestimate Q -values for actions with high-variance outcomes, which is irrational (not exploratory). It would lead the agent to take actions with uncertain outcomes not because it wants to learn but because it incorrectly believes things will turn out well. Exploration should come from the entropy term, not from optimistic transitions.
4. **How does the control-as-inference framework help with inverse RL compared to standard deterministic models?** Under a deterministic model, any deviation from optimal behavior requires a complex reward function to explain. Under control-as-inference, suboptimal behavior is naturally assigned non-zero (but lower) probability via the exponential reward weighting. This allows IRL to infer simpler, more parsimonious reward functions from imperfect demonstrations.

5. **Explain the connection between SAC’s entropy bonus and the control-as-inference graphical model. Where does the entropy term “come from” in the graphical model?** *The entropy term arises from the variational inference procedure. When we optimize a structured variational distribution $q(a_t|s_t)$ to lower-bound $\log p(\mathcal{O}_{1:T})$, the ELBO naturally decomposes into $\sum_t \mathbb{E}[r(s_t, a_t)] + \sum_t \mathcal{H}(q(\cdot|s_t))$. The entropy $\mathcal{H}(q)$ appears because the ELBO always includes the entropy of the variational distribution (from the derivation via Jensen’s inequality or KL divergence). This is not an ad-hoc addition – it’s a fundamental consequence of variational inference.*

Chapter 10

Inverse Reinforcement Learning & Reward Learning

10.1 Motivation: Why Learn Rewards?

In standard RL, we assume the reward function is given and our goal is to learn a policy that maximizes cumulative reward. **Inverse reinforcement learning (IRL)** flips this problem: given demonstrations of (near-)optimal behavior, learn the reward function that explains that behavior.

Intuition

Standard imitation learning copies the *actions* of the expert with no reasoning about *why* those actions are taken. Human imitation learning, by contrast, copies the *intent* of the expert – you might take very different actions to achieve the same goal. IRL captures this by recovering the underlying objective.

There are two complementary motivations for learning rewards:

From the imitation learning perspective: - Behavior cloning suffers from compounding errors (distributional shift – see Chapter 2). - If instead we learn the reward and then run RL, we explore the environment and discover consequences of actions, avoiding compounding errors entirely. - The reward function is often *simpler* and more *generalizable* than the policy. For example, in autonomous driving, learning “keep clearance from pedestrians” transfers to novel situations without needing demonstrations in every scenario.

From the RL perspective: - Many real-world tasks (autonomous driving, household robotics) have no obvious numeric reward signal. - Games like Atari have a score on screen, but what is the reward for driving safely? IRL lets us infer it from human behavior.

Professor’s Note

“The reward function can often be simpler and therefore more generalizable. If you figure out in the autonomous driving case that it’s a good idea to keep plenty of clearance from pedestrians, then you can figure out on your own how to keep clearance in lots of different situations without having seen demos in all those cases.”

10.2 Maximum Entropy IRL (MaxEnt IRL)

10.2.1 Setting Up the Problem

We use the **control-as-inference framework** from Chapter 9. Recall the probabilistic graphical model with optimality variables:

$$p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t) = \exp(r(\mathbf{s}_t, \mathbf{a}_t)) \quad (10.1)$$

The key insight of IRL: we now **parameterize** the reward with parameters ψ :

$$p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t, \psi) = \exp(r_\psi(\mathbf{s}_t, \mathbf{a}_t)) \quad (10.2)$$

Given: A dataset of trajectories $\{\tau_i\}$ sampled from an expert policy $\pi^\star$ that we assume is (soft-)optimal.

Goal: Find ψ such that the observed trajectories are maximally likely under the control-as-inference model.

10.2.2 Maximum Likelihood Objective

From the control-as-inference framework, the probability of a trajectory conditioned on optimality is:

$$p(\tau | \mathcal{O}_{1:T}, \psi) \propto p(\tau) \exp\left(\sum_t r_\psi(\mathbf{s}_t, \mathbf{a}_t)\right) \quad (10.3)$$

where $p(\tau) = p(\mathbf{s}_1) \prod_t p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$ is the trajectory probability under the dynamics (with uniform action prior).

We can drop $p(\tau)$ from the optimization since it does not depend on ψ . The maximum likelihood objective becomes:

$$\max_{\psi} \frac{1}{N} \sum_{i=1}^N \log p(\tau_i | \mathcal{O}_{1:T}, \psi) = \max_{\psi} \frac{1}{N} \sum_{i=1}^N r_\psi(\tau_i) - \log Z \quad (10.4)$$

where the **partition function** is:

$$Z = \int p(\tau) \exp(r_\psi(\tau)) d\tau \quad (10.5)$$

Intuition

The partition function Z is essential because without it, we could trivially maximize the objective by adding a large constant to all rewards. The normalizer ensures we are learning a proper probability distribution, not just inflating reward values.

10.2.3 Computing the Gradient

Taking the gradient of the objective (Eq. 10.4) with respect to ψ :

$$\nabla_{\psi} \mathcal{L} = \frac{1}{N} \sum_{i=1}^N \nabla_{\psi} r_{\psi}(\tau_i) - \frac{1}{Z} \int p(\tau) \exp(r_{\psi}(\tau)) \nabla_{\psi} r_{\psi}(\tau) d\tau \quad (10.6)$$

Step-by-step derivation of the second term:

1. Start with $\nabla_{\psi} \log Z = \frac{1}{Z} \frac{dZ}{d\psi}$.
2. Since $Z = \int p(\tau) \exp(r_{\psi}(\tau)) d\tau$, we get:

$$\frac{dZ}{d\psi} = \int p(\tau) \exp(r_{\psi}(\tau)) \nabla_{\psi} r_{\psi}(\tau) d\tau \quad (10.7)$$

3. Dividing by Z :

$$\frac{1}{Z} \int p(\tau) \exp(r_{\psi}(\tau)) \nabla_{\psi} r_{\psi}(\tau) d\tau = \int \underbrace{\frac{p(\tau) \exp(r_{\psi}(\tau))}{Z}}_{p(\tau|\mathcal{O}_{1:T}, \psi)} \nabla_{\psi} r_{\psi}(\tau) d\tau \quad (10.8)$$

4. This is simply an expectation under the **soft optimal policy** for the current reward:

$$= \mathbb{E}_{\tau \sim p(\tau|\mathcal{O}_{1:T}, \psi)} [\nabla_{\psi} r_{\psi}(\tau)] \quad (10.9)$$

Therefore, the gradient has the elegant form:

$$\boxed{\nabla_{\psi} \mathcal{L} = \mathbb{E}_{\tau \sim \pi^*} [\nabla_{\psi} r_{\psi}(\tau)] - \mathbb{E}_{\tau \sim p(\tau|\mathcal{O}_{1:T}, \psi)} [\nabla_{\psi} r_{\psi}(\tau)]} \quad (10.10)$$

Intuition

This gradient has a beautifully natural interpretation: - **First term (positive)**: Push up the reward of expert demonstrations. - **Second term (negative)**: Push down the reward of trajectories that are optimal under the *current* reward.

The reward function adjusts so that what the expert does looks better than what the current policy would do. At convergence, the expert demonstrations and policy samples are indistinguishable.

10.2.4 Estimating the Gradient in Practice

The first term is easy – just sample trajectories from the expert dataset and compute $\nabla_{\psi} r_{\psi}(\tau_i)$.

The second term is hard – it requires samples from $p(\tau|\mathcal{O}_{1:T}, \psi)$, i.e., the soft optimal policy under the current reward r_{ψ} . This is exactly the distribution that max-ent RL algorithms converge to (see Chapter 9).

This gives us a sketch of the algorithm:

$$\nabla_{\psi} \mathcal{L} \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\psi} r_{\psi}(\tau_i) - \frac{1}{M} \sum_{j=1}^M \nabla_{\psi} r_{\psi}(\tau_j) \quad (10.11)$$

where $\{\tau_i\}$ are expert samples and $\{\tau_j\}$ are samples from the current soft optimal policy.

Common Misconception

“We need to run RL to full convergence in the inner loop.” While the theoretically correct approach requires running max-ent RL to convergence for each reward update, this is prohibitively expensive. In practice, we only take a few policy improvement steps (or even one) and correct for the resulting bias using importance sampling.

10.2.5 Importance Sampling for Efficient Updates

If we only improve the policy a little (rather than solving RL to convergence), our samples $\{\tau_j\}$ come from a suboptimal policy π rather than from $p(\tau|\mathcal{O}_{1:T}, \psi)$. We correct this with importance weights:

$$\nabla_{\psi} \mathcal{L} \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\psi} r_{\psi}(\tau_i) - \frac{1}{\sum_j w_j} \sum_{j=1}^M w_j \nabla_{\psi} r_{\psi}(\tau_j) \quad (10.12)$$

where:

$$w_j = \frac{p(\tau_j) \exp(r_{\psi}(\tau_j))}{\pi(\tau_j)} \quad (10.13)$$

Simplifying the importance weight:

$$w_j = \frac{p(\mathbf{s}_1) \prod_t p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t) \exp(r_{\psi}(\mathbf{s}_t, \mathbf{a}_t))}{p(\mathbf{s}_1) \prod_t p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t) \pi(\mathbf{a}_t|\mathbf{s}_t)} = \frac{\exp(\sum_t r_{\psi}(\mathbf{s}_t, \mathbf{a}_t))}{\prod_t \pi(\mathbf{a}_t|\mathbf{s}_t)} \quad (10.14)$$

The initial state distribution and transition dynamics **cancel out** – we only need the current reward r_{ψ} (which we know, since we are learning it) and the current policy π (which we also know).

Professor’s Note

“Each policy update with respect to r_{ψ} brings us closer to the target distribution! So as you update π , these weights should become closer and closer to one. Updating your policy literally lowers the variance of the estimator.”

10.2.6 Algorithm 10.1: MaxEnt IRL (Practical Version)

Algorithm 10.1: MaxEnt IRL (Practical Version)

Input: Expert demonstrations $\{\tau_i\}$ from π^*

Initialize: reward parameters ψ , policy parameters θ

Repeat until convergence: 1. Run one step of max-ent RL on current reward r_ψ to update policy π_θ (e.g., one step of soft actor-critic or entropy-regularized policy gradient)

2. Sample trajectories $\{\tau_j\}$ from current policy π_θ

3. Compute importance weights for each τ_j : $w_j = \frac{\exp(\sum_t r_\psi(s_t, a_t))}{\prod_t \pi_\theta(a_t | s_t)}$

4. Update reward parameters via gradient ascent on ψ : $\psi \leftarrow \psi + \alpha \left[\frac{1}{N} \sum_i \nabla_\psi r_\psi(\tau_i) - \sum_j \frac{1}{w_j} \sum_j w_j \nabla_\psi r_\psi(\tau_j) \right]$

Intuition

This algorithm has an adversarial game interpretation. The reward function tries to make expert demonstrations look better and policy samples look worse. The policy tries to find trajectories that get high reward. The only equilibrium is when the policy matches the expert, because then the reward function cannot distinguish between the two.

10.3 Connection to GANs and GAIL

The adversarial structure of MaxEnt IRL bears a striking resemblance to **Generative Adversarial Networks (GANs)**:

	MaxEnt IRL	GAN
Generator	Policy $\pi_\theta(\tau)$	Generator $p_\theta(\mathbf{x} \mathbf{z})$
Discriminator	Reward function r_ψ	Classifier $D_\psi(\mathbf{x})$
Real data	Expert demonstrations π^*	Training images $p^*(\mathbf{x})$
Generator objective	Maximize reward	Fool discriminator
Discriminator objective	Distinguish expert vs. policy	Distinguish real vs. fake

GAIL (Generative Adversarial Imitation Learning) (Ho & Ermon) takes this analogy literally: replace the structured reward function with a plain binary classifier $D_\psi(\tau)$ that classifies trajectories as “expert” or “policy”:

$$\psi \leftarrow \arg \max_{\psi} \mathbb{E}_{\tau \sim p^*} [\log D_\psi(\tau)] + \mathbb{E}_{\tau \sim \pi_\theta} [\log(1 - D_\psi(\tau))] \quad (10.15)$$

$$\nabla_\theta \mathcal{L} \approx \frac{1}{M} \sum_{j=1}^M \nabla_\theta \log \pi_\theta(\tau_j) \log D_\psi(\tau_j) \quad (10.16)$$

Equivalence: If you choose the discriminator to have the specific form:

$$D(\tau) = \frac{\frac{1}{Z} \exp(r(\tau))}{\frac{1}{Z} \exp(r(\tau)) + \pi(\tau)} \quad (10.17)$$

then GAIL becomes equivalent to MaxEnt IRL.

Pros and cons of GAIL vs. MaxEnt IRL:

	GAIL	MaxEnt IRL
Simplicity	Simpler to set up	More complex optimization
At convergence	Discriminator knows <i>nothing</i> (outputs 0.5 everywhere)	Reward function captures meaningful structure
Reward reuse	Cannot reoptimize the “reward” for new settings	Can transfer learned reward to new environments

Professor’s Note

“It’s easier to judge than it is to actually do it, so you have to usually regularize the discriminator more to make this work well and give the policy a bit of an advantage.”

10.4 Learning Rewards from Human Preferences

An alternative to learning rewards from demonstrations is learning from **pairwise preferences**. This avoids requiring full demonstrations – instead, humans simply compare two options.

10.4.1 The Bradley-Terry Model

Given two trajectories τ_i and τ_j , we model the probability that a human prefers τ_i over τ_j as:

$$p(\tau_i \succ \tau_j) = \frac{\exp(r_\psi(\tau_i))}{\exp(r_\psi(\tau_i)) + \exp(r_\psi(\tau_j))} \quad (10.18)$$

Intuition

Intuition (The Optometrist Algorithm): This is exactly like visiting the eye doctor. They show you two lenses: “Is A better than B?” You pick one. They adjust and repeat. The key insight is that preferences are *stochastic* – 7 out of 10 people might prefer A, suggesting A is slightly better but not overwhelmingly so. The Bradley-Terry model captures this: if the rewards are close, the preference probability is near 50%.

Step-by-step simplification to the log-sigmoid form:

Starting from Eq. 10.18:

$$\log p(\tau_i \succ \tau_j) = \log \frac{\exp(r_\psi(\tau_i))}{\exp(r_\psi(\tau_i)) + \exp(r_\psi(\tau_j))} \quad (10.19)$$

$$= \log \left(\frac{\exp(r_\psi(\tau_i) - r_\psi(\tau_j))}{\exp(r_\psi(\tau_i) - r_\psi(\tau_j)) + 1} \right) \quad (10.20)$$

$$= \log \sigma(r_\psi(\tau_i) - r_\psi(\tau_j)) \quad (10.21)$$

where $\sigma(x) = \frac{\exp(x)}{\exp(x)+1} = \frac{1}{1+\exp(-x)}$ is the **sigmoid function**.

10.4.2 Training the Reward Model

Given a dataset of pairwise preferences $\{\tau_i \succ \tau_j\}_{i,j}$, we maximize the likelihood:

$$\psi \leftarrow \arg \max_{\psi} \sum_{i,j} \log \sigma(r_\psi(\tau_i) - r_\psi(\tau_j)) \quad (10.22)$$

This is a standard supervised learning objective – each data point is one preference label, and we maximize the probability of all observed preferences.

Professor’s Note

“This is also exactly the same as ELO scores in chess. If $r(\tau_i)$ and $r(\tau_j)$ are the ELO scores of the two trajectories, it’s basically the same thing. ELO score is a prediction of whether one player will beat another in a head-to-head game.”

Common Misconception: “You need a full ranking over trajectories.” No – only pairwise comparisons are needed. Full rankings are harder for humans and unnecessary. The same trajectory can appear in multiple pairwise comparisons.

10.5 Check Your Understanding

1. **Why does the MaxEnt IRL gradient have the form “expert expectation minus policy expectation”?** Explain where the second term comes from and what role the partition function plays.
2. **In the importance sampling version of MaxEnt IRL, the importance weights simplify to $\exp(\sum_t r_\psi) / \prod_t \pi(\mathbf{a}_t | \mathbf{s}_t)$.** Why do the dynamics terms cancel? What are the implications for variance as the policy improves?
3. **Compare and contrast MaxEnt IRL with GAIL.** Under what circumstances would you prefer one over the other? What happens to the discriminator at convergence in GAIL?
4. **In the Bradley-Terry model, what happens to the preference probability when the two trajectories have equal reward?** What does this tell us about how the model handles annotator disagreement?
5. **Why is learning the reward function potentially more transferable than learning the policy directly?** Give a concrete example (e.g., autonomous driving) to illustrate.

Answers

1. The gradient of the log-likelihood has two terms because of the partition function Z . The first term (expert expectation) comes from $\nabla_\psi r_\psi(\tau)$ evaluated on expert data. The second term comes from $\nabla_\psi \log Z$, which equals the expectation of $\nabla_\psi r_\psi(\tau)$ under the soft optimal policy for the current reward r_ψ (Eq. 10.9). Without Z , we could trivially maximize the objective by

inflating all rewards; Z normalizes the distribution and forces the gradient to be the *difference* between expert and policy expectations. At convergence, these match and the gradient is zero.

2. The importance weight $w_j = \frac{p(\tau) \exp(r_\psi(\tau))}{\pi(\tau)}$. Both numerator and denominator contain $p(\mathbf{s}_1) \prod_t p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$, which cancels, leaving $w_j = \frac{\exp(\sum_t r_\psi)}{\prod_t \pi(\mathbf{a}_t | \mathbf{s}_t)}$. As the policy improves toward soft optimality, $\pi(\mathbf{a}_t | \mathbf{s}_t) \rightarrow \exp(r_\psi(\mathbf{s}_t, \mathbf{a}_t)) / Z_t$, so the weights approach a constant – variance decreases as the policy gets better.
3. **GAIL** uses a generic binary classifier as the discriminator and is simpler to set up; at convergence the discriminator outputs 0.5 everywhere (it cannot distinguish expert from policy). **MaxEnt IRL** learns a structured reward function that captures meaningful task structure and can be transferred to new environments. Prefer GAIL when you just need policy matching; prefer MaxEnt IRL when you want a reusable reward function (e.g., transfer to a new robot morphology).
4. When $r_\psi(\tau_i) = r_\psi(\tau_j)$, the Bradley-Terry probability $p(\tau_i \succ \tau_j) = \sigma(0) = 0.5$. This means equally rewarded trajectories are equally likely to be preferred, matching our intuition. It also gracefully handles annotator disagreement: if 7/10 annotators prefer τ_i , this implies $r_\psi(\tau_i)$ is somewhat higher but not overwhelmingly so – the stochastic model captures noise in human judgments.
5. The reward function captures the *objective* (e.g., “maintain clearance from pedestrians”), which is a simpler, more compact representation than a full policy mapping states to actions. In autonomous driving, a learned reward “keep distance from pedestrians” transfers to new cities, weather conditions, and vehicle types without needing demonstrations in each. The policy (steering/braking commands) is highly situation-specific and would need retraining, but the reward generalizes because the underlying goal is the same.

Chapter 11

Reinforcement Learning for Large Language Models

11.1 Background: The Anatomy of LLM Training

Modern LLM training consists of two broad phases:

Pre-training: - Train on massive, diverse web data (Common Crawl, Wikipedia, etc.) - Objective: next-token prediction (cross-entropy loss) - This gives the LLM **knowledge** – facts, language patterns, reasoning capabilities - The resulting model is a “text completion engine,” not an assistant

Post-training: - Tell the model how to *use* its knowledge - **Supervised Fine-Tuning (SFT)** / **Instruction Tuning:** Train on curated (prompt, response) pairs showing desired behavior - **RL Fine-Tuning:** Optimize for a reward function using RL algorithms

Intuition

Pre-training is like reading every book in the library. Post-training is like an apprenticeship that teaches you how to apply that knowledge. Pre-trained models know a lot of stuff but do not yet know how to *use* it – they might respond to a question with a snarky forum post rather than a helpful answer.

Professor’s Note: “Sometimes even fine-tuning LLMs on *incorrect* answers actually gets them to answer questions correctly – but in the format shown in fine-tuning. This shows that pre-training provides the knowledge; post-training provides the format.”

11.1.1 Two Paradigms for RL with LLMs

	RL from Preferences (RLHF)	RL from Verifiers
Reward source	Learned from human preferences	Programmatic correctness check
Generality	Very general (any task)	Requires verifiable answers
Risk	May optimize for flattery / reward hacking	More robust but less general

	RL from Preferences (RLHF)	RL from Verifiers
Example	“Which response is more helpful?”	“Does the code pass the test suite?”
Used for	General assistants, safety alignment	Math reasoning, coding, “thinking” models

11.2 LLMs as MDPs

11.2.1 The One-Step (Contextual Bandit) Formulation

The simplest formulation treats the entire completion as a single action:

- **State $\mathbf{s}$:** The prompt / context / prefix
- **Action $\mathbf{a}$:** The entire completion / suffix (all generated tokens)
- **Reward $r(\mathbf{s}, \mathbf{a})$:** Score for that completion given the prompt

The policy factorizes via the chain rule of probability:

$$\pi_{\theta}(\mathbf{a}|\mathbf{s}) = p(x_5|x_{1:4}) \cdot p(x_6|x_{1:4}, x_5) \cdot \dots = \prod_{n=1}^N p(x_{|\mathbf{s}|+n}|x_{1:|\mathbf{s}|+n-1}) \quad (11.1)$$

The RL objective is simply:

$$\max_{\theta} \mathbb{E}_{\mathbf{s} \sim \mathcal{D}} \left[\mathbb{E}_{\mathbf{a} \sim \pi_{\theta}(\cdot|\mathbf{s})} [r(\mathbf{s}, \mathbf{a})] \right] \quad (11.2)$$

Intuition

Even though the action consists of many tokens, the LLM autoregressively generates them in sequence. From the RL algorithm’s perspective, this is just a one-step decision: given a prompt, produce a completion, receive a reward.

11.2.2 The Multi-Step (Token-Level) Formulation

Alternatively, treat each token as a separate time step:

- **State $\mathbf{s}_t$:** The prompt concatenated with all tokens generated so far
- **Action a_t :** The next token (from the vocabulary)
- **Transition:** Deterministic – append a_t to $\mathbf{s}_t$ to get $\mathbf{s}_{t+1}$
- **Reward:** Typically only at the final token (outcome reward), though intermediate “process rewards” are possible

Both formulations produce identical trajectory distributions, but the multi-step view is useful when:

- You want per-token value functions as baselines
- You want intermediate (process) rewards
- You want GAE-style advantage estimation

11.2.3 Why LLMs Are Technically POMDPs

Professor’s Note

“If you think of each token as an observation, it’s a very limited observation – just from seeing the word ‘hello’ it’s probably not really possible to figure out what the problem is. But if you look at the whole history of tokens, now you can probably actually figure out a decent action.”

At each time step, the LLM sees a single token as its “observation.” The full context (including the human’s intent, implicit constraints, etc.) is the hidden state. However, the full history of tokens serves as a sufficient statistic for the state, and the transformer architecture processes this entire history via self-attention.

Common Misconception

“POMDPs always require fundamentally different algorithms than MDPs.” In practice, for LLMs, the transformer’s attention over the full token history effectively implements a belief-state representation. The history of observations is a Markovian state representation, so standard MDP algorithms apply when conditioned on the full history.

Key properties of POMDPs relevant to the quiz: - POMDPs **can require stochastic optimal policies** (unlike MDPs where deterministic optimal policies always exist) - POMDPs **can require information-gathering actions** (probing to reduce uncertainty about the hidden state)

11.3 Policy Gradient for LLMs

11.3.1 The REINFORCE Estimator

Starting from the one-step formulation, the policy gradient is:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(\mathbf{a} | \mathbf{s}) \cdot r(\mathbf{s}, \mathbf{a})] \quad (11.3)$$

where:

$$\log \pi_{\theta}(\mathbf{a} | \mathbf{s}) = \sum_{n=1}^N \log p_{\theta}(x_{|\mathbf{s}|+n} | x_{1:|\mathbf{s}|+n-1}) \quad (11.4)$$

This is simply the **sum of log-probabilities of each token** in the completion – i.e., the negative cross-entropy loss.

To implement this: 1. Sample completions from the current LLM (generate answers) 2. Compute the reward for each completion 3. Compute the gradient of the cross-entropy loss, weighted by the reward 4. Update parameters

11.3.2 Importance-Sampled Policy Gradient / PPO

We can also use importance-weighted estimators, exactly as in Chapter 7:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim \bar{\pi}} \left[\frac{\pi_{\theta}(\mathbf{a}|\mathbf{s})}{\bar{\pi}(\mathbf{a}|\mathbf{s})} \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}|\mathbf{s}) \cdot r(\mathbf{s}, \mathbf{a}) \right] \quad (11.5)$$

where $\bar{\pi}$ is the sampling policy (e.g., the model’s parameters from a few gradient steps ago).

Why IS-PG / PPO is preferred for LLMs: Sampling from an LLM is very expensive (autoregressive generation). We want to take **many gradient steps per set of samples** to amortize the cost of generation. Importance-weighted methods like PPO allow this by reusing samples across multiple updates. (See Chapter 7 for PPO clipping details.)

This is different from robotics (where samples are expensive so you want Q-function methods) or games like AlphaGo (where samples are trivially cheap so on-policy methods are fine). LLMs hit a sweet spot where both sampling AND training are expensive, and IS-PG/PPO strikes the right balance.

11.3.3 Algorithm 11.1: PPO for LLMs

Algorithm 11.1: PPO for LLMs

Input: Prompt dataset $\mathcal{D}$, reward function r , reference model π_{ref}

Initialize: Policy π_{θ} (from SFT model), value function V_{ϕ}

Repeat:

1. For each prompt s in mini-batch from $\mathcal{D}$: - (a) Sample K completions: $a_1, \dots, a_K \sim \pi_{\theta}(\cdot|s)$ - (b) Compute rewards: $\bar{r}(s, a_k) = r(s, a_k) - \beta \cdot D_{\text{KL}}(\pi_{\theta}(\cdot|s) \parallel \pi_{\text{ref}}(\cdot|s))$
 2. Compute per-token advantages using GAE: $\delta_t = \bar{r}_t + V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$, $\hat{A}_t^{\text{GAE}} = \sum_{t' \geq t} (\gamma \lambda)^{t'-t} \delta_{t'}$
 3. Compute value targets: $V_{\text{target}, t} = \hat{A}_t^{\text{GAE}} + V_{\phi}(s_t)$
 4. Set $\bar{\pi} \leftarrow \pi_{\theta}$ (save current policy for importance weights)
 5. For $k = 1$ to K_{inner} (e.g., 10 inner gradient steps): - (a) Sample mini-batch of (s, a) pairs from buffer - (b) Compute clipped PPO objective (see Chapter 7) - (c) Update θ via gradient ascent - (d) Update ϕ by minimizing $\|V_{\phi}(s_t) - V_{\text{target}, t}\|^2$
-

11.4 Baselines for LLMs

11.4.1 Standard Approach: GAE with a Value Function

The standard approach (as in regular PPO) is to train a value function V_{ϕ} and use **Generalized Advantage Estimation (GAE)** (see Chapter 7 for full derivation):

$$\hat{A}_t^{GAE} = \sum_{t' \geq t} (\gamma \lambda)^{t'-t} \delta_{t'}, \quad \delta_t = r_t + \gamma V_\phi(\mathbf{s}_{t+1}) - V_\phi(\mathbf{s}_t) \quad (11.6)$$

The value function can be represented as: - A separate copy of the entire LLM with a scalar output head - A second “value head” added to the existing LLM (a small MLP on top of the last hidden layer)

Professor’s Note

“You typically don’t train the value function on the prompt. You only train on the suffix because you don’t need to know the value of the prompt.”

11.4.2 GRPO: Baselines Without Value Functions

Key insight for LLMs: Unlike physical environments (robots, cars), we can easily “teleport” back to any state and generate multiple completions. We simply feed the same prompt back into the LLM and sample again.

This enables a simple but effective baseline: for each prompt $\mathbf{s}_i$, sample K completions $\{\mathbf{a}_1, \dots, \mathbf{a}_K\}$ and use the **per-prompt average reward** as the baseline:

$$b(\mathbf{s}_i) = \frac{1}{K} \sum_{k=1}^K r(\mathbf{s}_i, \mathbf{a}_k) \quad (11.7)$$

The advantage estimate for completion k is then:

$$\hat{A}(\mathbf{s}_i, \mathbf{a}_k) = r(\mathbf{s}_i, \mathbf{a}_k) - b(\mathbf{s}_i) \quad (11.8)$$

Intuition

This baseline considers only *how good a completion is relative to other completions for the same prompt*. If all completions for a math problem are wrong, they all get negative advantage. If all are right, they all get near-zero advantage. This is much more powerful than a naive global average baseline because it’s an unbiased estimate of the value of that specific prompt.

This is called **GRPO (Group Relative Policy Optimization)** (Shao et al., 2024). It is essentially PPO without a value function, using the per-prompt average as the baseline.

11.4.3 Algorithm 11.2: GRPO

Algorithm 11.2: GRPO (Group Relative Policy Optimization)

Input: Prompt dataset $\mathcal{D}$, reward function r , reference model π_{ref}

Initialize: Policy π_θ (from SFT model)

Repeat:

1. For each prompt s_i in mini-batch from $\mathcal{D}$: - (a) Sample K completions: $\{a_1, \dots, a_K\} \sim \pi_\theta(\cdot|s_i)$ - (b) Compute rewards: $r_k = r(s_i, a_k)$ for $k = 1, \dots, K$ - (c) Compute baseline: $b(s_i) = \frac{1}{K} \sum_k r_k$ - (d) Compute advantages: $A_k = r_k - b(s_i)$ - (e) Optionally add KL penalty: $\bar{r}_k = r_k - \beta \cdot D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})$
2. Set $\bar{\pi} \leftarrow \pi_\theta$
3. For $k = 1$ to K_{inner} : - (a) Update θ using clipped IS-PG objective with advantages A_k and importance weights $\pi_\theta/\bar{\pi}$

Common Misconception

“GRPO is a fundamentally different algorithm from PPO.” GRPO is just PPO with a different (and arguably simpler) advantage estimator. The core update rule is the same importance-weighted, clipped policy gradient.

11.5 Reference Model Regularization (KL Penalty)

11.5.1 The Problem: Reward Hacking

If we optimize the reward without constraints, the model can learn to produce **gibberish that fools the reward model** into giving high scores. This is called **reward hacking** or **reward over-optimization**.

Examples: - A model trained on preference rewards might learn to be excessively flattering or verbose
- A model with a learned reward function might produce nonsense text that exploits weaknesses in the reward model - Without constraints, nothing prevents the model from “forgetting” English or producing offensive content

11.5.2 The Solution: KL Divergence Penalty

We add a **KL divergence** penalty between the current policy π_θ and a **reference model** π_{ref} (typically the SFT model before RL training):

$$\bar{r}(\mathbf{s}, \mathbf{a}) = r_\psi(\mathbf{s}, \mathbf{a}) - \beta \cdot D_{\text{KL}}(\pi_\theta(\mathbf{a}|\mathbf{s}) \parallel \pi_{\text{ref}}(\mathbf{a}|\mathbf{s})) \quad (11.9)$$

In practice, approximated per-sample as:

$$\bar{r}(\mathbf{s}, \mathbf{a}) \approx r_\psi(\mathbf{s}, \mathbf{a}) - \beta \cdot [\log \pi_\theta(\mathbf{a}|\mathbf{s}) - \log \pi_{\text{ref}}(\mathbf{a}|\mathbf{s})] \quad (11.10)$$

Intuition

The KL penalty says: “Maximize the reward, but all else being equal, try to remain a good language model. Don’t stray too far from the reference.” If the current policy produces a completion that is very unlikely under the reference model (e.g., random character sequences like “hhhhhjjj”), it receives a large negative reward contribution from the KL term, regardless of what the reward model says.

Professor’s Note: “The reference model typically will be fairly general, so it’ll say lots of things are probable, but outputting ‘hhhhhj’ is very improbable. So mostly this is just there to make the really improbable stuff have a very negative reward.”

Connection to control as inference (Chapter 9): The KL-regularized RL objective:

$$J(\theta) = \mathbb{E}_{\pi_\theta}[r(\mathbf{s}, \mathbf{a})] - \beta \cdot D_{KL}(\pi_\theta \parallel \pi_{\text{ref}}) \quad (11.11)$$

is equivalent to the maximum entropy RL objective with a non-uniform action prior π_{ref} instead of uniform. The entropy bonus $\mathcal{H}(\pi)$ in max-ent RL is the special case where π_{ref} is uniform.

11.6 The Full RLHF Pipeline

Putting everything together, the complete **Reinforcement Learning from Human Feedback (RLHF)** pipeline consists of three stages:

11.6.1 Stage 1: Supervised Fine-Tuning (SFT)

- Collect demonstration data: (prompt, desired response) pairs
- Fine-tune the pre-trained LLM on this data using standard cross-entropy loss
- This produces π_{SFT} , which serves as both the starting point for RL and the reference model

11.6.2 Stage 2: Reward Model Training

- Sample prompts and generate multiple completions from π_{SFT}
- Ask human labelers to compare pairs of completions: “Is A better than B?”
- Train reward model r_ψ using the Bradley-Terry objective (Eq. 10.22):

$$\psi \leftarrow \arg \max_{\psi} \sum_{i,j} \log \sigma(r_\psi(\tau_i) - r_\psi(\tau_j)) \quad (11.12)$$

11.6.3 Stage 3: RL Optimization

- Initialize policy $\pi_\theta \leftarrow \pi_{\text{SFT}}$, set $\pi_{\text{ref}} \leftarrow \pi_{\text{SFT}}$
- Train π_θ using PPO (or GRPO) with reward:

$$\bar{r}(\mathbf{s}, \mathbf{a}) = r_\psi(\mathbf{s}, \mathbf{a}) - \beta \cdot D_{KL}(\pi_\theta \| \pi_{\text{ref}}) \quad (11.13)$$

- Optionally: repeat stages 2-3 (collect new preferences from improved model)

11.6.4 Algorithm 11.3: Full RLHF Pipeline

Algorithm 11.3: Full RLHF Pipeline

Stage 1: SFT 1. Collect demonstrations $\{(s_i, a_i^*)\}$ from human labelers 2. Train π_{SFT} by minimizing cross-entropy: $-\sum_i \log \pi(a_i^* | s_i)$ 3. Set $\pi_{\text{ref}} \leftarrow \pi_{\text{SFT}}$

Stage 2: Reward Model 1. For each prompt s in dataset: sample completions $\tau_1, \tau_2, \dots$ from π_{SFT} 2. Ask humans: which is preferred? Record $\{\tau_i \succ \tau_j\}$ 3. Train r_ψ : $\max_\psi \sum_{i,j} \log \sigma(r_\psi(\tau_i) - r_\psi(\tau_j))$

Stage 3: RL Fine-Tuning 1. Initialize $\pi_\theta \leftarrow \pi_{\text{SFT}}$ 2. **Repeat:** - Sample prompts from dataset - Generate completions from π_θ - Compute modified reward: $\bar{r} = r_\psi - \beta \cdot [\log \pi_\theta - \log \pi_{\text{ref}}]$ - Update π_θ using PPO or GRPO with reward $\bar{r}$ - (Optionally: collect new preferences and update r_ψ)

Professor's Note

Professor's Note on the full pipeline (from Ouyang et al., 2022 / InstructGPT): Step 1 collects demonstration data and trains an SFT policy. Step 2 collects comparison data and trains a reward model. Step 3 optimizes the policy against the reward model using PPO. This is the pipeline that first turned models like GPT-3 into useful assistants like ChatGPT.

11.7 RL from Verifiers (Bonus)

An alternative to learned reward models: use **verifiable rewards** when available: - Math problems: check if the final answer matches ground truth - Code generation: run the code and check if tests pass - Factual QA: compare against known correct answers

This avoids reward hacking entirely because the reward is ground truth, not a learned approximation. It is used extensively in training “thinking” / “reasoning” models (e.g., DeepSeek-R1) that produce intermediate reasoning steps and are rewarded for reaching correct final answers.

11.8 Check Your Understanding

1. **Explain the two MDP formulations for LLMs (one-step vs. multi-step).** When might you prefer one over the other? What is the action space in each case?
2. **Why is importance-sampled PG / PPO preferred over on-policy REINFORCE for LLMs, and also preferred over off-policy Q-function methods?** Explain the computational tradeoffs. (Hint: think about the cost of both sampling and training.)
3. **What is GRPO and why is it possible for LLMs but not for robotics?** What property of the LLM environment allows us to compute a good baseline without a value function?
4. **Why is the KL penalty to a reference model necessary? Give two concrete failure modes that can occur without it.** How does this connect to the maximum entropy RL framework from Chapter 9?
5. **Walk through the three stages of the RLHF pipeline. At each stage, what is the input data, what is being optimized, and what is the output?**

Answers

1. **One-step (contextual bandit):** State = prompt, action = entire completion, reward given per completion. Action space is the set of all possible token sequences. **Multi-step (token-level):** State = prompt + tokens so far, action = next token (from vocabulary), transition is deterministic (append token). Prefer multi-step when you want per-token value functions, intermediate process rewards, or GAE-style advantage estimation. Prefer one-step for simplicity when only outcome reward is available.
2. On-policy REINFORCE requires fresh samples every gradient step, but LLM sampling (autoregressive generation) is expensive. IS-PG/PPO reuses samples across many inner gradient steps, amortizing sampling cost. Off-policy Q-function methods (DQN/SAC) would also reuse data, but they require a Q-function over the combinatorially large action space (all possible completions or all tokens in context), making them impractical for LLMs. PPO hits the sweet spot: reuses samples (unlike REINFORCE) without needing a Q-function over a huge discrete space (unlike DQN).
3. GRPO samples K completions per prompt and uses the per-prompt average reward as a baseline: $b(s_i) = \frac{1}{K} \sum_k r(s_i, a_k)$. This is possible for LLMs because you can “teleport” back to any state – just feed the same prompt again and sample a new completion. In robotics, you cannot reset the physical world to the exact same state on demand, so you need a learned value function baseline instead.
4. Without the KL penalty: (1) **Reward hacking** – the model can produce gibberish or degenerate text that exploits weaknesses in the reward model to get high scores. (2) **Mode collapse / forgetting** – the model can lose its language modeling capabilities entirely, producing incoherent text. The KL penalty $-\beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$ is equivalent to max-entropy RL (Chapter 9) with a non-uniform action prior π_{ref} instead of uniform. The entropy bonus $\mathcal{H}(\pi)$ in max-ent RL is the special case where π_{ref} is uniform.
5. **Stage 1 (SFT):** Input = (prompt, desired response) demonstration pairs. Optimize cross-entropy loss. Output = π_{SFT} , which also serves as π_{ref} . **Stage 2 (Reward Model):** Input = pairwise preference comparisons from human annotators on completions sampled from π_{SFT} . Optimize Bradley-Terry log-likelihood (Eq. 10.22). Output = reward model r_ψ . **Stage 3 (RL):** Input = prompts from dataset + reward model r_ψ . Optimize KL-regularized reward $\bar{r} = r_\psi - \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$ via PPO or GRPO. Output = final aligned policy π_θ .

Chapter 12

Model-Based Reinforcement Learning

Lectures 15-16 (March 11 & 13, 2026). Discussion Section 8.

12.1 Motivation: Why Learn a Model?

In **model-free RL** (policy gradients, actor-critic, Q-learning), we treat the environment as a black box: we never attempt to learn or represent the transition dynamics $p(s'|s, a)$. Instead, we directly learn a policy or value function through trial and error.

Model-based RL takes a different approach: learn a predictive model of the environment (a “learned simulator”), then use that model to derive a policy or plan.

Intuition

Think of model-free RL as learning to play chess purely through trial and error, never building any mental model of how pieces move. Model-based RL is like first learning the rules of chess, then reasoning about moves using those rules. If you can learn the rules accurately, you can potentially learn far more efficiently because you can “think ahead” without actually playing.

Key advantages of model-based RL:

1. **Sample efficiency:** If the model is accurate, you can generate unlimited synthetic experience, reducing the amount of real-world interaction needed.
2. **Leveraging prior data:** You can pre-train the model on data from other policies (π_β), then use it for planning or policy learning.
3. **Programmatic access:** Unlike black-box interaction, you can differentiate through the model, perform tree search, etc.

Key challenges:

1. The data distribution (from π_β) may not cover the states visited by the learned policy – **distributional shift**.
2. Model architecture design is domain-dependent (e.g., images vs. low-dimensional states).

3. Planning or RL inside a learned model can be expensive and susceptible to local optima.

12.2 The Basic Model-Based RL Loop

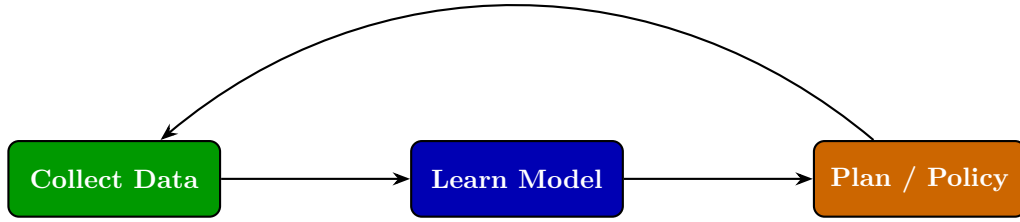


Figure 12.1: Basic Model-Based RL Loop: collect data, learn model, plan or improve policy.

The fundamental structure of model-based RL is a three-step procedure, often iterated:

The Model-Based RL Loop

1. **Step 1:** Collect data using π_β
2. **Step 2:** Learn dynamics model f from data $\mathcal{D}$
3. **Step 3:** Use model to recover policy π_f
4. **Step 4 (iterate):** Set $\pi_\beta \leftarrow \pi_f$, collect more data, return to Step 1

12.2.1 Step 1: Collect data

Gather transitions (s, a, s') using some behavior policy π_β . The quality and coverage of this data is critical.

12.2.2 Step 2: Learn the model

Train a function $f_\theta(s, a)$ to predict s' (deterministic case) or a distribution $p_\theta(s'|s, a)$ (stochastic case).

Deterministic model loss:

$$\min_{\theta} \mathcal{L}(\theta) = \sum_{i=1}^N \|f_\theta(s_i, a_i) - s'_i\|^2 \quad \dots(12.1)$$

Stochastic model loss (maximum likelihood):

$$\min_{\theta} \mathcal{L}(\theta) = - \sum_{i=1}^N \log p_\theta(s'_i | s_i, a_i) \quad \dots(12.2)$$

12.2.3 Step 3: Recover a policy

Use the learned model as a simulator and run your favorite RL algorithm (e.g., PPO), or use planning methods (Section 12.5).

12.2.4 Step 4: Iterate

Set $\pi_\beta \leftarrow \pi_f$, collect new data under the latest policy, add to dataset, retrain model, and repeat.

Professor's Note

“Retraining the model each iteration helps address distributional shift, analogous to DAgger in imitation learning. If you don’t iterate, the policy will exploit model errors in regions the data doesn’t cover.” (Lecture 15)

12.3 Distributional Shift in Model-Based RL

This is the central statistical challenge and is directly analogous to the distributional shift problem from imitation learning (see Chapter 6).

12.3.1 The Problem

When you train the model on data from π_β , the model is accurate under the state distribution $p_{\pi_\beta}(s)$. But when you optimize a policy π_f against this model, π_f visits a different state distribution $p_{\pi_f}(s)$. The model has no guarantee of accuracy under p_{π_f} .

Worse: the policy actively exploits model errors. Since π_f is optimized to maximize reward under f , if f makes an erroneously optimistic prediction somewhere, π_f will rush to exploit that error. This is analogous to the adversarial examples problem in supervised learning – the policy is essentially finding “adversarial inputs” to the model that produce inflated rewards.

Common Misconception

“Just training a really good model should fix distributional shift.” This is wrong. No matter how good your model is on its training distribution, a sufficiently powerful optimizer will find states/actions where the model makes positive errors. The problem is that the policy is specifically seeking the argmax of a learned function, which biases it toward the function’s largest errors in the positive direction. Even small, seemingly innocent model errors become catastrophic when they are specifically targeted by the policy optimizer.

12.3.2 The Fix: Iterate (Step 4)

Adding Step 4 – collecting data under π_f and retraining the model – addresses distributional shift over time, exactly like DAgger. However, there is a critical subtlety:

Do not train the policy to convergence against the model at each iteration. If you fully optimize π_f under the current model, the resulting policy will be extremely far from the data distribution and will exploit model errors maximally. Instead:

- Take only a few gradient steps (e.g., a few iterations of PPO) per model update.
- Use uncertainty-aware models (Section 12.4) to prevent the policy from venturing too far.

Intuition

Think of it as hiking up a mountain with an imperfect map. If you trust the map completely and run full speed toward where the map says the peak is, you might run off a cliff the map didn't show. Better to take a few careful steps, look around, update your map, then take a few more steps.

12.4 Uncertainty Estimation

To prevent the policy from exploiting model errors, we want the model to “know what it doesn't know.” This requires distinguishing between two fundamentally different types of uncertainty.

12.4.1 Aleatoric vs. Epistemic Uncertainty

Aleatoric (statistical) uncertainty: Inherent randomness in the environment. Even if you had infinite data, this uncertainty would remain. Example: the outcome of rolling a die.

$$\text{Aleatoric uncertainty} = \text{Var}_{s' \sim p(s'|s,a)}[s'] \quad \dots(12.3)$$

Epistemic (model) uncertainty: Uncertainty about which model is correct, due to limited data. This uncertainty can be reduced by collecting more data. Example: uncertainty about whether a coin is fair, given only 3 flips.

$$\text{Epistemic uncertainty} = \text{Var}_{\theta \sim p(\theta|D)}[f_{\theta}(s,a)] \quad \dots(12.4)$$

Common Misconception

“I can measure model uncertainty by looking at the variance of my model's predicted distribution $p_{\theta}(s'|s,a)$.” This is wrong! A model that outputs a mean and variance can be erroneously confident – it might predict zero variance even in regions where it has never seen data. The predicted variance captures aleatoric uncertainty (how noisy the data is), not epistemic uncertainty (how uncertain you are about the model itself). An overfitted model will have zero training error and predict zero variance everywhere, yet its predictions on unseen inputs are completely unreliable.

What we need for model-based RL is epistemic uncertainty, because we want to know: “Is this a region where my model has been trained and is reliable, or is this unexplored territory?”

12.4.2 Bayesian Neural Networks

The theoretically principled approach to epistemic uncertainty is to maintain a distribution over model parameters:

$$p(\theta|D) \propto p(D|\theta)p(\theta) \quad \dots(12.5)$$

A **Bayesian neural network (BNN)** approximates this posterior with a fully factorized Gaussian:

$$q(\theta) = \prod_i \mathcal{N}(\theta_i; \mu_i, \sigma_i^2) \quad \dots(12.6)$$

- Each weight stores two numbers: a mean μ_i and variance σ_i^2 (only 2x the parameters).
- Trained via variational inference (maximize the evidence lower bound / ELBO).
- Theoretically elegant but can be complex to implement and train.

12.4.3 Bootstrap Ensembles

A much more practical approach that is widely used in model-based RL:

Core idea: Train N independent models $f_{\theta_1}, \dots, f_{\theta_N}$ and see if they agree.

$$p(\theta|D) \approx \frac{1}{N} \sum_{i=1}^N \delta(\theta = \theta_i) \quad \dots(12.7)$$

Training procedure:

1. (Theoretical) For each model i , create a bootstrap dataset D_i by sampling from D with replacement, then train f_{θ_i} on D_i .
2. (Practical – what everyone actually does) Just initialize each model with different random weights and train them all on the same dataset D . The different initializations + stochasticity of SGD provide sufficient diversity.

Interpreting ensemble predictions: - If all N models agree on a prediction $\rightarrow$ high confidence (low epistemic uncertainty) - If models disagree $\rightarrow$ low confidence (high epistemic uncertainty)

Professor's Note

“The theoretical version with bootstrap resampling is a very pious thing to do, but nobody ever bothers. All people actually do is initialize 10 copies of their weights with Xavier initialization and train them on the same data. In principle not good enough; in practice, definitely good enough.” (Lecture 15)

12.4.4 Why Uncertainty-Aware Models Prevent Policy Exploitation

When we plan or optimize a policy using an uncertainty-aware model, we **average the reward over the ensemble samples** (or over the posterior). This hedges against uncertainty:

$$J(\pi) = \mathbb{E}_{\theta \sim p(\theta|D)} \left[\mathbb{E}_{\tau \sim \pi, f_\theta} \left[\sum_t r(s_t, a_t) \right] \right] \quad \dots(12.8)$$

If the model predicts that you *might* get something very good in a region but you also *might* fall off a cliff, averaging over these possibilities produces a moderate or low expected reward. The policy is therefore discouraged from visiting highly uncertain regions unless the expected outcome (averaged over model uncertainty) is genuinely good.

Intuition

Walking near a cliff with an uncertain map: if you average over your uncertainty about where the cliff edge is, walking close to the uncertain boundary has low expected reward (because some models predict you fall off). So you stay further back.

12.5 Planning with Learned Models

Once you have a learned model, you can use it to select actions without explicitly learning a policy. This is called **planning**.

12.5.1 Open-Loop vs. Closed-Loop Planning

Open-loop planning: Commit to a full sequence of actions $a_1, \dots, a_H$ upfront, then execute them without observing intermediate states.

$$\mathbf{A}^* = \arg \max_{\mathbf{A}} J(\mathbf{A}) = \arg \max_{a_1, \dots, a_H} \mathbb{E} \left[\sum_{t=1}^H r(s_t, a_t) \right] \quad \dots(12.9)$$

subject to $s_{t+1} = f(s_t, a_t)$ (deterministic) or $s_{t+1} \sim p_\theta(\cdot | s_t, a_t)$ (stochastic).

Closed-loop planning: Observe the state after each action and replan. This is equivalent to finding a policy. Much better in stochastic environments because it can react to new information.

Professor's Note

“Imagine I give you a quiz. In open-loop, you have to commit to your answers before seeing the questions. In closed-loop, you see each question and then answer it. Obviously closed-loop is better whenever there is uncertainty about what you will observe.” (Lecture 16)

Practical compromise: Plan open-loop for a short horizon (e.g., $H=10$ steps), execute a few steps, observe the new state, then replan. This is called **model-predictive control (MPC)** and is very widely used.

12.5.2 Random Shooting

The simplest planning method:

Algorithm: Random Shooting

1. Sample N random action sequences $\{A_i\}_{i=1}^N$
2. For each A_i , roll out the model to compute $J(A_i)$
3. Select $A^* = \arg \max_{A_i} J(A_i)$

Pros: Embarrassingly parallel, no gradients needed, ~3 lines of code. **Cons:** Exponentially worse with dimensionality. Only works for low-dimensional action spaces ($\dim < \sim 30\text{-}50$) and broad reward landscapes.

12.5.3 Cross-Entropy Method (CEM)

An iterative refinement of random shooting:

Algorithm: Cross-Entropy Method (CEM)

Input: Initial distribution $p(A)$ (e.g., Gaussian), number of samples N , number of elites M , number of iterations K

1. **For** $k = 1$ to K **do:**
2. Sample N action sequences $\{A^{(i)}\}_{i=1}^N \sim p(A)$
3. Evaluate $J(A^{(i)})$ for each sequence using the model
4. Select top M sequences (elites): $\{A^{(i_j)}\}_{j=1}^M$ with highest J values
5. Refit $p(A)$ to the elites: $\mu \leftarrow$ mean of elites, $\Sigma \leftarrow$ covariance of elites
6. **End for**
7. **Return:** μ (or best elite)

Key insight: Instead of sampling uniformly, CEM iteratively narrows the search distribution toward high-reward regions. Each iteration, it fits a new Gaussian to the top- M “elite” samples, concentrating future samples near promising areas.

Practical notes: - Typically use a Gaussian distribution with diagonal or full covariance. - 5-10 iterations with $N=100\text{-}1000$ samples often works well. - CMAES (Covariance Matrix Adaptation Evolutionary Strategies) is CEM + momentum.

Intuition

CEM is like a “zoom and enhance” strategy. First, cast a wide net. Then zoom into the region where the best samples were found. Then zoom in again. Each iteration gives you a higher-resolution view of the good region.

12.5.4 Planning with Ensembles

When using a bootstrap ensemble for planning, the objective becomes:

$$J(a_1, \dots, a_H) = \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H r(s_t^{(i)}, a_t) \quad \dots(12.10)$$

where for each sample i , we choose a single model θ_i from the ensemble and roll out the entire trajectory using that model.

Critical detail: Sample a different model for each *trajectory*, not for each *time step*.

Common Misconception

“I should resample a different ensemble member at every time step to get more diversity.” This is wrong. Each θ_i represents a possible world. In one possible world, the left door is locked and the right door is unlocked. In another world, it’s the opposite. But there is no world where the locked/unlocked status randomly changes every time step. Resampling per step creates an impossible dynamics that doesn’t correspond to any consistent world.

Why average, not max? (Quiz question, Lecture 16)

If you take the *maximum* reward over ensemble samples rather than the average, you are being optimistic – choosing the world that gives the best outcome. This defeats the purpose of uncertainty estimation and allows the policy to exploit the most optimistic model, which is exactly the model exploitation problem we are trying to avoid.

12.5.5 Why Backpropagating Through Dynamics is Ill-Conditioned

Since we have a differentiable model f_θ , one might think: “Why not just backpropagate gradients through the model to optimize the policy directly?” This gives what is called the **pathwise gradient**:

$$\nabla_\theta J(\theta) = \sum_{t=1}^H \frac{d\mathbf{a}_t}{d\theta} \frac{ds_{t+1}}{d\mathbf{a}_t} \left(\sum_{t'=t+1}^H \frac{dr_{t'}}{ds_{t'}} \prod_{t''=t+2}^{t'} \frac{ds_{t''}}{d\mathbf{a}_{t''-1}} \frac{d\mathbf{a}_{t''-1}}{ds_{t''-1}} + \frac{ds_{t''}}{ds_{t''-1}} \right) \quad \dots(12.11)$$

This involves a **product of $O(H)$ Jacobian matrices**. Each Jacobian ds_{t+1}/ds_t is a matrix. When you multiply many matrices together:

- If eigenvalues > 1 : the product **explodes** (gradient explosion)
- If eigenvalues < 1 : the product **vanishes** (gradient vanishing)

This is the same problem as vanishing/exploding gradients in RNNs, and it makes the optimization problem **ill-conditioned**. In practice, the policy gradient (likelihood ratio) estimator is often more stable despite its higher variance, because it avoids this Jacobian product.

Professor’s Note

“Generally backprop through learned dynamics does not work well unless you’re in a very special case – dynamics close to linear, eigenvalues close to 1, or very short horizon.” (Lecture 16)

12.6 Dyna-Style Methods: Short Model Rollouts

12.6.1 The Curse of Long Rollouts

When generating trajectories through a learned model, errors compound over time – exactly the same compounding error as in imitation learning (see Chapter 6). The error grows **quadratically** with the horizon H :

$$\text{Total error} \sim O(\epsilon H^2) \quad \dots(12.12)$$

where ϵ is the per-step model error. Doubling the rollout length quadruples the error.

12.6.2 The Dyna Framework

Key insight: Instead of rolling out full trajectories from the initial state, start short rollouts from states sampled from the replay buffer.

Algorithm: Dyna-Style Model-Based RL

1. Collect initial data $\mathcal{D}$ by running π_0
2. **While** not converged **do**:
3. Train dynamics model f_θ on $\mathcal{D}$
4. **For** K steps **do**:
5. Sample state s from $\mathcal{D}$ (replay buffer)
6. Use π_θ to select action a
7. Simulate $s' \sim f_\theta(\cdot|s, a)$, $r = r(s, a)$
8. (Optionally repeat for N more steps to get a short rollout of length N)
9. Add simulated transitions to model buffer $\mathcal{D}_{\text{model}}$
10. **End for**
11. Train policy π_θ using BOTH real data $\mathcal{D}$ AND model data $\mathcal{D}_{\text{model}}$ with off-policy RL (e.g., SAC, Q-learning)
12. Run π_θ in real environment, add to $\mathcal{D}$
13. **End while**

Why this works: - Short rollouts (1-10 steps) have low compounding error. - Starting from replay buffer states means you still see diverse situations, including those from later in trajectories. - Mixing real and model data helps the policy hedge – it can't fully exploit model errors because it also trains on real data.

Intuition

Rather than trying to simulate an entire game of chess from the opening move (which would accumulate huge errors), just simulate 2-3 moves ahead from positions you've actually encountered in real games. This gives you useful planning signal with minimal error accumulation.

12.6.3 Parallel Process View

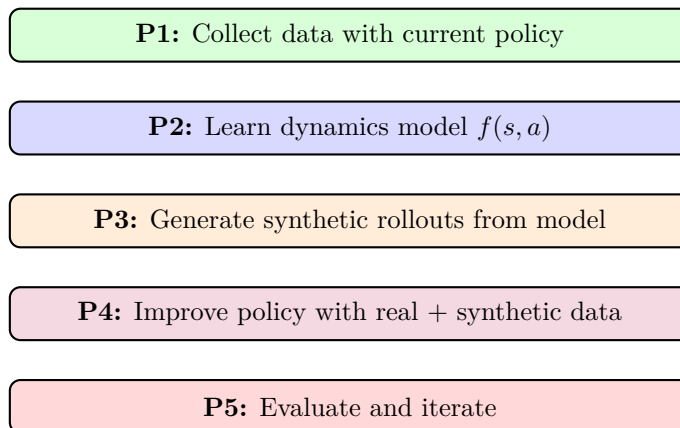


Figure 12.2: Dyna: Parallel process view of model-based RL.

A practical implementation looks like a set of parallel processes:

- **Process 1 – Data Collection:** Periodically run π in real environment, add to buffer $\mathcal{D}$
- **Process 2 – Target Network Updates:** Periodically sync target networks
- **Process 3 – RL Training:** Sample from both $\mathcal{D}$ and $\mathcal{D}_{\text{model}}$, update actor/critic
- **Process 4 – Model Training:** Load from $\mathcal{D}$, update f_{θ}
- **Process 5 – Model Rollouts:** Sample states from $\mathcal{D}$, run short rollouts with f_{θ} , store in $\mathcal{D}_{\text{model}}$

Concrete algorithms following this template include: - **MBA** (Model-Based Acceleration, Gu et al. 2016) - **MVE** (Model-Based Value Expansion, Feinberg et al. 2018) - **MBPO** (Model-Based Policy Optimization, Janner et al. 2019)

They differ in rollout length, which policy generates rollouts, and how model vs. real data are mixed.

12.7 Latent State Space Models / Sequence VAEs

12.7.1 Motivation

For high-dimensional observations (e.g., images), learning dynamics directly in observation space is difficult: - Pixel-level predictions are expensive and often inaccurate. - Pixel representations are non-smooth (object moving one pixel causes discontinuous changes). - Full state information may not be available from a single observation (partial observability / POMDPs).

Solution: Learn a compact latent state space z and model dynamics in that space.

12.7.2 The Variational Framework

This is a direct application of the VAE framework (see Chapter 9) to sequential data:

Observed variables: Observations $o_1, \dots, o_H$

Latent variables: States $z_1, \dots, z_H$

Prior (dynamics):

$$p(z_1, \dots, z_H | a_1, \dots, a_{H-1}) = p(z_1) \prod_{t=1}^{H-1} p_\phi(z_{t+1} | z_t, a_t) \quad \dots (12.13)$$

Key point: The prior is **conditioned on actions**. You get a different distribution over state sequences depending on what actions were taken. This is different from a standard VAE where the prior is fixed (e.g., $N(0, I)$). Here, the transition probabilities $p_\phi(z_{t+1} | z_t, a_t)$ are learned as part of the prior.

Decoder (observation model):

$$p(o_1, \dots, o_H | z_1, \dots, z_H) = \prod_{t=1}^H p_\psi(o_t | z_t) \quad \dots (12.14)$$

Each observation depends only on the corresponding state (conditional independence given states – this enforces the Markov property).

Encoder (approximate posterior): Several choices:

Encoder Type	Input	Accuracy	Complexity
Full smoothing posterior	All $o_1, \dots, o_H$ and $a_1, \dots, a_{H-1}$	Highest	Highest
Filtering posterior	$o_1, \dots, o_t$ and $a_1, \dots, a_{t-1}$	Medium	Medium
Single-step encoder	o_t only	Lowest	Lowest

12.7.3 The ELBO for Sequence Models

The training objective is the Evidence Lower Bound (ELBO), with the sequential structure plugged in:

$$\mathcal{L} = \mathbb{E}_{q(z_{1:H} | o_{1:H}, a_{1:H-1})} \left[\sum_t \log p_\psi(o_t | z_t) + \sum_t \log p_\phi(z_{t+1} | z_t, a_t) + H[q] \right] \quad \dots (12.15)$$

- First term: reconstruction quality (decoder)
- Second term: dynamics consistency (prior)
- Third term: encoder entropy (prevents posterior collapse)

12.7.4 Deterministic Encoder (Simplified Version)

For pure representation learning (when you don't care about uncertainty), use a deterministic encoder $g(o_t) \rightarrow z_t$. The ELBO simplifies to:

$$\mathcal{L} = \sum_t \log p_\phi(g(o_{t+1})|g(o_t), a_t) + \sum_t \log p_\psi(o_t|g(o_t)) \quad \dots(12.16)$$

This is just: (1) learn an autoencoder, and (2) learn dynamics in the latent space. No reparameterization trick needed – pure backprop.

12.7.5 Using Latent Models for RL

Once trained, the latent model can be used with any model-based RL algorithm:

1. Encode observations into latent states: $z_t = g(o_t)$
 2. Evaluate target values: $y = r(z_t, a_t) + \gamma \cdot V(z_{t+1})$
 3. Update Q-function and policy in latent space
 4. Optionally: generate synthetic rollouts in latent space for Dyna-style augmentation
-

12.8 Check Your Understanding

1. **Why does retraining the model each iteration help with distributional shift?** What is the analogy to imitation learning?
2. **Explain why an uncertainty-aware model prevents the policy from exploiting model errors.** What would happen if you used the maximum predicted reward across ensemble members instead of the average?
3. **You have a stochastic environment. Should you use open-loop or closed-loop planning? Why?** Give a concrete example where open-loop planning would fail badly.
4. **In a Dyna-style algorithm, why do we start model rollouts from states in the replay buffer rather than from the initial state?** What would go wrong with full-length rollouts?
5. **In a latent state space model, why is the prior conditioned on actions?** What would the prior represent if actions were omitted?

Answers

1. Retraining the model on data from the current policy π_f makes the model accurate under $p_{\pi_f}(s)$, the states the policy actually visits. This is analogous to DAgger in imitation learning, where the expert relabels states visited by the learned policy. Without iteration, the model is only accurate under $p_{\pi_\beta}(s)$, and the optimized policy exploits model errors in unvisited regions.
2. When planning with an ensemble, we average reward over multiple models. If one model predicts a great outcome and another predicts a terrible one, the average is mediocre, discouraging the policy from visiting that uncertain region. Using the *maximum* instead of the average would be optimistic – it picks the most favorable model, which is exactly the model exploitation problem we are trying to prevent. The policy would rush toward regions where at least one model is erroneously optimistic.

3. In stochastic environments, open-loop planning fails because you commit to actions before observing outcomes. Example: navigating a T-intersection where traffic randomly comes from left or right. Open-loop must commit to “go left” or “go right” before seeing the traffic. Closed-loop observes traffic first and reacts. Use closed-loop (or MPC as a practical compromise) in stochastic environments.
4. Full-length model rollouts accumulate compounding error $O(\epsilon H^2)$, becoming unreliable for long horizons. Starting from replay buffer states keeps rollouts short (1-10 steps) with low error, while still covering diverse situations from different trajectory stages. It is the same principle as why behavior cloning suffers less from compounding error on short horizons.
5. The prior is conditioned on actions because different action sequences produce different state trajectories. The prior $p_\phi(z_{t+1}|z_t, a_t)$ represents the learned dynamics – how the latent state evolves given an action. Without action conditioning, the prior would represent the unconditional distribution of next states (marginalizing over all possible actions), which is not useful for planning or control since you could not reason about the consequences of specific actions.

Chapter 13

Offline Reinforcement Learning

Lectures 17-18 (March 18 & 20, 2026). Discussion Section 9.

13.1 What is Offline RL?

Offline RL (also called **batch RL**) learns a policy from a fixed, pre-collected dataset without any further interaction with the environment.

- **On-Policy RL:** collect $\rightarrow$ improve $\rightarrow$ discard $\rightarrow$ collect $\rightarrow \dots$
- **Off-Policy RL:** collect $\rightarrow$ buffer $\rightarrow$ improve $\rightarrow$ collect more $\rightarrow \dots$
- **Offline RL:** given fixed buffer $\mathcal{D} \rightarrow$ improve (NO more collection)

The dataset $D = \{(s_i, a_i, r_i, s'_i)\}_{i=1}^N$ was collected by some unknown **behavior policy** π_β . We do NOT assume π_β is known, optimal, or even a single policy – D might come from a mixture of sources.

13.1.1 Why Study Offline RL?

1. **Safety-critical domains:** Autonomous driving, healthcare – you can't deploy untested policies.
2. **Expensive interaction:** Robotics, long-horizon tasks where data collection takes hours/years.
3. **Leveraging existing data:** Historical logs, human demonstrations, previously collected datasets.
4. **Offline pre-training for online fine-tuning:** Use offline RL to get a good initialization, then fine-tune with online RL.

13.1.2 How is Offline RL Different from Imitation Learning?

Imitation learning matches the data distribution (minimize divergence from expert behavior). **Offline RL maximizes reward** (find the best policy, even if it differs from the data).

In imitation learning, you assume the data is near-optimal and try to copy it. In offline RL, the data may be highly suboptimal, and you need to extract the best possible behavior by: 1. **Filtering**: Identifying which parts of the data are good vs. bad. 2. **Stitching**: Combining good sub-behaviors from different trajectories (the “transitive property” – if you’ve seen $A \rightarrow B$ and $B \rightarrow C$, deduce you can do $A \rightarrow B \rightarrow C$).

Professor’s Note

“Imitation learning is like copying everything your teacher does. Offline RL is like watching many different people attempt a task – some succeed, some fail – and figuring out which actions lead to success, then combining the best parts.” (Lecture 17)

13.2 Why Standard Q-Learning Fails Offline: OOD Action Overestimation

This is the central challenge of offline RL. Let’s see why naively applying off-policy methods (like SAC or DQN) to a fixed dataset fails catastrophically.

13.2.1 The Problem

In Q-learning, the Bellman backup is:

$$Q(s, a) \leftarrow r(s, a) + \gamma \mathbb{E}_{s' \sim p(s'|s, a)} \left[\max_{a' \in \mathcal{A}} Q(s', a') \right] \quad \dots(13.1)$$

The Q-function is trained on states and actions from π_β (the training distribution). But the max over a' queries the Q-function at actions chosen to maximize Q – these actions may be **out-of-distribution (OOD)**, i.e., very different from anything in D .

Illustration: OOD Action Overestimation

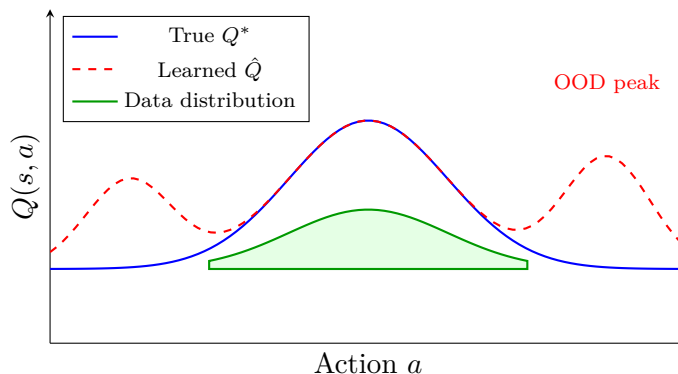


Figure 13.1: OOD Action Overestimation: learned Q-function has spurious peaks outside the data distribution.

- The true $Q(s, a)$ has well-defined peaks and valleys within the data distribution.
- The learned $Q(s, a)$ matches the true function within the in-distribution region, but develops **spurious peaks** in the out-of-distribution (OOD) region where no training data exists.
- The policy maximizes Q , so it selects the spurious OOD peak. This erroneously high value gets backed up through the Bellman equation, infecting the entire Q -function.

13.2.2 Empirical Evidence

Running SAC in offline mode (no data collection) on standard benchmarks: - **Actual reward:** stays around -250 throughout training - **Predicted Q-values:** explode to 10^{20} (!)

The Q -function becomes delusionally optimistic about OOD actions, while the policy achieves nothing.

13.2.3 This Affects All RL Methods

This is not just a Q -learning problem:

Method	Where OOD Problem Shows Up
Q -learning / Actor-critic	Policy selects OOD actions that have inflated Q values
Model-based RL	Policy visits OOD states where model makes erroneously good predictions
Importance-sampled policy gradient	Importance weights explode for OOD actions, variance becomes infinite

Common Misconception

“The distributional shift problem only affects value-based methods.” Wrong. Every RL method has some version of this problem. Value-based methods overestimate Q -values for OOD actions. Model-based methods make inaccurate predictions for OOD states. Policy gradient methods suffer from importance weight explosion. The solutions differ, but the underlying issue – counterfactual queries about unseen actions – is universal.

13.2.4 Why Q -Function Methods are Preferred for Offline RL

Despite the OOD challenge, Q -function-based methods are generally preferred for offline RL because:

1. **Off-policy by design:** Q -learning and actor-critic methods can learn from data collected by any policy, making them natural fits for the offline setting (cf. Chapter 8).
2. **No importance weights needed:** Unlike importance-sampled policy gradients, Q -function methods don’t require knowing π_β to compute importance ratios.
3. **Sample reuse:** Every transition can be used multiple gradient steps, unlike on-policy methods.

The key is to fix the OOD overestimation problem while retaining these benefits. The rest of this chapter covers three broad approaches to doing so.

13.3 Approach 1: Explicit Policy Constraint Methods

Core idea: Constrain the learned policy π_θ to stay “close” to the behavior policy π_β , preventing it from selecting OOD actions.

Explicit vs. Implicit Policy Constraints (Lecture 18). Offline RL methods fall into two categories based on *how* they avoid OOD actions:

- **Explicit policy constraints** (this section, §13.3): directly add a regularizer $D(\pi_\theta, \pi_\beta)$ to the actor loss or reward. You need to estimate π_β (via behavioral cloning or similar), and the constraint is visible in the objective. Examples: AC+BC, AWR/AWAC, BEAR, BRAC.
- **Implicit policy constraints** (§13.4, §13.5): never write down $D(\pi_\theta, \pi_\beta)$. Instead, modify the value-learning procedure so that the resulting policy is “automatically” close to π_β . Examples: IQL (never queries OOD actions), CQL (pushes OOD Q-values down).

“Instead of explicitly adding a regularizer to the policy like getting it to maximize the constraint, we do something implicit that results in a policy that by construction has to satisfy the constraint.” — Sergey Levine, Lecture 18

13.3.1 The Constrained Actor Objective

Start with the standard actor-critic objective and add a constraint:

$$\max_{\theta} \mathbb{E}_{s \sim D, a \sim \pi_\theta(\cdot|s)}[Q(s, a)] \quad \text{subject to} \quad D(\pi_\theta(\cdot|s), \pi_\beta(\cdot|s)) \leq \epsilon \quad \forall s \quad \dots(13.2)$$

where D is some divergence measure. The choice of divergence has significant implications.

13.3.2 Forward KL vs. Reverse KL

KL divergence is asymmetric, so there are two choices:

Forward KL (mode-covering):

$$D_{KL}(\pi_\beta || \pi_\theta) = \mathbb{E}_{a \sim \pi_\beta(\cdot|s)} \left[\log \frac{\pi_\beta(a|s)}{\pi_\theta(a|s)} \right] \quad \dots(13.3)$$

- If π_θ assigns zero probability to any action that π_β produces $\rightarrow$ loss = infinity
- Forces π_θ to *cover all modes* of π_β
- Equivalent to maximum likelihood / behavioral cloning loss (up to constant)
- **Does NOT require knowing** π_β – just need samples from it

Reverse KL (mode-seeking):

$$D_{KL}(\pi_\theta \parallel \pi_\beta) = \mathbb{E}_{a \sim \pi_\theta(\cdot|s)} \left[\log \frac{\pi_\theta(a|s)}{\pi_\beta(a|s)} \right] \quad \dots(13.4)$$

- If π_θ assigns non-zero probability to any action where $\pi_\beta = 0 \rightarrow$ loss = infinity
- Forces π_θ to *seek the best mode* of π_β
- Expands to: $-H[\pi_\theta] - \mathbb{E}_{\pi_\theta}[\log \pi_\beta]$
- **Requires estimating** π_β (or its log-density)

Professor's Note

“On the surface, reverse KL seems preferred because it prevents taking any action with zero probability under π_β . But forward KL is often used in practice because it’s much easier to implement – it’s literally a one-line change to add $\log \pi_\theta$ of data samples to your actor loss.” (Lecture 18)

Which is better for offline RL?

Reverse KL is theoretically preferable (it avoids OOD actions more directly), but forward KL is simpler to implement. In practice, the ideal constraint is actually neither – it’s a **support constraint** that says “any action is fine as long as it has non-trivial probability under π_β .” But support constraints are very hard to implement in practice.

13.3.3 Reverse KL is Convenient: Add $\log \pi_\beta$ to Reward

A key insight is that the reverse KL constraint can be implemented by **modifying the reward function** in a max-entropy RL framework:

$$\tilde{r}(s, a) = r(s, a) + \lambda \log \pi_\beta(a|s) \quad \dots(13.5)$$

Derivation: The reverse KL divergence $D_{KL}(\pi_\theta \parallel \pi_\beta)$ expanded is:

$$D_{KL}(\pi_\theta \parallel \pi_\beta) = \mathbb{E}_{\pi_\theta}[\log \pi_\theta(a|s)] - \mathbb{E}_{\pi_\theta}[\log \pi_\beta(a|s)] \quad \dots(13.6)$$

The first term is the negative entropy $-H[\pi_\theta]$, which is already handled in max-entropy RL (SAC). The second term is just $-\mathbb{E}_{\pi_\theta}[\log \pi_\beta(a|s)]$, which can be added to the reward.

When added to the reward (not just the actor objective), the constraint accounts for **future divergences** too: the policy is discouraged from going to states where it would *want* to take OOD actions in the future.

13.3.4 AC+BC (Actor-Critic + Behavioral Cloning)

The simplest policy constraint method, using forward KL:

Actor objective:

$$\max_{\theta} \mathbb{E}_{s \sim D, a \sim \pi_{\theta}(\cdot|s)}[Q(s, a)] + \lambda \mathbb{E}_{(s, a) \sim D}[\log \pi_{\theta}(a|s)] \quad \dots(13.7)$$

The second term is just the behavioral cloning loss on the dataset.

Algorithm: AC+BC (Actor-Critic + Behavioral Cloning)

Input: Dataset $\mathcal{D}$, hyperparameter λ

Repeat:

1. Sample batch $\{(s_i, a_i, r_i, s'_i)\}$ from $\mathcal{D}$
2. Compute target values: $y_i = r_i + \gamma \cdot Q_{\text{target}}(s'_i, a')$, where $a' \sim \pi_{\theta}(\cdot|s'_i)$
3. Update Q by minimizing: $\mathcal{L}_Q = \frac{1}{B} \sum_i (Q(s_i, a_i) - y_i)^2$
4. Update actor by maximizing: $\mathcal{L}_{\text{actor}} = \mathbb{E}_{a \sim \pi_{\theta}(\cdot|s)}[Q(s, a)] + \lambda \cdot \mathbb{E}_{(s, a) \sim \mathcal{D}}[\log \pi_{\theta}(a|s)]$
5. **Until converged**

Variants: SAC+BC, DDPG+BC, TD3+BC – just add the BC term to the existing actor loss.

Pros: Extremely simple (half a line of code change). No need to estimate π_{β} . **Cons:** Forward KL is mode-covering, which may spread probability mass over suboptimal modes. Sensitive to λ .

13.3.5 Advantage-Weighted Regression (AWR)

AWR uses an **implicit** policy constraint via importance-weighted behavioral cloning.

Derivation: Start with the constrained optimization (reverse KL):

$$\max_{\pi} \mathbb{E}_{s \sim D, a \sim \pi(\cdot|s)}[Q(s, a)] \quad \text{s.t.} \quad D_{KL}(\pi(\cdot|s) \parallel \pi_{\beta}(\cdot|s)) \leq \epsilon \quad \dots(13.8)$$

Solving via Lagrangian duality yields the optimal (non-parametric) policy:

$$\pi^*(a|s) \propto \pi_{\beta}(a|s) \exp\left(\frac{1}{\lambda} A(s, a)\right) \quad \dots(13.9)$$

where $A(s, a) = Q(s, a) - V(s)$ is the advantage and λ is the Lagrange multiplier.

Intuition

The optimal constrained policy is the behavior policy reweighted by the exponentiated advantage. Actions with high advantage get upweighted; actions with low advantage get downweighted. It's like selecting the best parts of the data and imitating those more.

Since we can't directly sample from π^* , we train π_{θ} to match it via importance-weighted behavioral cloning:

$$\max_{\theta} \mathbb{E}_{(s, a) \sim D} \left[\exp\left(\frac{1}{\lambda} A(s, a)\right) \log \pi_{\theta}(a|s) \right] \quad \dots(13.10)$$

Algorithm: Advantage-Weighted Regression (AWR / AWAC)

Input: Dataset $\mathcal{D}$, temperature λ

Repeat:

1. Sample batch $\{(s_i, a_i, r_i, s'_i)\}$ from $\mathcal{D}$
2. Update Q -function (standard Bellman regression): $\mathcal{L}_Q = \frac{1}{B} \sum_i (Q(s_i, a_i) - y_i)^2$
3. Update V -function (optional): $\mathcal{L}_V = \frac{1}{B} \sum_i (V(s_i) - \mathbb{E}_{a \sim \pi} [Q(s_i, a)])^2$, or use single-sample estimate $V(s) \approx Q(s, a')$ where $a' \sim \pi_\theta(\cdot|s)$
4. Compute advantages: $A_i = Q(s_i, a_i) - V(s_i)$
5. Update actor (weighted max-likelihood): $\mathcal{L}_{\text{actor}} = \sum_i \exp(A_i/\lambda) \cdot \log \pi_\theta(a_i|s_i)$
6. **Until converged**

Pros: Very stable (it's just weighted supervised learning). Simple to implement. **Cons:** - Bad actions get weight ~ 0 , so they effectively “disappear” rather than being actively avoided. This means AWR learns slowly because it only has positive signal. - The derivation uses reverse KL for the constraint but forward KL for the projection to π_θ – a “weird middle ground” between mode-seeking and mode-covering. - Can struggle to improve significantly over the behavior policy.

Professor's Note

“AWR is great if you want imitation learning plus plus – a slightly improved version of imitation learning. Very stable, easy to get working, but can learn pretty slowly and struggle to improve over the behavior policy.” (Lecture 18)

13.4 Approach 2: Implicit Policy Constraints — Implicit Q-Learning (IQL)

13.4.1 Core Idea: In-Sample Bellman Backup

This is an implicit policy constraint method. Rather than adding a divergence term $D(\pi_\theta, \pi_\beta)$ to the objective, IQL structurally avoids OOD actions in the value-learning process — the constraint emerges “by construction.”

IQL takes a radically different approach: **never query the Q-function with out-of-distribution actions.** Every action that appears in the Q-function update comes directly from the dataset.

Standard Q-learning target: $Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a')$ – the max over ALL actions causes OOD problems.

IQL target: $Q(s, a) \leftarrow r + \gamma V(s')$ – the V function implicitly approximates the max using only in-sample data.

13.4.2 Expectile Regression

The key innovation: instead of training $V(s)$ with mean squared error (which gives the *mean* of Q-values), use an **asymmetric loss** that gives an *upper expectile*:

$$L_\tau^2(u) = \begin{cases} \tau \cdot u^2 & \text{if } u \geq 0 \quad (\text{underestimation penalty}) \\ (1 - \tau) \cdot u^2 & \text{if } u < 0 \quad (\text{overestimation penalty}) \end{cases} \quad \dots(13.11)$$

where $u = V(s) - Q(s, a)$, and $\tau \in (0.5, 1)$ controls how aggressively V targets the upper end.

V-function loss:

$$\min_V \mathbb{E}_{(s,a) \sim D} [L_\tau^2(V(s) - Q(s, a))] \quad \dots(13.12)$$

- When $\tau = 0.5$: standard MSE, V learns the *mean* (i.e., $V = \mathbb{E}[Q]$ = value of behavior policy).
- When $\tau \rightarrow 1$: V learns the *maximum* Q-value seen in the data at each state.

Intuition

Imagine at state s , the data contains two actions: one leading to $Q = +10$, another to $Q = -10$. Standard MSE gives $V = 0$ (the mean). With $\tau = 0.9$, the loss penalizes underestimation 9x more than overestimation, so V shifts toward $+10$ (say, ~ 9.7). This implicitly finds the best action without ever querying the Q-function at novel actions.

13.4.3 Why This Isn't Over-Optimistic

The asymmetric loss only applies to the V regression. The Q-function still uses standard MSE regression onto $r + \gamma V(s')$. So:

- **V** is optimistic over *actions* (finds the upper expectile of Q across actions in the data)
- **Q** is risk-neutral over *transitions* (takes the expected value over next states)

This matches exactly what we want in RL: maximize over actions, expectation over transitions. And since V only sees actions from the dataset, there is no OOD action querying.

13.4.4 The Complete IQL Algorithm

Algorithm: Implicit Q-Learning (IQL)

Input: Dataset $\mathcal{D}$, expectile $\tau \in (0.5, 1)$

Repeat:

1. Sample batch $\{(s_i, a_i, r_i, s'_i)\}$ from $\mathcal{D}$
2. Compute target values: $y_i = r_i + \gamma \cdot V_\psi(s'_i)$
3. Update Q-function (standard MSE): $\mathcal{L}_Q = \frac{1}{B} \sum_i (Q_\phi(s_i, a_i) - y_i)^2$
4. Update V-function (expectile regression): $\mathcal{L}_V = \frac{1}{B} \sum_i L_\tau^2(V_\psi(s_i) - Q_\phi(s_i, a_i))$, where

$$L_\tau^2(u) = |\tau - \mathbf{1}(u < 0)| \cdot u^2$$

5. Extract policy (e.g., via AWR): $\mathcal{L}_\pi = \frac{1}{B} \sum_i \exp\left(\frac{Q(s_i, a_i) - V(s_i)}{\lambda}\right) \cdot \log \pi_\theta(a_i | s_i)$

6. **Until converged**

Key properties: - π_θ **never appears in the critic update** (lines 3-5). The critic is completely independent of the actor. - **No expectations under π_θ anywhere** except in the final policy extraction step (line 6). - Can train the critic first, then extract the policy separately.

Challenges: Policy extraction (step 6) is the weak link. AWR has the issues discussed earlier. Using SAC-style actor objectives re-introduces the OOD problem. This is an area of active research.

13.5 Approach 3: Conservative Q-Learning (CQL)

13.5.1 Core Idea: Push Down OOD Action Values

Instead of constraining the policy (Approach 1) or avoiding OOD queries (Approach 2), CQL directly modifies the Q-function so that OOD actions have low Q-values.

Inspiration from adversarial training: The OOD overestimation problem is like adversarial examples – the policy finds “adversarial actions” that fool the Q-function into high predictions. CQL is inspired by adversarial training: find those adversarial actions and explicitly reduce their Q-values.

13.5.2 CQL Objective (Version 1 – Naive)

$$\min_Q \underbrace{\alpha \mathbb{E}_{s \sim D} \left[\max_{\mu} \mathbb{E}_{a \sim \mu(\cdot|s)} [Q(s, a)] \right]}_{\text{push down Q for adversarial actions}} + \underbrace{\mathbb{E}_{(s, a, s') \sim D} [(Q(s, a) - y)^2]}_{\text{standard Bellman error}} \quad \dots(13.13)$$

The first term finds actions μ that have high Q-values and pushes them down. Problem: this also pushes down Q-values for *in-distribution* actions, leading to systematic underestimation.

13.5.3 CQL Objective (Version 2 – Improved)

Add a term that pushes Q-values **up** for actions in the dataset:

$$\min_Q \underbrace{\alpha \mathbb{E}_{s \sim D} \left[\mathbb{E}_{a \sim \mu(\cdot|s)} [Q(s, a)] \right]}_{\text{push down OOD}} - \underbrace{\alpha \mathbb{E}_{(s, a) \sim D} [Q(s, a)]}_{\text{push up in-distribution}} + \underbrace{\mathbb{E}_{(s, a, s') \sim D} [(Q(s, a) - y)^2]}_{\text{Bellman error}} \quad \dots(13.14)$$

How this works: - If μ selects an action **far from the data**: push-down term activates, push-up term does not cancel it -> Q-value is reduced. - If μ selects an action **close to data**: push-down and push-up terms roughly cancel -> Q-value is unaffected.

Theoretical guarantee: With large enough α , the expected Q-value under the policy is guaranteed to be a lower bound on the true Q-value. This means CQL is **pessimistic** – it would rather underestimate than overestimate, which prevents the catastrophic overestimation spirals.

13.5.4 Practical Implementation

For the adversarial distribution μ , add entropy regularization: μ proportional to $\exp(Q)$.

Discrete actions: The expectation under μ becomes the **log-sum-exp** of Q-values (no explicit μ needed):

$$\mathbb{E}_{a \sim \mu}[Q(s, a)] = \log \sum_a \exp(Q(s, a)) \quad \dots(13.15)$$

This is a “soft maximum” – it penalizes the overall magnitude of Q-values, with more penalty on the largest values.

Continuous actions: Sample actions from the current policy, weight by importance weights ($\exp(Q)$).

Algorithm: Conservative Q-Learning (CQL)

Input: Dataset $\mathcal{D}$, conservatism coefficient α

Repeat:

1. Sample batch $\{(s_i, a_i, r_i, s'_i)\}$ from $\mathcal{D}$
2. Compute target values: $y_i = r_i + \gamma \cdot \mathbb{E}_{a' \sim \pi}[Q_{\text{target}}(s'_i, a')]$
3. Update Q-function with CQL regularizer: $\mathcal{L}_Q = \frac{1}{B} \sum_i (Q(s_i, a_i) - y_i)^2 + \alpha \cdot [\log \sum_a \exp(Q(s_i, a))]$ (discrete) or $\alpha \cdot \frac{1}{B} \sum_i Q(s_i, a_i)$
4. Update policy (standard actor-critic): $\mathcal{L}_\pi = -\mathbb{E}_{a \sim \pi_\theta(\cdot|s)}[Q(s, a)]$
5. **Until converged**

Professor’s Note

“CQL methods have the most elegant theoretical properties among offline RL methods – you can prove bounded regret. But they can be more complex to code up than IQL or AC+BC. If you just want a convenient method, try IQL. If you want to prove theorems, study CQL.” (Lecture 18)

13.5.5 Flow Matching + Q Ranking (“Flow Q-Learning”)

Lecture 18 also discussed a practical way to combine **expressive action models** (diffusion / flow matching) with offline Q-functions. The key point is that the flow model is **not** usually trained directly with the RL objective. Instead:

1. Train a critic with an offline RL method that avoids catastrophic OOD behavior (e.g., IQL).
2. Train a flow-matching policy $\pi_{\text{flow}}(a|s)$ on the dataset with pure behavioral cloning, so $\pi_{\text{flow}} \approx \pi_\beta$.
3. At test time, sample several candidate actions from π_{flow} and pick the one with the highest Q-value:

$$a^*(s) = \arg \max_{i \in \{1, \dots, N\}} Q(s, a^{(i)}), \quad a^{(i)} \sim \pi_{\text{flow}}(\cdot | s) \quad \dots(13.16)$$

This is often described as **best-of-N**, **sample-and-rank**, **rejection sampling**, or **guess-and-check**. It is not a separate Bellman backup like IQL or CQL; rather, the critic is trained by one offline RL method, and the flow model is used as a powerful **action proposal distribution**.

Intuition

A Gaussian actor tends to spread probability mass into low-density regions between modes. A flow-matching model can represent the multimodal behavior distribution much more faithfully. Then the Q-function only has to **rank plausible actions from the dataset support**, instead of extrapolating to arbitrary OOD actions.

Strengths:

- Handles multimodal continuous action distributions much better than a simple Gaussian actor.
- Avoids backpropagating the RL objective through the full diffusion / flow-matching sampling process.
- Very simple to use in practice if you already have an offline critic and a flow model.

Weaknesses:

- Action selection is a search procedure over sampled candidates, not a direct optimizer.
- Can scale poorly with action dimension because the best action may require many samples to discover.
- Performance depends on both pieces being good: the flow model must cover strong modes in the data, and the Q-function must rank them accurately.

More advanced variants distill the flow model into a simpler actor and then optimize that actor with Q-learning / actor-critic objectives, but the core lecture idea is the simple **sample plausible actions, then rank with Q** recipe above.

13.6 Model-Based Offline RL

13.6.1 Why It's Challenging

Model-based offline RL faces a double distributional shift problem. You train a model on data from π_β . When you optimize a policy against this model:

1. **OOD actions:** The policy may select actions not seen in the data (same as Q-function methods).
2. **OOD states:** The model rollouts may visit states not in the data, compounding errors.

The OOD *state* problem is especially insidious because: - In value-based methods, the policy only queries Q at states from the dataset (or next-states from the dataset). - In model-based methods, multi-step rollouts can drift into completely novel states where the model is unreliable.

Common Misconception

“Model-based offline RL should work better than value-based offline RL because models give you more information.” In practice, value-based methods are often preferred for offline RL because they only need single-step predictions (Q-values), whereas models need to be accurate over multi-step rollouts. The compounding error problem (Section 12.6.1) is particularly severe when you can’t correct it by collecting more data.

13.6.2 Solutions

The same principles apply: - **Policy constraints** to keep rollouts close to π_β ’s state distribution. - **Pessimistic reward/model penalties** in uncertain regions (using bootstrap ensembles). - **Short rollouts** (Dyna-style) to limit error accumulation.

But model-based offline RL remains challenging because the model can produce OOD states that wouldn’t arise in value-based approaches. Generally, **Q-function-based methods are preferred for pure offline RL**, while model-based approaches shine more in the online or offline-to-online setting where you can correct model errors through additional data collection.

13.7 Summary and Comparison of Offline RL Methods

Method	Principle	Requires π_β ?	Key Advantage	Key Weakness
AC+BC	Forward KL constraint	No (just samples)	Extremely simple	Mode-covering; sensitive to λ
AWR / AWAC	Reverse KL constraint (implicit)	No (importance weights)	Stable; weighted BC	Slow improvement; ignores bad actions
IQL	In-sample Bellman; expectile regression	No	No OOD Q-queries ever	Policy extraction is delicate
CQL	Pessimistic Q regularization	No	Strong theoretical guarantees	More complex; may underestimate
Flow Matching + Q Ranking	BC-fit expressive action model; rank sampled actions with Q	No explicit density (just samples)	Handles multimodality; avoids diffusion-through-Q training	Search cost; limited by candidate set

Professor’s Note

Professor’s Note on the slogan of offline RL: “Maximize return while staying close to the dataset.” Every offline RL method implements some version of this principle. The methods differ in *how* they enforce staying close to the data. (Discussion Section 9)

13.8 Connection to Previous Chapters

- **Distributional shift** (Chapter 6): The same phenomenon from imitation learning reappears. In IL, the policy visits unseen states and takes bad actions. In offline RL, the Q-function or model is evaluated on unseen actions and produces unreliable predictions. The solutions (Dagger / iterative data collection) are analogous to online RL, but in offline RL we cannot collect more data.
 - **Off-policy RL** (Chapter 8): Offline RL is the extreme case of off-policy learning where the behavior policy is completely fixed. The replay buffer techniques, importance sampling, and off-policy actor-critic methods from Chapter 8 are the starting point for offline RL methods.
 - **Maximum entropy RL** (Chapter 9): The reverse KL policy constraint in offline RL is intimately connected to max-entropy RL. Adding $\log \pi_\beta$ to the reward in a SAC-style algorithm yields the reverse KL constraint. The temperature parameter λ plays the same role as the entropy coefficient α in SAC.
-

13.9 Check Your Understanding

1. **Explain the difference between imitation learning and offline RL.** If you have a dataset of expert demonstrations, would you prefer imitation learning or offline RL? What if the dataset contains a mixture of expert and novice behavior?
2. **Why do Q-function methods suffer from OOD action overestimation in offline RL?** Draw a picture showing how a Q-function trained on limited data can produce spuriously high values. Explain why this problem is self-reinforcing through Bellman backups.
3. **Compare forward KL and reverse KL constraints for offline RL.** Why is forward KL easier to implement? Why is reverse KL theoretically preferable? How does the reverse KL constraint connect to max-entropy RL?
4. **Explain how IQL avoids querying OOD actions.** What role does expectile regression play? Why isn’t the asymmetric loss for V over-optimistic?
5. **In CQL, why is the “push up in-distribution” term important?** What would happen if you only had the “push down OOD” term without the counterbalancing “push up” term?

Answers

1. **Imitation learning** matches the data distribution (minimizes divergence from demonstrations). **Offline RL** maximizes reward, potentially finding better behavior than exists in the data. With expert-only data, imitation learning is simpler and sufficient. With mixed-quality data, offline RL is preferred because it can identify which parts are good (filtering) and combine good

sub-behaviors from different trajectories (stitching), whereas imitation learning would copy the bad behavior too.

2. The Q-function is trained on (s, a) pairs from the dataset, so it is accurate for in-distribution actions. But the Bellman backup computes $\max_{a'} Q(s', a')$, querying the Q-function at actions chosen to maximize it – these are likely OOD. The Q-function can produce spuriously high values for OOD actions (like a neural network extrapolating wildly outside its training data). This is self-reinforcing: the inflated Q-value at (s', a') gets backed up into $Q(s, a)$ via $r + \gamma \max_{a'} Q(s', a')$, inflating $Q(s, a)$, which in turn inflates Q-values at predecessor states, creating a positive feedback loop that drives Q-values to astronomical levels.
3. **Forward KL** $D_{\text{KL}}(\pi_\beta \| \pi_\theta)$ is easier because it only requires samples from π_β (i.e., the dataset) – it reduces to the behavioral cloning loss $-\mathbb{E}_{(s,a) \sim D}[\log \pi_\theta(a|s)]$. **Reverse KL** $D_{\text{KL}}(\pi_\theta \| \pi_\beta)$ is theoretically preferable because it directly penalizes π_θ for placing mass on actions where $\pi_\beta = 0$ (OOD actions), but it requires estimating $\log \pi_\beta$. The reverse KL connects to max-entropy RL: it decomposes into $-H[\pi_\theta] - \mathbb{E}_{\pi_\theta}[\log \pi_\beta]$; the entropy term is already in SAC, and the second term can be added to the reward as $\tilde{r}(s, a) = r(s, a) + \lambda \log \pi_\beta(a|s)$.
4. IQL replaces $\max_{a'} Q(s', a')$ in the Bellman target with $V(s')$, where V is trained via expectile regression on $(V(s) - Q(s, a))$ using only dataset actions. The asymmetric loss with $\tau > 0.5$ penalizes underestimation more than overestimation, pushing V toward the upper end of the Q-value distribution *over actions already in the data*. This is not over-optimistic because (a) only in-sample actions are considered, (b) the Q-function itself uses standard MSE with target $r + \gamma V(s')$ (risk-neutral over transitions), and (c) the optimism is only over actions (which is what we want in RL – take the best action), not over states/transitions.
5. Without the “push up in-distribution” term, CQL would push down Q-values for *all* actions, including good in-distribution ones. This causes systematic underestimation everywhere, making the policy overly conservative – it would avoid taking any action at all, even well-supported ones. The “push up” term $-\alpha \mathbb{E}_{(s,a) \sim D}[Q(s, a)]$ counterbalances by raising Q-values for actions in the dataset, so that the net effect is: Q-values are reduced only for OOD actions (where push-down dominates) while remaining accurate for in-distribution actions (where push-down and push-up cancel).

Chapter 14

Exploration

14.1 The Exploration Problem

14.1.1 Why Exploration Matters

Not all RL tasks are equally hard. Games like Breakout (Chapter 7, DQN) are “easy” because rewards are **immediate and frequent** – you get a reward every time you break a brick. But games like **Montezuma’s Revenge** are nearly impossible for standard RL algorithms because:

- Getting the key = reward, opening the door = reward
- Getting killed by the skull = **no penalty** (the reward is simply zero)
- Finishing the game only **weakly correlates** with rewarding events
- The agent must execute a **long temporally-extended sequence** of individually unrewarding actions

A naive DQN agent will learn to pick up the key, die (key reappears), pick up the key again, die again – repeating forever. This is locally reward-maximizing but globally terrible.

Intuition

Imagine playing the card game Mao: “the only rule you may be told is this one.” You incur penalties when you break rules you don’t know, and you must discover them through trial and error. This is what RL feels like to the agent – the entire world is a game where the rules (dynamics and reward structure) are unknown and must be discovered by experiencing outcomes.

14.1.2 Exploration vs. Exploitation

Two equivalent framings of the exploration problem:

1. How can an agent discover high-reward strategies requiring temporally extended sequences of individually non-rewarding behaviors?

2. How can an agent decide whether to try new behaviors (exploration) or continue doing the best known thing (exploitation)?

These are actually the **same problem**: if the agent only exploits, it gets stuck in local optima (like the key-pickup loop). If it explores, it might find globally better strategies.

Definition (Exploitation): Doing what you *know* will yield the highest reward.

Definition (Exploration): Doing things you haven't done before, hoping to discover even higher reward.

Professor's Note

"Don't be fooled by the visuals – Montezuma's Revenge *looks* easier than Breakout to a human because we understand what keys and skulls mean. But to the RL agent, which has no prior knowledge, Montezuma's Revenge is incomparably harder."

14.1.3 The Spectrum of Tractability

Exploration problems range from theoretically tractable to intractable:

Setting	Tractability	Approach
Multi-armed bandits (1-step, stateless)	Theoretically tractable	Formalize as POMDP; optimal strategies known
Contextual bandits (1-step with states)	Tractable	POMDP identification; policy learning is trivial
Small, tabular MDPs	Moderate	Bayesian model identification; reason about value of information
Large MDPs with known structure	Hard	Some theoretical results possible
Large MDPs without known structure (e.g., Atari)	Theoretically intractable	Heuristic methods inspired by simpler settings

The strategy: figure out what works optimally in simple settings, then heuristically apply those ideas to complex settings.

14.2 Multi-Armed Bandits

14.2.1 Setup

A **multi-armed bandit** is the simplest possible RL problem – a single-step, stateless MDP:

- Action space: $A = \{\text{pull}_1, \text{pull}_2, \dots, \text{pull}_n\}$ (n arms)

- Each arm i has an unknown reward distribution:

$$r(a_i) \sim p_{\theta_i}(r_i) \quad (14.1)$$

- Example: Bernoulli bandit where $p(r_i = 1) = \theta_i$ and $p(r_i = 0) = 1 - \theta_i \dots$ (Eq. 14.2)
- The parameters $\theta = [\theta_1, \dots, \theta_n]$ are drawn from some prior $p(\theta)$ but are otherwise **unknown**

Intuition

Think of n slot machines in a casino, each with a different (unknown) probability of paying out. Your goal: find the best machine and play it as much as possible.

14.2.2 Bandits as POMDPs

The bandit exploration problem defines a **POMDP** with: - Hidden state: $s = [\theta_1, \dots, \theta_n]$ - Belief state: $\hat{p}(\theta_1, \dots, \theta_n)$ - your current estimate of the parameters - Observations: rewards received from pulling arms

Solving this POMDP yields the **optimal exploration strategy**. However, this is overkill – the belief state is huge, and we can do very well with much simpler strategies.

Common Misconception

“A board game like chess is a bandit problem.” No! Board games have sequential decision-making with state transitions. Bandits are specifically **one-step** problems with no state. This is a key distinction – bandits have no dynamics, no state transitions, just actions and rewards (see Chapter 4 for MDP structure).

14.2.3 Regret

Definition (Regret): The regret after H time steps measures how much worse your exploration strategy performs compared to an omniscient agent that always plays the best arm:

$$\text{Reg}(H) = H \cdot \mathbb{E}[r(a^*)] - \sum_{t=1}^H r(a_t) \quad (14.3)$$

where: - $H \cdot \mathbb{E}[r(a^*)]$ = expected reward from always playing the best action (an upper bound on performance) - $\sum_{t=1}^H r(a_t)$ = actual reward accumulated by the agent’s strategy

Since the first term is constant, **minimizing regret is equivalent to maximizing total reward**.

14.3 Upper Confidence Bound (UCB) – Optimism in the Face of Uncertainty

14.3.1 The UCB Algorithm

Keep track of the average reward $\hat{\mu}_a$ for each action a . The key idea:

Pure exploitation:

$$a = \arg \max_a \hat{\mu}_a \quad (14.4)$$

Optimistic estimate (UCB):

$$a = \arg \max_a [\hat{\mu}_a + C \cdot \sigma_a] \quad (14.5)$$

where σ_a is some measure of uncertainty about arm a .

Intuition

Try each arm until you are **sure** it's not great. Give uncertain arms a “benefit of the doubt” bonus. As $C \rightarrow \infty$, you only explore; as $C \rightarrow 0$, you only exploit.

14.3.2 The Classic UCB1 Formula

From Auer et al., “Finite-time analysis of the multiarmed bandit problem”:

$$a = \arg \max_a \left[\hat{\mu}_a + \sqrt{\frac{2 \ln H}{N(a)}} \right] \quad (14.6)$$

where: - $N(a)$ = number of times action a has been selected - H = total number of steps so far

Key properties: 1. The bonus decreases as $N(a)$ increases (more pulls = more confidence = less bonus) 2. The $1/\sqrt{N}$ scaling comes from Hoeffding’s inequality (see Chapter 15) 3. **Regret is** $O(\log H)$ – provably as good as any algorithm (asymptotically optimal)

Professor’s Note

“Even though UCB looks pretty simple and pretty ad hoc, in fact it is asymptotically as good – up to a constant – as the optimal POMDP strategy can get. Of course, this is for the simplified multi-armed bandit case.”

14.4 From Bandits to Deep RL: Count-Based Exploration

14.4.1 Exploration Bonuses in MDPs

The UCB bonus $\sqrt{2 \ln H / N(a)}$ is an **exploration bonus**. We can generalize this to MDPs:

Count-based exploration: Use $N(s, a)$ or $N(s)$ to add an exploration bonus:

$$r^+(s, a) = r(s, a) + B(N(s)) \quad (14.7)$$

where B is a bonus function that **decreases with** $N(s)$.

Use $r^+(s, a)$ instead of $r(s, a)$ with any model-free algorithm (DQN, PPO, Actor-Critic, etc.).

Advantages: Simple addition to any RL algorithm. **Disadvantages:** Need to tune bonus weight; need to define what a “count” means in large state spaces.

14.4.2 Common Bonus Functions

Method	Bonus $B(N(s))$	Reference
UCB	$\sqrt{2 \ln n / N(s)}$	Auer et al.
MBIE-EB	$\sqrt{1 / N(s)}$	Strehl & Littman, 2008
BEB	$1 / N(s)$	Kolter & Ng, 2009

All decrease with $N(s)$ but at different rates. Bellemare et al. (2016) used the MBIE-EB form ($1/\sqrt{N}$).

14.5 Why Count-Based Exploration Fails in Deep RL

14.5.1 The Problem: $N(s) = 1$ Almost Always

In deep RL with high-dimensional state spaces (images, continuous states):

- You **never see the same state twice** – exact pixel equality almost never occurs
- In continuous state spaces, floating-point states are never exactly repeated
- So $N(s) = 1$ for essentially every state, making the bonus constant and useless

Common Misconception

“Count-based exploration works fine in deep RL because we just count states.” No! In large/continuous state spaces, every state has count 1, so the bonus is always the same and provides no useful exploration signal. This is the fundamental challenge that pseudo-counts and prediction error methods address.

14.5.2 The Insight: Similarity Matters

Some states are **more similar** than others. Even if the exact pixel configuration differs, a state where the agent is in the same room with the key in a slightly different position should be considered “familiar.” We need a notion of **soft counts** that leverages state similarity.

14.6 Pseudo-Counts and Density Models

14.6.1 The Idea

Fit a density model $p_\theta(s)$ (or $p_\theta(s, a)$) to the states seen so far:

- $p_\theta(s)$ might be **high** even for a new state s if s is similar to previously seen states
- Use $p_\theta(s)$ to derive a **pseudo-count** $\hat{N}(s)$ that generalizes discrete counts

14.6.2 From Densities to Pseudo-Counts

In small discrete MDPs, counts and probabilities are related by:

$$P(s) = \frac{N(s)}{n} \quad (14.8)$$

After seeing s one more time:

$$P'(s) = \frac{N(s) + 1}{n + 1} \quad (14.9)$$

For density models, we set up two equations with two unknowns ($\hat{N}(s)$ and $\hat{n}$):

$$p_\theta(s_i) = \frac{\hat{N}(s_i)}{\hat{n}} \quad (14.10)$$

$$p_{\theta'}(s_i) = \frac{\hat{N}(s_i) + 1}{\hat{n} + 1} \quad (14.11)$$

where θ' is the model retrained on $D \cup \{s_i\}$.

14.6.3 Solving for the Pseudo-Count

Solving the two equations and two unknowns yields:

$$\hat{N}(s_i) = \hat{n} \cdot p_\theta(s_i) \quad (14.12)$$

$$\hat{n} = \frac{(1 - p_{\theta'}(s_i))}{(p_{\theta'}(s_i) - p_\theta(s_i))} \cdot p_\theta(s_i) \quad (14.13)$$

14.6.4 The Pseudo-Count Algorithm (Bellemare et al., 2016)

1. Fit model $p_\theta(s)$ to all states D seen so far
2. Take a step i and observe s_i
3. Fit new model $p_{\theta'}(s)$ to $D \cup \{s_i\}$
4. Use $p_\theta(s_i)$ and $p_{\theta'}(s_i)$ to estimate $\hat{N}(s)$ via Eq. 14.12-14.13
5. Set $r_i^+ = r_i + B(\hat{N}(s))$ where B is the MBIE-EB bonus $1/\sqrt{\hat{N}}$

14.6.5 What Kind of Model?

The density model $p_\theta(s)$ must: - Be able to **output densities** (not just generate samples) - Does NOT need to produce great samples (unlike GANs) - Does NOT even need to output great densities - just needs to tell if a state is **novel or not**

Bellemare et al. used the CTS model (conditioning each pixel on its top-left neighborhood). Other options: stochastic neural networks, compression-length models, VAEs, etc.

14.7 Prediction Error as Exploration Bonus

14.7.1 The Key Insight

We don't actually need a full density model. We just need to tell if a state is **novel or not**. A much simpler approach: use **prediction error** as a proxy for novelty.

14.7.2 General Framework

Let $f^*(s, a)$ be some **target function**. Given buffer $D = \{(s_i, a_i)\}$, fit a neural network $\hat{f}_\theta(s, a)$ to approximate f^* .

Use the prediction error as the exploration bonus:

$$E(s, a) = \|\hat{f}_\theta(s, a) - f^*(s, a)\|^2 \quad (14.14)$$

Why this works: - In regions with many data points (familiar states), $\hat{f}$ fits f^* well -> **low error** -> low bonus - In regions with few data points (novel states), $\hat{f}$ fits f^* poorly -> **high error** -> high bonus

14.7.3 Dynamics Prediction Error

One natural choice: set $f^*(s, a) = s'$ (the next state).

Train a dynamics model $\hat{f}_\theta(s, a)$ to predict the next state. Use the prediction error $\|\hat{f}_\theta(s, a) - s'\|^2$ as the exploration bonus.

Problem: The dynamics can be inherently stochastic (e.g., a noisy TV problem), leading to high prediction error even in familiar states. The agent might get “stuck” watching a noisy TV because the prediction error is always high there.

14.7.4 Random Network Distillation (RND)

Solution (Burda et al., 2018): Use a **fixed random neural network** as the target function.

$$f^*(s, a) = f_\phi(s, a), \text{ where } \phi \text{ is a } \mathbf{random, fixed} \text{ parameter vector} \quad (14.15)$$

Then train $\hat{f}_\theta(s, a)$ to match $f_\phi(s, a)$ on the replay buffer data. Use:

$$E(s, a) = \|\hat{f}_\theta(s, a) - f_\phi(s, a)\|^2 \quad (14.16)$$

Why RND works: - The random network f_ϕ defines an arbitrary but **deterministic** function - In familiar states (many training examples), $\hat{f}_\theta$ matches f_ϕ well $\rightarrow$ low bonus - In novel states, $\hat{f}_\theta$ hasn't been trained there $\rightarrow$ high error $\rightarrow$ high bonus - Avoids the “noisy TV” problem because f_ϕ is deterministic

Intuition

RND is like a student trying to memorize a teacher's random answers. On questions the student has studied (familiar states), they get the answer right. On new questions (novel states), they get it wrong. The amount of wrongness measures novelty.

RND is generally the **most widely used deep RL exploration method today** due to its simplicity and effectiveness.

14.8 Thompson Sampling

14.8.1 The Idea

Thompson sampling is a **Bayesian exploration** strategy for bandits:

1. Maintain a posterior distribution over the parameters θ of each arm
2. At each step, **sample** θ from the posterior: $\theta \sim p(\theta \mid \text{data}) \dots$ (Eq. 14.17)
3. Act greedily with respect to the sampled parameters: $a = \arg \max_a \mathbb{E}[r \mid a, \theta] \dots$ (Eq. 14.18)
4. Observe reward, update posterior

Intuition

Instead of being optimistic about all uncertain arms (UCB), randomly sample what you think the world looks like, and act optimally in that sampled world. Arms you're uncertain about will sometimes be sampled with high θ (leading to exploration) and sometimes with low θ (leading to exploitation), automatically balancing the tradeoff.

14.8.2 Properties

- Achieves **optimal regret bounds** (matching UCB asymptotically)
 - Natural Bayesian approach – automatically handles the exploration-exploitation tradeoff
 - Can be extended to MDPs by maintaining posteriors over dynamics or value functions (see posterior sampling RL)
 - In deep RL, approximated by methods like bootstrapped DQN (maintain an ensemble of Q-functions, randomly select one to follow each episode)
-

14.9 Information Gain and Bayesian Exploration

14.9.1 Information Gain

Instead of using counts or prediction errors, we can directly reason about **information gain**: choose actions that maximize the expected reduction in uncertainty about the environment.

Definition (Information Gain): The mutual information between the next observation and the unknown parameters:

$$\text{IG}(a) = I(\theta; r \mid a, \text{history}) \quad (14.19)$$

$$= H(\theta \mid \text{history}) - \mathbb{E}_{r|a}[H(\theta \mid \text{history}, r, a)] \quad (14.20)$$

Choose actions that maximize information gain: $a = \arg \max_a \text{IG}(a)$.

14.9.2 Bayesian Exploration in MDPs

For full MDPs, the optimal exploration strategy can be framed as a **POMDP** where: - The hidden state includes the unknown MDP parameters (transition probabilities, rewards) - The belief state is the posterior over these parameters - Solving this POMDP gives the **Bayes-optimal** exploration strategy

This is intractable for large MDPs but provides the theoretical foundation for practical methods.

Professor's Note

“Solving the POMDP is overkill for bandits – simpler methods like UCB work just as well. For large MDPs, even these principled methods break down, and we resort to heuristics inspired by the theory.”

14.9.3 Connection to Other Methods

Method	What it approximates
UCB / Count-based	Optimism (upper bound on value)
Pseudo-counts	Soft version of count-based optimism
RND / Prediction error	Heuristic novelty detection
Thompson sampling	Posterior sampling
Information gain	Optimal Bayesian strategy

Each is an increasingly accurate (but increasingly complex) approximation to the Bayes-optimal exploration strategy. In practice, the simpler methods (RND, counts) work well enough.

14.10 Check Your Understanding

1. **Why can't a standard DQN agent solve Montezuma's Revenge?** Explain the exploration challenge in terms of reward sparsity and temporal extent of the task.

Answer: Rewards in Montezuma's Revenge are extremely sparse – the agent must execute a long sequence of individually unrewarding actions (navigate rooms, avoid enemies) before receiving any reward signal. A standard DQN with ϵ -greedy exploration will almost never stumble upon these reward events by chance. The agent gets stuck in locally reward-maximizing loops (e.g., repeatedly picking up a respawning key) because random exploration is insufficient to discover the temporally extended strategies required.

2. **Why does count-based exploration (using $N(s)$) fail in deep RL with image observations?** What is the pseudo-count approach and how does it address this problem?

Answer: In high-dimensional state spaces (e.g., images), every state is essentially unique – you never visit the exact same pixel configuration twice, so $N(s) = 1$ for all states and the exploration bonus is constant and uninformative. Pseudo-counts address this by fitting a density model $p_\theta(s)$ to observed states and deriving a soft count $\hat{N}(s)$ that generalizes discrete counts. States similar to many previously seen states get high pseudo-counts (low bonus), while genuinely novel states get low pseudo-counts (high bonus), even if the exact state has never been visited.

3. **In the UCB formula $a = \arg \max_a [\hat{\mu}_a + \sqrt{2 \ln H / N(a)}]$, what happens to the bonus term as $N(a)$ grows large? What does this mean intuitively for exploration behavior?**

Answer: The bonus $\sqrt{2 \ln H / N(a)}$ decreases as $N(a)$ grows (scaling as $1/\sqrt{N(a)}$). Intuitively, the more you pull an arm, the more confident you are about its mean reward, so the “benefit of the doubt” bonus shrinks. Eventually, well-explored arms are selected only if their estimated mean $\hat{\mu}_a$ is genuinely high, while under-explored arms retain large bonuses and are preferentially tried. This naturally transitions from exploration to exploitation.

4. **Compare RND and dynamics prediction error as exploration bonuses. What is the “noisy TV” problem, and why does RND avoid it?**

Answer: Both use neural network prediction error as a novelty signal: high error means the state is unfamiliar. Dynamics prediction error trains a model to predict the next state s' given (s, a) . The “noisy TV” problem arises when environment dynamics are inherently stochastic (e.g., a TV showing random static) – prediction error remains permanently high in these regions regardless of how often they are visited, trapping the agent. RND avoids this by using a fixed random network f_ϕ as the target instead of the next state. Since f_ϕ is deterministic, the learned network f_θ can eventually match it on frequently visited states, and the error genuinely reflects novelty rather than irreducible stochasticity.

5. **A student claims: “Playing a board game like chess against an opponent is a multi-armed bandit problem because you choose between different opening moves.” Why is this incorrect?** (Hint: bandits are single-step problems with no state transitions.)

Answer: Multi-armed bandits are single-step, stateless problems: you choose an action (pull an arm), receive a reward, and the episode ends. Chess involves sequential decision-making with state transitions – after choosing an opening move, the board state changes, the opponent responds, and you must make many more decisions. The outcome depends on the entire sequence of moves, not just the first one. Chess is an MDP (or game-theoretic MDP), not a bandit.

Chapter 15

RL Theory

15.1 What is the Point of RL Theory?

15.1.1 Types of Theoretical Questions

Given an RL algorithm, we might ask:

1. **Sample complexity:** If I run this algorithm with N samples and k iterations, how good is the result?
2. **Convergence:** Does my algorithm converge to the optimal solution?
3. **Error scaling:** How does the error scale with problem parameters (horizon, state space size, etc.)?
4. **Exploration efficiency:** What is the regret of my exploration algorithm?

15.1.2 The Nature of Theoretical Results

Results are typically **worst-case bounds under assumptions**:

$$\|\hat{Q}_k - Q^*\| \leq \epsilon, \text{ with probability at least } 1 - \delta, \text{ if } N \geq f(\epsilon, \delta) \quad (15.1)$$

where: - $\hat{Q}_k$ = learned Q-function at iteration k - Q^* = optimal Q-function - N = number of samples
- $f(\epsilon, \delta)$ = sample complexity function (hopefully polynomial in ϵ and δ)

Professor's Note

“Don’t take someone seriously if they say their RL algorithm has ‘provable guarantees’ – the assumptions are always unrealistic. Theory is at best a rough guide to what might happen. We use precise theory to get imprecise qualitative conclusions about how various factors influence performance.”

15.1.3 Important Notational Distinction

- $\hat{Q}_k$: The **learned** Q-function (a neural network or table in the computer) at iteration k
- Q^{π_k} : The **true** Q-function of the policy at iteration k (exists theoretically, generally unknown)
- Q^* : The true Q-function of the **optimal** policy ($= Q^{\pi^*}$)

Common Misconception

“ $\hat{Q}_k$ and Q^{π_k} are the same thing.” No! $\hat{Q}_k$ is what your algorithm produces (which may have errors), while Q^{π_k} is the true value of the policy your algorithm produces. Showing $\hat{Q}_k$ is close to Q^* is different from showing Q^{π_k} is close to Q^* – the latter is what we really care about.

15.1.4 Assumptions in RL Theory

Effective analysis requires strong assumptions:

For exploration: Worst-case exploration is extremely hard, so we either: - Study exploration directly (regret bounds scaling with $|S|$, $|A|$, H) - Abstract away exploration entirely

For learning: The “oracle exploration” / “generative model” assumption: - Assume we can sample $s' \sim P(s'|s, a)$ for **any** $(s, a) \dots$ (Eq. 15.2) - Equivalent to assuming a simulator where we can “teleport” to any state - Not realistic, but allows us to focus on analyzing learning vs. exploration

15.1.5 What We Learn From Theory

Even under unrealistic assumptions, theory provides: - Understanding of how errors scale with problem parameters (γ , $|S|$, $|A|$, N) - Whether difficulty is linear, quadratic, or exponential in various quantities - Qualitative rules of thumb for algorithm design and hyperparameter selection

15.2 Useful Identities and Notation

15.2.1 Vector Notation for Value Functions

Assuming finite, discrete state and action spaces (see Chapter 4):

- Q^π is a **vector** of size $|S| \times |A|$
- r is a **vector** of size $|S| \times |A|$
- P is a **matrix** of size $|S||A| \times |S|$ (transition probabilities)
- V^π is a **vector** of size $|S|$

15.2.2 Bellman Equation in Matrix Form

The Bellman equation $Q^\pi(s, a) = r(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s')$ becomes:

$$Q^\pi = r + \gamma P V^\pi \tag{15.3}$$

15.2.3 State-Action Transition Matrix

Define P^π (size $|S||A| \times |S||A|$) as the state-action transition matrix under policy π :

$$P^\pi(s', a' | s, a) = \pi(a' | s') \cdot P(s' | s, a) \quad (15.4)$$

Then the Bellman equation becomes:

$$Q^\pi = r + \gamma P^\pi Q^\pi \quad (15.5)$$

15.2.4 Closed-Form Policy Evaluation

Solving the Bellman equation algebraically:

$$\begin{aligned} \text{Step 1: } Q^\pi - \gamma P^\pi Q^\pi &= r \\ \text{Step 2: } (I - \gamma P^\pi) Q^\pi &= r \\ \text{Step 3: } Q^\pi &= (I - \gamma P^\pi)^{-1} r \end{aligned} \quad (15.6)$$

This is theoretically correct but **impractical** (requires inverting a potentially huge/infinite matrix, and we typically don't know P). However, it is extremely useful for theoretical analysis.

Intuition

Policy evaluation is a **linear operation** – the Q-function is a linear function of the reward vector, with $(I - \gamma P^\pi)^{-1}$ acting as the “evaluation operator.”

15.2.5 Size of Q-values

If rewards are roughly of size 1, then Q-values are roughly:

$$\|Q^\pi\|_\infty \sim \sum_{t=0}^{\infty} \gamma^t \cdot 1 = \frac{1}{1 - \gamma} \quad (15.7)$$

In the finite-horizon case: $\|Q^\pi\|_\infty \sim H$ (the horizon length).

Key insight: $1/(1 - \gamma)$ plays the role of an **effective horizon**. Whenever you see $1/(1 - \gamma)$ in a theoretical result, think of it as the length of the episode.

15.3 Value Iteration Convergence Proof – Contraction Mapping

15.3.1 The Bellman Optimality Operator

Value iteration (Chapter 4): $V \leftarrow \max_a [r + \gamma PV]$

Define the **Bellman optimality operator** $\mathcal{T}$:

$$\mathcal{T}V = \max_a [r + \gamma PV] \quad (15.8)$$

This is a **nonlinear** operator (due to the max).

Fact: V^* is a **fixed point** of $\mathcal{T}$: $\mathcal{T}V^* = V^* \dots$ (Eq. 15.9)

15.3.2 Key Inequality

For any two functions f and g :

$$|\max_x f(x) - \max_x g(x)| \leq \max_x |f(x) - g(x)| \quad (15.10)$$

Intuition

The difference between two maxima can't exceed the maximum difference. (Draw two curves and look at the gaps – this becomes geometrically obvious.)

15.3.3 Proof: The Bellman Operator is a γ -Contraction

Theorem (Contraction): For any two value functions V and U :

$$\|\mathcal{T}V - \mathcal{T}U\|_\infty \leq \gamma \|V - U\|_\infty \quad (15.11)$$

Proof:

Step 1. Start with the pointwise difference for a particular state s :

$$\begin{aligned} |\mathcal{T}V(s) - \mathcal{T}U(s)| &= \left| \max_a [r(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s')] \right. \\ &\quad \left. - \max_a [r(s, a) + \gamma \sum_{s'} P(s'|s, a) U(s')] \right| \end{aligned} \quad (15.12)$$

Step 2. Apply the inequality (Eq. 15.10) – difference of maxes $\leq$ max of difference:

$$\leq \max_a \left| r(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s') - r(s, a) - \gamma \sum_{s'} P(s'|s, a) U(s') \right| \quad (15.13)$$

Step 3. Cancel the reward terms:

$$= \gamma \max_a \left| \sum_{s'} P(s'|s, a) (V(s') - U(s')) \right| \quad (15.14)$$

Step 4. Since $P(s'|s, a)$ are probabilities (non-negative, sum to 1), the weighted sum is bounded by the maximum entry:

$$\leq \gamma \max_{s'} |V(s') - U(s')| \quad (15.15)$$

Step 5. Recognize this as the infinity norm:

$$= \gamma \|V - U\|_\infty \quad (15.16)$$

Step 6. Since this holds for **all states** s , it holds for the maximum over s (the infinity norm of the left side):

$$\|\mathcal{T}V - \mathcal{T}U\|_\infty \leq \gamma \|V - U\|_\infty \quad \text{QED} \quad (15.17)$$

Intuition

Applying $\mathcal{T}$ to any two value functions **squeezes them together** by a factor of γ . If $\gamma = 0.9$, after one application of $\mathcal{T}$, V and U are at most 90% as far apart as before.

15.3.4 Proof: Value Iteration Converges to V^*

Theorem: $\lim_{k \rightarrow \infty} \|\mathcal{T}^k V - V^*\|_\infty = 0$ for any initial V .

Proof:

Step 1. Use the fact that V^* is a fixed point (Eq. 15.9):

$$\|\mathcal{T}V - V^*\|_\infty = \|\mathcal{T}V - \mathcal{T}V^*\|_\infty \quad (15.18)$$

Step 2. Apply the contraction result (Eq. 15.17):

$$\leq \gamma \|V - V^*\|_\infty \quad (15.19)$$

Step 3. Apply $\mathcal{T}$ k times:

$$\|\mathcal{T}^k V - V^*\|_\infty \leq \gamma^k \|V - V^*\|_\infty \quad (15.20)$$

Step 4. Since $0 < \gamma < 1$, $\gamma^k \rightarrow 0$ as $k \rightarrow \infty$:

$$\lim_{k \rightarrow \infty} \|\mathcal{T}^k V - V^*\|_\infty \rightarrow 0 \quad \text{QED} \quad (15.21)$$

Intuition

Value iteration converges because $\mathcal{T}$ is a contraction mapping with fixed point V^* . Each iteration brings us γ times closer. After k iterations, our error is at most γ^k of the initial error – geometric convergence.

Professor’s Note on the Infinity Norm: “You can think of the infinity norm as the worst-case error over all states. Showing convergence in infinity norm is strong – it means convergence everywhere. However, in practice with sampling, infinity-norm results are typically impossible to achieve because you can’t guarantee good performance on states you’ve never visited.”

15.4 Sample Complexity Without Exploration

15.4.1 The Setup

Assume **oracle exploration** (generative model assumption): for every (s, a) , we can sample $s' \sim P(s'|s, a)$ N times.

Simple model-based algorithm:

1. Estimate dynamics: $\hat{P}(s'|s, a) = \#(s, a, s')/N \dots$ (Eq. 15.22)
2. Given any policy π , use $\hat{P}$ to compute $\hat{Q}^\pi = (I - \gamma\hat{P}^\pi)^{-1}r \dots$ (Eq. 15.23)

Question: How close is $\hat{Q}^\pi$ to Q^π ?

15.4.2 Concentration Inequalities

To relate samples to errors, we need **concentration inequalities** from statistics.

Hoeffding’s Inequality: If $X_1, \dots, X_n$ are i.i.d. random variables with mean μ , and $X_i \in [b_-, b_+]$, then:

$$P(\bar{X}_n \geq \mu + \epsilon) \leq e^{-2n\epsilon^2/(b_+ - b_-)^2} \quad (15.24)$$

Interpretation: If we estimate μ with n samples, the probability we’re off by more than ϵ is at most $2e^{-2n\epsilon^2/(b_+ - b_-)^2}$.

Solving for ϵ : **error scales as $1/\sqrt{n}$** ... (Eq. 15.25)

This is fundamental – it’s why machine learning works. Sample-based estimates have error that drops rapidly with more samples.

Concentration for Discrete Distributions: For estimating transition probabilities $\hat{P}(s'|s, a)$:

$$\|\hat{P}(\cdot|s, a) - P(\cdot|s, a)\|_1 \leq c\sqrt{\frac{|S|\log(1/\delta)}{N}}, \text{ with prob } 1 - \delta \quad (15.26)$$

15.4.3 The Simulation Lemma

Theorem (Simulation Lemma): The difference between the true Q-function and the learned Q-function is:

$$Q^\pi - \hat{Q}^\pi = \gamma(I - \gamma\hat{P}^\pi)^{-1}(P - \hat{P})V^\pi \quad (15.27)$$

Proof:

Step 1. Write out both Q-functions using the closed-form:

$$Q^\pi = (I - \gamma P^\pi)^{-1}r \quad (15.28)$$

$$\hat{Q}^\pi = (I - \gamma\hat{P}^\pi)^{-1}r \quad (15.29)$$

Step 2. Take the difference and multiply Q^π by the identity $(I - \gamma\hat{P}^\pi)^{-1}(I - \gamma\hat{P}^\pi)$:

$$Q^\pi - \hat{Q}^\pi = (I - \gamma\hat{P}^\pi)^{-1}(I - \gamma\hat{P}^\pi)Q^\pi - (I - \gamma\hat{P}^\pi)^{-1}r \quad (15.30)$$

Step 3. Factor out $(I - \gamma\hat{P}^\pi)^{-1}$:

$$= (I - \gamma\hat{P}^\pi)^{-1} \left[(I - \gamma\hat{P}^\pi)Q^\pi - r \right] \quad (15.31)$$

Step 4. Substitute $r = (I - \gamma P^\pi)Q^\pi$:

$$= (I - \gamma\hat{P}^\pi)^{-1} \left[(I - \gamma\hat{P}^\pi) - (I - \gamma P^\pi) \right] Q^\pi \quad (15.32)$$

Step 5. The I 's cancel:

$$= (I - \gamma\hat{P}^\pi)^{-1} \gamma(P^\pi - \hat{P}^\pi)Q^\pi \quad (15.33)$$

Step 6. Since $P^\pi = P \cdot \Pi$ and $\Pi \cdot Q^\pi = V^\pi$:

$$= \gamma(I - \gamma\hat{P}^\pi)^{-1}(P - \hat{P})V^\pi \quad \text{QED} \quad (15.34)$$

Intuition

The Q-function error is itself like a Q-function – it's the value you'd get if your "reward" were the value-weighted dynamics error $(P - \hat{P})V^\pi$. Dynamics error matters more in high-value states and less in low-value states.

15.4.4 Bounding the Evaluation Operator

Lemma: For any transition matrix P^π and any vector v :

$$\|(I - \gamma P^\pi)^{-1}v\|_\infty \leq \frac{\|v\|_\infty}{1 - \gamma} \quad (15.35)$$

Proof:

Step 1. Let $w = (I - \gamma P^\pi)^{-1}v$, so $v = (I - \gamma P^\pi)w$.

Step 2. By the triangle inequality:

$$\|v\|_\infty = \|(I - \gamma P^\pi)w\|_\infty \geq \|w\|_\infty - \gamma\|P^\pi w\|_\infty \quad (15.36)$$

Step 3. Since $\|P^\pi\|_\infty \leq 1$ (rows sum to 1, entries between 0 and 1):

$$\geq \|w\|_\infty - \gamma\|w\|_\infty = (1 - \gamma)\|w\|_\infty \quad (15.37)$$

Step 4. Rearranging: $\|w\|_\infty \leq \|v\|_\infty / (1 - \gamma)$ QED ... (Eq. 15.38)

Intuition

The “Q-function” corresponding to a “reward” v has entries at most $1/(1 - \gamma)$ times larger. This is the geometric series again – summing discounted rewards multiplies by at most $1/(1 - \gamma)$.

15.5 The Main Error Bound

15.5.1 Putting It All Together

Combining the simulation lemma and the bound on the evaluation operator:

$$\|Q^\pi - \hat{Q}^\pi\|_\infty = \|\gamma(I - \gamma \hat{P}^\pi)^{-1}(P - \hat{P})V^\pi\|_\infty$$

Step 1. Apply the evaluation operator bound (Eq. 15.35):

$$\leq \frac{\gamma}{1 - \gamma} \|(P - \hat{P})V^\pi\|_\infty \quad (15.39)$$

Step 2. Bound by the worst-case dynamics error:

$$\leq \frac{\gamma}{1 - \gamma} \left(\max_{s,a} \|P(\cdot|s,a) - \hat{P}(\cdot|s,a)\|_1 \right) \|V^\pi\|_\infty \quad (15.40)$$

Step 3. Bound $\|V^\pi\|_\infty \leq R_{\max}/(1 - \gamma)$ (assuming $R_{\max} = 1$):

$$\leq \frac{\gamma}{(1-\gamma)^2} \left(\max_{s,a} \|P(\cdot|s,a) - \hat{P}(\cdot|s,a)\|_1 \right) \quad (15.41)$$

Step 4. Substitute the concentration inequality (Eq. 15.26), using the union bound to handle the max over s, a :

$$\|Q^\pi - \hat{Q}^\pi\|_\infty \leq \frac{\gamma}{(1-\gamma)^2} \cdot c_2 \sqrt{\frac{|S| \log(1/\delta)}{N}} \quad (15.42)$$

with probability at least $1 - \delta$.

15.5.2 Interpretation of the Error Bound

The final bound has the form:

$$\epsilon = \frac{\gamma}{(1-\gamma)^2} \cdot c_2 \sqrt{\frac{|S| \log(1/\delta)}{N}} \quad (15.43)$$

Key observations:

Factor	Effect	Interpretation
$1/(1-\gamma)^2$	Error grows quadratically in the horizon	Each Bellman backup accumulates error; longer episodes = more error
$\sqrt{ S /N}$	Error decreases as $1/\sqrt{N}$	More samples = lower error (standard statistical scaling)
$\sqrt{ S }$	Error increases with state space size	Larger state spaces require more samples
$\log(1/\delta)$	Logarithmic dependence on confidence	High confidence requires only logarithmically more samples

Key Result: Error scales as $1/(1-\gamma)^2 = \text{quadratic in the effective horizon}$. This is a fundamental result in RL theory.

Common Misconception

“The error scales linearly with the horizon because there are H time steps.” No! The error is **quadratic** in the horizon because the evaluation operator (which amplifies errors by $1/(1-\gamma)$) is applied to dynamics errors that are themselves accumulated over the horizon. One factor of $1/(1-\gamma)$ comes from the evaluation operator and one from bounding $\|V^\pi\|_\infty$.

15.6 Implications for Q^* and the Optimal Policy

15.6.1 From Q^π to Q^*

We showed $\|Q^\pi - \hat{Q}^\pi\|_\infty \leq \epsilon$ for all π . What about Q^* vs $\hat{Q}^*$?

Using the identity $|\sup_x f(x) - \sup_x g(x)| \leq \sup_x |f(x) - g(x)|$:

$$\|Q^* - \hat{Q}^*\|_\infty = \|\sup_\pi Q^\pi - \sup_\pi \hat{Q}^\pi\|_\infty \leq \sup_\pi \|Q^\pi - \hat{Q}^\pi\|_\infty \leq \epsilon \quad (15.44)$$

15.6.2 From Q^* to the Policy Performance

What about $\|Q^* - Q^{\hat{\pi}^*}\|_\infty$ where $\hat{\pi}^*$ is the greedy policy w.r.t. $\hat{Q}^*$?

By the triangle inequality:

$$\begin{aligned} \|Q^* - Q^{\hat{\pi}^*}\|_\infty &= \|Q^* - \hat{Q}^{\hat{\pi}^*} + \hat{Q}^{\hat{\pi}^*} - Q^{\hat{\pi}^*}\|_\infty \\ &\leq \|Q^* - \hat{Q}^{\hat{\pi}^*}\|_\infty + \|\hat{Q}^{\hat{\pi}^*} - Q^{\hat{\pi}^*}\|_\infty \\ &\leq 2\epsilon \end{aligned} \quad (15.45)$$

The first term is bounded by ϵ (since $\hat{Q}^*$ maximizes over $\hat{Q}^\pi$), and the second term is bounded by ϵ (from our main result, both have the **same policy**).

Intuition

The policy learned from approximate dynamics is at most 2ϵ suboptimal. The factor of 2 comes from two sources of error: (1) the estimated Q-function being wrong about the optimal action, and (2) the estimated Q-function being wrong about the value of that action.

15.7 Approximation Error vs. Sampling Error in Fitted Q-Iteration

15.7.1 The Setting

Recall **fitted Q-iteration** (Chapter 7):

Exact Q-iteration: $Q_{k+1} \leftarrow \mathcal{T}Q_k$ (apply the Bellman optimality operator)

Approximate fitted Q-iteration:

$$\hat{Q}_{k+1} \leftarrow \arg \min_{\hat{Q}} \|\hat{Q} - \hat{\mathcal{T}}\hat{Q}_k\| \quad (15.46)$$

where $\hat{\mathcal{T}}$ is the **approximate** Bellman operator using sample-based estimates:

$$\hat{\mathcal{T}}Q = \hat{r} + \gamma \hat{P} \max_a Q \quad (15.47)$$

with $\hat{r}(s, a) = \frac{1}{N(s, a)} \sum_i \delta((s_i, a_i) = (s, a)) r_i$ and $\hat{P}(s'|s, a) = N(s, a, s')/N(s, a)$.

15.7.2 Two Sources of Error

1. **Sampling error:** $\mathcal{T} \neq \hat{\mathcal{T}}$ (because we use sample-based estimates of r and P)
2. **Approximation error:** $\hat{Q}_{k+1} \neq \hat{\mathcal{T}}\hat{Q}_k$ (because our function class can't perfectly represent the target)

Common Misconception

“Approximation error only appears with function approximation (neural networks).” Correct! In the tabular case, $\hat{Q}_{k+1} = \hat{\mathcal{T}}\hat{Q}_k$ exactly, so there is no approximation error. Approximation error appears **only** when you use a restricted function class (neural nets, linear functions, etc.) that cannot perfectly represent the Bellman backup.

15.7.3 Analyzing Sampling Error

The difference between $\hat{\mathcal{T}}$ and $\mathcal{T}$ at a single (s, a) :

$$\begin{aligned} |\hat{\mathcal{T}}Q(s, a) - \mathcal{T}Q(s, a)| &= |\hat{r}(s, a) - r(s, a) + \gamma(\mathbb{E}_{\hat{P}}[\max_{a'} Q(s', a')] - \mathbb{E}_P[\max_{a'} Q(s', a')])| \\ &\leq |\hat{r}(s, a) - r(s, a)| + \gamma \left| \sum_{s'} (\hat{P}(s'|s, a) - P(s'|s, a)) \max_{a'} Q(s', a') \right| \end{aligned} \quad (15.48)$$

By Hoeffding's inequality for the reward term and the concentration inequality for the dynamics term:

$$\|\hat{\mathcal{T}}Q - \mathcal{T}Q\|_{\infty} \leq 2R_{\max}c_1 \sqrt{\frac{\log(|S||A|/\delta)}{2N}} + c_2 \|Q\|_{\infty} \sqrt{\frac{\log(|S|/\delta)}{N}} \quad (15.49)$$

15.7.4 Analyzing Approximation Error

Assume approximation error is bounded: $\|\hat{Q}_{k+1} - \mathcal{T}\hat{Q}_k\|_{\infty} \leq \epsilon_k \dots$ (Eq. 15.50)

(Note: this is a **strong assumption** – it says the function approximator can represent the Bellman backup to within ϵ_k at every state-action pair.)

Step 1. Add and subtract $\mathcal{T}\hat{Q}_{k-1}$:

$$\|\hat{Q}_k - Q^*\|_{\infty} = \|\hat{Q}_k - \mathcal{T}\hat{Q}_{k-1} + \mathcal{T}\hat{Q}_{k-1} - Q^*\|_{\infty} \quad (15.51)$$

Step 2. Triangle inequality:

$$\leq \|\hat{Q}_k - \mathcal{T}\hat{Q}_{k-1}\|_{\infty} + \|\mathcal{T}\hat{Q}_{k-1} - Q^*\|_{\infty} \quad (15.52)$$

Step 3. Use the assumption (Eq. 15.50) and the contraction property (Eq. 15.17):

$$\leq \epsilon_{k-1} + \gamma \|\hat{Q}_{k-1} - Q^*\|_{\infty} \quad (15.53)$$

Step 4. Unroll the recursion:

$$\begin{aligned}\|\hat{Q}_k - Q^*\|_\infty &\leq \epsilon_{k-1} + \gamma\epsilon_{k-2} + \gamma^2\epsilon_{k-3} + \cdots + \gamma^k\|\hat{Q}_0 - Q^*\|_\infty \\ &\leq \sum_{i=0}^{k-1} \gamma^i \epsilon_{k-i-1} + \gamma^k\|\hat{Q}_0 - Q^*\|_\infty\end{aligned}\tag{15.54}$$

Step 5. As $k \rightarrow \infty$:

$$\begin{aligned}\lim_{k \rightarrow \infty} \|\hat{Q}_k - Q^*\|_\infty &\leq \sum_{i=0}^{\infty} \gamma^i \max_k \epsilon_k = \frac{1}{1-\gamma} \max_k \epsilon_k \\ &= \frac{1}{1-\gamma} \|\epsilon\|_\infty\end{aligned}\tag{15.55}$$

Key Result: With approximation error only, the final error is amplified by $1/(1-\gamma)$ (one factor of the horizon). The approximation error does NOT converge to zero – it accumulates and is bounded by $1/(1-\gamma)$ times the worst per-iteration approximation error.

15.7.5 Putting Sampling and Approximation Error Together

The per-iteration error ϵ_k decomposes as:

$$\|\hat{Q}_k - \mathcal{T}\hat{Q}_{k-1}\|_\infty \leq \|\hat{Q}_k - \hat{\mathcal{T}}\hat{Q}_{k-1}\|_\infty + \|\hat{\mathcal{T}}\hat{Q}_{k-1} - \mathcal{T}\hat{Q}_{k-1}\|_\infty\tag{15.56}$$

- First term: **approximation error** (function class limitation)
- Second term: **sampling error** (finite samples)

The total error compounds: error “compounds” with horizon, over iterations, and due to sampling.

Error Source	When It Appears	Scaling
Sampling error	Always (finite samples)	$O\left(\frac{1}{1-\gamma} \cdot \frac{1}{\sqrt{N}}\right)$
Approximation error	Only with function approximation	$O\left(\frac{1}{1-\gamma} \cdot \max_k \epsilon_k\right)$

Professor’s Note

“So far we needed strong assumptions (infinity norm). More advanced results can be derived with p-norms under some distribution: $\|\hat{Q}_k - Q^*\|_{p,\mu} = (\mathbb{E}_{(s,a) \sim \mu} [|\hat{Q}_k(s,a) - Q^*(s,a)|^p])^{1/p}$. These are more realistic but much harder to prove.”

15.8 Check Your Understanding

1. **State the contraction property of the Bellman optimality operator. What are the two key consequences for value iteration convergence?** (Hint: applying $\mathcal{T}$ to two value functions brings them closer; applying $\mathcal{T}$ to V brings it closer to V^* .)

Answer: The contraction property states $\|\mathcal{T}V - \mathcal{T}U\|_\infty \leq \gamma\|V - U\|_\infty$ for any value functions V, U . Consequence 1: Any two value functions are brought closer together by a factor of γ after applying $\mathcal{T}$, so there can be at most one fixed point. Consequence 2: Since V^* is the fixed point ($\mathcal{T}V^* = V^*$), repeated application gives $\|\mathcal{T}^k V - V^*\|_\infty \leq \gamma^k \|V - V^*\|_\infty \rightarrow 0$, guaranteeing convergence to V^* from any initialization.

2. **Why does the error bound for model-based RL scale as $1/(1 - \gamma)^2$? Where does each factor of $1/(1 - \gamma)$ come from?**

Answer: The first factor of $1/(1 - \gamma)$ comes from the evaluation operator bound: $(I - \gamma\hat{P}^\pi)^{-1}$ amplifies errors by at most $1/(1 - \gamma)$, reflecting how dynamics errors accumulate over the effective horizon. The second factor comes from bounding $\|V^\pi\|_\infty \leq 1/(1 - \gamma)$, since Q-values scale with the sum of discounted rewards. Together, they give the quadratic scaling $1/(1 - \gamma)^2$.

3. **A classmate says: “Approximation error occurs whenever we use finite samples.” Is this correct? When exactly does approximation error appear?** (Hint: tabular vs. function approximation.)

Answer: Incorrect. Using finite samples causes *sampling error*, not approximation error. Approximation error occurs only when using a restricted function class (neural networks, linear models, etc.) that cannot perfectly represent the Bellman backup target. In the tabular case, the function class can represent any function over the finite state-action space, so there is zero approximation error – only sampling error remains.

4. **In the simulation lemma $Q^\pi - \hat{Q}^\pi = \gamma(I - \gamma\hat{P}^\pi)^{-1}(P - \hat{P})V^\pi$, explain intuitively why the error depends on V^π . Why is dynamics error in high-value states worse than in low-value states?**

Answer: The term $(P - \hat{P})V^\pi$ weights the dynamics error by the value of the next state. Errors in predicting transitions to high-value states matter more because those transitions affect the expected return more – mistakenly routing the agent toward or away from high-reward regions has a large impact on cumulative reward. Errors in transitions to low-value states are less consequential because those states contribute little to the total return regardless.

5. **For a model-based RL algorithm with $\gamma = 0.99$, roughly how many times more samples would you need compared to $\gamma = 0.9$ to achieve the same error ϵ ?** (Hint: consider the $1/(1 - \gamma)^2$ scaling.)

Answer: The error scales as $1/(1 - \gamma)^2 \cdot 1/\sqrt{N}$. For $\gamma = 0.9$: $1/(1 - 0.9)^2 = 100$. For $\gamma = 0.99$: $1/(1 - 0.99)^2 = 10,000$. The ratio is $10,000/100 = 100$. Since N appears as $1/\sqrt{N}$, to compensate for 100x larger constant, we need N to increase by $100^2 = 10,000$ times. So $\gamma = 0.99$ requires roughly **10,000 times** more samples than $\gamma = 0.9$.

Chapter 16

Algorithm Summary & Comparison

This chapter provides a comprehensive reference table for all RL algorithms covered in CS 185/285. For each algorithm, we list its key properties, equations, strengths, weaknesses, and when to use it. Cross-references to the chapters where each algorithm is discussed in depth are provided.

16.1 Imitation Learning Algorithms

16.1.1 Behavioral Cloning (BC)

Property	Value
Category	Imitation Learning
On/Off-policy	Off-policy (learns from demonstrations)
Key idea	Supervised learning on expert demonstrations: learn $\pi(a \mid s)$ directly
Objective/Loss	$L = -\mathbb{E}_{(s,a) \sim D_{\text{expert}}} [\log \pi_{\theta}(a \mid s)]$
Key equations	$\theta^* = \arg \max_{\theta} \sum_i \log \pi_{\theta}(a_i \mid s_i)$
Strengths	Extremely simple; no environment interaction needed; widely used in practice
Weaknesses	Distributional shift (compounding errors); performance degrades as $O(T^2)$ with horizon
When to use	Abundant expert data; short horizons; as initialization for RL
Intuition	“Copy the teacher” – learn to map observations to actions by mimicking demonstrations
Chapter ref	Chapter 2

16.1.2 DAgger (Dataset Aggregation)

Property	Value
Category	Imitation Learning (interactive)
On/Off-policy	On-policy data collection with off-policy labels
Key idea	Iteratively collect data with the learned policy, get expert labels, aggregate datasets
Objective/Loss	Same as BC, but on aggregated dataset $D = D_1 \cup D_2 \cup \dots$
Key equations	1. Train π on D 2. Run π , collect states 3. Query expert for actions 4. $D \leftarrow D \cup \text{new data}$; Repeat
Strengths	Addresses distributional shift; performance degrades as $O(T)$ instead of $O(T^2)$
Weaknesses	Requires interactive expert; expensive to query expert repeatedly
When to use	When an interactive expert is available; when BC has distributional shift problems
Intuition	“Practice with feedback” – drive the car yourself, but have the instructor correct you when you go off-track
Chapter ref	Chapter 2

16.2 Policy Gradient Algorithms

16.2.1 REINFORCE

Property	Value
Category	Policy Gradient
On/Off-policy	On-policy
Key idea	Estimate policy gradient using complete episode returns
Objective/Loss	$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [\sum_t r(s_t, a_t)]$
Key equations	$\nabla J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [\sum_t \nabla \log \pi_\theta(a_t s_t) \cdot (\sum_{t'=t}^T \gamma^{t'-t} r(s_{t'}, a_{t'}))]$
Strengths	Simple; unbiased gradient estimates; works with any differentiable policy
Weaknesses	Very high variance; requires many samples; on-policy only
When to use	Simple problems; when you need a baseline implementation; educational purposes
Intuition	“Trial and error” – try things, increase probability of actions that led to good outcomes, decrease for bad
Chapter ref	Chapter 5

16.2.2 REINFORCE with Baseline

Property	Value
Category	Policy Gradient
On/Off-policy	On-policy
Key idea	Subtract a state-dependent baseline $b(s_t)$ from returns to reduce variance
Objective/Loss	Same as REINFORCE
Key equations	$\nabla J(\theta) = \mathbb{E}[\sum_t \nabla \log \pi_\theta(a_t s_t) \cdot (R_t - b(s_t))]$; optimal $b(s_t) \approx V^\pi(s_t)$
Strengths	Significantly lower variance than REINFORCE; still unbiased
Weaknesses	Still high variance compared to actor-critic; still on-policy
When to use	When REINFORCE has too much variance; stepping stone toward actor-critic
Intuition	“Grade on a curve” – instead of asking “was this good?”, ask “was this better than average?”
Chapter ref	Chapter 5

16.2.3 Vanilla Actor-Critic

Property	Value
Category	Actor-Critic
On/Off-policy	On-policy
Key idea	Use a learned value function (critic) as baseline and for bootstrapping
Objective/Loss	Actor: $J(\theta) = \mathbb{E}[\sum_t \nabla \log \pi(a_t s_t) \cdot A^\pi(s_t, a_t)]$ Critic: $L(\phi) = \mathbb{E}[(V_\phi(s_t) - y_t)^2]$
Key equations	$A^\pi(s_t, a_t) \approx r(s_t, a_t) + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ (TD error as advantage)
Strengths	Lower variance than REINFORCE; can update online (single-step)
Weaknesses	Biased (due to bootstrapping); critic errors affect actor; on-policy
When to use	Continuous action spaces; when sample efficiency matters more than REINFORCE
Intuition	“Director and critic” – the actor proposes actions, the critic evaluates them
Chapter ref	Chapter 6

16.2.4 A2C / A3C (Advantage Actor-Critic / Asynchronous)

Property	Value
Category	Actor-Critic
On/Off-policy	On-policy
Key idea	Parallel workers collecting experience; advantage function for lower variance
Objective/Loss	Same as vanilla actor-critic with advantage estimation
Key equations	$A(s_t, a_t) = \sum_{l=0}^{k-1} \gamma^l r_{t+l} + \gamma^k V(s_{t+k}) - V(s_t)$ (n-step advantage)
Strengths	Parallelizable; stable training; good balance of bias/variance

Property	Value
Weaknesses	On-policy (sample inefficient); synchronization overhead (A3C)
When to use	When parallel compute is available; general-purpose on-policy RL
Intuition	“Many cooks in the kitchen” – multiple agents explore simultaneously and share learnings
Chapter ref	Chapter 6

16.2.5 GAE (Generalized Advantage Estimation)

Property	Value
Category	Advantage Estimation (used within actor-critic methods)
On/Off-policy	On-policy
Key idea	Exponentially-weighted average of n-step advantage estimates; λ controls bias-variance
Objective/Loss	N/A (estimation technique, not standalone algorithm)
Key equations	$A_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$ where $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$
Strengths	Smooth bias-variance tradeoff via λ : $\lambda = 0$ gives TD (low variance, biased); $\lambda = 1$ gives MC (unbiased, high variance)
Weaknesses	Adds a hyperparameter λ ; still requires on-policy data
When to use	Within PPO or any actor-critic method; almost always beneficial
Intuition	“Weighted average of opinions” – combine short-term (biased but low variance) and long-term (unbiased but high variance) advantage estimates
Chapter ref	Chapter 6

16.2.6 TRPO (Trust Region Policy Optimization)

Property	Value
Category	Policy Gradient (constrained)
On/Off-policy	On-policy
Key idea	Constrain policy updates to a trust region (KL divergence constraint) for monotonic improvement
Objective/Loss	$\max_{\theta} \mathbb{E}_{s \sim d^{\pi_{\text{old}}}, a \sim \pi_{\text{old}}} \left[\frac{\pi_{\theta}(a s)}{\pi_{\theta_{\text{old}}}(a s)} A^{\pi_{\text{old}}}(s, a) \right]$ s.t. $D_{\text{KL}}(\pi_{\theta_{\text{old}}} \ \pi_{\theta}) \leq \delta$
Key equations	Solved via conjugate gradient + line search on natural gradient direction
Strengths	Theoretically motivated; monotonic improvement guarantee; stable
Weaknesses	Complex implementation (conjugate gradient, line search); computationally expensive
When to use	When stability is critical; continuous control; when PPO is not available
Intuition	“Small careful steps” – take the biggest step you can while guaranteeing the policy doesn’t degrade

Property	Value
Chapter ref	Chapter 9

16.2.7 Natural Policy Gradient

Property	Value
Category	Policy Gradient
On/Off-policy	On-policy
Key idea	Use the Fisher information matrix to take steepest ascent steps in distribution space (not parameter space)
Objective/Loss	Same as policy gradient
Key equations	$\theta \leftarrow \theta + \alpha F^{-1} \nabla J(\theta)$ $F = \mathbb{E}[\nabla \log \pi \cdot (\nabla \log \pi)^T]$ (Fisher information matrix)
Strengths	Invariant to parameterization; more stable than vanilla gradient descent
Weaknesses	Computing/inverting Fisher matrix is expensive; approximations needed
When to use	Theoretical understanding of TRPO; when parameter-space gradients perform poorly
Intuition	“Move in the direction that changes the <i>behavior</i> the most, not the <i>parameters</i> the most”
Chapter ref	Chapter 9

16.2.8 PPO (Proximal Policy Optimization)

Property	Value
Category	Policy Gradient (constrained, approximate)
On/Off-policy	On-policy
Key idea	Approximate TRPO using clipping or KL penalty for simpler implementation
Objective/Loss	PPO-Clip: $L = \mathbb{E}[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)]$ PPO-KL: $L = \mathbb{E}[r_t(\theta)A_t - \beta D_{\text{KL}}]$
Key equations	$r_t(\theta) = \pi_\theta(a_t s_t) / \pi_{\theta_{\text{old}}}(a_t s_t)$ (importance ratio)
Strengths	Simple to implement; stable; strong empirical performance; the most widely used on-policy method
Weaknesses	On-policy (sample inefficient); clipping can be overly conservative; hyperparameter-sensitive
When to use	Default choice for on-policy RL; continuous and discrete action spaces; RLHF for LLMs
Intuition	“Guardrails on policy updates” – allow improvement but clip extreme changes to prevent catastrophic updates
Chapter ref	Chapter 9

16.3 Value-Based Algorithms

16.3.1 Q-Learning (Tabular)

Property	Value
Category	Value-Based
On/Off-policy	Off-policy
Key idea	Learn Q^* directly by iteratively applying the Bellman optimality equation
Objective/Loss	$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$
Key equations	Bellman optimality: $Q^*(s, a) = r(s, a) + \gamma \mathbb{E}[\max_{a'} Q^*(s', a')]$
Strengths	Off-policy; converges to Q^* in tabular case with proper exploration
Weaknesses	Only for discrete actions; tabular version doesn't scale; exploration depends on ϵ -greedy
When to use	Small discrete MDPs; educational purposes; as foundation for DQN
Intuition	“Learn the optimal cheat sheet” – directly estimate how good each state-action pair is under the best possible policy
Chapter ref	Chapter 7

16.3.2 DQN (Deep Q-Network)

Property	Value
Category	Value-Based (Deep)
On/Off-policy	Off-policy
Key idea	Use neural network to approximate Q-function; stabilize with replay buffer and target network
Objective/Loss	$L(\phi) = \mathbb{E}_{(s,a,r,s') \sim D} [(Q_\phi(s, a) - (r + \gamma \max_{a'} Q_{\phi'}(s', a')))^2]$
Key equations	ϕ' = target network (updated slowly); ϵ -greedy exploration
Strengths	Off-policy (sample efficient); works from raw pixels; replay buffer reuse
Weaknesses	Discrete actions only; overestimation bias; unstable training possible
When to use	Discrete action spaces (Atari); when sample efficiency matters
Intuition	“Q-learning with a neural net brain” – same idea as tabular Q-learning but with a neural network and tricks for stability
Chapter ref	Chapter 7

16.3.3 Double DQN

Property	Value
Category	Value-Based (Deep)
On/Off-policy	Off-policy
Key idea	Decouple action selection from evaluation to reduce overestimation bias
Objective/Loss	$L(\phi) = \mathbb{E}[(Q_\phi(s, a) - y)^2]$ where $y = r + \gamma Q_{\phi'}(s', \arg \max_{a'} Q_\phi(s', a'))$
Key equations	Use online net to select action: $a^* = \arg \max_{a'} Q_\phi(s', a')$ Use target net to evaluate: $Q_{\phi'}(s', a^*)$
Strengths	Reduces overestimation bias; simple modification to DQN; better performance
Weaknesses	Still discrete actions only; doesn't completely eliminate bias
When to use	Whenever you use DQN – Double DQN is almost always better
Intuition	“Second opinion” – use one Q-network to choose the action and another to evaluate it, preventing optimistic self-deception
Chapter ref	Chapter 7

16.3.4 Value Iteration

Property	Value
Category	Dynamic Programming
On/Off-policy	N/A (requires known model)
Key idea	Iteratively apply Bellman optimality operator until convergence
Objective/Loss	$V(s) \leftarrow \max_a [r(s, a) + \gamma \sum_{s'} P(s' s, a) V(s')]$
Key equations	$\mathcal{T}V = \max_a [r + \gamma PV]$ $\ \mathcal{T}^k V - V^*\ _\infty \leq \gamma^k \ V - V^*\ _\infty$
Strengths	Guaranteed convergence (contraction mapping); finds optimal policy
Weaknesses	Requires known model and discrete states/actions; $O(S ^2 A)$ per iteration
When to use	Small MDPs with known dynamics; as theoretical tool
Intuition	“Backwards reasoning” – figure out what the last step should be, then the second-to-last, and so on
Chapter ref	Chapters 4, 15

16.3.5 Policy Iteration

Property	Value
Category	Dynamic Programming
On/Off-policy	N/A (requires known model)
Key idea	Alternate between policy evaluation (find V^π) and policy improvement ($\pi \leftarrow$ greedy w.r.t. V^π)
Objective/Loss	Evaluation: $V^\pi(s) = r(s, \pi(s)) + \gamma \sum_{s'} P(s' s, \pi(s)) V^\pi(s')$ Improvement: $\pi(s) = \arg \max_a Q^\pi(s, a)$

Property	Value
Key equations	Converges in at most $ A ^{ S }$ iterations (finite policy space)
Strengths	Converges faster than VI in practice; monotonic improvement at each step
Weaknesses	Requires known model; policy evaluation can be expensive
When to use	Small MDPs with known dynamics; when faster convergence than VI is desired
Intuition	“Evaluate and improve” – figure out how good your current plan is, then update it greedily
Chapter ref	Chapter 4

16.3.6 Fitted Value Iteration (Fitted VI)

Property	Value
Category	Value-Based (with function approximation)
On/Off-policy	Off-policy
Key idea	Approximate value iteration using a function approximator and samples
Objective/Loss	$V_\phi \leftarrow \arg \min_\phi \sum_i (V_\phi(s_i) - \max_a [r(s_i, a) + \gamma \sum_{s'} \hat{P}(s' s_i, a) V_{\phi_{\text{old}}}(s')])^2$
Key equations	Uses samples to approximate the Bellman backup
Strengths	Scales to large state spaces with function approximation
Weaknesses	No convergence guarantees with general function approximation; requires model
When to use	Large state spaces with known or learned model
Intuition	“Value iteration with a neural net” – approximate the Bellman backup using a function approximator
Chapter ref	Chapters 7, 15

16.3.7 Fitted Q-Iteration (FQI)

Property	Value
Category	Value-Based (with function approximation)
On/Off-policy	Off-policy
Key idea	Q-learning with function approximation, using a batch of data
Objective/Loss	$\hat{Q}_{k+1} \leftarrow \arg \min_{\hat{Q}} \ \hat{Q} - \hat{\mathcal{T}}\hat{Q}_k\ ^2$
Key equations	Error bound: $\lim_{k \rightarrow \infty} \ \hat{Q}_k - Q^*\ _\infty \leq \frac{1}{1-\gamma} \max_k \epsilon_k$ (approximation error) + sampling terms
Strengths	Off-policy; uses replay buffer; foundation for DQN
Weaknesses	Approximation error accumulates as $\frac{1}{1-\gamma} \cdot \epsilon$; may diverge with some function classes
When to use	Off-policy learning with function approximation; batch RL

Property	Value
Intuition	“Iterate Q-learning with a function approximator” – each step fits a new Q-function to the backed-up values
Chapter ref	Chapters 7, 15

16.4 Continuous Action Algorithms

16.4.1 DDPG (Deep Deterministic Policy Gradient)

Property	Value
Category	Actor-Critic (off-policy, deterministic)
On/Off-policy	Off-policy
Key idea	Actor-critic for continuous actions using a deterministic policy with Q-function
Objective/Loss	Critic: $L = \mathbb{E}[(Q_\phi(s, a) - (r + \gamma Q_{\phi'}(s', \mu_{\theta'}(s'))))^2]$ Actor: $J = \mathbb{E}[Q_\phi(s, \mu_\theta(s))]$
Key equations	$\nabla_\theta J = \mathbb{E}[\nabla_a Q_\phi(s, a) _{a=\mu_\theta(s)} \cdot \nabla_\theta \mu_\theta(s)]$ (deterministic policy gradient theorem)
Strengths	Handles continuous actions; off-policy; sample efficient
Weaknesses	Deterministic policy (less exploration); overestimation bias; brittle training
When to use	Continuous control tasks; when sample efficiency is important
Intuition	“Q-learning with a continuous action optimizer” – the actor learns to output the action that maximizes the critic
Chapter ref	Chapter 8

16.4.2 TD3 (Twin Delayed DDPG)

Property	Value
Category	Actor-Critic (off-policy, deterministic)
On/Off-policy	Off-policy
Key idea	Three tricks on DDPG: twin Q-networks (min), delayed actor updates, target policy smoothing
Objective/Loss	Critic ($i = 1, 2$): $L = \mathbb{E}[(Q_{\phi_i}(s, a) - y)^2]$ where $y = r + \gamma \min_{j=1,2} Q_{\phi'_j}(s', \tilde{a})$
Key equations	$\tilde{a} = \mu_{\theta'}(s') + \text{clip}(\mathcal{N}(0, \sigma), -c, c)$ (target policy smoothing)
Strengths	Addresses DDPG’s overestimation; more stable; state-of-the-art for continuous control
Weaknesses	Deterministic policy; more hyperparameters; target smoothing noise needs tuning

Property	Value
When to use	Continuous control; default choice when SAC is not used
Intuition	“DDPG with safety nets” – use the more pessimistic of two critics, update the actor less often, and add noise for robustness
Chapter ref	Chapter 8

16.4.3 SAC (Soft Actor-Critic)

Property	Value
Category	Actor-Critic (off-policy, maximum entropy)
On/Off-policy	Off-policy
Key idea	Maximize reward + entropy: learn a stochastic policy that’s as random as possible while still being good
Objective/Loss	$J(\pi) = \sum_t \mathbb{E}[r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot s_t))]$ Critic: $L = \mathbb{E}[(Q_\phi(s, a) - (r + \gamma(\min_j Q_{\phi'_j}(s', a') - \alpha \log \pi(a' s'))))^2]$
Key equations	Actor: $\theta^* = \arg \min_\theta \mathbb{E}_{s \sim D}[D_{\text{KL}}(\pi_\theta(\cdot s) \ \exp(Q_\phi(s, \cdot))/Z)]$
Strengths	Stochastic policy (natural exploration); stable; robust to hyperparameters; state-of-the-art
Weaknesses	Temperature α needs tuning (or auto-tuning); more complex than TD3
When to use	Default choice for continuous control; when exploration matters; robust tasks
Intuition	“Be as random as you can while still being good” – entropy bonus encourages diverse behaviors and robust policies
Chapter ref	Chapters 8, 10

16.5 Model-Based RL Algorithms

16.5.1 Dyna

Property	Value
Category	Model-Based RL
On/Off-policy	Off-policy
Key idea	Learn a dynamics model; generate synthetic rollouts from real states; combine with model-free RL
Objective/Loss	Model: $L = \mathbb{E}[\ f_\theta(s, a) - s'\ ^2]$; then use synthetic data in any off-policy RL algorithm
Key equations	1. Collect real data 2. Train model 3. Generate short rollouts from real states 4. Add to replay buffer 5. Update policy with model-free RL

Property	Value
Strengths	More sample efficient; uses model to augment data; short rollouts limit model error
Weaknesses	Model errors can compound; needs uncertainty estimation; more complex
When to use	When sample efficiency is critical; when environment interaction is expensive
Intuition	“Imagine and learn” – build a mental model of the world, imagine what would happen, and learn from your imagination
Chapter ref	Chapter 11

16.5.2 CEM / Random Shooting (Model Predictive Control)

Property	Value
Category	Model-Based RL (planning)
On/Off-policy	N/A (model-based planning)
Key idea	Sample random action sequences, evaluate with learned model, pick best (CEM: iteratively refine)
Objective/Loss	$a^* = \arg \max_a \sum_t r(s_t, a_t)$ evaluated under the learned model
Key equations	CEM: 1. Sample action sequences from $\mathcal{N}(\mu, \sigma)$ 2. Evaluate under learned model 3. Refit μ, σ to top- K sequences; Repeat
Strengths	Derivative-free; works with any model; simple to implement
Weaknesses	Short planning horizons; can’t handle long-term credit assignment; replanning at every step
When to use	Short-horizon tasks; when a good dynamics model is available
Intuition	“Try random plans and pick the best” – like a chess player who considers a few moves ahead and picks the most promising
Chapter ref	Chapter 11

16.5.3 MBPO (Model-Based Policy Optimization)

Property	Value
Category	Model-Based RL (Dyna-style)
On/Off-policy	Off-policy
Key idea	Dyna with principled model usage: short model rollouts from real data states, with monotonic improvement guarantee
Objective/Loss	Same as Dyna but with theoretical analysis of rollout length vs. model error tradeoff
Key equations	Optimal rollout length balances model error accumulation vs. distributional shift
Strengths	Principled; sample efficient; theoretically motivated rollout horizon selection

Property	Value
Weaknesses	Requires ensemble of models for uncertainty; still limited by model quality
When to use	When maximum sample efficiency is needed; continuous control with learned models
Intuition	“Dyna done right” – use the model for imagination, but be careful about how far you imagine
Chapter ref	Chapter 11

16.6 Inverse RL and Reward Learning

16.6.1 MaxEnt IRL (Maximum Entropy Inverse RL)

Property	Value
Category	Inverse Reinforcement Learning
On/Off-policy	N/A (reward learning)
Key idea	Infer reward function from demonstrations assuming expert maximizes entropy-regularized return
Objective/Loss	$\max_r \mathbb{E}_{\pi_{\text{expert}}} [\sum r(s, a)] - \log Z(r)$ where $Z(r) = \int \exp(\sum r(s, a)) d\tau$ (partition function)
Key equations	$\nabla_r L = \mathbb{E}_{\pi_{\text{expert}}} [\nabla_r r] - \mathbb{E}_{\pi_r} [\nabla_r r]$ (match feature expectations between expert and learned policy)
Strengths	Principled probabilistic framework; recovers transferable reward functions
Weaknesses	Requires solving RL in inner loop; computationally expensive; reward ambiguity
When to use	When you want a reward function (not just a policy); transfer learning; understanding expert behavior
Intuition	“What are they optimizing?” – watch someone act and figure out what reward function makes their behavior look optimal
Chapter ref	Chapter 12

16.7 Offline RL Algorithms

16.7.1 AWR (Advantage-Weighted Regression)

Property	Value
Category	Offline RL / Off-policy Policy Gradient
On/Off-policy	Off-policy
Key idea	Weight behavioral cloning by exponentiated advantage: upweight good actions, downweight bad
Objective/Loss	$L(\theta) = \mathbb{E}_{(s,a) \sim D} [\exp(A^\pi(s,a)/\beta) \cdot \log \pi_\theta(a s)]$
Key equations	Equivalent to KL-constrained policy improvement; β controls constraint strength
Strengths	Simple; no min-max optimization; works offline; stable
Weaknesses	Conservative; limited by data quality; advantage estimation can be noisy
When to use	Offline RL; when simplicity is desired; as component in RLHF pipelines
Intuition	“Weighted imitation” – imitate the data, but pay more attention to the good parts and less to the bad parts
Chapter ref	Chapter 13

16.7.2 CQL (Conservative Q-Learning)

Property	Value
Category	Offline RL
On/Off-policy	Off-policy (offline)
Key idea	Learn conservative Q-values: push down Q-values for OOD actions, push up for in-distribution actions
Objective/Loss	$L = \mathbb{E}_{(s,a,r,s') \sim D} [(Q(s,a) - y)^2]$ $+ \alpha (\mathbb{E}_{s \sim D, a \sim \mu} [Q(s,a)] - \mathbb{E}_{(s,a) \sim D} [Q(s,a)])$
Key equations	CQL penalty: minimize Q under sampling distribution μ (e.g., uniform), maximize Q under data distribution
Strengths	Provides lower bound on Q-values; prevents overestimation of OOD actions; principled
Weaknesses	Conservative (may be too pessimistic); α requires tuning; additional computation
When to use	Offline RL when overestimation of OOD actions is a concern; fixed datasets
Intuition	“When in doubt, be pessimistic” – assume untried actions are worse than they appear to prevent exploitation of Q-function errors
Chapter ref	Chapter 13

16.7.3 IQL (Implicit Q-Learning)

Property	Value
Category	Offline RL
On/Off-policy	Off-policy (offline)

Property	Value
Key idea	Learn Q-function without querying OOD actions by using expectile regression on V-function
Objective/Loss	V-loss: $L_\tau(V) = \mathbb{E}_{(s,a) \sim D} [\tau - \mathbf{1}(Q(s,a) - V(s) < 0) \cdot (Q(s,a) - V(s))^2]$ Q-loss: standard Bellman with V as target
Key equations	$V(s)$ approximates a high quantile of $Q(s,a)$ under the data distribution (τ close to 1 emphasizes upper quantile)
Strengths	Never queries OOD actions; simple; no explicit policy constraint needed; extracts policy via AWR
Weaknesses	Expectile regression can be tricky; τ requires tuning; may be overly conservative
When to use	Offline RL; when avoiding OOD action queries is important; combined with AWR for policy extraction
Intuition	“Learn the best you can from what you’ve seen” – estimate value by looking at the upper quantile of observed Q-values, without ever imagining actions you haven’t tried
Chapter ref	Chapter 13

16.7.4 Flow Matching + Q Ranking (“Flow Q-Learning”)

Property	Value
Category	Offline RL / Policy Extraction
On/Off-policy	Off-policy (offline or offline-to-online warm start)
Key idea	Fit a flow-matching policy to the dataset with BC, sample N plausible actions, and choose the one with the highest Q-value
Objective/Loss	Train π_{flow} with BC; act via $a^*(s) = \arg \max_i Q(s, a^{(i)})$ where $a^{(i)} \sim \pi_{\text{flow}}(\cdot s)$
Key equations	Best-of- N / sample-and-rank action selection with a critic trained by IQL, CQL, or another offline RL method
Strengths	Respects data support when $\pi_{\text{flow}} \approx \pi_\beta$; handles multimodal actions well; no need to backprop through diffusion sampling
Weaknesses	Search over sampled actions can be expensive; quality depends on candidate coverage and critic ranking accuracy
When to use	Offline or offline-to-online continuous control with multimodal actions; when Gaussian actors are too restrictive
Intuition	“Generate plausible actions, then let the critic rank them”
Chapter ref	Chapter 13

16.8 LLM/RLHF-Related Algorithms

16.8.1 GRPO (Group Relative Policy Optimization)

Property	Value
Category	Policy Gradient (for LLMs)
On/Off-policy	On-policy
Key idea	REINFORCE-style policy gradient for LLMs using group-level relative rewards as advantages
Objective/Loss	$L = -\mathbb{E}[\sum_t \min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t) - \beta D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})]$
Key equations	$A_i = \frac{r_i - \text{mean}(r_{\text{group}})}{\text{std}(r_{\text{group}})}$ (advantage is normalized within a group of completions)
Strengths	No need for a learned value function; simple; effective for LLM alignment
Weaknesses	High variance from group-level normalization; sensitive to group size
When to use	RLHF / LLM fine-tuning; when training a separate value function is impractical
Intuition	“Grade on a curve within each prompt” – generate multiple completions for a prompt, rank them by reward, and increase probability of relatively better ones
Chapter ref	Chapter 10

16.9 Exploration Algorithms

16.9.1 RND (Random Network Distillation)

Property	Value
Category	Exploration
On/Off-policy	Can be added to any algorithm
Key idea	Use prediction error of a learned network trying to match a fixed random network as exploration bonus
Objective/Loss	Bonus: $E(s) = \ f_\theta(s) - f_\phi(s)\ ^2$ where ϕ is fixed random, θ is learned
Key equations	$r^+(s, a) = r(s, a) + \eta \ f_\theta(s) - f_\phi(s)\ ^2$
Strengths	Simple; no density estimation needed; avoids noisy TV problem; effective
Weaknesses	Bonus weight η requires tuning; may not work well in all environments
When to use	Sparse reward environments; hard exploration problems (Montezuma’s Revenge); default exploration method

Property	Value
Intuition	“Surprise detector” – if a student neural net can’t predict a random teacher’s answers at a state, that state must be unfamiliar
Chapter ref	Chapter 14

16.10 Master Comparison Table

Algorithm	Category	On/Off	Key Idea	Error Scaling	Best For
BC	Imitation	Off	Supervised learning on demos	$O(T^2)$ compounding	Simple tasks, lots of demos
Dagger	Imitation	On+Off	Interactive aggregation	$O(T)$	Interactive expert available
REINFORCE	PG	On	Monte Carlo PG	High variance	Simple/educational
REINFORCE+PG	PG	On	PG with baseline	Medium variance	Stepping stone
Actor-Critic	AC	On	Learned critic baseline	Lower variance, biased	Continuous actions
A2C/A3C	AC	On	Parallel actor-critic	Scalable	Parallel compute
GAE	Estimation	On	λ -weighted advantages	Tunable bias/variance	Within any AC method
TRPO	PG	On	KL-constrained updates	Monotonic improvement	When stability critical
Natural PG	PG	On	Fisher-weighted gradients	Param-invariant	Theoretical foundation
PPO	PG	On	Clipped importance ratio	Practical and stable	Default on-policy
Q-Learning	Value	Off	Bellman optimality	Converges (tabular)	Small discrete MDPs
DQN	Value	Off	Neural Q + replay + target	N/A	Discrete actions (Atari)
Double DQN	Value	Off	Decouple select/evaluate	Less overestimation	Always over DQN

Algorithm	Category	On/Off	Key Idea	Error Scaling	Best For
DDPG	AC	Off	Deterministic policy gradient	N/A	Continuous control
TD3	AC	Off	Twin Q + delay + smoothing	Less overestimation	Continuous control
SAC	AC	Off	MaxEnt actor-critic	Robust	Default continuous
Value Iter.	DP	N/A	Bellman operator iteration	$O(\gamma^k)$ convergence	Small known MDPs
Policy Iter.	DP	N/A	Evaluate then improve	Fast convergence	Small known MDPs
Fitted VI	Value+FA	Off	VI with function approx	$O(\frac{\epsilon}{1-\gamma})$	Large state, known model
Fitted QI	Value+FA	Off	Q-iteration with FA	$O(\frac{\epsilon}{1-\gamma})$	Off-policy + FA
Dyna	MB	Off	Model-augmented replay	Depends on model quality	Sample efficiency
CEM/Shooting	MB Plan	N/A	Sample and refit action seqs	Short horizon only	Short-horizon planning
MBPO	MB	Off	Principled Dyna	Theoretical guarantees	Max sample efficiency
MaxEnt IRL	IRL	N/A	Infer reward from demos	N/A	Reward learning
AWR	Offline	Off	Advantage-weighted BC	Conservative	Simple offline RL
CQL	Offline	Off	Conservative Q -values	Lower bound on Q	Offline, prevent OOD
IQL	Offline	Off	Expectile regression	Conservative	Offline, no OOD queries
GRPO	PG/LLM	On	Group-relative advantages	N/A	LLM alignment
RND	Exploration	Any	Random net prediction error	N/A	Sparse reward exploration

16.11 Decision Flowchart: Which Algorithm Should I Use?

Step 1: Do you have demonstrations? - Yes, and that's all -> **Behavioral Cloning** (or **Dagger** if interactive expert available) - Yes, and you want a reward function -> **MaxEnt IRL** - No -> continue

Step 2: Do you have a fixed dataset (no environment interaction)? - Yes -> **Offline RL: CQL** (conservative Q-values), **IQL** (avoid OOD), or **AWR** (simple) - No -> continue

Step 3: Is your action space discrete or continuous? - Discrete -> **DQN / Double DQN** (off-policy, sample efficient) - Continuous -> continue

Step 4: How much do you care about sample efficiency? - A lot (expensive interactions) -> **SAC** or **TD3** (off-policy) or **Model-Based** (**MBPO**, **Dyna**) - Not much (cheap simulation) -> **PPO** (on-policy, simple, stable)

Step 5: Is exploration a major challenge (sparse rewards)? - Yes -> Add **RND** or pseudo-count exploration bonus to your chosen algorithm

Step 6: Are you fine-tuning an LLM? - Yes -> **PPO** or **GRPO** with reward model

Professor's Note

"In practice, PPO and SAC are the two workhorses. PPO for on-policy settings (LLMs, when you can afford many environment interactions), SAC for off-policy continuous control. DQN for discrete actions. Everything else is for specialized situations."

16.12 Key Themes Across Algorithms

16.12.1 The Bias-Variance Tradeoff (Chapters 5-6)

- Monte Carlo methods (REINFORCE): **unbiased**, high variance
- TD methods (Actor-Critic with bootstrapping): **biased**, lower variance
- GAE with λ : smooth interpolation between the two

16.12.2 On-Policy vs. Off-Policy (Chapters 5-8)

- On-policy (PPO, A2C): simpler theory, stable, but sample inefficient
- Off-policy (SAC, DQN, TD3): sample efficient (reuse old data), but harder to stabilize

16.12.3 The Deadly Triad (Chapter 7)

Function approximation + bootstrapping + off-policy learning can cause **divergence**. Solutions: target networks (DQN), clipping (PPO), entropy regularization (SAC).

16.12.4 Exploration vs. Exploitation (Chapter 14)

- Epsilon-greedy: simple but undirected
- UCB / Count-based: principled for small state spaces
- Pseudo-counts / RND: heuristic but effective for deep RL
- Thompson sampling: Bayesian, theoretically elegant

16.12.5 The $1/(1 - \gamma)$ Factor (Chapter 15)

This “effective horizon” appears everywhere in RL theory: - Q-values scale as $1/(1 - \gamma)$ (Eq. 15.7) - Approximation error amplified by $1/(1 - \gamma)$ (Eq. 15.55) - Model-based RL error scales as $1/(1 - \gamma)^2$ (Eq. 15.43) - Longer horizons = harder problems, universally

Chapter 17

Appendix: Key Identities and Formulas

This appendix collects the most important identities used throughout the course. These appear repeatedly in derivations, proofs, and exam problems.

17.1 A.1 The Log-Derivative Trick (REINFORCE Trick)

For any differentiable probability distribution $p_\theta(x) > 0$:

$$\nabla_\theta p_\theta(x) = p_\theta(x) \nabla_\theta \log p_\theta(x)$$

Proof: From the chain rule, $\frac{d}{d\theta} \log p_\theta = \frac{\nabla_\theta p_\theta}{p_\theta}$, so $\nabla_\theta p_\theta = p_\theta \nabla_\theta \log p_\theta$. $\square$

Why it matters: Converts the gradient of an expectation (hard) into an expectation of a gradient (easy, estimable by sampling):

$$\nabla_\theta \mathbb{E}_{x \sim p_\theta}[f(x)] = \mathbb{E}_{x \sim p_\theta}[\nabla_\theta \log p_\theta(x) \cdot f(x)]$$

Used in: Policy gradients (Ch 4), REINFORCE, actor-critic, entropy gradient derivation.

17.2 A.2 Score Function Has Zero Expectation

$$\mathbb{E}_{a \sim \pi_\theta(a|s)}[\nabla_\theta \log \pi_\theta(a|s)] = 0$$

Proof:

$$\sum_a \pi_\theta(a|s) \nabla_\theta \log \pi_\theta(a|s) = \sum_a \nabla_\theta \pi_\theta(a|s) = \nabla_\theta \underbrace{\sum_a \pi_\theta(a|s)}_{=1} = \nabla_\theta 1 = 0 \quad \square$$

Why it matters: This is why baselines work and why constant terms vanish from policy gradients.

17.3 A.3 Baselines Don't Introduce Bias

For any function $b(s)$ that does not depend on a :

$$\mathbb{E}_{a \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a|s) \cdot b(s)] = b(s) \cdot \underbrace{\mathbb{E}_{a \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a|s)]}_{=0} = 0$$

Consequence: We can subtract any baseline from the reward without changing the expected gradient:

$$\mathbb{E}[\nabla_\theta \log \pi_\theta \cdot (r - b)] = \mathbb{E}[\nabla_\theta \log \pi_\theta \cdot r]$$

The baseline only affects **variance**, not bias.

Used in: REINFORCE with baseline (Ch 4), advantage estimation, GRPO (Ch 11).

17.4 A.4 Policy Gradient Theorem

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_{t=0}^{H-1} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \Psi_t \right]$$

where Ψ_t can be any of: - $\sum_{t'=0}^{H-1} r(s_{t'}, a_{t'})$ — total trajectory reward - $\sum_{t'=0}^{H-1} r(s_{t'}, a_{t'}) - b$ — with baseline - $\sum_{t'=t}^{H-1} r(s_{t'}, a_{t'})$ — reward-to-go (lower variance, still unbiased) - $r(s_t, a_t) + \gamma V^\pi(s_{t+1}) - V^\pi(s_t)$ — TD advantage (biased if V is approximate) - $\hat{A}_t^{\text{GAE}}$ — GAE advantage (tunable bias-variance)

Key insight: The dynamics terms $p(s_{t+1}|s_t, a_t)$ drop out of $\nabla_\theta \log p_\theta(\tau)$ because they don't depend on θ .

17.5 A.5 Importance Sampling

$$\mathbb{E}_{x \sim p}[f(x)] = \mathbb{E}_{x \sim q}\left[\frac{p(x)}{q(x)}f(x)\right]$$

For policy gradients (off-policy):

$$\nabla_{\theta} J(\theta') \approx \mathbb{E}_{s \sim p_{\theta}, a \sim \pi_{\theta}} \left[\frac{\pi_{\theta'}(a|s)}{\pi_{\theta}(a|s)} \nabla_{\theta'} \log \pi_{\theta'}(a|s) \hat{A}^{\pi_{\theta}}(s, a) \right]$$

Important: The state marginal ratio $p_{\theta'}(s)/p_{\theta}(s)$ is intractable and is typically dropped (valid when $\pi_{\theta'} \approx \pi_{\theta}$).

Used in: PPO, TRPO, off-policy policy gradients (Ch 7).

17.6 A.6 KL Divergence

$$D_{\text{KL}}(p\|q) = \mathbb{E}_{x \sim p} \left[\log \frac{p(x)}{q(x)} \right] = \mathbb{E}_{x \sim p} [\log p(x) - \log q(x)]$$

Properties: - $D_{\text{KL}}(p\|q) \geq 0$, with equality iff $p = q$ - **Not symmetric:** $D_{\text{KL}}(p\|q) \neq D_{\text{KL}}(q\|p)$ in general - **Forward KL** $D_{\text{KL}}(p\|q)$: mode-covering (q must cover all of p) - **Reverse KL** $D_{\text{KL}}(q\|p)$: mode-seeking (q collapses onto one mode of p)

Used in: Trust regions (Ch 7), variational inference (Ch 8), offline RL constraints (Ch 13).

17.7 A.7 Total Variation Divergence

$$D_{\text{TV}}(p, q) = \frac{1}{2} \sum_x |p(x) - q(x)|$$

Bounded: $0 \leq D_{\text{TV}} \leq 1$.

Relationship to KL via Pinsker's inequality:

$$D_{\text{TV}}(p, q) \leq \sqrt{\frac{1}{2} D_{\text{KL}}(p\|q)}$$

Used in: Distributional shift bound (Ch 2), trust region theory (Ch 7), simulation lemma (Ch 15).

17.8 A.8 Jensen's Inequality

For a **concave** function ϕ (e.g., $\log$):

$$\phi(\mathbb{E}[X]) \geq \mathbb{E}[\phi(X)]$$

Specifically for $\log$:

$$\log \mathbb{E}[X] \geq \mathbb{E}[\log X]$$

Used in: ELBO derivation (Ch 8) — this is why the ELBO is a **lower** bound on $\log p(x)$.

Reverse (for convex ϕ): $\phi(\mathbb{E}[X]) \leq \mathbb{E}[\phi(X)]$.

17.9 A.9 Evidence Lower Bound (ELBO)

For a latent variable model $p(x) = \int p(x|z)p(z) dz$ with variational distribution $q(z)$:

$$\log p(x) = D_{\text{KL}}(q(z)||p(z|x)) + \underbrace{\mathbb{E}_{z \sim q}[\log p(x|z) + \log p(z) - \log q(z)]}_{\text{ELBO } \mathcal{L}(p,q)}$$

Since $D_{\text{KL}} \geq 0$:

$$\log p(x) \geq \mathcal{L}(p, q) = \mathbb{E}_{z \sim q}[\log p(x|z)] - D_{\text{KL}}(q(z)||p(z))$$

Tight when: $q(z) = p(z|x)$ (the true posterior).

Used in: VAEs (Ch 8), control as inference (Ch 9), latent state-space models (Ch 12).

17.10 A.10 Bellman Equations

Policy evaluation (for a fixed π):

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim p(s'|s, a)}[V^\pi(s')]$$

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(a|s)}[Q^\pi(s, a)]$$

Bellman optimality:

$$Q^*(s, a) = r(s, a) + \gamma \mathbb{E}_{s'}[\max_{a'} Q^*(s', a')]$$

$$V^*(s) = \max_a Q^*(s, a)$$

Matrix form (finite, discrete):

$$Q^\pi = (I - \gamma P^\pi)^{-1} r$$

17.11 A.11 Bellman Operator Contraction

The Bellman optimality operator $\mathcal{B}$ defined by $(\mathcal{B}V)(s) = \max_a[r(s, a) + \gamma \sum_{s'} p(s'|s, a)V(s')]$ is a γ -contraction in $\|\cdot\|_\infty$:

$$\|\mathcal{B}V - \mathcal{B}\bar{V}\|_\infty \leq \gamma \|V - \bar{V}\|_\infty$$

Consequence: Value iteration converges: $\|\mathcal{B}^k V - V^*\|_\infty \leq \gamma^k \|V - V^*\|_\infty \rightarrow 0$.

Warning: This only holds in $\|\cdot\|_\infty$ (tabular). With function approximation, the regression step projects in $\|\cdot\|_2$, and the composition $\Pi\mathcal{B}$ is **not** a contraction in any norm.

17.12 A.12 Entropy

$$\mathcal{H}(\pi(\cdot|s)) = -\mathbb{E}_{a \sim \pi}[\log \pi(a|s)] = -\sum_a \pi(a|s) \log \pi(a|s)$$

Gradient of entropy (derived using product rule + score function identity):

$$\nabla_\theta \mathcal{H}(\pi_\theta(\cdot|s)) = \mathbb{E}_{a \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(a|s) \cdot (-\log \pi_\theta(a|s) - 1)]$$

$$= \mathbb{E}_{a \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(a|s) \cdot (-\log \pi_\theta(a|s))]$$

(The -1 drops out by identity A.2.)

Used in: Maximum entropy RL (Ch 9), SAC (Ch 6), entropy-regularized policy gradient (exam Q3).

17.13 A.13 Soft Bellman Equations (Maximum Entropy RL)

$$Q^{\text{soft}}(s, a) = r(s, a) + \gamma \mathbb{E}_{s'}[V^{\text{soft}}(s')]$$

$$V^{\text{soft}}(s) = \alpha \log \sum_a \exp\left(\frac{1}{\alpha} Q^{\text{soft}}(s, a)\right) \quad (\text{soft max})$$

$$\pi^*(a|s) = \exp\left(\frac{1}{\alpha}(Q^{\text{soft}}(s, a) - V^{\text{soft}}(s))\right) = \exp\left(\frac{1}{\alpha} A^{\text{soft}}(s, a)\right)$$

Connection: As $\alpha \rightarrow 0$, soft max $\rightarrow$ hard max, and we recover standard RL.

17.14 A.14 Offline RL: KL Constraint as Reward Bonus

To enforce $D_{\text{KL}}(\pi \parallel \pi_\beta) \leq \varepsilon$ (reverse KL constraint on policy), add a bonus to the reward:

$$\tilde{r}(s, a) = r(s, a) + \lambda \log \pi_\beta(a|s)$$

Then run max-entropy RL with this modified reward. The $\log \pi_\beta$ term penalizes actions that the data-collection policy π_β wouldn't take (OOD actions).

For entropy regularization (SAC): $b(s, a) = -\log \pi(a|s)$ (encourages exploration).

For offline RL (KL constraint): $b(s, a) = \log \pi_\beta(a|s)$ (stays close to data).

17.15 A.15 Simulation Lemma (Model Error $\rightarrow$ Policy Regret)

If $D_{\text{KL}}(p(s'|s, a) \parallel f(s'|s, a)) < \varepsilon^2$ for all s, a , then the regret of the optimal policy π_F^* learned in the model F when deployed in the true MDP M satisfies:

$$\tilde{R}^M(\pi_F^*) \leq \frac{2\varepsilon\gamma}{(1-\gamma)^2}$$

Key scaling: Error grows as $\frac{1}{(1-\gamma)^2}$ — quadratic in the effective horizon. This is why model-based RL is harder for long-horizon tasks.

17.16 A.16 Distributional Shift Bound (Behavioral Cloning)

If $\mathbb{E}_{s_t \sim p_{\text{train}}} [\pi_\theta(a_t \neq \pi^*(s_t) | s_t)] \leq \varepsilon$, then:

$$D_{\text{TV}}(p_{\text{train}}, p_{\pi_\theta}) \leq \varepsilon t$$

$$J_{\text{cost}} = \sum_{t=1}^H \mathbb{E}[c(s_t, a_t)] \leq \varepsilon H + \varepsilon H(H+1) = O(\varepsilon H^2)$$

With DAgger: $O(\varepsilon H)$ (linear, not quadratic).

Key insight: The H^2 comes from two sources: εH mistakes from the policy itself, plus εH^2 additional mistakes from distributional shift (visiting unfamiliar states).

17.17 A.17 Value Function Notation Guide

Throughout this course, many variants of Q and V appear. This section clarifies every variant and when each is used.

17.17.1 The Core Definitions

Symbol	Name	Definition	Meaning
$Q^\pi(s, a)$	Q-function of π	$\mathbb{E} \left[\sum_{t'=t}^H \gamma^{t'-t} r(s_{t'}, a_{t'}) \mid s_t = s, a_t = a \right]$	True expected return from taking a in s , then following π forever
$V^\pi(s)$	Value function of π	$\mathbb{E}_{a \sim \pi} [Q^\pi(s, a)]$	True expected return from s under π (average over actions)
$A^\pi(s, a)$	Advantage of π	$Q^\pi(s, a) - V^\pi(s)$	How much better a is than average under π

Key relationships: - $V^\pi(s) = \mathbb{E}_{a \sim \pi} [Q^\pi(s, a)]$ (V is the expected Q) - $\mathbb{E}_{a \sim \pi} [A^\pi(s, a)] = 0$ (advantage averages to zero) - $Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} [V^\pi(s')]]$ (Bellman equation)

17.17.2 Optimal Value Functions

Symbol	Meaning
$Q^*(s, a)$	Q-function of the optimal policy π^* . Equals $Q^{\pi^*}(s, a)$. Satisfies the Bellman optimality equation: $Q^*(s, a) = r(s, a) + \gamma \mathbb{E}_{s'}[\max_{a'} Q^*(s', a')]$
$V^*(s)$	Value function of the optimal policy. $V^*(s) = \max_a Q^*(s, a)$
$A^*(s, a)$	Advantage under optimal policy. $A^*(s, a) = Q^*(s, a) - V^*(s)$. Note: $A^*(s, \pi^*(s)) = 0$ since the optimal action IS the average

Recovering the optimal policy: $\pi^*(a|s) = \mathbf{1}[a = \arg \max_a Q^*(s, a)]$ or equivalently $\arg \max_a A^*(s, a)$.

17.17.3 Learned / Approximate Value Functions

Symbol	Meaning	Where it appears
$Q_\theta(s, a)$ or $Q_\phi(s, a)$	Neural network approximation of Q, parameterized by θ or ϕ	DQN, SAC, fitted Q-iteration
$V_\phi(s)$ or $\hat{V}(s)$	Neural network approximation of V	Actor-critic, GAE
$\hat{Q}_k$	Learned Q-function at iteration k of an algorithm	RL theory (Ch 15)
$Q_{\bar{\theta}}(s, a)$	Target network Q-function (frozen copy of Q_θ , updated slowly)	DQN, SAC, TD3

Critical distinction (from Ch 15): $\hat{Q}_k \neq Q^{\pi_k}$ in general. $\hat{Q}_k$ is what your algorithm computes (may have approximation and sampling error). Q^{π_k} is the true Q-function of the policy your algorithm is currently using. These differ because of function approximation errors.

17.17.4 Policy-Specific Q-Functions

Symbol	Meaning
$Q^{\pi_\theta}(s, a)$	True Q-function of the parameterized policy π_θ

Symbol	Meaning
$Q^{\pi_\beta}(s, a)$	True Q-function of the behavior policy π_β (the policy that collected the offline data)
$Q^{\pi_k}(s, a)$	True Q-function of the policy at iteration k
$Q^{\pi_{\text{old}}}(s, a)$	True Q-function of the “old” policy (used in importance-sampled policy gradients)

In offline RL (Ch 13): - When IQL uses $\tau = 0.5$ (symmetric expectile): it learns Q^{π_β} (the behavior policy’s Q) - When IQL uses $\tau \rightarrow 1$ (asymmetric expectile): it approximates Q^* (the optimal Q) - The distinction matters because Q^{π_β} tells you how good the data-collection policy is, while Q^* tells you the best possible performance

17.17.5 Soft / Entropy-Regularized Value Functions

Symbol	Meaning
$Q^{\text{soft}}(s, a)$	Soft Q-function: accounts for future entropy in addition to reward
$V^{\text{soft}}(s)$	$= \alpha \log \sum_a \exp(Q^{\text{soft}}(s, a)/\alpha)$ (log-sum-exp, a “soft max”)
$A^{\text{soft}}(s, a)$	$= Q^{\text{soft}}(s, a) - V^{\text{soft}}(s)$. The policy is $\pi(a s) = \exp(A^{\text{soft}}/\alpha)$

Connection: As temperature $\alpha \rightarrow 0$: soft max $\rightarrow$ hard max, and $Q^{\text{soft}} \rightarrow Q^*$.

17.17.6 Summary: How They All Relate

$$\underbrace{Q^*}_{\text{optimal, true}} \quad \underbrace{Q^{\pi_k}}_{\text{policy k, true}} \quad \underbrace{\hat{Q}_k}_{\text{policy k, learned}} \quad \underbrace{Q_\theta}_{\text{neural net params}} \quad \underbrace{Q_{\bar{\theta}}}_{\text{target net}}$$

- **Theory cares about:** $Q^* - Q^{\pi_k}$ (how far is our policy from optimal?)
- **Algorithms compute:** $\hat{Q}_k$ (which approximates Q^{π_k} or Q^* , depending on the algorithm)
- **The gap** $\hat{Q}_k - Q^{\pi_k}$ is the **approximation + sampling error** (Ch 15)
- **DQN/SAC** use Q_θ (online network) and $Q_{\bar{\theta}}$ (target network) to stabilize learning

Chapter 18

Appendix B: Practice Midterm Solutions and Additional Problems

This appendix provides detailed solutions to the practice midterm from the course, plus additional problems in the same format for extra preparation.

18.1 B.1 Answer Key (Practice Midterm)

Problem	Answer	Topic
1.1	(e)	BC failure probability formulation
1.2	(c)	TV divergence bound step
1.3	(b)	Tightest upper bound on failure
2.1	(d)	Open-loop vs closed-loop state occupancy
2.2	(d)	CEM properties
3	(b)	Entropy-regularized policy gradient
4	(a), (g)	Sim-to-real regret bounds
5.1	(d)	Policy gradient computation
5.2	(f)	Optimal return
5.3	(c)	Initial θ for optimal policy
6.1	(a)	IL regret bound $O(T^2\varepsilon)$
6.2	(b)	Terminal reward only $O(T\varepsilon)$
7.1	(a)	IQL unbiased for Q^*
7.2	(a), (b), (c), (d)	IQL unbiased for Q^{π_β}
8	(b), (d), (f), (j), (k), (l)	LLM MDP properties
9.1	$\geq, ?$	Chunked MDP comparison (general)
9.2	$=, =$	Chunked MDP (deterministic)
10	(d)	KL constraint as reward bonus

18.2 B.2 Detailed Solutions

18.2.1 Problem 1: Distribution Shift for Success/Failure Tasks

Setup: Train π_θ via BC on data from a deterministic optimal policy π^* that always succeeds. The training assumption is $\mathbb{E}_{s_t \sim p_{\text{train}}}[\pi_\theta(a_t \neq \pi^*(s_t)|s_t)] \leq \varepsilon$. We want to bound $p_\theta(\tau \text{ failed})$.

Part (1) — Answer: (e)

The policy fails when its final state s_H is not in the success set Ω . The final state is sampled under π_θ 's distribution (not the training distribution), so:

$$p_\theta(\tau \text{ failed}) = \sum_{s_H} p_{\pi_\theta}(s_H) \mathbf{1}(s_H \notin \Omega)$$

This is answer (e). Options (a)-(c) use p_{train} which is wrong — at test time we roll out π_θ and see states from p_{π_θ} . Option (d) forgets the sum. Option (f) has $\mathbf{1}(s_H \in \Omega)$ which measures success, not failure.

Part (2) — Answer: (c)

Following the BC bound derivation (Chapter 2, Eqs 14-17), we write:

$$p_\theta(\tau \text{ failed}) = \sum_{s_H} p_{\pi_\theta}(s_H) \mathbf{1}(s_H \notin \Omega)$$

Add and subtract $p_{\text{train}}(s_H)$:

$$= \sum_{s_H} (p_{\pi_\theta}(s_H) - p_{\text{train}}(s_H) + p_{\text{train}}(s_H)) \mathbf{1}(s_H \notin \Omega)$$

Upper-bound by taking absolute value of the difference:

$$\leq \sum_{s_H} (|p_{\pi_\theta}(s_H) - p_{\text{train}}(s_H)| + p_{\text{train}}(s_H)) \mathbf{1}(s_H \notin \Omega)$$

This matches (c). Note: indicator is $\mathbf{1}(s_H \notin \Omega)$ (failure), ruling out (a) and (b).

Part (3) — Answer: (b)

Continuing from part (2):

$$p_\theta(\tau \text{ failed}) \leq \sum_{s_H} |p_{\pi_\theta}(s_H) - p_{\text{train}}(s_H)| \cdot \mathbf{1}(\cdot) + \sum_{s_H} p_{\text{train}}(s_H) \mathbf{1}(s_H \notin \Omega)$$

- **First term:** bounded by $\sum_{s_H} |p_{\pi_\theta}(s_H) - p_{\text{train}}(s_H)| \leq \varepsilon H$ (from the TV divergence bound at $t = H$).
- **Second term:** equals 0 because π^* always succeeds, so $p_{\text{train}}(s_H) = 0$ for $s_H \notin \Omega$.

Therefore $p_\theta(\tau \text{ failed}) \leq \varepsilon H$, giving answer (b).

This is $O(\varepsilon H)$, **not** $O(\varepsilon H^2)$! The failure probability scales linearly because the expert never reaches failure states in the training data — only the shift matters, not the per-step cost.

18.2.2 Problem 2: Open Loop Control

Part (1) — Answer: (d) (*which is NOT true*)

- (a) **TRUE:** Open-loop policies can be non-stationary (time-varying). E.g., “do a_1 at $t=0$, a_2 at $t=1$ ” is not stationary.
- (b) **TRUE:** With deterministic dynamics and a fixed initial state, you can plan the optimal action sequence in advance (same as running the closed-loop policy).
- (c) **TRUE:** Example: a stochastic MDP where the optimal action depends on an observed random outcome. Open-loop can’t react, closed-loop can.
- (d) **FALSE (answer):** Given an open-loop policy $\mathbf{A}$, there may be no Markovian closed-loop policy π with the same state occupancy. Reason: the open-loop’s visited states can depend on the specific action sequence, which a Markovian π can’t fully replicate from state alone.

Part (2) — Answer: (d)

- (a) **FALSE:** CEM is a local method and can converge to local optima. It is not guaranteed to find the global optimum.
- (b) **FALSE:** CEM with uniform $p(A_t)$ on iteration 1 is random shooting for iteration 1 only — but on later iterations CEM refits to elites, making it different.
- (c) **FALSE:** CEM works with any updating scheme; “elites-dependent” is CEM’s specific update but not a convergence requirement.
- (d) **TRUE:** CEM is gradient-free — it only evaluates the model, doesn’t differentiate through it. This is its main advantage (see Ch 12.5 and Ch 16 Algorithm Summary).
- (e) **FALSE:** CEM works for continuous action spaces (using Gaussian distributions).

18.2.3 Problem 3: Entropy-Regularized Policy Gradient

Answer: (b)

See Appendix A.12 for the full derivation. For the bandit case $J(\theta) = \mathbb{E}_s[\mathbb{E}_{a \sim \pi_\theta}[r(s, a)] + \mathcal{H}(\pi_\theta(\cdot|s))]$:

- Reward term gradient: $\mathbb{E}[\nabla_\theta \log \pi_\theta(a|s) \cdot (r(s, a) - b)]$
- Entropy term gradient: $\mathbb{E}[\nabla_\theta \log \pi_\theta(a|s) \cdot (-\log \pi_\theta(a|s))]$ (derived in A.12)

Combining:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_\theta \log \pi_\theta(a_i|s) [r(s, a_i) - \log \pi_\theta(a_i|s) - b]$$

The “effective reward” is $r(s, a) - \log \pi_\theta(a|s)$, which penalizes overly confident (peaked) policies.

18.2.4 Problem 4: Learning in Simulation

Answer: (a) and (g) (*multiple may apply*)

By the simulation lemma (Appendix A.15 and Ch 15), if $D_{\text{KL}}(P\|f) < \varepsilon^2$, then:

By Pinsker's inequality:

$$D_{\text{TV}}(P, f) \leq \sqrt{\frac{1}{2} D_{\text{KL}}(P\|f)} < \sqrt{\frac{1}{2}} \varepsilon = \varepsilon/\sqrt{2}$$

The regret scales as:

$$\tilde{R}^M(\pi_F^*) \leq \frac{2\gamma}{(1-\gamma)^2} \cdot D_{\text{TV}}(P, f) \leq \frac{2\gamma \cdot \varepsilon/\sqrt{2}}{(1-\gamma)^2} = \frac{\sqrt{2}\gamma\varepsilon}{(1-\gamma)^2} = \varepsilon\gamma\sqrt{2}(1-\gamma)^{-2}$$

This matches (a). Answer (g) $\varepsilon\sqrt{2}(1-\gamma)^{-5/2}$ is a looser bound that also holds (it weakens the tighter bound by adding factors), so it is also valid.

Key insight: The $(1-\gamma)^{-2}$ scaling is quadratic in the effective horizon — this is a fundamental result.

18.2.5 Problem 5: Computing the Policy Gradient

Setup: MDP with $\pi_\theta(a_1|s) = 1 - \theta$, $\pi_\theta(a_2|s) = \theta$. Rewards: $r = 1$ on edge $s_3 \rightarrow s_0$ (from a_1), $r = 2/\gamma$ on edge $s_4 \rightarrow s_0$ (from a_1). From s_0 : $a_1 \rightarrow s_1$, $a_2 \rightarrow s_2$. From s_1 : $a_1 \rightarrow s_3$, $a_2 \rightarrow$ back to s_1 or similar. (Exact dynamics in problem figure.)

Part (1) — Answer: (d)

The policy gradient for this MDP (after computing trajectories and their probabilities):

$$\nabla_\theta J(\theta) = \frac{4\theta\gamma - 1}{1 - \gamma^3}$$

Part (2) — Answer: (f)

Setting gradient to zero: $4\theta\gamma - 1 = 0 \implies \theta^* = \frac{1}{4\gamma}$. But we also need $\theta \in [0, 1]$.

Plugging back into $J(\theta)$ gives $J(\theta^*) = \frac{2\gamma}{1-\gamma^3}$.

Part (3) — Answer: (c)

For $\gamma = 3/4$: $\theta^* = \frac{1}{4 \cdot 3/4} = \frac{1}{3}$.

Gradient direction: $\nabla J > 0$ when $\theta > 1/3$. So gradient ascent (the formula says ascent $\theta \leftarrow \theta + \alpha \nabla J$) from $\theta > 1/3$ moves toward larger $\theta = 1$ (one extreme). Actually the optimum at $\theta^* = 1/3$ is a critical point.

To recover the optimal policy via gradient ascent, we need to start with $\theta > 1/3$ so gradient pushes us up toward the optimal boundary. Hence answer (c): $\theta > 1/3$.

(Note: this depends on the exact landscape of $J(\theta)$ for this specific MDP. Check that $\theta = 1$ is optimal via second-order analysis or by direct computation at boundary.)

18.2.6 Problem 6: Imitation Learning Analysis

Part (1) — Answer: (a) $O(T^2\varepsilon)$

This is the standard BC distributional shift bound (Ch 2, Eq 18). The reward is bounded ($|r| \leq 1$) at **every** time step, and the compounding errors give $O(\varepsilon T^2)$ total regret.

Part (2) — Answer: (b) $O(T\varepsilon)$

When the reward only depends on the **last** state, the terminal-only reward is like Problem 1.3 above: the bound improves to $O(T\varepsilon)$.

Intuition: If only the final state matters, we only “pay” for distributional shift once (at s_T), not at every intermediate step. The TV divergence bound gives $D_{\text{TV}}(p_{\text{train}}, p_{\pi_\theta}) \leq \varepsilon T$, so the regret is at most $2 \cdot |r| \cdot \varepsilon T = O(T\varepsilon)$.

18.2.7 Problem 7: IQL Variants

See Chapter 13 (Offline RL) for IQL background. Variants (a) and (b) train both Q and V; variants (c) and (d) train only Q.

Part (1) — Answer: (a)

As $\tau \rightarrow 1$, expectile regression approximates the maximum (upper bound of the distribution). - (a) trains V_ψ to upper-bound $Q_\phi(s, a)$ across actions — this approximates $\max_a Q(s, a)$, which is the Bellman optimality backup. Unbiased in stochastic environments because the expectation over s' is handled by direct regression of Q . - (b) trains V_ψ to be upper-bounded by Q , which is wrong direction. - (c), (d): single-step expectile in Q-loss conflates the max over action distributions with the max over next-state outcomes. This causes **bias in stochastic environments** because the expectile also takes an upper-bound over s' outcomes, not just actions.

Part (2) — Answer: (a), (b), (c), (d) — all four

When $\tau = 0.5$, expectile loss reduces to MSE (symmetric, no upper bounding). All four variants become standard SARSA-style behavioral Q-function regression, which is unbiased.

18.2.8 Problem 8: LLM MDPs

Answers: (b), (d), (f), (j), (k), (l)

LLM MDP has **deterministic dynamics** (concatenation) and **finite discrete actions** (tokens).

- (a) **FALSE:** Baseline reduces variance (always helpful).
- (b) **TRUE:** Dynamics are deterministic concatenation — no benefit to learning them.
- (c) **FALSE:** Training $V(s)$ with TD or MC is possible.

- (d) **TRUE:** With deterministic dynamics, $Q(s, a) = r(s, a) + V(s')$ is directly computable from V — no separate network needed.
- (e) **FALSE:** Bias-variance tradeoff always exists (MC vs TD, baselines, etc.).
- (f) **TRUE:** Control-as-inference needs variational only when dynamics are stochastic (to fix the optimistic backup). Deterministic $\implies$ no optimism problem $\implies$ no variational fix needed.
- (g) **FALSE:** DQN works with discrete actions — LLM actions are discrete tokens.
- (h) **FALSE:** TRPO works with any policy class including LLMs.
- (i) **FALSE:** Offline RL is widely used for LLMs (e.g., DPO, offline RLHF).
- (j) **TRUE:** Next-token prediction on demonstrations = maximum likelihood of expert actions = behavioral cloning.
- (k) **TRUE:** With deterministic dynamics, value iteration and policy iteration produce the same updates.
- (l) **TRUE:** With deterministic dynamics, one policy improvement step suffices because the greedy improvement is exact.
- (m) **FALSE:** Maximizing action entropy at each state does **not** maximize the final state entropy — this is a subtle issue (related to concentration of $p^\pi(s_H)$).

18.2.9 Problem 9: Multi-Step Dynamics Models

Part (1) — Answer: $J^{(1)} \geq J^{(2)}$ **and** $J^{(2)} ? J^{(3)}$

- $J^{(1)} \geq J^{(2)}$: The 1-step (standard) optimal policy can always replicate any 2-chunk policy (just execute the chunked actions sequentially), but has more reaction points. So $J^{(1)} \geq J^{(2)}$.
- $J^{(2)} ? J^{(3)}$: The reaction time points don't nest! $M^{(2)}$ reacts at $t = 0, 2, 4, \dots$ while $M^{(3)}$ reacts at $t = 0, 3, 6, \dots$. Neither strictly dominates the other's decision schedule. Relation is **indeterminate** (?).

Part (2) — Answer: $J^{(1)} = J^{(2)}$ **and** $J^{(2)} = J^{(3)}$

With **deterministic dynamics** and a fixed initial state, you can plan everything optimally in advance — reactions don't help. All three give the same optimal value:

$$J^{(1)}(\pi_{(1)}^*) = J^{(2)}(\pi_{(2)}^*) = J^{(3)}(\pi_{(3)}^*)$$

18.2.10 Problem 10: Managing Distribution Shift in Offline RL

Answer: (d) $b(s, a) = \log \pi_\beta(a|s) - \log \pi(a|s)$

We want to enforce $D_{\text{KL}}(\pi \| \pi_\beta) = \mathbb{E}_{a \sim \pi}[\log \pi - \log \pi_\beta]$ as a Lagrangian penalty in the reward.

$$-\lambda D_{\text{KL}}(\pi \| \pi_\beta) = -\lambda \mathbb{E}_{a \sim \pi}[\log \pi(a|s) - \log \pi_\beta(a|s)]$$

$$= \lambda \mathbb{E}_{a \sim \pi} [\log \pi_\beta(a|s) - \log \pi(a|s)]$$

So the reward bonus is $b(s, a) = \log \pi_\beta(a|s) - \log \pi(a|s)$ — answer (d).

Intuition: This is like SAC’s entropy bonus $-\log \pi(a|s)$ plus an additional $\log \pi_\beta(a|s)$ term that pulls the policy toward actions the behavior policy would take (OOD action penalty).

18.3 B.3 Additional Practice Problems

These problems are in the same style as the practice midterm, covering topics across all chapters.

18.3.1 Problem A1: Baselines in Policy Gradient

Which of the following statements about baselines in REINFORCE are true? Select all that apply.

- (a) A constant baseline equal to the expected return is optimal for variance reduction.
- (b) A state-dependent baseline $b(s)$ introduces bias if it depends on the current state.
- (c) The learned value function $\hat{V}_\phi(s)$ is a valid baseline.
- (d) Subtracting a baseline that depends on the action a (not just state) preserves unbiasedness.
- (e) Baselines can reduce variance to zero in principle.

Answer: (c)

- (a) FALSE: The variance-minimizing constant baseline is $b^* = \mathbb{E}[\|\nabla \log \pi\|^2 r] / \mathbb{E}[\|\nabla \log \pi\|^2]$ (or $\mathbb{E}[x^2 r] / \mathbb{E}[x^2]$ in 1D), not the mean reward.
 - (b) FALSE: State-dependent baselines are unbiased (Appendix A.3).
 - (c) TRUE: $\hat{V}_\phi(s)$ is state-dependent.
 - (d) FALSE: Action-dependent baselines introduce bias because they correlate with $\nabla \log \pi_\theta(a|s)$.
 - (e) FALSE: The irreducible variance from MC returns always remains.
-

18.3.2 Problem A2: Bellman Operator Contraction

The Bellman optimality operator $\mathcal{B}$ satisfies $\|\mathcal{B}V_1 - \mathcal{B}V_2\|_\infty \leq \gamma\|V_1 - V_2\|_\infty$. Which of the following are true consequences?

- (a) Value iteration converges at rate γ^k .
- (b) The fixed point V^* is unique.
- (c) Fitted value iteration (with neural network projection) also contracts in $\|\cdot\|_\infty$.
- (d) Policy iteration converges in a finite number of steps for tabular MDPs.
- (e) Q-learning with linear function approximation always converges.

Answer: (a), (b), (d)

- (a) TRUE: Contraction implies geometric convergence.
- (b) TRUE: Banach fixed-point theorem.
- (c) FALSE: Projection uses $\|\cdot\|_2$, composition is not a contraction (see Ch 6.11).
- (d) TRUE: Tabular policy iteration has finitely many policies and improves monotonically.
- (e) FALSE: Q-learning with function approximation can diverge (deadly triad).

18.3.3 Problem A3: Variational Inference

For the ELBO $\mathcal{L}(p, q) = \mathbb{E}_{z \sim q}[\log p(x|z) + \log p(z) - \log q(z)]$, which is true?

- (a) $\mathcal{L}(p, q) = \log p(x)$ always.
- (b) $\mathcal{L}(p, q) \leq \log p(x)$ with equality iff $q(z) = p(z)$.
- (c) $\mathcal{L}(p, q) \leq \log p(x)$ with equality iff $q(z) = p(z|x)$.
- (d) Maximizing $\mathcal{L}$ w.r.t. q makes q closer to the prior $p(z)$.

Answer: (c)

The ELBO gap is exactly $D_{\text{KL}}(q(z)||p(z|x))$. The bound is tight when q equals the **true posterior**, not the prior.

18.3.4 Problem A4: Off-Policy Actor-Critic

Which are required for off-policy actor-critic to work correctly?

- (a) Use a Q-function $Q_\phi(s, a)$ instead of value function $V_\phi(s)$.
- (b) Sample actions for the policy gradient from the replay buffer.
- (c) Sample actions for the policy gradient from the **latest** policy π_θ .
- (d) Use importance sampling weights for both the Q-update and the policy update.
- (e) Use a target Q-network for stability.

Answer: (a), (c), (e)

- (a) TRUE: Q is defined per (s,a) pair, so replay-buffer states can be used directly.
- (b) FALSE: Buffer actions are from old policies; we need fresh actions from π_θ .
- (c) TRUE: Step 5 of the off-policy AC algorithm samples $a^\pi \sim \pi_\theta(\cdot|s_i)$.
- (d) FALSE: IS weights are only needed for the Q-update (to correct for old actions); the policy update uses freshly sampled actions so no IS needed.
- (e) TRUE: Target networks stabilize TD learning.

18.3.5 Problem A5: Soft Actor-Critic

In SAC, the policy objective is:

$$J(\pi) = \sum_t \mathbb{E}[r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t))]$$

Which of the following describes the role of α ?

- (a) Learning rate.
- (b) Discount factor substitute.
- (c) Temperature: high α favors exploration, low α favors exploitation.
- (d) Clipping threshold for the KL divergence.
- (e) Lagrangian multiplier for a target entropy constraint.

Answer: (c), (e)

(c) **TRUE:** Higher α weights the entropy term more heavily.

(e) **TRUE:** In modern SAC, α is auto-tuned as a Lagrangian multiplier for a target entropy (typically $-\dim(\mathcal{A})$).

18.3.6 Problem A6: Importance Sampling in PPO

In the PPO clipped objective $L = \mathbb{E}[\min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t)]$ where $r_t = \pi_{\theta'}(a_t|s_t)/\pi_{\theta}(a_t|s_t)$, which are true?

- (a) Clipping prevents the importance ratio from growing too large.
- (b) Clipping is applied asymmetrically based on the sign of $\hat{A}_t$.
- (c) PPO-Clip exactly enforces a KL constraint.
- (d) Clipping is equivalent to a trust region method.
- (e) Clipping removes the need for a baseline.

Answer: (a), (b)

- (a) **TRUE:** That's the main purpose.
 - (b) **TRUE:** For $\hat{A}_t > 0$, clip at $1 + \varepsilon$ (prevents over-increasing); for $\hat{A}_t < 0$, clip at $1 - \varepsilon$.
 - (c) **FALSE:** Clipping is a **heuristic** that approximates a KL constraint but doesn't enforce it exactly (unlike TRPO).
 - (d) **FALSE:** TRPO enforces KL exactly via second-order optimization; PPO is simpler but weaker.
 - (e) **FALSE:** Baselines still help reduce variance.
-

18.3.7 Problem A7: Distributional Shift in Model-Based RL

You train a dynamics model f_{θ} on data $\mathcal{D}$ from behavior policy π_{β} , then optimize a policy π_f against f_{θ} . Which of the following help prevent the policy from exploiting model errors?

- (a) Use an ensemble of models and average the planning objective over ensemble samples.
- (b) Train the model with maximum likelihood on more data.

(c) Constrain the policy to stay close to π_β (e.g., KL constraint).

(d) Re-collect data using the current π_f each iteration.

(e) Use a Bayesian neural network for the model.

(f) Train the model with adversarial loss.

Answer: (a), (c), (d), (e)

(a) TRUE: Ensembles capture epistemic uncertainty; averaging penalizes uncertain regions.

(b) FALSE: More data doesn't solve the targeted exploitation problem — the adversarial policy can still find small remaining errors.

(c) TRUE: Prevents OOD exploration of the model.

(d) TRUE: This is the Dyna/iterate fix — exactly like DAgger.

(e) TRUE: Bayesian NNs give epistemic uncertainty (but harder to train).

(f) FALSE: Adversarial training is for robustness, not for preventing policy-induced drift.

18.3.8 Problem A8: Inverse Reinforcement Learning

In MaxEnt IRL, the reward gradient is:

$$\nabla_\psi \mathcal{L} = \mathbb{E}_{\pi^*}[\nabla_\psi r_\psi(\tau)] - \mathbb{E}_{\pi_\theta}[\nabla_\psi r_\psi(\tau)]$$

Which are true?

(a) The first term pushes the reward up on expert trajectories.

(b) The second term pushes the reward down on policy trajectories.

(c) At convergence, the expert and learned policy have the same expected reward gradient.

(d) MaxEnt IRL requires solving an RL problem in the inner loop.

(e) MaxEnt IRL assumes experts are perfectly optimal.

Answer: (a), (b), (c), (d)

(a) TRUE: Maximum likelihood over expert demonstrations.

- (b) TRUE: “Energy-based” model interpretation.
 - (c) TRUE: At the optimum, expectations match.
 - (d) TRUE: Need to compute $\pi_\theta = \text{optimal policy under current reward at each step}$.
 - (e) FALSE: MaxEnt IRL assumes experts are *approximately* optimal (exponential-advantage distribution), not perfectly so — that’s the whole point of the MaxEnt formulation.
-

18.3.9 Problem A9: GRPO (Group Relative Policy Optimization)

GRPO for LLM fine-tuning: - Samples K completions for each prompt. - Computes advantage as $A_i = (r_i - \bar{r})/\text{std}(r_{\text{group}})$. - Uses PPO-style clipped objective + KL penalty to reference model.

Why does GRPO not need a value function?

- (a) The value function is approximated by the reward model.
- (b) Multiple completions per prompt provide a per-prompt baseline without needing V .
- (c) LLMs are single-step MDPs so $V = Q$.
- (d) GRPO replaces Q with the reward model directly.

Answer: (b)

Sampling K completions from the same prompt lets you estimate the baseline $\bar{r}$ empirically — this works because you can sample many times from the same state (prompt), which is not possible in general RL.

18.3.10 Problem A10: Control as Inference

In control-as-inference, the optimality variable is defined as $p(\mathcal{O}_t = 1 | s_t, a_t) = \exp(r(s_t, a_t))$.

Why is this exponential form chosen?

- (a) It’s the unique form that makes inference tractable.
- (b) It leads to a soft max over Q-values, recovering MaxEnt RL.
- (c) It ensures the optimal policy is deterministic.
- (d) It makes rewards non-negative.

(e) It makes the backward message recursion work out algebraically.

Answer: (b), (e)

(b) **TRUE:** Running inference with $\exp(r)$ gives $\pi(a|s) \propto \exp(Q(s, a)/\alpha)$ — the soft max.

(e) **TRUE:** The exponential form leads to clean recursions involving log-sum-exp.

(a) **FALSE:** Other forms are possible.

(c) **FALSE:** The optimal policy is *stochastic* (soft max).

(d) **FALSE:** Rewards must be non-**positive** (so $\exp(r) \leq 1$), not non-negative.

18.3.11 Problem A11: Exploration

Which of the following exploration methods use “prediction error as novelty”?

(a) Epsilon-greedy.

(b) UCB bonus based on visit counts.

(c) Random Network Distillation (RND).

(d) Thompson sampling.

(e) Dynamics prediction error as intrinsic reward.

(f) Pseudo-counts from density models.

Answer: (c), (e)

(c) **TRUE:** RND uses prediction error of a fixed random network as bonus.

(e) **TRUE:** Forward dynamics prediction error signals unfamiliar state-action pairs.

(a) **FALSE:** Epsilon-greedy is undirected noise.

(b) **FALSE:** UCB is count-based.

(d) **FALSE:** Thompson sampling is posterior-based.

(f) **FALSE:** Pseudo-counts are count-based / density-model based.

18.3.12 Problem A12: Advantage Estimators

Given a learned $\hat{V}_\phi^\pi$, which advantage estimators are **unbiased**?

(a) $\hat{A}(s_t, a_t) = \sum_{t'=t}^H \gamma^{t'-t} r(s_{t'}, a_{t'}) - \hat{V}_\phi^\pi(s_t)$ (MC return - learned V baseline)

- (b) $\hat{A}(s_t, a_t) = r(s_t, a_t) + \gamma \hat{V}_\phi^\pi(s_{t+1}) - \hat{V}_\phi^\pi(s_t)$ (1-step TD)
- (c) $\hat{A}(s_t, a_t) = \hat{Q}_\phi(s_t, a_t) - \hat{V}_\phi^\pi(s_t)$ (learned Q - learned V)
- (d) $\hat{A}(s_t, a_t) = \sum_{t'=t}^H \gamma^{t'-t} r(s_{t'}, a_{t'}) - b_t$ (MC - constant baseline)

Answer: (a), (d)

Key principle: **learned functions can be subtracted as baselines (unbiased), but replace the reward-to-go they introduce bias.**

- (a) TRUE: $\hat{V}$ is subtracted as state-dependent baseline.
 - (b) FALSE: $\hat{V}(s_{t+1})$ replaces the future return (not just subtracted), so errors propagate.
 - (c) FALSE: Both Q and V are learned; bias compounds.
 - (d) TRUE: Constant baseline is unbiased.
-

18.3.13 Problem A13: Offline RL OOD Problem

In offline RL, standard Q-learning fails because:

- (a) The data is not i.i.d.
- (b) The max operator over actions selects out-of-distribution (OOD) actions with overestimated Q-values.
- (c) The replay buffer is too small.
- (d) Gradients through the max are unstable.
- (e) OOD actions have lower variance estimates in the Q-function.

Answer: (b)

Without online exploration, the policy cannot correct for Q-function overestimation errors on OOD actions. The $\max_{a'} Q(s', a')$ in the Bellman backup finds actions with the highest estimated Q, which tend to be OOD actions with spurious high values. This is the central problem that CQL, IQL, AWR, etc. address (Ch 13).

18.3.14 Problem A14: Natural Policy Gradient

The natural policy gradient is:

$$\theta \leftarrow \theta + \alpha F(\theta)^{-1} \nabla_{\theta} J(\theta)$$

where $F(\theta)$ is the Fisher information matrix. Which are true?

- (a) The natural gradient approximately enforces a KL divergence constraint.
- (b) Computing $F(\theta)^{-1}$ directly requires $O(|\theta|^2)$ memory.
- (c) $F(\theta) = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta} \cdot (\nabla_{\theta} \log \pi_{\theta})^T]$.
- (d) The natural gradient is invariant to reparameterization of the policy.
- (e) TRPO uses conjugate gradient to avoid explicitly inverting F .

Answer: (a), (b), (c), (d), (e)

All true — these are standard properties of the natural gradient / TRPO.

18.3.15 Problem A15: RLHF Reward Hacking

Why do we add a KL penalty $-\beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$ to the RLHF objective?

- (a) To reduce training time.
- (b) To prevent the policy from drifting too far from the supervised fine-tuned model.
- (c) To discourage reward hacking (finding inputs that maximize reward model but aren't good).
- (d) To make the optimization convex.
- (e) To enforce that the policy is stochastic.

Answer: (b), (c)

(b) TRUE: Stays close to a known-good starting point.

(c) TRUE: Reward models are imperfect; the KL penalty prevents finding degenerate reward-model maximizers.

(a) FALSE: It is not added to reduce training time.

(d) FALSE: It does not make the optimization convex.

(e) FALSE: It does not enforce stochasticity by itself.

This concludes the additional practice problems. For further preparation, re-derive the bounds and gradients throughout Chapters 2, 4, 7, 8, 9, 13, and 15 — these are the chapters most likely to appear on exams.